

```
#include <iostream>
#include <vector>
#include "gen/core.h"
#include "gen/app.h"
```

```
int main() {
    gen::App app;
    app.setTitle("Mi App");
    app.init();
    app.run();
    return 0;
}
```



Introducción a

GEN FRAMEWORK

DESARROLLO DE APLICACIONES
MULTIPLATAFORMA EN C/C++



MULTIPLATAFORMA



C/C++



CODE FAST,
RUN FASTER...

RÁPIDO DESARROLLO • FÁCIL DE USAR • POTENTE Y ESCALABLE

Introducción a GEN Framework

Desarrollo de aplicaciones multiplataforma en C/C++

Contenidos

Capítulo I: C/C++, multiplataforma y el sentido de GEN Framework	14
1. El punto de partida	14
2. El origen del lenguaje C	15
3. C y C++ no son el mismo lenguaje	15
4. La evolución de C++	16
5. C/C++ frente a otros lenguajes	16
6. El problema real: la plataforma	17
7. Qué aporta GEN Framework	18
8. Desarrollo multiplataforma sin renunciar al rendimiento.....	18
9. La ausencia de una librería única en C++.....	19
10. Una sola aplicación, varias plataformas	19
11. La importancia de una arquitectura propia.....	20
12. C/C++ como decisión estratégica	21
13. Qué se puede aprender en este libro	22
Capítulo II: Preparando el entorno de desarrollo	22
1. Un capítulo orientado a la práctica	22
2. La página oficial del framework.....	23
3. La carpeta Projects como espacio de trabajo.....	24
4. Organización recomendada en Windows.....	25
5. Organización recomendada en Linux	26
6. Descarga mediante Git	27
7. Herramientas necesarias	28
8. Por qué usamos estas herramientas	29
9. Windows como entorno de desarrollo.....	29
10. Linux como entorno de desarrollo	30
11. WSL2 como entorno mixto Windows/Linux.....	31
12. Docker y Docker Compose.....	35
13. Los scripts de Common como punto de entrada.....	36
14. Convención de nombres en los scripts	36
15. Modificadores de los scripts.....	37
16. Modificadores de stage	37
17. Modificadores de plataforma en Windows	38

18. Modificadores de plataforma en Linux.....	39
19. Modificadores de modo	39
20. Modificadores de compilador	40
21. Modificadores de control de memoria.....	41
22. Modificadores de trazas	41
23. Modificadores de feedback	42
24. Modificador DOCKERS	42
25. Combinación de modificadores.....	43
26. Tabla resumen de modificadores	44
27. CMake como infraestructura interna	44
28. Visual Studio y Visual Studio Code.....	45
29. Proyectos propios al mismo nivel que GEN_FrameWork.....	45
30. Errores habituales durante la preparación.....	46
31. Cómo enfrentarse a un error de compilación	47
32. El entorno como parte del aprendizaje	48
33. Referencia rápida de compilación	50
Capítulo III: Visión general y arquitectura del framework.....	52
1. Una visión global antes de entrar en el detalle	52
2. La esencia modular de GEN Framework.....	52
3. El papel de las factorías	53
4. Factorías y aislamiento de plataforma	54
5. El confinamiento del código de plataforma	54
6. XFACTORY: eXtender Utils	55
7. DIOFACTORY: Data Input/Output	55
8. GRPFACTORY: Graphic Factory	56
9. INPFACTORY: Input Factory	57
10. SNDFACTORY: Sound Factory	57
11. Grupos adicionales y servicios transversales.....	57
12. Compression	58
13. Cipher	58
14. AppFlow	59
15. Scripts	59
16. UserInterface	59
17. Databases	60

18. Relación entre módulos.....	60
19. Cómo debe aproximarse el programador a esta arquitectura	60
20. Resumen de arquitectura	61
Capítulo IV: Primera aplicación con GEN Framework.....	63
1. El objetivo del capítulo	63
2. Las tres modalidades principales de aplicación.....	63
3. Por qué no escribimos la función main	64
4. El ciclo de vida de una aplicación con GEN Framework	64
5. Uso base sin la modalidad AppFlow de consola o gráfica	65
6. Aplicación de consola con AppFlow	65
7. Máquina de estados finitos con XFSMACHINE	66
8. Estructura típica de una aplicación de consola	68
9. XPATHMANAGER y la carpeta assets.....	68
10. Configuración principal de la aplicación	69
11. Mapeo de variables de configuración	70
12. Relación entre AppFlow, configuración y trazas	71
13. CMakeLists.txt mínimo para una aplicación de consola.....	71
14. Aplicación gráfica con AppFlowGraphics.....	72
15. Añadir UserInterface a una aplicación gráfica.....	73
16. Estructura típica de una aplicación gráfica.....	73
17. Esqueleto recomendado de una primera aplicación.....	75
18. Por qué este esqueleto importa	75
Capítulo V: XUtils, las utilidades extendidas del framework	77
1. El origen de XUtils.....	77
2. XUtils como base común del framework.....	77
3. XFACTORY y el aislamiento de plataforma	78
4. Mapa general de XUtils.....	78
5. Tipos básicos, cadenas, buffers y rutas	79
6. Colecciones y estructuras de datos	80
7. Tiempo, sistema y objetos dependientes de plataforma.....	81
8. Ficheros, directorios y XPATHSMANAGER.....	82
9. Formatos de fichero soportados en XUtils	83
10. Configuración con XFILECFG	84
11. Eventos, observadores y publicación	84

12. Máquina de estados finitos	85
13. Trazas, logs, memoria y feedback.....	86
14. Concurrencia, hilos y planificación	86
15. Serialización, codificación y estructuras binarias	87
16. Internacionalización, traducción y textos.....	87
17. Licencias, firmware, almacenamiento y recursos especiales	88
18. Módulos auxiliares y de integración.....	88
19. Cómo estudiar XUtils desde DevTestsConsole	89
20. Criterios de uso en una aplicación con GEN Framework.....	89
21. Resumen práctico	90
Capítulo VI: DataIO, entrada y salida de datos en GEN Framework	92
1. El origen de DataIO.....	92
2. DataIO como segundo bloque de GEN Framework.....	92
3. DIOFACTORY y el aislamiento de plataforma	92
4. Mapa general de DataIO.....	93
5. Direcciones, identificadores y utilidades de red.....	94
6. Streams: la abstracción principal de comunicación	94
7. Enumeración de dispositivos y descriptores	95
8. TLS y streams seguros.....	95
9. Protocolos de alto nivel.....	96
10. DIOCoreProtocol y DIOProtocol	96
11. Web, cliente HTTP y servidor embebido	97
12. ScraperWeb y obtención de información externa	97
13. Nodes: nodos, items, valores y handlers.....	97
14. Drivers, I2C, SPI y dispositivos físicos	98
15. Cámaras, notificaciones, alertas y actualización	98
16. Diagnóstico, conectividad y seguridad operativa.....	99
17. Relación con XUtils, AppFlow y las demás factorías.....	99
18. Catálogo de clases y módulos de DataIO.....	99
19. Cómo estudiar DataIO desde DevTestsConsole	106
20. Criterios de uso en una aplicación con GEN Framework.....	106
21. Resumen práctico	107
Capítulo VII: Graphic, el sistema gráfico de GEN Framework.....	109
1. El tercer bloque que dio forma a GEN Framework.....	109

2. GRPFACTORY y el aislamiento de plataforma	109
3. Graphic dentro de una aplicación AppFlowGraphics	110
4. Pantallas, viewports y propiedades gráficas	110
5. Canvas 2D, colores, paths y áreas de reconstrucción.....	111
6. Bitmaps, secuencias y formatos raster.....	112
7. Ficheros vectoriales: base común, SVG y DXF	112
8. Renderizado vectorial sobre canvas	113
9. OpenGL ES, EGL y presentación acelerada	113
10. Vídeo y extracción de frames	114
11. Gráficas estadísticas	114
12. Graphic y UserInterface.....	115
13. Mapa general de Graphic	115
14. Catálogo de clases y módulos de Graphic	116
15. Cómo estudiar Graphic desde los ejemplos	119
16. Criterios de uso en una aplicación con GEN Framework.....	119
17. Resumen práctico.....	120
Capítulo VIII: Input, dispositivos de entrada y abstracción de interacción.....	121
1. El bloque Input dentro de GEN Framework	121
2. Un bloque nacido después del sistema gráfico	121
3. La idea clave: descomponer los dispositivos en cursores y botones.....	121
4. INPFACTORY y el aislamiento de plataforma.....	122
5. INPMANAGER: el punto de acceso a los dispositivos.....	122
6. INPDEVICE: un dispositivo reducido a elementos comunes.....	123
7. INPBUTTON: pulsación, estado y transición	123
8. INPCURSOR: posición, cambio y selección	124
9. INPCURSORMOTIONPOINT e INPCURSORMOTION.....	124
10. El patrón de uso en las aplicaciones gráficas.....	125
11. Input y UserInterface.....	126
12. INPCAPTURE e INPCAPTURE_XEVENT	126
13. INPSIMULATE: entrada generada por software	127
14. Clases de plataforma	127
15. Relación entre clases	128
16. Ejemplo de esqueleto de entrada en una aplicación gráfica.....	128
17. Input en aplicaciones sin AppFlow	129

18. Errores habituales al usar Input.....	130
19. Catálogo resumido de clases del bloque Input.....	130
20. Resumen práctico	130
Capítulo IX: Sound, sonido y reproducción en GEN Framework.....	132
1. El último bloque factory de GEN Framework	132
2. SNDFACTORY y el aislamiento de plataforma	132
3. Mapa general del bloque Sound.....	133
4. SNDITEM: la unidad reproducible	133
5. SNDPLAYCFG: volumen y pitch por reproducción	134
6. Sonidos desde fichero: SNDFILE, OGG y WAV	134
7. SNDNOTE: sonidos generados por frecuencia y duración.....	135
8. Eventos de sonido.....	135
9. SNDOPENALFACTORY: implementación de reproducción	135
10. Buffer, source y play item.....	136
11. La máquina de estados de reproducción.....	137
12. Sound dentro de una aplicación gráfica	137
13. Sincronización de pisadas en Canvas2D	137
14. assets/sounds y organización de recursos	138
15. Patrón de cierre y limpieza	138
16. Relación con otros módulos del framework.....	139
17. Catálogo resumido de clases del bloque Sound	139
18. Criterios de uso en una aplicación con GEN Framework.....	140
19. Resumen práctico	140
Capítulo X: Cipher, encriptación y hashes en GEN Framework	142
1. El objetivo del capítulo	142
2. Hash, checksum, cifrado y firma no son lo mismo	142
3. Mapa general del bloque Cipher	142
4. La clase HASH.....	143
5. Checksums y hashes rápidos	144
6. Hashes criptográficos generales.....	144
7. HASHCOMP128V1	145
8. La clase CIPHER.....	146
9. Cifrado simétrico	146
10. Claves y representación interna	147

11. Ficheros de claves GKF y PEM.....	148
12. RSA y CIPHERRSA	148
13. Curve25519 / X25519	149
14. Certificados X.509 y raíces de confianza	150
15. Relación con assets y XPATHMANAGER	150
16. Activación del bloque en CMake	151
17. Catálogo de clases del bloque Cipher.....	151
18. Cómo estudiar Cipher desde DevTestsConsole	152
19. Criterios de uso en una aplicación con GEN Framework.....	152
20. Resumen práctico	153
Capítulo XI: Compress, compresión de datos y ficheros en GEN Framework	155
1. El objetivo del capítulo	155
2. El bloque Compress dentro de GEN Framework	155
3. Compresión de datos frente a compresión de ficheros	155
4. Mapa general de clases	156
5. COMPRESSBASE: la interfaz común.....	156
6. COMPRESSMANAGER: creación de algoritmos	157
7. COMPRESS_ZIP: compresión deflate sobre buffers	157
8. COMPRESS_GZ: compresión compatible con GZip.....	158
9. COMPRESS_LZW: compresión LZW y soporte de uso tipo GIF.....	158
10. COMPRESS_LZRW1KH: compresión rápida y ligera.....	159
11. XFILEZIP: paquetes ZIP y ficheros	159
12. Relación con assets y XPATHMANAGER	160
13. Ejemplo completo: comprimir y descomprimir un texto	160
14. Compresión, cifrado y orden de operaciones	161
15. Activación mediante macros y compilación	162
16. Errores habituales al usar Compress	162
17. Catálogo resumido del bloque Compress.....	162
18. Criterios de uso en una aplicación con GEN Framework.....	163
19. Resumen práctico	163
Capítulo XII: Databases, bases de datos SQL en GEN Framework	164
1. El papel de Databases dentro del framework	164
2. Por qué un conector común es importante	164
3. DB_SQL_FACTORY y selección del motor	165

4. Organización del bloque SQL.....	165
5. DB_SQL_DATABASE: la abstracción de base de datos.....	166
6. DB_SQL_CONNECTION: opciones de conexión	166
7. Configuración de base de datos en la aplicación.....	167
8. Integración con assets y XPATHMANAGER.....	167
9. DB_SQL_QUERY: construcción, binding y ejecución	168
10. Transacciones, commit, rollback y savepoints	168
11. Creación y gestión de tablas.....	169
12. DB_SQL_RESULT, DB_SQL_ROW y DB_SQL_ROWHEADER	169
13. DB_SQL_VARIANT: tipos de datos SQL en una forma común	170
14. DB_SQL_ERROR y gestión de errores	170
15. SQLite: base local en fichero.....	171
16. MySQL y MariaDB: conexión cliente/servidor.....	171
17. PostgreSQL: conector relacional avanzado	172
18. Ejemplo completo de flujo básico	172
19. Activación mediante CMake.....	173
20. Relación con AppFlow, DIO, XUtils y Cipher	173
21. Catálogo resumido del bloque Databases.....	174
22. Criterios de uso en una aplicación con GEN Framework.....	174
23. Errores habituales al usar Databases.....	175
24. Resumen práctico	175
Capítulo XIII: Scripts, algoritmos dinámicos y plugins en tiempo real	176
1. El papel de Scripts dentro de GEN Framework.....	176
2. De algoritmos compilados a algoritmos cargables.....	176
3. Tres lenguajes soportados.....	177
4. La clase SCRIPT como punto común	177
5. Carga, rutas y assets/scripts	178
6. Configuración de la lista de scripts.....	178
7. SCRIPT_CACHE: cache de scripts	179
8. Eventos y errores de script	179
9. Bibliotecas de funciones: SCRIPT_LIB y SCRIPT_LIB_FUNCTION	180
10. Bibliotecas internas estándar	180
11. Bibliotecas opcionales e integración con módulos de GEN Framework	181
12. SCRIPT_LNG_JAVASCRIPT	182

13. SCRIPT_LNG_LUA.....	182
14. SCRIPT_LNG_G: el lenguaje .GEN	183
15. Integración en ScriptsExample	184
16. Activación mediante CMake.....	184
17. Ampliar el sistema con funciones propias	185
18. Ejemplos de uso.....	185
19. Relación con otros módulos	186
20. Criterios de uso.....	186
21. Errores habituales al usar Scripts	186
22. Catálogo resumido del bloque Scripts	186
23. Resumen práctico.....	187
Capítulo XIV: UserInterface, widgets y diseño de interfaces multiplataforma.....	188
1. El lugar de UserInterface dentro de GEN Framework	188
2. Arquitectura general y flujo de ejecución	188
3. Carga de una interfaz y ciclo mínimo	189
4. UI_MANAGER y UI_LAYOUT: gestión global y composición.....	189
5. UI_ELEMENT: base común de todos los widgets.....	190
6. Geometría, límites y adaptación de coordenadas.....	190
7. Configuración XML y estilos declarativos	191
8. Colores, textos, animaciones y fondos compartidos.....	191
9. Skins y renderizado sobre el canvas	192
10. Propiedades reutilizables: selección, edición y scroll.....	192
11. Widgets de contenido: texto, imágenes y animación	193
12. Widgets de selección: Option, Button y CheckBox.....	193
13. MultiOption y representación por estados	193
14. Contenedores, formularios, menús y listas	194
15. Edición de texto y teclado virtual	195
16. Indicadores de progreso lineales, radiales y por imagen	195
17. Eventos de entrada y salida	196
18. Textos automáticos, traducción y datos dinámicos	196
19. Interfaces adaptables a pantallas y displays diferentes	197
20. Actualización, áreas de reconstrucción y rendimiento	197
21. Ejemplo integrado basado en UI_Options.....	198
22. Catálogo completo del bloque UserInterface.....	198

23. Errores habituales al trabajar con UserInterface	199
24. Criterios de diseño para una aplicación con GEN Framework.....	199
25. Resumen práctico	199
Capítulo XV: ThirdPartyLibraries, crecer sobre cimientos abiertos	200
1. Criterios de selección e integración.....	200
2. Alcance de esta revisión	200
3. Inventario general	201
4. Gráficos, imágenes y tipografías.....	201
5. Sonido y multimedia.....	203
6. Lenguajes de script y utilidades de C++	203
7. Compresión y formatos contenedores.....	204
8. Bases de datos SQL.....	204
9. Captura de red y funciones específicas de Windows	205
10. Hardware embebido y SDK de plataforma	206
11. Pruebas unitarias	207
12. Discrepancias detectadas en el inventario	207
13. Lectura práctica de las licencias	207
14. Política recomendada para mantener ThirdPartyLibraries	208
15. Resumen práctico	208
16. Fuentes de la revisión	208
Anexo I: Guía de estilo de código C++ en GEN Framework	209
Resumen global del estilo de GEN Framework	209
Nombres de ficheros y módulos.....	209
Estructura de una cabecera.....	210
Estructura de un fichero .cpp	211
Separadores de sección	212
Indentación, llaves y espaciado	212
Convenciones de nombres	213
Ciclo de vida de una clase	213
Patrón singleton en GEN Framework	214
Documentación Doxygen	215
Tipos públicos y API	216
Memoria y propiedad de objetos	216
Reglas de includes	216

Flujo de control.....	217
Código de plataforma y factorías	217
Trazas y diagnóstico.....	217
Estilos que se deben evitar	217
Plantilla mínima de módulo.....	218
Lista de comprobación para nuevos módulos.....	219
Regla final de estilo.....	219
Anexo II: Utilidades — eXtended Trace Monitor	219
Qué es una traza y qué problema resuelve	220
Trazas y logs no son lo mismo	220
Arquitectura del sistema de trazas de GEN Framework.....	220
Configuración básica desde una aplicación con GEN Framework.....	221
Emisión de mensajes, colores y bloques de datos	221
Lectura de la ventana principal	222
Organización por máquina y aplicación.....	224
Filtros por IP, texto, nivel, color y tipo.....	224
Ventana de variables importantes	225
Pausa, limpieza y guardado de una captura	225
Transporte remoto, UDP, pipeline y UART	225
Control del volumen y coste de las trazas	226
Uso recomendado durante una sesión de diagnóstico	226
Seguridad y despliegue.....	226
Clases y componentes relacionados.....	226
Resumen práctico	227
Anexo III: Preguntas frecuentes — Preguntas y respuestas	228
1. Conceptos fundamentales.....	228
2. Primeros pasos	229
3. Arquitectura, módulos y aplicaciones	229
4. Plataformas, compilación y rendimiento.....	230
5. Calidad, diagnóstico y seguridad	231
6. Bibliotecas de terceros, licencias y mantenimiento	232
7. Adopción y criterios de uso	233
Anexo IV: Glosario de términos	235

Capítulo I: C/C++, multiplataforma y el sentido de GEN Framework

1. El punto de partida

Imagine una aplicación que se desarrolla y depura en un PC, se ejecuta también en una Raspberry Pi y debe llegar a un dispositivo con recursos mucho más limitados. Cambia el procesador, cambia el sistema operativo —o incluso desaparece—, cambian las herramientas y cambian las APIs. Sin embargo, la idea de la aplicación, sus reglas y la mayor parte de su código deberían seguir siendo reconocibles. Esa continuidad es el problema que GEN Framework se propuso resolver.

No se trata de obtener el mismo binario para todos los destinos ni de fingir que todas las máquinas son iguales. Se trata de conseguir algo más valioso: que una misma arquitectura pueda sobrevivir al cambio de plataforma; que el código que expresa lo que hace la aplicación no quede enterrado bajo detalles de ventanas, ficheros, hilos, comunicaciones, gráficos, dispositivos o sistemas operativos.

GEN Framework es un framework. Esta afirmación es esencial. No es una carpeta repleta de librerías independientes ni una colección de funciones sin relación entre sí. Un framework establece un lenguaje común entre sus piezas: define cómo se nombran, cómo se inicializan, cómo colaboran, de qué dependen y cómo pueden sustituirse sus implementaciones. Una función básica sirve de apoyo a una clase; varias clases forman un servicio; varios servicios, coordinados, permiten construir una aplicación completa.

Esa arquitectura no apareció terminada en un único momento. GEN Framework fue creciendo durante años al ritmo de proyectos concretos. Primero surgieron utilidades para evitar repetir tareas; después fue necesario ordenarlas, separar lo común de lo específico de cada plataforma y controlar mejor sus relaciones. Cuando aquellas piezas empezaron a compartir tipos, convenciones, ciclos de vida y mecanismos de extensión, dejaron de ser soluciones aisladas y se convirtieron en un sistema.

Cada avance respondió a una necesidad que ya existía fuera del propio framework: leer una configuración, controlar un dispositivo, intercambiar datos, dibujar una interfaz, registrar una incidencia, almacenar información o trasladar una aplicación a otro entorno. En GEN Framework, una funcionalidad no se incorpora únicamente porque resulte interesante sobre el papel. Se incorpora porque ha hecho falta para resolver un problema real y porque puede convertirse en una pieza reutilizable para el siguiente proyecto.

Por eso los tests son necesarios, pero no son la única prueba. Las clases y los módulos de GEN Framework han pasado también por compilaciones distintas, restricciones de hardware, instalaciones, actualizaciones, comunicaciones imperfectas y años de mantenimiento. Hay aplicaciones basadas en el framework funcionando en proyectos reales en distintos lugares del mundo. Esa experiencia acumulada no elimina todos los errores, pero sí obliga a que las decisiones de diseño respondan a la realidad y no solo a un ejemplo ideal.

Tres ideas han guiado esa evolución: orden, control de dependencias y atomización de funcionalidades. Juntas permiten desarrollar en «modo Lego»: construir piezas pequeñas, con un cometido claro y conexiones previsibles, para combinarlas después en conjuntos cada vez mayores. El propio GEN Framework está construido así, y la misma forma de trabajo queda disponible para las aplicaciones que se apoyan en él.

Este capítulo explica por qué esa combinación —C/C++, una arquitectura modular y una capa multiplataforma nacida de proyectos reales— ofrece una base especialmente potente. Antes de recorrer un catálogo muy amplio de clases y módulos, conviene comprender la idea que los mantiene unidos. Cuando esa idea se entiende, GEN Framework deja de parecer una gran cantidad de código y empieza a verse como una caja de piezas con la que es posible construir mucho más de lo que cada pieza haría por separado.

2. El origen del lenguaje C

El lenguaje C nació a principios de los años setenta de la mano de Dennis Ritchie, en los laboratorios Bell y en estrecha relación con Unix. Su origen fue profundamente práctico: hacía falta un lenguaje capaz de escribir sistemas con una eficiencia cercana a la del ensamblador, pero cuyo código pudiera trasladarse entre máquinas con un esfuerzo razonable. Esa búsqueda de equilibrio entre control y portabilidad cambió la informática.

El resultado fue un lenguaje pequeño en sus mecanismos fundamentales y enorme en sus posibilidades. El lenguaje C permite describir algoritmos y estructuras de datos y, al mismo tiempo, trabajar cerca de la memoria, los dispositivos y las llamadas del sistema. No interpone una maquinaria obligatoria entre el programa y la plataforma. Esa proximidad exige responsabilidad, pero también permite decidir con precisión cómo se utilizan los recursos.

La portabilidad del lenguaje C nunca significó que cualquier programa pudiera compilarse sin cambios en cualquier lugar. Significó algo más realista y duradero: separar una parte importante del programa de las instrucciones de una máquina concreta y confiar su traducción a un compilador. Desde entonces, la disponibilidad de un compilador del lenguaje C se ha convertido en una de las primeras señales de que una plataforma puede abrirse al desarrollo de software propio.

Décadas después, el lenguaje C continúa presente en sistemas operativos, firmware, controladores, bibliotecas, toolchains y APIs nativas. También es el punto de encuentro de tecnologías muy diferentes: incluso cuando un componente está escrito en otro lenguaje, es frecuente que exponga o consuma una interfaz compatible con el lenguaje C. Esa capacidad de integración resulta crucial para un framework que debe convivir con sistemas operativos, SDK, bibliotecas gráficas, dispositivos y código existente.

GEN Framework hereda del lenguaje C esa vocación de llegar a la máquina real. No intenta sustituir el hardware por una abstracción imaginaria; intenta aislar lo que cambia y conservar lo que puede ser común. Allí donde una plataforma ofrece un servicio, el framework puede adaptarlo. Allí donde no lo ofrece, puede proporcionar una implementación propia o una alternativa adecuada al entorno.

3. C y C++ no son el mismo lenguaje

En este libro utilizamos la expresión C/C++ para referirnos a una base tecnológica compartida, pero conviene ser precisos: C y C++ son lenguajes distintos, con estándares y modelos propios. Están estrechamente relacionados, comparten buena parte de su ecosistema y pueden interoperar con facilidad, pero no deben confundirse.

El lenguaje C organiza de forma natural el programa alrededor de funciones, estructuras y módulos. C++ amplía ese terreno con clases, plantillas, sobrecarga, polimorfismo, lambdas, evaluación en tiempo de compilación y otras herramientas. Es un lenguaje multiparadigma: permite elegir entre un estilo procedural, orientado a objetos, genérico o funcional según el problema, y combinar esos estilos cuando aporta claridad.

Para GEN Framework, la aportación decisiva de C++ es la posibilidad de crear abstracciones sin renunciar al control. Una clase puede reunir datos, comportamiento e invariantes; una interfaz puede separar el servicio de su implementación; una factoría puede seleccionar la variante adecuada para una plataforma; y RAII puede vincular la vida de un recurso —memoria, fichero, socket o bloque— a la vida de un objeto.

Esto permite que el código complejo nazca de elementos sencillos. Una clase pequeña no necesita conocer la aplicación completa. Solo necesita cumplir bien su contrato y depender de las piezas imprescindibles. Otra clase puede apoyarse en ella para añadir un nivel de comportamiento, y así sucesivamente. La complejidad no desaparece: se reparte en lugares donde puede entenderse, probarse y sustituirse.

GEN Framework utiliza C++ para expresar su arquitectura y mantiene la capacidad de relacionarse con el lenguaje C cuando la plataforma o una biblioteca lo requiere. Esa convivencia ofrece un puente entre dos mundos: la

estabilidad de las interfaces nativas y la potencia de las abstracciones modernas. El programador puede trabajar con conceptos de alto nivel sin perder la posibilidad de descender hasta el sistema cuando sea necesario.

La elección no elimina la exigencia del lenguaje. C/C++ permiten hacer mucho, incluido cometer errores costosos. El papel del framework es ofrecer caminos repetibles, ciclos de vida claros y componentes ya contrastados para que esa libertad se convierta en capacidad de construcción, no en improvisación permanente.

4. La evolución de C++

Hablar hoy de C++ no es hablar del lenguaje de los años noventa. Sus revisiones modernas han incorporado movimiento de objetos, punteros inteligentes, lambdas, concurrencia, constexpr, rangos, conceptos, módulos y numerosas mejoras en la biblioteca estándar. El objetivo sigue siendo singular: ofrecer herramientas de alto nivel capaces de producir código nativo eficaz.

Una buena abstracción en C++ puede resolverse en gran medida durante la compilación. Esto hace posible escribir una interfaz clara sin pagar necesariamente una penalización relevante en ejecución. Es una propiedad especialmente valiosa en una arquitectura que debe servir tanto a equipos con recursos abundantes como a entornos donde cada milisegundo, cada asignación y cada kilobyte cuentan.

El lenguaje evoluciona mediante estándares internacionales y múltiples implementaciones. Ninguna empresa posee el lenguaje C ni C++ ni decide por sí sola su futuro. Fabricantes, proyectos abiertos, especialistas y usuarios participan en un ecosistema donde conviven intereses distintos. Esa pluralidad no evita los desacuerdos, pero reduce la dependencia estratégica de un único proveedor.

Al mismo tiempo, C++ conserva una compatibilidad histórica poco común. Código escrito hace años puede continuar siendo útil, mientras las nuevas partes adoptan mecanismos más expresivos y seguros. Para un framework que ha evolucionado junto a aplicaciones de larga vida, esa continuidad importa tanto como cualquier característica reciente.

GEN Framework aprovecha la evolución del lenguaje con criterio práctico: una característica es valiosa cuando mejora la claridad, la seguridad o el rendimiento y cuando está disponible en los compiladores y plataformas que deben soportarse. La modernidad no consiste en usar toda novedad, sino en disponer de más herramientas y elegir la que mejor resuelve cada problema.

5. C/C++ frente a otros lenguajes

No existe un lenguaje óptimo para todos los proyectos. Python puede ser magnífico para automatización y prototipado; Java y C# ofrecen entornos gestionados y ecosistemas empresariales muy completos; Kotlin y Swift se integran de forma natural en sus plataformas principales; Rust propone garantías diferentes para el software de sistemas. Elegir una tecnología consiste en aceptar conscientemente sus ventajas y sus compromisos.

GEN Framework se orienta a aplicaciones donde coinciden varias necesidades: rendimiento nativo, control de recursos, integración con hardware o APIs de bajo nivel, vida útil prolongada y posibilidad de alcanzar plataformas muy diferentes. En ese terreno, C/C++ ofrecen una combinación excepcionalmente amplia.

Cuando aparece una nueva arquitectura o un nuevo sistema, es muy habitual que el primer toolchain útil incluya el lenguaje C y, con frecuencia, C++. Esto ocurre porque los propios sistemas, SDK y controladores se construyen en gran parte alrededor de ellos. La consecuencia práctica es una cobertura enorme: desde microcontroladores hasta servidores, pasando por ordenadores personales, dispositivos móviles y sistemas industriales.

Esa cobertura no depende de un solo compilador. GEN Framework puede contrastarse con familias de herramientas distintas, entre ellas GCC, Clang, Microsoft Visual C++ y Embarcadero C++Builder, además de

toolchains específicos de fabricantes. Compilar una misma arquitectura con implementaciones diferentes ayuda a descubrir supuestos no portables, extensiones accidentales y errores que un único compilador podría no revelar.

La diversidad de compiladores también ofrece libertad operativa. Un equipo puede utilizar la herramienta adecuada para su destino sin tener que entregar el control de toda la aplicación a una única empresa o máquina virtual. Los lenguajes C y C++ están definidos por estándares; los compiladores compiten en diagnósticos, optimización, plataformas soportadas e integración con sus entornos.

La compilación nativa permite generar aplicaciones muy optimizadas en velocidad y tamaño. No garantiza por sí sola un programa rápido —la arquitectura y los algoritmos siguen siendo decisivos—, pero coloca al desarrollador entre las mejores condiciones posibles para conseguirlo. No impone un recolector de basura, una máquina virtual o un runtime de gran tamaño cuando el proyecto no los necesita.

C/C++ son, además, compatibles con una enorme variedad de paradigmas y tecnologías. Pueden exponer una API sencilla a otros lenguajes, integrar código procedural, organizar sistemas orientados a objetos, utilizar programación genérica y llamar directamente a bibliotecas del sistema. Esa capacidad para ocupar el centro de una arquitectura heterogénea es difícil de igualar.

El precio de esa libertad es una mayor responsabilidad. La memoria, los recursos, las dependencias y el proceso de compilación deben gestionarse con rigor. GEN Framework no niega ese coste; lo organiza. Proporciona una base común para que los equipos no tengan que resolver desde cero los mismos problemas de infraestructura en cada aplicación.

6. El problema real: la plataforma

En GEN Framework, una plataforma no es solo un sistema operativo ni solo un procesador. Es la combinación concreta de hardware —basado en un microcontrolador o un microprocesador— y entorno de ejecución sobre la que debe funcionar el programa. Puede ser un PC x86-64 con Windows, una Raspberry Pi ARM con Linux, un dispositivo Android o un microcontrolador con un sistema operativo de tiempo real. También puede ser hardware sin sistema operativo, donde la aplicación se ejecuta directamente con el soporte mínimo disponible.

A esa combinación concreta la llamamos target o destino. El objetivo de GEN Framework no es compilar para una palabra genérica como «Linux» o «ARM», sino hacer que el programa funcione en el destino real: con su procesador, su sistema —si existe—, su compilador, sus bibliotecas, sus permisos y sus dispositivos.

Mientras un programa es pequeño, la plataforma puede parecer invisible. En cuanto crece, aparece al gestionar rutas y permisos, abrir una ventana, obtener tiempo preciso, crear hilos, dibujar, usar un puerto serie, acceder a una cámara, comunicarse por red, cargar recursos o responder a eventos. Cada API introduce tipos, reglas y ciclos de vida propios.

Si todos esos detalles entran directamente en la lógica de la aplicación, el código se llena de condiciones de compilación y excepciones locales. Un bloque sirve para Windows, otro para Linux, otro para Android y otro para una placa concreta. Al principio parece un atajo; con el tiempo, cada cambio obliga a razonar sobre todos los destinos a la vez y la aplicación deja de expresar con claridad su propósito.

El problema real no es que las plataformas sean diferentes. Deben serlo: tienen capacidades, límites y modelos de seguridad distintos. El problema es permitir que esas diferencias se propaguen sin control por todo el programa. Una arquitectura multiplataforma debe concentrarlas en puntos conocidos y ofrecer al resto del código contratos estables.

GEN Framework sitúa esas decisiones en capas específicas, adaptadores y factorías. La aplicación solicita un servicio mediante una interfaz común; el framework selecciona o construye la implementación que corresponde al target. Así, lo que cambia queda localizado y lo que define a la aplicación puede conservarse.

Portar no consiste únicamente en lograr que el compilador termine sin errores. Consiste en preservar el comportamiento, respetar los recursos del destino y poder mantener el resultado. La definición precisa de plataforma obliga a pensar desde el principio en la máquina donde vivirá el programa, no solo en el ordenador donde se escribió.

7. Qué aporta GEN Framework

GEN Framework aporta una arquitectura común entre la aplicación y la plataforma. Sus módulos no se limitan a agrupar utilidades: comparten convenciones, tipos, patrones de creación, trazas, gestión de errores y relaciones de dependencia. Aprender una parte prepara al programador para comprender la siguiente.

La diferencia entre un framework y una colección de librerías se descubre al seguir una operación de extremo a extremo. Una funcionalidad compleja puede comenzar en una clase básica de cadenas, buffers o tiempo; apoyarse después en ficheros, streams o eventos; incorporar un protocolo o un formato; y terminar formando parte de un servicio visible para la aplicación. Cada nivel utiliza el anterior en lugar de volver a resolver el mismo problema.

Esa composición hace que el alcance crezca de forma acumulativa. Las utilidades extendidas sostienen la entrada y salida de datos; las capas de datos alimentan comunicaciones, configuración y almacenamiento; los sistemas gráficos e Input permiten interacción; Sound, Cipher, Compress, Databases, Scripts y UserInterface añaden capacidades especializadas. No son islas: forman niveles de una misma construcción.

El número de clases importa porque amplía el terreno ya resuelto, pero la relación entre ellas importa todavía más. Una clase aislada ahorra unas líneas. Un conjunto coherente puede ahorrar una arquitectura entera: decisiones de inicialización, adaptación a plataforma, gestión de recursos, trazabilidad y extensión que, de otro modo, cada proyecto tendría que inventar y mantener.

GEN Framework también ofrece una manera de crecer. Una aplicación puede empezar utilizando unas pocas clases y añadir módulos a medida que aparecen nuevas necesidades. No es preciso activar todo el framework ni convertir cada programa en un sistema gigantesco. La atomización permite seleccionar piezas y conservar una dirección arquitectónica común.

El framework practica internamente lo que propone a sus usuarios. Sus funcionalidades complejas se construyen sobre componentes más básicos, y las aplicaciones pueden seguir el mismo patrón. Una clase del proyecto puede apoyarse en servicios de GEN Framework, mantener un cometido pequeño y convertirse a su vez en una pieza para subsistemas mayores.

El resultado es una doble aceleración. Se avanza más rápido porque muchas necesidades ya tienen una solución y porque esas soluciones encajan entre sí. Y se mantiene mejor porque la aplicación comparte un vocabulario y una estructura reconocibles con otros proyectos desarrollados sobre GEN Framework.

GEN Framework no pretende ocultar todo ni impedir que el programador llegue al sistema. Pretende que esa decisión sea explícita. La ruta habitual debe ser cómoda, eficaz y portable; cuando un caso excepcional exige acceso nativo, la arquitectura debe permitir localizarlo sin contaminar el resto del código.

8. Desarrollo multiplataforma sin renunciar al rendimiento

En algunos entornos, la portabilidad se obtiene colocando la aplicación sobre una máquina virtual, un intérprete o un motor completo. Es una estrategia válida, pero no es la única. GEN Framework elige conservar la compilación nativa de C/C++ y realizar la adaptación dentro de sus capas.

Esto permite que cada target utilice el compilador, las optimizaciones y las APIs más adecuadas. Una misma interfaz de fichero puede resolverse con llamadas diferentes; un canvas puede apoyarse en backends distintos;

una comunicación puede adaptarse a los servicios disponibles. La aplicación observa un contrato común, mientras el framework aprovecha las capacidades reales de la plataforma.

El control también alcanza al tamaño y al consumo. En un PC, unos megabytes adicionales pueden ser irrelevantes; en un sistema embebido, pueden impedir el despliegue. La arquitectura modular y la compilación condicional por target ayudan a incluir lo necesario sin imponer componentes que el programa no va a utilizar.

Abstraer no significa ignorar el coste. Una capa útil reduce complejidad y conserva un comportamiento previsible. C++ permite que muchas decisiones se resuelvan en compilación y que las llamadas críticas permanezcan cerca del sistema. Cuando hace falta optimizar, el programador puede medir, identificar el nivel adecuado y actuar sin abandonar toda la arquitectura.

La eficacia que busca GEN Framework tiene dos dimensiones. La aplicación debe ser rápida y ajustada en ejecución, pero el equipo también debe ser eficaz al desarrollarla. Reutilizar componentes probados, evitar duplicar implementaciones y mantener una sola lógica común reduce el tiempo hasta el resultado sin convertir el rendimiento en una concesión posterior.

No hay una promesa mágica de velocidad: un mal algoritmo sigue siendo un mal algoritmo. Lo que sí ofrece la base tecnológica es la posibilidad de alcanzar un rendimiento muy alto, controlar los recursos y mantener esa capacidad en destinos diferentes. GEN Framework intenta que la portabilidad amplíe el alcance del programa, no que rebaje sus ambiciones.

9. La ausencia de una librería única en C++

La biblioteca estándar de C++ proporciona contenedores, algoritmos, memoria, concurrencia, tiempo y muchas otras herramientas esenciales. Deliberadamente, no define una solución universal para ventanas, gráficos, sonido, dispositivos, protocolos, bases de datos, interfaces de usuario o todos los servicios que necesita una aplicación completa.

La respuesta habitual es reunir una librería para cada necesidad. Puede funcionar, pero pronto aparecen estilos incompatibles, tipos duplicados, modelos de errores distintos, dependencias transitivas y procesos de compilación que no coinciden. El proyecto termina integrando piezas excelentes que, sin una arquitectura, no forman necesariamente un sistema excelente.

GEN Framework no rechaza las librerías de terceros. Las utiliza cuando aportan una solución madura y su licencia abierta permite hacerlo con seguridad. Siempre que es viable, procura integrarlas como código fuente para conservar el control de la compilación multiplataforma. La selección considera simultáneamente funcionalidad, portabilidad, mantenimiento y licencia; no basta con que una biblioteca resuelva el caso inmediato.

La integración es la palabra importante. La librería externa queda situada detrás de las convenciones y abstracciones del framework, de modo que no imponga su vocabulario a toda la aplicación. Si cambia el backend, si un target necesita otra implementación o si la dependencia debe actualizarse, el impacto permanece contenido.

De esta forma, GEN Framework ocupa el espacio que el estándar no pretende cubrir: ofrece un entorno coherente para construir aplicaciones. No compite con la biblioteca estándar; se apoya en ella, la complementa y organiza las capacidades adicionales que la práctica exige.

10. Una sola aplicación, varias plataformas

La promesa central puede expresarse con sencillez: desarrollar una aplicación sobre una base común y compilarla para los targets soportados sin reescribir su lógica principal. No significa producir el mismo ejecutable, sino construir un ejecutable nativo para cada destino a partir de una arquitectura compartida.

La lógica de negocio, los modelos, los protocolos propios y los flujos de la aplicación deben permanecer en el nivel común. Los detalles de ventana, dispositivo, almacenamiento o sistema se delegan en GEN Framework o se encapsulan en adaptadores del proyecto. Esa frontera convierte la portabilidad en una propiedad del diseño, no en una campaña de correcciones al final.

El beneficio se multiplica con el tiempo. Una mejora en el código común llega a todos los destinos. Una incidencia corregida en un servicio compartido deja de repetirse. Una nueva plataforma puede añadirse implementando los puntos de adaptación necesarios, sin clonar la aplicación entera.

También cambia la forma de probar. El equipo puede desarrollar con comodidad en un entorno principal y verificar periódicamente otros compiladores y targets. Cada combinación actúa como una perspectiva distinta sobre el código: descubre tamaños asumidos, orden de bytes, APIs accidentales, rutas, codificaciones o comportamientos indefinidos que podrían permanecer ocultos.

Naturalmente, un dispositivo táctil no se usa como un PC y un microcontrolador no ofrece los servicios de un servidor. La parte común no debe negar esas diferencias. Debe separar las capacidades esenciales de las decisiones de presentación y de hardware, permitiendo que cada destino se adapte sin fracturar el núcleo.

Cuando esa disciplina se mantiene, ampliar el alcance de una aplicación deja de equivaler a mantener productos desconectados. Se mantiene una familia de ejecutables nacidos del mismo diseño. Esa es una de las ventajas más profundas de GEN Framework: convertir la variedad de plataformas en una dimensión controlada del proyecto.

11. La importancia de una arquitectura propia

Un framework necesita una arquitectura porque la complejidad no se vence acumulando funciones, sino organizando relaciones. En GEN Framework, esa organización se apoya en tres principios que se refuerzan entre sí y que pueden aplicarse igualmente al código de cualquier aplicación.

Orden

Orden significa que cada responsabilidad tiene un lugar reconocible. Los nombres, directorios, módulos, capas y ciclos de vida siguen criterios comunes. El orden reduce el número de decisiones improvisadas y permite que un programador pueda orientarse en una parte desconocida del sistema a partir de lo aprendido en otra.

No es una preferencia estética. Cuando un proyecto crece, encontrar dónde debe vivir una funcionalidad y quién debe iniciarla o destruirla afecta directamente a la fiabilidad. Un sistema ordenado hace visibles sus límites; uno desordenado obliga a reconstruirlos mentalmente cada vez que se modifica.

Control de dependencias

Controlar dependencias significa saber qué necesita cada componente, impedir relaciones innecesarias y mantener la dirección correcta entre capas. Las partes básicas no deberían depender de las más especializadas. La lógica de una aplicación no debería conocer detalles nativos que el framework puede resolver. Y una biblioteca externa no debería propagarse por todo el código si puede quedar detrás de una interfaz.

Una dependencia explícita se puede compilar, probar, sustituir y revisar. Una dependencia accidental se descubre tarde, a menudo al portar o actualizar. GEN Framework trata las dependencias como parte del diseño porque son las líneas invisibles que determinan si un conjunto de clases puede evolucionar o queda inmovilizado.

Atomización de funcionalidades

Atomizar consiste en dar a cada clase o función un propósito reducido y completo. Una pieza pequeña es más fácil de comprender, probar y reutilizar. También puede combinarse de maneras que su autor no necesitaba en el

proyecto original. La clave no es dividir por dividir, sino encontrar unidades con responsabilidad clara y una interfaz estable.

Las clases pequeñas permiten crear conjuntos grandes porque limitan el número de cosas que deben entenderse al mismo tiempo. Un buffer puede alimentar un stream; un stream puede transportar un protocolo; un cliente de protocolo puede participar en un servicio; y ese servicio puede formar parte de una aplicación. Cada nivel añade significado sin obligar a los anteriores a conocer el resultado final.

Desarrollar en modo Lego

La metáfora de Lego no significa que cualquier pieza encaje en cualquier sitio. Los bloques funcionan porque sus conexiones tienen una forma definida. En software, esas conexiones son interfaces, tipos, dependencias y contratos de ciclo de vida. Cuanto más claros son, mayor es la libertad para construir.

GEN Framework aplica este modo de desarrollo sobre sí mismo: sus servicios mayores se levantan sobre elementos básicos del propio framework. Y lo extiende al usuario: las clases de una aplicación pueden convertirse en nuevos bloques que combinan servicios existentes sin romper su separación. El resultado no es un programa monolítico, sino una estructura capaz de crecer por composición.

Orden sin atomización produce compartimentos demasiado grandes. Atomización sin control de dependencias produce una maraña de piezas diminutas. Dependencias controladas sin orden pueden seguir siendo difíciles de localizar. Los tres principios juntos crean una arquitectura que puede ampliarse sin perder legibilidad.

12. C/C++ como decisión estratégica

Elegir C/C++ para GEN Framework no fue una decisión nostálgica, sino estratégica. Son lenguajes disponibles en una variedad extraordinaria de plataformas, capaces de producir código nativo muy eficiente y de integrarse con casi cualquier API relevante. Además, su estandarización y la existencia de compiladores independientes evitan que el futuro del framework quede ligado a un único proveedor.

La compatibilidad con diferentes compiladores forma parte de esa estrategia. GCC, Clang, Microsoft Visual C++ y C++Builder no son copias idénticas: interpretan diagnósticos, extensiones y optimizaciones desde implementaciones distintas. Conseguir que una base de código funcione con varias de ellas aumenta su robustez y reduce dependencias involuntarias.

Sin embargo, una buena base tecnológica no basta. GEN Framework está diseñado por programadores para programadores. Sus utilidades, funciones y clases intentan resultar efectivas y cómodas en el trabajo diario: resolver tareas completas, mantener patrones reconocibles y ofrecer resultados grandes con una cantidad razonable de código de aplicación.

Esa comodidad no procede de ocultar decisiones importantes, sino de haberlas tomado muchas veces. Las piezas del framework nacen de proyectos concretos y se refinan al reutilizarse. Cuando una necesidad aparece de nuevo, el objetivo no es copiar la solución anterior, sino convertirla en un componente mejor definido para que el siguiente proyecto pueda empezar más arriba.

Los tests unitarios y de desarrollo comprueban comportamientos controlados. Las aplicaciones desplegadas añaden otra clase de evidencia: funcionamiento prolongado, datos imperfectos, máquinas distintas, usuarios reales y mantenimiento. GEN Framework se apoya en ambas. Una prueba protege una condición; la experiencia revela qué condiciones importan.

La estrategia completa, por tanto, no es simplemente «usar C++». Es combinar una tecnología portable y eficaz con una arquitectura ordenada, dependencias controladas, piezas pequeñas y experiencia de producción. C/C++ proporcionan el alcance y el control; GEN Framework convierte esas capacidades en una forma repetible de construir aplicaciones.

13. Qué se puede aprender en este libro

Este libro no es un inventario de miles de nombres para memorizar. Es un recorrido por la forma en que GEN Framework transforma piezas pequeñas en aplicaciones completas. Presenta la estructura general, el entorno de compilación y el ciclo de una aplicación; después recorre los módulos que aportan utilidades, datos, gráficos, entrada, sonido, cifrado, compresión, bases de datos, scripts e interfaces de usuario.

En cada etapa convendrá hacer tres preguntas: ¿qué responsabilidad resuelve esta pieza?, ¿de qué depende? y ¿con qué otras piezas puede combinarse? Esas preguntas son más valiosas que la memoria, porque permiten orientarse incluso ante una clase que todavía no se conoce.

En estas páginas se explica cómo separar la lógica de aplicación de la plataforma, cómo elegir un target, cómo aprovechar implementaciones nativas mediante una interfaz común y cómo extender la arquitectura sin convertirla en un conjunto de excepciones. También se muestra dónde entran las librerías de terceros y por qué funcionalidad, licencia y posibilidad de compilación forman una sola decisión técnica.

El objetivo práctico es ambicioso: poder construir más deprisa sin renunciar al control; llegar a más plataformas sin multiplicar la lógica; reutilizar sin copiar; y obtener aplicaciones nativas, rápidas y ajustadas al entorno donde deben funcionar.

GEN Framework no promete que el software complejo deje de ser complejo. Promete que esa complejidad puede ordenarse. Ofrece muchas clases y un alcance amplio, pero su verdadera potencia aparece cuando se comprende cómo se relacionan. Entonces una nueva necesidad ya no obliga a empezar de cero: puede resolverse combinando bloques existentes y añadiendo solo la pieza que falta.

Ese es el camino que comienza aquí. Del primer tipo básico al servicio más especializado; del ordenador de desarrollo al target real; de una clase pequeña a una aplicación capaz de vivir durante años. Si la idea resulta estimulante, es porque las posibilidades también lo son: construir software serio, eficaz y portable sin perder de vista cada una de las piezas que lo hacen posible.

Capítulo II: Preparando el entorno de desarrollo

1. Un capítulo orientado a la práctica

En el capítulo anterior vimos por qué C y C++ siguen siendo lenguajes fundamentales para el desarrollo de software multiplataforma, eficiente y cercano al sistema. También presentamos la razón de ser de GEN Framework: proporcionar una capa de abstracción que permita desarrollar aplicaciones en C/C++ para distintas plataformas sin tener que reescribir la lógica de la aplicación para cada sistema operativo.

Ahora debemos dar un paso más práctico. Antes de estudiar las clases internas, los módulos, la arquitectura o los ejemplos del framework, necesitamos preparar correctamente el entorno de desarrollo. Este capítulo tiene precisamente ese objetivo: explicar qué herramientas necesitamos, cómo debemos organizar los repositorios, dónde conviene colocar nuestros propios proyectos y cuál es el procedimiento recomendado para realizar una primera compilación.

En C/C++, el entorno de desarrollo es una parte esencial del proyecto. No basta con escribir código fuente. Es necesario saber qué compilador se va a utilizar, cómo se localizan las cabeceras, cómo se enlazan las librerías, qué sistema de construcción interviene, qué configuración se está generando, qué dependencias externas se necesitan y qué diferencias existen entre Windows, Linux, WSL2, Android o Docker.

En lenguajes más gestionados, muchas de estas decisiones quedan ocultas detrás del entorno de ejecución, del gestor de paquetes o del IDE. En C/C++, y especialmente en un framework multiplataforma como GEN Framework, esas decisiones son visibles y forman parte del conocimiento que debe adquirir el desarrollador.

Esto no significa que el usuario tenga que dominar desde el primer día Visual Studio, GCC, Clang, CMake, Docker, Docker Compose, Ninja, WSL2 y todos los scripts del framework. Significa que debe comprender qué papel cumple cada herramienta y cuál es el flujo recomendado para empezar sin perderse en detalles innecesarios.

La idea más importante de este capítulo es sencilla: aunque GEN Framework utiliza CMake como base interna para organizar la construcción de proyectos, la primera compilación no debe hacerse invocando CMake directamente. El punto de entrada recomendado son los scripts de compilación situados en la parte común del framework, especialmente `compile.bat` en Windows y `compile.bash` en Linux.

Estos scripts existen para simplificar el trabajo. Preparan rutas, variables, opciones, configuraciones, plataformas y comandos internos. Permiten compilar todos los ejemplos o seleccionar uno concreto. También permiten trabajar por etapas, ejecutando solo la configuración con CMake, solo la compilación o la fase de test.

Por tanto, el lector debe quedarse desde el principio con esta regla:

```
Para empezar con GEN Framework, primero se compila desde la carpeta Common usando compile.bat o compile.bash.
```

El estudio detallado de CMake aparece en una etapa posterior. El primer contacto práctico se realiza con los scripts del propio framework.

2. La página oficial del framework

El punto de entrada natural para obtener GEN Framework es su página web oficial:

```
https://www.genframework.com
```

Desde la web se puede acceder a la información general del proyecto y a los repositorios donde se mantiene el código fuente. GEN Framework no debe entenderse como una única carpeta con unos cuantos ficheros, sino como un conjunto de repositorios organizados alrededor de una arquitectura común.

Esta separación es importante. En proyectos pequeños puede parecer cómodo tenerlo todo en una sola carpeta. Sin embargo, cuando un framework crece y debe incluir núcleo, código común, librerías de terceros, ejemplos, pruebas, recursos, utilidades y documentación, la separación por repositorios ayuda a mantener el sistema ordenado.

Una organización típica de GEN Framework incluye repositorios como:

```
GEN
Common
ThirdPartyLibraries
Examples
Tests
Utilities
Resources
Documents
```

Cada uno tiene una función concreta dentro del ecosistema.

El repositorio principal de GEN Framework contiene el núcleo principal del framework. Aquí se encuentran las clases base, los módulos principales, las abstracciones de plataforma, las capas de gráficos, comunicaciones, sistema, ficheros, trazas y otros componentes fundamentales.

Common contiene elementos comunes y desempeña un papel muy importante en la compilación. Para el usuario que empieza, esta carpeta es clave porque desde ella se lanzan los scripts principales de construcción: `compile.bat` en Windows y `compile.bash` en Linux.

ThirdPartyLibraries contiene librerías de terceros utilizadas por GEN Framework. En C/C++ es habitual depender de bibliotecas externas para gráficos, compresión, formatos de imagen, parsers, motores de renderizado o herramientas auxiliares. Estas librerías pueden requerir compilaciones específicas según plataforma, compilador y arquitectura.

Examples contiene aplicaciones de ejemplo. Este repositorio es esencial durante el aprendizaje, porque permite ver aplicaciones reales construidas sobre GEN Framework. Los ejemplos no son simples fragmentos de código, sino pequeñas aplicaciones que muestran cómo se inicializa, organiza y utiliza el framework.

Tests contiene pruebas destinadas a validar funcionalidades concretas. En un framework en evolución, los tests son fundamentales para detectar regresiones y comprobar que los cambios no rompen comportamientos anteriores.

Utilities contiene herramientas auxiliares. Algunas pueden servir para generar recursos, comprobar configuraciones, depurar módulos o realizar tareas de apoyo durante el desarrollo.

Resources agrupa recursos utilizados por ejemplos, aplicaciones o herramientas: imágenes, fuentes, configuraciones, plantillas u otros elementos externos al código fuente.

Documents puede contener documentación, notas técnicas, referencias o material adicional del proyecto.

3. La carpeta Projects como espacio de trabajo

Una decisión importante al preparar el entorno es dónde colocar el framework y dónde colocar nuestros propios proyectos.

La recomendación de este libro es utilizar una carpeta general llamada Projects como espacio de trabajo. Dentro de ella se coloca, por un lado, la carpeta completa del framework, denominada GEN_FrameWork, y, por otro, los proyectos propios.

Es importante entender bien esta idea: los proyectos propios no deben colocarse dentro de la carpeta GEN_FrameWork. Deben estar al mismo nivel que ella.

La estructura recomendada sería:

```
Projects/
|
├── GEN_FrameWork/
|   ├── GEN/
|   ├── Common/
|   ├── ThirdPartyLibraries/
|   ├── Examples/
|   ├── Tests/
|   ├── Utilities/
|   ├── Resources/
|   └── Documents/
|
├── MiProyecto/
├── OtroProyecto/
└── PruebasGEN/
```

En este esquema, Projects es la carpeta raíz general de trabajo. Dentro de ella, GEN_FrameWork contiene todos los repositorios que forman parte del framework. Los proyectos propios, como MiProyecto, OtroProyecto o PruebasGEN, son carpetas hermanas de GEN_FrameWork.

Esta organización tiene varias ventajas.

La primera es la separación clara entre el framework y las aplicaciones. El framework se mantiene en su propia carpeta, con sus repositorios internos organizados. Los proyectos propios no contaminan esa estructura y pueden evolucionar de forma independiente.

La segunda es la facilidad para usar rutas relativas. Un proyecto propio situado en Projects/MiProyecto puede acceder al framework mediante rutas relativas como:

```
../GEN_FrameWork/GEN
../GEN_FrameWork/Common
../GEN_FrameWork/ThirdPartyLibraries
../GEN_FrameWork/Resources
```

Esto evita depender de rutas absolutas de una máquina concreta. No queremos que un proyecto dependa de una ruta como:

```
C:\Users\Usuario\Desktop\Temporal\GEN_FrameWork
```

Ese tipo de rutas hacen que el proyecto solo funcione en un equipo concreto. En cambio, si usamos rutas relativas desde una estructura común, el proyecto puede moverse a otro disco, a otro usuario o a otra máquina manteniendo la misma relación entre carpetas.

La tercera ventaja es que podemos tener varios proyectos propios usando el mismo framework sin duplicarlo. Por ejemplo:

```
Projects/
|
├─ GEN_FrameWork/
├─ AplicacionIndustrial/
├─ HerramientaDiagnostico/
├─ EjemploCliente/
└─ PruebasGraficas/
```

Todos estos proyectos pueden referenciar la misma carpeta GEN_FrameWork.

La cuarta ventaja es la limpieza del control de versiones. El framework puede mantenerse con sus propios repositorios Git, mientras que cada proyecto propio puede tener su propio repositorio independiente. Esto es especialmente importante en trabajos profesionales, donde una aplicación puede tener su ciclo de vida, ramas, versiones y entregas separadas del framework.

Por tanto, la regla de organización recomendada es:

```
La carpeta GEN_FrameWork contiene el framework.
Los proyectos propios se colocan al mismo nivel que GEN_FrameWork, dentro de Projects.
```

4. Organización recomendada en Windows

En Windows, una ubicación cómoda podría ser:

```
E:\Projects\
```

Dentro de ella colocaríamos:

```
E:\Projects\GEN_FrameWork\
E:\Projects\MiProyecto\
E:\Projects\OtroProyecto\
```

La carpeta del framework contendría los repositorios principales:

```
E:\Projects\GEN_Framework\GEN
E:\Projects\GEN_Framework\Common
E:\Projects\GEN_Framework\ThirdPartyLibraries
E:\Projects\GEN_Framework\Examples
E:\Projects\GEN_Framework\Tests
E:\Projects\GEN_Framework\Utilities
E:\Projects\GEN_Framework\Resources
E:\Projects\GEN_Framework\Documents
```

Un proyecto propio quedaría fuera de GEN_Framework, pero dentro de la misma carpeta general Projects:

```
E:\Projects\MiProyecto
```

Desde MiProyecto, las rutas relativas hacia el framework serían:

```
..\GEN_Framework\GEN
..\GEN_Framework\Common
..\GEN_Framework\ThirdPartyLibraries
..\GEN_Framework\Resources
```

También podríamos usar la unidad del sistema:

```
C:\Projects\GEN_Framework\
C:\Projects\MiProyecto\
```

Lo importante no es la unidad de disco, sino la relación entre carpetas.

Una estructura incorrecta sería:

```
E:\Projects\GEN_Framework\Projects\MiProyecto
```

Aunque podría hacerse funcionar con ajustes manuales, no es la organización recomendada en este libro. Mezcla los desarrollos propios dentro de la carpeta del framework y puede complicar la actualización, limpieza o mantenimiento del árbol.

La estructura recomendada es:

```
E:\Projects\GEN_Framework
E:\Projects\MiProyecto
```

5. Organización recomendada en Linux

En Linux podemos usar una estructura equivalente dentro del directorio personal del usuario:

```
/home/usuario/Projects/
```

Dentro de esa carpeta:

```
/home/usuario/Projects/GEN_Framework/
/home/usuario/Projects/MiProyecto/
/home/usuario/Projects/OtroProyecto/
```

La carpeta del framework contendría:

```
/home/usuario/Projects/GEN_Framework/GEN
/home/usuario/Projects/GEN_Framework/Common
/home/usuario/Projects/GEN_Framework/ThirdPartyLibraries
/home/usuario/Projects/GEN_Framework/Examples
/home/usuario/Projects/GEN_Framework/Tests
/home/usuario/Projects/GEN_Framework/Utilities
/home/usuario/Projects/GEN_Framework/Resources
/home/usuario/Projects/GEN_Framework/Documents
```

Desde un proyecto propio situado en:

```
/home/usuario/Projects/MiProyecto
```

podríamos acceder al framework mediante:

```
../GEN_FrameWork/GEN
../GEN_FrameWork/Common
../GEN_FrameWork/ThirdPartyLibraries
../GEN_FrameWork/Resources
```

Esta organización es limpia, portable y fácil de entender. Si el proyecto se mueve a otra máquina Linux, basta con mantener la misma relación entre GEN_FrameWork y los proyectos propios.

6. Descarga mediante Git

La forma más recomendable de descargar GEN Framework es utilizar Git. Git permite clonar los repositorios, actualizarlos, trabajar con ramas, revisar cambios y mantener una copia local sincronizada.

En Windows, un ejemplo de descarga podría ser:

```
cd /d E:\
mkdir Projects
cd Projects

mkdir GEN_FrameWork
cd GEN_FrameWork

git clone https://gitlab.genframework.com/gen-framework/gen.git GEN
git clone https://gitlab.genframework.com/gen-framework/common.git Common
git clone https://gitlab.genframework.com/gen-framework/thirdpartylibraries.git ThirdPartyLibraries
git clone https://gitlab.genframework.com/gen-framework/examples.git Examples
git clone https://gitlab.genframework.com/gen-framework/tests.git Tests
git clone https://gitlab.genframework.com/gen-framework/utilities.git Utilities
git clone https://gitlab.genframework.com/gen-framework/resources.git Resources
git clone https://gitlab.genframework.com/gen-framework/documents.git Documents
```

Después, si queremos crear un proyecto propio, volveríamos a la carpeta general Projects:

```
cd /d E:\Projects
mkdir MiProyecto
```

El resultado sería:

```
E:\Projects\GEN_FrameWork
E:\Projects\MiProyecto
```

En Linux:

```
mkdir -p ~/Projects
cd ~/Projects

mkdir GEN_FrameWork
cd GEN_FrameWork

git clone https://gitlab.genframework.com/gen-framework/gen.git GEN
git clone https://gitlab.genframework.com/gen-framework/common.git Common
git clone https://gitlab.genframework.com/gen-framework/thirdpartylibraries.git ThirdPartyLibraries
git clone https://gitlab.genframework.com/gen-framework/examples.git Examples
git clone https://gitlab.genframework.com/gen-framework/tests.git Tests
git clone https://gitlab.genframework.com/gen-framework/utilities.git Utilities
```

```
git clone https://gitlab.genframework.com/gen-framework/resources.git Resources
git clone https://gitlab.genframework.com/gen-framework/documents.git Documents
```

Y para crear un proyecto propio:

```
cd ~/Projects
mkdir MiProyecto
```

El resultado:

```
~/Projects/GEN_FrameWork
~/Projects/MiProyecto
```

Si el usuario solo quiere descargar y compilar, HTTPS suele ser suficiente. Si se va a contribuir al proyecto o trabajar con permisos de escritura, puede ser preferible usar SSH.

Para actualizar un repositorio:

```
git pull
```

Para actualizar todos los repositorios del framework, se puede entrar en cada carpeta de GEN_FrameWork y ejecutar git pull. También es posible crear scripts auxiliares que actualicen todo el árbol, pero al principio basta con entender que cada carpeta es un repositorio independiente.

Durante el aprendizaje inicial es recomendable no modificar directamente el código interno del framework salvo que se trabaje en una rama específica. Lo mejor es estudiar los ejemplos, crear pruebas propias y empezar nuevas aplicaciones como carpetas hermanas de GEN_FrameWork, dentro de Projects.

Como alternativa a la descarga manual de cada repositorio, en la sección de descargas de la web de GEN Framework (<https://www.genframework.com/#downloads>) están disponibles dos scripts: uno para Windows y otro para Linux. Cada script descarga automáticamente todos los repositorios de GEN Framework. Solo es necesario utilizar el que corresponda al sistema operativo desde el que se vaya a trabajar, que servirá como base para la compilación.

7. Herramientas necesarias

Para trabajar con GEN Framework se necesitan varias herramientas. Algunas son imprescindibles y otras son recomendables según la plataforma o el tipo de compilación.

La primera herramienta es un compilador de C/C++. En Windows se utiliza habitualmente Microsoft Visual C++, incluido con Visual Studio. En Linux se emplean GCC o Clang. Clang también se puede usar en Windows, aunque al principio suele resultar más sencillo trabajar con el compilador de Microsoft dentro del entorno de Visual Studio.

La segunda herramienta es CMake. GEN Framework utiliza CMake como infraestructura interna para describir cómo se construyen los proyectos. CMake permite generar configuraciones para distintos compiladores y plataformas.

La tercera herramienta es Ninja. Ninja es un sistema de construcción rápido que suele utilizarse junto con CMake. CMake genera los ficheros necesarios y Ninja ejecuta la compilación.

La cuarta herramienta es Git, necesaria para descargar y actualizar los repositorios.

La quinta herramienta es un entorno de edición o desarrollo. Aquí destacan Visual Studio y Visual Studio Code.

Visual Studio es un IDE completo para Windows. Incluye depurador, integración con el compilador de Microsoft, gestión de soluciones, análisis, inspección de memoria y muchas herramientas útiles para C++.

Visual Studio Code es un editor más ligero, extensible y multiplataforma. Funciona muy bien con CMake, Git, WSL2, Docker y extensiones para C/C++. No sustituye automáticamente al compilador; hay que instalar el compilador por separado.

La sexta herramienta importante es Docker. Docker permite crear entornos aislados mediante contenedores. En GEN Framework puede ser útil para compilación cruzada, para crear entornos reproducibles y para compilar librerías de terceros en Linux cuando el sistema principal no tiene instaladas todas las dependencias necesarias.

Junto a Docker, Docker Compose permite definir y lanzar entornos de compilación de forma más cómoda mediante ficheros de configuración.

8. Por qué usamos estas herramientas

Las herramientas propuestas no son arbitrarias. Forman parte del ecosistema real del desarrollo C/C++ moderno.

Visual Studio es una de las mejores opciones para desarrollar y depurar aplicaciones C++ en Windows. Su depurador, su integración con MSVC y su madurez lo convierten en una herramienta muy cómoda para esta plataforma.

Visual Studio Code es especialmente útil cuando queremos trabajar de forma multiplataforma. Permite editar en Windows, Linux o WSL2 y no impone una estructura cerrada de proyecto. Puede trabajar sobre carpetas, integrarse con Git, lanzar terminales, conectarse a WSL2 y usar extensiones de CMake o Docker.

GCC y Clang son compiladores ampliamente utilizados en Linux y en muchos entornos de desarrollo. GCC tiene una presencia histórica enorme en el mundo GNU/Linux. Clang, basado en LLVM, ofrece diagnósticos muy claros, buena integración con herramientas modernas y soporte para múltiples plataformas.

CMake permite que el proyecto no dependa de un único IDE. En lugar de mantener manualmente proyectos distintos para Visual Studio, Makefiles para Linux o configuraciones separadas para cada entorno, CMake ofrece una descripción común del sistema de construcción.

Ninja acelera la compilación y se integra muy bien con CMake. En proyectos grandes, la velocidad del sistema de construcción puede marcar una diferencia importante.

Git permite mantener el código organizado y actualizado. En un framework compuesto por varios repositorios, es una herramienta imprescindible.

Docker permite reproducir entornos. Esto es especialmente valioso cuando una compilación depende de versiones concretas de herramientas, librerías del sistema o toolchains de compilación cruzada.

Docker Compose permite definir esos entornos de forma más ordenada, evitando comandos largos y difíciles de recordar.

Todas estas herramientas tienen además una característica común: permiten automatizar. Y la automatización es esencial en un framework multiplataforma. Si una compilación depende de pasos manuales que cada usuario ejecuta de forma diferente, el resultado es frágil. Si la compilación se puede repetir mediante scripts, CMake y contenedores, el proyecto resulta más estable y mantenible.

9. Windows como entorno de desarrollo

Windows es una plataforma natural para empezar con GEN Framework si el usuario ya utiliza Visual Studio.

Una instalación básica en Windows debería incluir:

```
Visual Studio Community o superior
Carga de trabajo de desarrollo de escritorio con C++
Git
CMake
Ninja
Visual Studio Code, opcional pero recomendable
```

Docker Desktop, opcional para trabajar con contenedores

Durante la instalación de Visual Studio es importante seleccionar la carga de trabajo de C++. No basta con instalar Visual Studio como editor general. GEN Framework necesita el compilador de C/C++, el enlazador, el SDK de Windows y las herramientas de construcción.

Una vez instalado, se puede comprobar si Git, CMake o Ninja están disponibles:

```
git --version
cmake --version
ninja --version
```

Para comprobar el compilador de Microsoft:

```
cl
```

Si cl no se reconoce, es posible que no estemos usando una consola con las variables de Visual Studio configuradas o que no se hayan instalado las herramientas de C++.

Sin embargo, aunque estas comprobaciones son útiles, el usuario no debería empezar compilando manualmente con CMake. El flujo recomendado en Windows es ir a la carpeta Common dentro de GEN_FrameWork y usar compile.bat.

Por ejemplo:

```
cd /d E:\Projects\GEN_FrameWork\Common
compile.bat
```

El script compile.bat es el punto de entrada recomendado para la compilación en Windows. Desde ahí se pueden lanzar compilaciones de ejemplos, preparar la configuración y ejecutar etapas concretas del proceso.

10. Linux como entorno de desarrollo

Linux es una plataforma esencial para GEN Framework. Muchos sistemas embebidos, servidores, Raspberry Pi, equipos industriales y entornos de desarrollo utilizan Linux. Por eso es importante que el framework pueda compilar y ejecutarse correctamente en este sistema.

En una distribución basada en Debian o Ubuntu, una instalación inicial podría ser:

```
sudo apt update
sudo apt install build-essential git cmake ninja-build clang
```

build-essential instala GCC, G++ y herramientas básicas de compilación. Clang se instala aparte si queremos utilizar LLVM como compilador.

También pueden ser necesarias otras dependencias del sistema, especialmente para ejemplos gráficos, comunicaciones, audio, dispositivos o librerías de terceros. Estas dependencias pueden variar según la distribución y la versión.

El flujo inicial en Linux debe partir también de la carpeta Common dentro de GEN_FrameWork:

```
cd ~/Projects/GEN_FrameWork/Common
chmod +x compile.bash
./compile.bash
```

Al igual que en Windows, el script permite compilar todos los ejemplos o seleccionar uno concreto. Además, en Linux aparece una opción especialmente interesante: usar Docker. En determinados casos, puede ser recomendable o necesario compilar dentro de un contenedor, especialmente cuando intervienen librerías de terceros o entornos de compilación cruzada.

Para ello, el script puede aceptar el modificador DOCKERS, escrito en mayúsculas.

El usuario debe entender que Docker no es obligatorio para todos los casos. Si el entorno Linux tiene todas las dependencias instaladas y la compilación nativa funciona, puede usarse directamente. Pero cuando se trabaja con librerías de terceros, compilación cruzada o entornos que deben ser reproducibles, Docker se convierte en una herramienta muy potente.

11. WSL2 como entorno mixto Windows/Linux

WSL2, el Subsistema de Windows para Linux, permite ejecutar un entorno Linux dentro de Windows. Para GEN Framework es una herramienta muy interesante porque nos permite trabajar en un mismo equipo con dos mundos que normalmente estarían separados: por un lado Windows, Visual Studio, herramientas gráficas y depuración nativa; por otro lado Linux, GCC, Clang, Bash, CMake, Docker y los scripts propios de compilación del framework.

Esta combinación resulta especialmente útil en un framework multiplataforma. Podemos desarrollar en Windows, compilar con las herramientas de Microsoft, abrir el mismo código con Visual Studio Code y, al mismo tiempo, lanzar compilaciones Linux desde WSL2. Esto permite validar desde fases tempranas que el código no depende accidentalmente de una única plataforma.

Una instalación básica de WSL2 puede realizarse desde PowerShell:

```
wsl --install
```

Una vez instalada la distribución Linux, como Ubuntu, se preparan las herramientas principales:

```
sudo apt update
sudo apt install build-essential git cmake ninja-build clang
```

A partir de ahí, se puede compilar GEN Framework desde Linux usando los scripts de la carpeta Common:

```
cd ~/Projects/GEN_Framework/Common
./compile.bash inibase
```

O, cuando interese usar el flujo basado en Docker:

```
cd ~/Projects/GEN_Framework/Common
./compile.bash inibase DOCKERS
```

Sin embargo, WSL2 introduce una cuestión importante que conviene entender bien: el rendimiento del sistema de ficheros.

Desde Linux dentro de WSL2 podemos trabajar de dos maneras principales. La primera es guardar los fuentes dentro del propio sistema de ficheros Linux, por ejemplo:

```
/home/usuario/Projects/GEN_Framework
/home/usuario/Projects/MiProyecto
```

La segunda es trabajar sobre una unidad de Windows montada dentro de Linux:

```
/mnt/c/Projects/GEN_Framework
/mnt/c/Projects/MiProyecto
```

Desde el punto de vista del rendimiento puro de compilación en Linux, la primera opción suele ser claramente más rápida. Cuando los fuentes están dentro del sistema de ficheros Linux de WSL2, normalmente sobre el disco virtual de la distribución, las operaciones de lectura, escritura, creación de ficheros temporales, generación de objetos y recorrido de directorios son mucho más eficientes.

Esto es especialmente relevante en C/C++. Una compilación no consiste en leer un único fichero grande. Al contrario, una compilación real puede abrir miles de ficheros pequeños: cabeceras, fuentes, objetos, dependencias generadas, ficheros intermedios, scripts, cachés y librerías. Este patrón de acceso intensivo penaliza mucho cualquier capa de traducción lenta entre sistemas de ficheros.

El problema aparece cuando se usa WSL2 como entorno mixto real. Si los repositorios se copian o clonan una vez en Windows y otra dentro de Linux, la compilación Linux resulta más rápida, pero introduce otro problema: los fuentes quedan duplicados.

Esa duplicación no es una buena solución general. Tener dos árboles de código independientes significa tener dos lugares donde pueden existir modificaciones. Podemos cambiar un fichero en Windows y olvidarnos de replicarlo en Linux. Podemos corregir algo en WSL2 y no tenerlo en la copia de Windows. Podemos compilar una versión creyendo que corresponde al último código, cuando en realidad pertenece a una copia desactualizada.

En proyectos pequeños puede parecer manejable; en un framework grande o en una aplicación real, termina siendo una fuente de errores. Por tanto, aunque colocar los fuentes dentro del sistema Linux de WSL2 sea lo más rápido desde el punto de vista de la compilación, nuestra recomendación para GEN Framework debe ser más equilibrada: evitar duplicar fuentes siempre que sea posible. Es preferible tener un único árbol de trabajo bien definido y asumir conscientemente las consecuencias de rendimiento, antes que trabajar con dos copias que pueden divergir.

El problema de rendimiento al acceder desde WSL2 a rutas de Windows, como `/mnt/c`, es conocido. WSL2 separa más claramente el entorno Linux del entorno Windows que WSL1. Esa arquitectura tiene ventajas importantes, porque WSL2 ejecuta un kernel Linux real dentro de una máquina virtual ligera, pero el acceso cruzado entre sistemas de ficheros tiene un coste.

En la interoperabilidad entre Windows y Linux dentro de WSL2 intervienen mecanismos basados en 9P, el protocolo asociado al sistema Plan 9. Plan 9 fue un sistema operativo de investigación desarrollado en Bell Labs. Su nombre procede de la película de Ed Wood *Plan 9 from Outer Space*, una obra de culto famosa precisamente por su mala reputación cinematográfica. La anécdota puede resultar curiosa, pero en este contexto lo importante no es el origen del nombre, sino sus consecuencias prácticas: para cargas de trabajo con muchos accesos a ficheros pequeños, como una compilación C/C++, esta capa de interoperabilidad no es una buena opción desde el punto de vista del rendimiento.

En la práctica, esto convierte el rendimiento de `/mnt/c` en uno de los problemas endémicos de WSL2 frente a WSL1. WSL1 tenía otra arquitectura y, en algunos escenarios de acceso a ficheros Windows desde Linux, podía comportarse mejor. WSL2 mejora muchos aspectos técnicos, especialmente al disponer de un kernel Linux real, pero el acceso cruzado a ficheros de Windows puede ser claramente más lento.

Por tanto, tenemos tres estrategias posibles.

La primera estrategia es trabajar dentro del sistema de ficheros Linux de WSL2. Es la opción más rápida para compilar desde Linux:

```
/home/usuario/Projects/GEN_Framework  
/home/usuario/Projects/MiProyecto
```

Su ventaja es el rendimiento. Su inconveniente es que, si también queremos trabajar con herramientas Windows de forma directa sobre el mismo árbol, podemos encontrarnos con flujos menos cómodos. Visual Studio Code lo resuelve bastante bien mediante su integración con WSL, pero otras herramientas Windows pueden resultar menos naturales.

La segunda estrategia es trabajar sobre una unidad Windows desde WSL2:

```
/mnt/c/Projects/GEN_Framework  
/mnt/c/Projects/MiProyecto
```

Su ventaja es que Windows y Linux ven el mismo árbol de fuentes de forma directa. No hay duplicación. La misma carpeta se puede abrir desde Windows, editar desde herramientas Windows y compilar desde WSL2. Su inconveniente es el rendimiento: compilar desde Linux sobre `/mnt/c` puede ser sensiblemente más lento.

La tercera estrategia, que es la que recomendamos como punto de equilibrio para muchos escenarios de GEN Framework, es mantener un único árbol de trabajo en Windows, preferiblemente en una unidad preparada para desarrollo, y paliar el coste de WSL2 con técnicas de optimización como caché de compilación y Dev Drive.

GEN Framework está preparado para aprovechar técnicas de Compiler Cache. La idea de un compilador con caché es evitar recompilar innecesariamente ficheros que no han cambiado. En un proyecto C/C++ grande, muchas compilaciones incrementales repiten trabajo. Si el sistema puede detectar que una unidad de compilación ya fue procesada con los mismos parámetros, puede reutilizar el resultado anterior. Esto reduce tiempo de compilación y disminuye parte del impacto de un sistema de ficheros más lento.

El uso de caché de compilación no elimina el problema de fondo de /mnt/c, porque todavía se producen accesos al sistema de ficheros, pero sí puede reducir mucho el coste en compilaciones incrementales. En un flujo real de desarrollo, donde se modifica una parte pequeña del código y se recompila muchas veces, esta mejora puede ser muy significativa.

Además, en Windows 11 existe otra herramienta interesante: Dev Drive. Un Dev Drive es un volumen de almacenamiento pensado para cargas de trabajo de desarrollo. Está basado en ReFS y utiliza optimizaciones específicas para escenarios como repositorios, compilaciones, restauración de paquetes, ficheros temporales y operaciones intensivas de desarrollo.

En la práctica, un Dev Drive puede ser una unidad separada, por ejemplo:

```
D:\Projects\
```

o:

```
G:\Projects\
```

Dentro de ella podríamos mantener la estructura recomendada:

```
D:\Projects\
|
|--- GEN_Framework\
|   |--- GEN\
|   |--- Common\
|   |--- Examples\
|   |--- ThirdPartyLibraries\
|   |--- ...
|
|--- MiProyecto\
```

Desde Windows trabajaríamos sobre:

```
D:\Projects\GEN_Framework
D:\Projects\MiProyecto
```

Desde WSL2 accederíamos a la misma estructura mediante:

```
/mnt/d/Projects/GEN_Framework
/mnt/d/Projects/MiProyecto
```

Esta estrategia mantiene un único árbol de fuentes. No obliga a duplicar repositorios. Permite usar herramientas Windows de forma natural y, al mismo tiempo, lanzar compilaciones desde WSL2. El coste de acceso cruzado sigue existiendo, pero se puede reducir utilizando una unidad optimizada para desarrollo y apoyándose en caché de compilación.

Por tanto, una recomendación práctica para GEN Framework en equipos Windows modernos podría ser:

```
Usar una carpeta Projects en una unidad Dev Drive.
Colocar GEN_Framework dentro de esa carpeta.
Colocar los proyectos propios al mismo nivel que GEN_Framework.
Usar Visual Studio o Visual Studio Code desde Windows.
```

```
Usar WSL2 para compilaciones Linux cuando sea necesario.  
Activar Compiler Cache cuando el flujo de compilación lo permita.  
Evitar duplicar fuentes salvo en casos muy concretos.
```

La estructura final quedaría así:

```
D:\Projects\  
|  
├── GEN_Framework\  
|   ├── GEN\  
|   ├── Common\  
|   ├── ThirdPartyLibraries\  
|   ├── Examples\  
|   ├── Tests\  
|   ├── Utilities\  
|   ├── Resources\  
|   └── Documents\  
|  
├── MiProyecto\  
└── OtroProyecto\
```

Y desde WSL2:

```
/mnt/d/Projects/  
|  
├── GEN_Framework/  
└── MiProyecto/
```

Para compilar desde WSL2 usando ese árbol compartido:

```
cd /mnt/d/Projects/GEN_Framework/Common  
./compile.bash inibase INTEL64 DEBUG COMPILE
```

O usando Docker:

```
cd /mnt/d/Projects/GEN_Framework/Common  
./compile.bash inibase INTEL64 DEBUG COMPILE DOCKERS
```

Esta no es la opción más rápida en términos absolutos frente a mantener todo dentro de `/home/usuario`, pero evita el problema de gestionar dos árboles de fuentes. En proyectos reales, esta estabilidad suele ser más importante que ganar algunos segundos a costa de introducir riesgo de divergencia.

Si el objetivo es una compilación Linux intensiva, automatizada o de integración continua, entonces sí puede tener sentido trabajar dentro del sistema de ficheros Linux o dentro de contenedores Docker, donde todo el árbol se monta y se compila de forma controlada. Pero para el desarrollo diario mixto Windows/Linux, especialmente cuando se alternan Visual Studio, Visual Studio Code, scripts Bash y herramientas Windows, la prioridad debe ser mantener una única fuente de verdad.

En resumen, WSL2 es una herramienta muy valiosa para GEN Framework, pero hay que usarla con criterio. Su integración de sistemas de ficheros no es perfecta. El acceso a rutas Windows desde Linux puede ser lento por la capa de interoperabilidad. Duplicar fuentes mejora el rendimiento, pero empeora el mantenimiento. Por eso nuestra recomendación es utilizar un único árbol de trabajo, preferiblemente en una unidad Dev Drive cuando se trabaja en Windows 11, y apoyarse en técnicas como Compiler Cache, Docker y una buena estructura de carpetas para reducir el impacto de rendimiento sin sacrificar la coherencia del proyecto.

12. Docker y Docker Compose

Docker permite crear entornos de ejecución y compilación aislados mediante contenedores. En el contexto de GEN Framework, Docker puede utilizarse para crear entornos de compilación reproducibles, para compilación cruzada y para construir librerías de terceros que requieren configuraciones específicas.

En C/C++, el resultado de una compilación no depende únicamente del código fuente. También depende del compilador, del enlazador, de las cabeceras instaladas, de las librerías disponibles, de la versión del sistema, de las rutas y de las opciones usadas. Esto hace que dos equipos puedan obtener resultados diferentes si sus entornos no son iguales.

Docker ayuda a resolver ese problema. Una imagen con las herramientas necesarias proporciona un entorno muy parecido a cualquier equipo que la ejecute. Esto reduce los errores difíciles de reproducir.

En GEN Framework, Docker puede ser especialmente útil en Linux por varias razones.

La primera es la compilación de librerías de terceros. Algunas bibliotecas externas pueden requerir paquetes concretos, versiones específicas o herramientas adicionales. Instalar todo eso en el sistema principal puede no ser deseable. Un contenedor permite encapsular esas dependencias.

La segunda es la compilación cruzada. Si necesitamos generar binarios para una arquitectura distinta, podemos preparar un contenedor con el toolchain correspondiente.

La tercera es la integración continua. Los mismos contenedores que se utilizan localmente pueden ejecutarse en servidores automáticos para comprobar que el framework compila correctamente.

La cuarta es la limpieza del sistema. En lugar de instalar muchas dependencias en la máquina principal, se mantienen dentro de la imagen Docker.

Docker Compose permite definir estos entornos mediante ficheros de configuración. En lugar de ejecutar comandos largos de Docker, podemos describir servicios, volúmenes, imágenes y comandos en un fichero Compose.

Un flujo conceptual podría ser:

```
cd ~/Projects/GEN_Framework
docker compose build
docker compose run --rm gen-build
```

Dentro del contenedor, el flujo debería seguir usando los scripts de GEN Framework:

```
cd /workspace/Common
./compile.bash inibase
```

O, si el propio script está preparado para invocar el contenedor desde fuera:

```
cd ~/Projects/GEN_Framework/Common
./compile.bash inibase DOCKERS
```

Este último caso es más cómodo para el usuario, porque mantiene el mismo punto de entrada: `compile.bash`.

Lo importante es no pensar en Docker como sustituto del framework ni como sustituto de CMake. Docker proporciona el entorno. Los scripts de GEN Framework organizan la compilación. CMake genera la configuración interna. El compilador construye el binario.

Cada herramienta cumple su papel.

13. Los scripts de Common como punto de entrada

Uno de los aspectos más importantes de GEN Framework es que la compilación inicial debe hacerse desde la carpeta Common.

En Windows:

```
cd /d E:\Projects\GEN_Framework\Common
compile.bat
```

En Linux:

```
cd ~/Projects/GEN_Framework/Common
./compile.bash
```

O, en WSL2 usando un árbol único en una unidad de Windows:

```
cd /mnt/d/Projects/GEN_Framework/Common
./compile.bash
```

Estos scripts son el punto de entrada recomendado porque concentran la lógica necesaria para compilar el framework y sus ejemplos. Permiten evitar errores derivados de invocar CMake directamente con parámetros incompletos, rutas incorrectas o variables de entorno no preparadas.

El usuario que empieza no tiene por qué saber todavía qué opciones internas necesita CMake, dónde deben generarse las carpetas de construcción, qué rutas relativas se usan, qué compilador se selecciona o cómo se estructuran los binarios de salida. Esa información la gestionan los scripts.

Una característica importante es que los scripts pueden recibir el nombre de un ejemplo. Esto permite compilar solo ese ejemplo.

Por ejemplo, en Windows:

```
compile.bat inibase
```

En Linux:

```
./compile.bash inibase
```

Y en Linux usando Docker:

```
./compile.bash inibase DOCKERS
```

Este mecanismo es especialmente útil durante el aprendizaje. Si estamos estudiando inibase, no necesitamos compilar todos los ejemplos del framework cada vez. Podemos centrarnos en uno, modificarlo, recompilarlo y ejecutarlo.

También es útil para depuración. Si un ejemplo concreto falla, podemos compilar solo ese ejemplo y revisar la salida con más claridad.

14. Convención de nombres en los scripts

Antes de utilizar los scripts de compilación es importante conocer una convención básica de GEN Framework: el nombre del programa, aplicación o ejemplo que se pasa al script debe escribirse siempre en minúsculas.

Por ejemplo, aunque el ejemplo pueda aparecer documentado como IniBase, al llamar al script de compilación se debe escribir:

```
compile.bat inibase
```

o en Linux:

```
./compile.bash inibase
```

Los modificadores, en cambio, se escriben siempre en mayúsculas. Esto permite distinguir claramente entre el nombre del programa que se quiere compilar y las opciones especiales que modifican el comportamiento del script.

El criterio general es:

```
compile.bat programa_en_minusculas MODIFICADOR_EN_MAYUSCULAS
compile.bash programa_en_minusculas MODIFICADOR_EN_MAYUSCULAS
```

Ejemplos correctos:

```
compile.bat inibase INTEL64 DEBUG COMPILE
./compile.bash inibase INTEL64 DEBUG COMPILE
./compile.bash inibase ARM64 RELEASE CLANG DOCKERS
```

Ejemplos incorrectos:

```
compile.bat IniBase intel64 debug compile
./compile.bash IniBase docker
```

La diferencia es importante. El programa va en minúsculas. Los modificadores van en mayúsculas.

15. Modificadores de los scripts

Los scripts de compilación de GEN Framework permiten modificar el comportamiento del proceso de construcción mediante argumentos. Estos argumentos se pasan después del nombre del programa o ejemplo.

La sintaxis general puede entenderse así:

```
script programa modificador modificador modificador ...
```

En Windows:

```
compile.bat inibase INTEL64 DEBUG COMPILE
```

En Linux:

```
./compile.bash inibase INTEL64 DEBUG COMPILE
```

No todos los modificadores tienen sentido en todas las plataformas. Algunos pertenecen al entorno Windows, otros al entorno Linux, otros al modo de compilación, otros al compilador y otros a funcionalidades internas del framework.

También es importante entender que los scripts pueden tener valores por defecto. Si el usuario ejecuta:

```
compile.bat inibase
```

o:

```
./compile.bash inibase
```

El script aplica la configuración predeterminada definida por el framework. Esa configuración puede ser suficiente para una primera prueba. Cuando se necesita controlar con precisión la arquitectura, el modo, el compilador, las trazas o el control de memoria, se utilizan los modificadores.

16. Modificadores de stage

Los modificadores de stage indican qué parte del proceso de construcción queremos ejecutar.

Los valores disponibles son:

```
CMAKE
COMPILE
TEST
```

CMAKE ejecuta la fase de configuración. En esta etapa se preparan las carpetas de construcción, se detectan compiladores, se configuran opciones, se localizan dependencias y se generan los ficheros necesarios para que el sistema de construcción pueda trabajar.

Ejemplo en Windows:

```
compile.bat inibase CMAKE
```

Ejemplo en Linux:

```
./compile.bash inibase CMAKE
```

COMPILE ejecuta la fase de compilación. En esta etapa se compilan los ficheros fuente, se generan los objetos, se enlazan las librerías y se crea el binario final.

Ejemplo en Windows:

```
compile.bat inibase COMPILE
```

Ejemplo en Linux:

```
./compile.bash inibase COMPILE
```

TEST ejecuta la fase de pruebas, si está disponible para el proyecto o configuración seleccionada. Esta fase permite validar componentes, comprobar comportamientos o ejecutar pruebas automatizadas.

Ejemplo en Windows:

```
compile.bat inibase TEST
```

Ejemplo en Linux:

```
./compile.bash inibase TEST
```

Separar el proceso en stages tiene varias ventajas. Si solo hemos cambiado opciones de configuración, podemos ejecutar CMAKE. Si ya está todo configurado y hemos cambiado código fuente, podemos ejecutar COMPILE. Si queremos comprobar comportamiento después de compilar, podemos ejecutar TEST.

Una compilación completa puede ejecutar varias fases internamente, pero los stages permiten controlar el proceso cuando necesitamos más precisión.

17. Modificadores de plataforma en Windows

Los modificadores de plataforma indican para qué arquitectura o destino queremos compilar. En Windows, los valores disponibles son:

```
INTEL32  
INTEL64  
ANDROID32  
ANDROID64
```

INTEL32 selecciona una compilación para arquitectura Intel de 32 bits. Este destino puede ser necesario cuando queremos generar binarios compatibles con sistemas de 32 bits o validar comportamiento en esa arquitectura.

Ejemplo:

```
compile.bat inibase INTEL32 DEBUG COMPILE
```

INTEL64 selecciona una compilación para arquitectura Intel de 64 bits. Este es probablemente el destino más habitual en Windows moderno.

Ejemplo:

```
compile.bat inibase INTEL64 DEBUG COMPILE
```

ANDROID32 selecciona una compilación para Android de 32 bits, cuando el entorno de compilación y las herramientas necesarias estén configuradas.

Ejemplo conceptual:

```
compile.bat inibase ANDROID32 DEBUG COMPILE
```

ANDROID64 selecciona una compilación para Android de 64 bits.

Ejemplo conceptual:

```
compile.bat inibase ANDROID64 DEBUG COMPILE
```

Los modificadores Android dependen de que el entorno Android esté preparado correctamente: SDK, NDK, toolchain, rutas y configuración de CMake. No deben confundirse con una compilación Windows normal. Permiten seleccionar un destino de compilación, pero requieren que las herramientas específicas existan.

18. Modificadores de plataforma en Linux

En Linux, los valores disponibles son:

```
INTEL64
ARM32
ARM64
RPI32
RPI64
```

INTEL64 selecciona una compilación para arquitectura Intel o AMD de 64 bits. Es el destino habitual para un PC Linux moderno o una máquina WSL2 sobre Windows.

Ejemplo:

```
./compile.bash inibase INTEL64 DEBUG COMPILE
```

ARM32 selecciona una compilación para arquitectura ARM de 32 bits. Puede ser útil en sistemas embebidos, placas antiguas o entornos específicos que todavía trabajen en 32 bits.

Ejemplo:

```
./compile.bash inibase ARM32 DEBUG COMPILE
```

ARM64 selecciona una compilación para arquitectura ARM de 64 bits. Es habitual en placas modernas, sistemas embebidos recientes o entornos Linux sobre ARM64.

Ejemplo:

```
./compile.bash inibase ARM64 DEBUG COMPILE
```

RPI32 selecciona una compilación orientada a Raspberry Pi de 32 bits.

Ejemplo:

```
./compile.bash inibase RPI32 DEBUG COMPILE
```

RPI64 selecciona una compilación orientada a Raspberry Pi de 64 bits.

Ejemplo:

```
./compile.bash inibase RPI64 DEBUG COMPILE
```

En estos casos es especialmente importante distinguir entre compilación nativa y compilación cruzada. Ejecutar el script directamente en una Raspberry Pi puede corresponder a una compilación nativa. Compilar desde un PC Linux para generar binarios destinados a Raspberry Pi es una compilación cruzada y normalmente requiere toolchains, Docker o configuración adicional.

19. Modificadores de modo

Los modificadores de modo indican si queremos generar una compilación orientada a desarrollo o una compilación orientada a distribución.

Los valores disponibles son:

```
DEBUG  
RELEASE
```

DEBUG genera una compilación pensada para desarrollar. Normalmente incluye información de depuración, permite inspeccionar variables, seguir la ejecución paso a paso y detectar errores con más facilidad. También puede incluir comprobaciones adicionales o menor optimización.

Ejemplo en Windows:

```
compile.bat inibase INTEL64 DEBUG COMPILE
```

Ejemplo en Linux:

```
./compile.bash inibase INTEL64 DEBUG COMPILE
```

RELEASE genera una compilación pensada para ejecución final. Aplica optimizaciones, reduce información de depuración y genera binarios más adecuados para distribución o pruebas de rendimiento.

Ejemplo en Windows:

```
compile.bat inibase INTEL64 RELEASE COMPILE
```

Ejemplo en Linux:

```
./compile.bash inibase INTEL64 RELEASE COMPILE
```

Durante el aprendizaje se recomienda empezar con DEBUG, porque facilita la depuración. Cuando la aplicación ya compila y funciona correctamente, se puede probar RELEASE para medir rendimiento o generar una versión más cercana a distribución.

No es recomendable medir rendimiento real usando una compilación DEBUG. Tampoco es cómodo depurar un problema complejo usando únicamente RELEASE. Cada modo tiene su propósito.

20. Modificadores de compilador

Los modificadores de compilador indican si queremos utilizar Clang o no.

Los valores disponibles son:

```
CLANG  
NOTCLANG
```

CLANG solicita el uso de Clang como compilador, siempre que esté disponible para la plataforma seleccionada y que el entorno esté preparado.

Ejemplo en Linux:

```
./compile.bash inibase INTEL64 DEBUG CLANG COMPILE
```

Ejemplo en Windows, si el entorno lo soporta:

```
compile.bat inibase INTEL64 DEBUG CLANG COMPILE
```

NOTCLANG indica que no se desea usar Clang. En Linux esto puede significar usar GCC. En Windows puede significar usar el compilador Microsoft, dependiendo del flujo configurado.

Ejemplo:

```
./compile.bash inibase INTEL64 DEBUG NOTCLANG COMPILE
```

La elección entre CLANG y NOTCLANG puede ser importante para detectar diferencias entre compiladores. Un código que compila con MSVC puede mostrar advertencias distintas con Clang. Un código que compila con GCC puede revelar otros problemas con Clang. Por eso, en un framework multiplataforma, probar varios compiladores es una práctica muy saludable.

Clang suele ofrecer mensajes de diagnóstico muy claros y es útil para encontrar problemas de compatibilidad. GCC es una referencia fundamental en Linux. MSVC es esencial en Windows. GEN Framework debe ser capaz de convivir con estos entornos.

21. Modificadores de control de memoria

Los modificadores de memoria indican si se activa o no el sistema de control de memoria del framework.

Los valores disponibles son:

```
MEMCTRL
NOTMEMCTRL
```

MEMCTRL activa el control de memoria, si la configuración y el framework lo soportan. Este modo puede ser útil durante el desarrollo para detectar fugas, asignaciones incorrectas, liberaciones no realizadas o comportamientos relacionados con el ciclo de vida de objetos y recursos.

Ejemplo:

```
compile.bat inibase INTEL64 DEBUG MEMCTRL COMPILE
```

En Linux:

```
./compile.bash inibase INTEL64 DEBUG MEMCTRL COMPILE
```

NOTMEMCTRL desactiva ese control de memoria.

Ejemplo:

```
compile.bat inibase INTEL64 DEBUG NOTMEMCTRL COMPILE
```

En Linux:

```
./compile.bash inibase INTEL64 DEBUG NOTMEMCTRL COMPILE
```

En C/C++, la memoria es una parte crítica del desarrollo. Aunque C++ moderno ofrece herramientas como RAII, contenedores estándar y punteros inteligentes, un framework de bajo nivel puede necesitar mecanismos adicionales de diagnóstico. Activar el control de memoria durante el desarrollo puede ayudar a encontrar errores que de otro modo aparecerían más tarde y serían más difíciles de localizar.

Sin embargo, estos mecanismos pueden introducir coste adicional o generar información extra. Por eso es útil poder activarlos o desactivarlos mediante modificadores.

22. Modificadores de trazas

Los modificadores de trazas controlan la generación o comportamiento del sistema de trazas.

Los valores disponibles son:

```
TRACE
TRACENOTINTER
NOTTRACE
```

TRACE activa el sistema de trazas. Las trazas son fundamentales para diagnosticar el comportamiento de una aplicación, especialmente cuando trabajamos con varios sistemas, hilos, comunicaciones, módulos gráficos o plataformas distintas.

Ejemplo:

```
compile.bat inibase INTEL64 DEBUG TRACE COMPILE
```

En Linux:

```
./compile.bash inibase INTEL64 DEBUG TRACE COMPILE
```

TRACENOTINTER activa una configuración de trazas sin la parte intermedia, interactiva o interconectada que pueda estar asociada al sistema de trazas del framework, según la implementación concreta del script. Es una opción útil cuando se desea conservar trazabilidad, pero evitar determinados mecanismos adicionales de comunicación, intermediación o salida.

Ejemplo:

```
compile.bat inibase INTEL64 DEBUG TRACENOTINTER COMPILE
```

En Linux:

```
./compile.bash inibase INTEL64 DEBUG TRACENOTINTER COMPILE
```

NOTTRACE desactiva el sistema de trazas.

Ejemplo:

```
compile.bat inibase INTEL64 RELEASE NOTTRACE COMPILE
```

En Linux:

```
./compile.bash inibase INTEL64 RELEASE NOTTRACE COMPILE
```

Durante el desarrollo, TRACE puede ser muy útil. En versiones de distribución o pruebas de rendimiento, puede interesar NOTTRACE para reducir ruido, dependencias o coste de ejecución.

El valor TRACENOTINTER permite una configuración intermedia: no es lo mismo desactivar todas las trazas que mantener trazas pero sin determinados mecanismos auxiliares.

23. Modificadores de feedback

Los modificadores de feedback controlan si el proceso o la aplicación genera información adicional de retroalimentación durante la compilación o ejecución, según cómo lo utilice el framework.

Los valores disponibles son:

```
FEEDBACK  
NOTFEEDBACK
```

FEEDBACK activa la generación de feedback. Puede resultar útil durante desarrollo, pruebas, validación de scripts o diagnóstico de comportamiento.

Ejemplo:

```
compile.bat inibase INTEL64 DEBUG FEEDBACK COMPILE
```

En Linux:

```
./compile.bash inibase INTEL64 DEBUG FEEDBACK COMPILE
```

NOTFEEDBACK desactiva ese feedback adicional.

Ejemplo:

```
compile.bat inibase INTEL64 RELEASE NOTFEEDBACK COMPILE
```

En Linux:

```
./compile.bash inibase INTEL64 RELEASE NOTFEEDBACK COMPILE
```

La utilidad de este modificador depende del contexto. En fases de desarrollo puede ser interesante disponer de más información. En compilaciones limpias, automatizadas o de distribución, puede interesar reducir la salida a lo imprescindible.

24. Modificador DOCKERS

Además de los grupos anteriores, en Linux puede utilizarse el modificador:

DOCKERS

DOCKERS indica que la compilación debe realizarse utilizando el flujo basado en Docker preparado por el framework.

Ejemplo:

```
./compile.bash inibase DOCKERS
```

También puede combinarse con otros modificadores:

```
./compile.bash inibase INTEL64 DEBUG CLANG MEMCTRL TRACE FEEDBACK COMPILE DOCKERS
```

O para ejecutar solo la configuración CMake dentro del flujo Docker:

```
./compile.bash inibase INTEL64 DEBUG CMAKE DOCKERS
```

El uso de DOCKERS puede ser especialmente importante para compilación cruzada o para compilar librerías de terceros en Linux. Si una dependencia necesita un entorno concreto, el contenedor puede proporcionar ese entorno de forma reproducible.

Aunque el modificador se llame DOCKERS, conceptualmente se refiere al uso de Docker como entorno de compilación.

25. Combinación de modificadores

Los modificadores permiten construir órdenes bastante expresivas. Por ejemplo, si queremos compilar inibase en Windows para Intel 64 bits, en modo Debug, usando la fase de compilación, con control de memoria y trazas activadas, podríamos escribir:

```
compile.bat inibase INTEL64 DEBUG COMPILE MEMCTRL TRACE
```

Si queremos configurar con CMake antes de compilar:

```
compile.bat inibase INTEL64 DEBUG CMAKE MEMCTRL TRACE
```

Y después compilar:

```
compile.bat inibase INTEL64 DEBUG COMPILE MEMCTRL TRACE
```

En Linux, una orden equivalente podría ser:

```
./compile.bash inibase INTEL64 DEBUG COMPILE MEMCTRL TRACE
```

Con Clang:

```
./compile.bash inibase INTEL64 DEBUG CLANG COMPILE MEMCTRL TRACE
```

Con Docker:

```
./compile.bash inibase INTEL64 DEBUG CLANG COMPILE MEMCTRL TRACE DOCKERS
```

Para una compilación Release sin control de memoria, sin trazas y sin feedback:

```
./compile.bash inibase INTEL64 RELEASE NOTMEMCTRL NOTTRACE NOTFEEDBACK COMPILE
```

La idea general es que los argumentos describen el tipo de compilación que queremos realizar:

programa	→ inibase
plataforma	→ INTEL64
modo	→ DEBUG o RELEASE
compilador	→ CLANG o NOTCLANG
memoria	→ MEMCTRL o NOTMEMCTRL
trazas	→ TRACE, TRACENOTINTER o NOTTRACE
feedback	→ FEEDBACK o NOTFEEDBACK
stage	→ CMAKE, COMPILE o TEST
entorno	→ DOCKERS, si se usa Docker en Linux

El nombre del programa siempre permanece en minúsculas. Los modificadores siempre permanecen en mayúsculas.

26. Tabla resumen de modificadores

La siguiente tabla resume los modificadores principales:

Grupo	Valores	Uso
Stage	CMAKE, COMPILE, TEST	Selecciona la fase del proceso de construcción
Plataforma Windows	INTEL32, INTEL64, ANDROID32, ANDROID64	Selecciona destino de compilación en Windows
Plataforma Linux	INTEL64, ARM32, ARM64, RPI32, RPI64	Selecciona destino de compilación en Linux
Modo	DEBUG, RELEASE	Selecciona compilación de desarrollo o distribución
Compilador	CLANG, NOTCLANG	Activa o desactiva uso de Clang
Memoria	MEMCTRL, NOTMEMCTRL	Activa o desactiva control de memoria
Trazas	TRACE, TRACENOTINTER, NOTTRACE	Configura el sistema de trazas
Feedback	FEEDBACK, NOTFEEDBACK	Activa o desactiva información adicional
Docker	DOCKERS	Usa flujo de compilación con Docker en Linux

Esta tabla no sustituye a la ayuda del propio script. La versión concreta de los scripts puede ampliar, limitar o ajustar combinaciones. Aun así, resume la filosofía general: los scripts permiten describir la compilación mediante modificadores claros, legibles y escritos en mayúsculas.

27. CMake como infraestructura interna

Aunque en este capítulo insistimos en que la primera compilación debe hacerse con los scripts, es importante entender qué papel cumple CMake.

CMake es la infraestructura que permite describir cómo se construye el proyecto de forma portable. No es un compilador. No sustituye a Visual Studio, GCC o Clang. Su trabajo consiste en generar los ficheros de construcción necesarios para el entorno elegido.

En Windows puede generar proyectos de Visual Studio o configuraciones para Ninja. En Linux puede generar Makefiles o ficheros Ninja. En entornos más avanzados puede trabajar con toolchains específicos para compilación cruzada.

GEN Framework utiliza CMake porque necesita una forma ordenada de describir módulos, dependencias, opciones, rutas, configuraciones y plataformas. Mantener manualmente proyectos separados para cada sistema sería difícil y propenso a errores.

Sin embargo, CMake tiene muchas opciones. Invocarlo directamente sin conocer la estructura del framework puede provocar problemas:

```
Rutas incorrectas.
Variables no definidas.
Carpetas de salida mal ubicadas.
Compilador equivocado.
Configuración Debug o Release incorrecta.
Dependencias no localizadas.
Librerías de terceros no preparadas.
```

Por eso los scripts de Common son tan importantes. No eliminan CMake, sino que lo encapsulan. El usuario trabaja con una entrada sencilla y el script se encarga de llamar a CMake de la forma adecuada.

El estudio de los ficheros CMake resulta especialmente útil cuando ya se conoce mejor el framework. Permite entender cómo se agregan nuevos proyectos, cómo se definen módulos, cómo se enlazan librerías y cómo se adaptan configuraciones para distintas plataformas.

El orden de aprendizaje recomendado es:

```
Primero usar los scripts.
Después entender qué hacen los scripts.
Finalmente estudiar CMake en detalle.
```

28. Visual Studio y Visual Studio Code

Visual Studio y Visual Studio Code tienen nombres parecidos, pero son herramientas distintas.

Visual Studio es un IDE completo para Windows. Está especialmente orientado al desarrollo con el compilador de Microsoft. Ofrece depuración avanzada, inspección de memoria, análisis, integración con soluciones y un entorno muy potente para aplicaciones Windows.

Visual Studio Code es un editor extensible y multiplataforma. No incluye por sí mismo un compilador del lenguaje C++, pero se integra muy bien con compiladores externos, CMake, Git, WSL2, Docker y terminales. Es especialmente útil cuando queremos trabajar de forma parecida en Windows y Linux.

Para GEN Framework, ambas herramientas pueden convivir perfectamente.

Un flujo habitual podría ser:

```
Usar Visual Studio para depurar aplicaciones Windows.
Usar Visual Studio Code para editar código y trabajar con scripts.
Usar Visual Studio Code conectado a WSL2 para compilar en Linux.
Usar Docker desde terminal cuando se necesite un entorno aislado.
```

Lo importante es que el proyecto no dependa exclusivamente de un IDE. GEN Framework debe poder compilarse mediante scripts y CMake, no solo pulsando un botón en Visual Studio. Esta independencia es esencial para mantener la naturaleza multiplataforma del framework.

29. Proyectos propios al mismo nivel que GEN_FrameWork

Cuando el usuario ya ha compilado ejemplos oficiales, el siguiente paso natural es crear un proyecto propio.

La recomendación es colocarlo dentro de la carpeta general Projects, pero no dentro de GEN_FrameWork.

La estructura sería:

```
Projects/
|
├── GEN_FrameWork/
|   ├── GEN/
|   ├── Common/
|   ├── Examples/
|   └── ...
└── MiProyecto/
```

Esta aplicación propia podría tener una estructura interna como:

```
MiProyecto/
|
├── CMakeLists.txt
└── Sources/
```

```
| Headers/  
| Resources/  
└ Scripts/
```

La estructura exacta depende del tipo de aplicación, pero el punto importante es que la aplicación está cerca del framework y puede acceder a él mediante rutas relativas.

Por ejemplo, desde Projects/MiProyecto podríamos referenciar:

```
../GEN_FrameWork/GEN  
../GEN_FrameWork/Common  
../GEN_FrameWork/Resources  
../GEN_FrameWork/ThirdPartyLibraries
```

Esto evita depender de rutas absolutas de una máquina concreta. Si otro usuario tiene el árbol completo en otro disco o en otra carpeta, las rutas relativas funcionan mientras se mantenga la estructura general:

```
Projects/  
├ GEN_FrameWork/  
└ MiProyecto/
```

También facilita que las aplicaciones propias puedan seguir el mismo estilo que los ejemplos oficiales. El lector puede estudiar un ejemplo en GEN_FrameWork/Examples, crear una versión propia en Projects/MiProyecto y modificarla sin tocar el original.

Esta separación es importante. Los ejemplos oficiales deben mantenerse como referencia limpia. Los proyectos propios deben evolucionar de forma independiente.

Además, esta organización permite tener múltiples proyectos propios usando una única copia del framework:

```
Projects/  
├ GEN_FrameWork/  
├ MiProyecto/  
├ HerramientaInterna/  
├ AplicacionCliente/  
└ PruebasRender/
```

Cada proyecto puede tener su propio repositorio Git, su propio ciclo de versiones y su propia estructura, pero todos pueden usar el mismo GEN_FrameWork situado en la carpeta común Projects.

Esta organización también ayuda cuando se desea actualizar el framework. Podemos actualizar GEN_FrameWork sin mezclar esos cambios con los proyectos propios. Y podemos actualizar un proyecto propio sin tocar los repositorios del framework.

Por tanto, la recomendación final es:

```
No colocar los proyectos propios dentro de GEN_FrameWork.  
Colocarlos dentro de Projects, al mismo nivel que GEN_FrameWork.  
Usar rutas relativas desde el proyecto hacia GEN_FrameWork.
```

30. Errores habituales durante la preparación

La preparación del entorno puede fallar por muchos motivos. Algunos errores son muy comunes.

Uno de los primeros es no tener Git instalado o no tenerlo en el PATH. Si `git --version` no funciona, no es posible clonar ni actualizar los repositorios.

Otro error habitual en Windows es instalar Visual Studio sin las herramientas de C++. Visual Studio puede instalarse para muchos tipos de desarrollo, pero GEN Framework necesita el compilador C++, el SDK de Windows y las herramientas de construcción.

También es frecuente instalar Visual Studio Code pensando que ya incluye compilador. Visual Studio Code es un editor. Necesita GCC, Clang, MSVC u otro compilador externo.

Otro problema habitual es tener CMake instalado, pero no disponible en el PATH, o tener una versión antigua.

También puede faltar Ninja si los scripts lo utilizan como generador.

En Linux, pueden faltar paquetes de desarrollo. Un ejemplo gráfico puede necesitar cabeceras de X11, OpenGL u otras dependencias. Un módulo concreto puede necesitar librerías adicionales.

En Docker, puede ocurrir que el servicio no esté iniciado, que el usuario no tenga permisos para ejecutar contenedores o que Docker Compose no encuentre el fichero de configuración.

En WSL2, un error común es no decidir correctamente dónde estará el árbol principal de fuentes. Trabajar dentro de /home/usuario puede ser más rápido para compilar desde Linux, pero dificulta mantener el mismo flujo desde Windows si se quieren usar herramientas Windows de forma directa. Trabajar desde /mnt/c o /mnt/d permite un único árbol compartido, pero puede ser más lento. Duplicar fuentes en ambos lados suele ser una mala práctica salvo en casos muy controlados.

También puede fallar la compilación si los repositorios no están al mismo nivel dentro de GEN_FrameWork. Por ejemplo, si el repositorio principal está en una carpeta y Common en otra completamente distinta, los scripts pueden no encontrar lo que esperan.

Otro error habitual es colocar los proyectos propios dentro de GEN_FrameWork y después mezclar ficheros propios con los repositorios del framework. Aunque esto pueda funcionar temporalmente, no es una buena práctica. La estructura recomendada es que el proyecto propio sea hermano de GEN_FrameWork dentro de Projects.

Comandos útiles para comprobar el entorno:

```
git --version
cmake --version
ninja --version
gcc --version
g++ --version
clang --version
docker --version
docker compose version
```

En Windows:

```
cl
```

Para comprobar Docker:

```
docker run hello-world
```

Si estos comandos fallan, el problema debe resolverse antes de culpar al framework. La primera tarea siempre es validar las herramientas básicas.

31. Cómo enfrentarse a un error de compilación

Cuando aparece un error de compilación, la reacción natural del principiante es pensar que todo está mal. En C/C++, sin embargo, los errores deben analizarse con método.

Primero hay que localizar el primer error real. Muchas veces una compilación produce decenas o cientos de mensajes, pero todos derivan del primero. Si falta una cabecera, aparecen después muchos errores adicionales porque el compilador ya no conoce tipos, clases o funciones.

Segundo, hay que comprobar si el error aparece durante CMAKE, durante COMPILE o durante TEST. No es lo mismo un error de configuración que un error de código o un fallo de prueba.

Si falla CMAKE, el problema suele estar en rutas, dependencias, compilador o configuración.

Si falla COMPILE, el problema suele estar en código fuente, cabeceras, opciones del compilador o incompatibilidades.

Si falla TEST, el código puede haber compilado correctamente, pero algún comportamiento no es el esperado.

Tercero, hay que comprobar si el error ocurre en compilación nativa o con Docker. Si ocurre solo en nativa, tal vez falte una dependencia local. Si ocurre solo en Docker, tal vez el contenedor no tenga acceso correcto al árbol de trabajo o falte algo en la imagen.

Cuarto, hay que reducir el caso. Si estamos compilando todos los ejemplos, podemos intentar compilar solo uno:

```
compile.bat inibase
```

o:

```
./compile.bash inibase
```

Si ese ejemplo compila, el entorno básico funciona y el problema puede estar en una dependencia específica de otro ejemplo.

Quinto, hay que comprobar la estructura de carpetas. Si los scripts están pensados para ejecutarse desde GEN_FrameWork/Common, debemos respetar esa ubicación. Si un proyecto propio usa rutas relativas hacia GEN_FrameWork, debemos comprobar que realmente está situado al mismo nivel que la carpeta del framework.

Por ejemplo, esta estructura es correcta:

```
Projects/
├── GEN_FrameWork/
└── MiProyecto/
```

Esta otra no es la recomendada para proyectos propios:

```
Projects/
├── GEN_FrameWork/
│   ├── GEN/
│   ├── Common/
│   └── MiProyecto/
```

Este método de análisis evita perder tiempo y ayuda a distinguir entre errores de instalación, errores de estructura, errores de configuración y errores reales de código.

32. El entorno como parte del aprendizaje

En GEN Framework, preparar el entorno no es un trámite secundario. Es parte del aprendizaje real.

Un programador que trabaja con C/C++ debe saber que el compilador importa, el enlazador importa, las rutas importan, las librerías importan, la arquitectura importa y la configuración importa.

GEN Framework intenta reducir la cantidad de código específico de plataforma que debe escribir el usuario. Pero no puede eliminar por completo la realidad de cada sistema. Windows, Linux, Android, WSL2 o Docker tienen diferencias. El framework las abstrae en la medida de lo posible, pero el entorno de construcción debe estar bien preparado.

Los scripts de Common ayudan a que esa preparación sea más sencilla. En lugar de empezar escribiendo comandos CMake largos y frágiles, el usuario dispone de una entrada controlada:

```
compile.bat
```

o:

```
./compile.bash
```

Después puede concretar:

```
compile.bat inibase
```

o:

```
./compile.bash inibase DOCKERS
```

Con el entorno ya conocido, también puede usar stages:

```
CMAKE  
COMPILE  
TEST
```

Este diseño permite aprender de forma progresiva. Primero se compila. Después se entiende el script. Después se estudia CMake. Después se crean proyectos propios.

La estructura de carpetas también forma parte de ese aprendizaje. Mantener GEN_FrameWork como carpeta del framework y los proyectos propios como carpetas hermanas dentro de Projects ayuda a conservar una separación sana entre base tecnológica y aplicaciones.

En WSL2, además, el entorno obliga a tomar decisiones prácticas sobre dónde se colocan las fuentes. La opción de máximo rendimiento Linux es usar /home/usuario, pero la opción más coherente para un desarrollo mixto Windows/Linux puede ser mantener un único árbol en una unidad Windows, preferiblemente Dev Drive, y compensar el coste con Compiler Cache. Esta decisión no es puramente técnica; también afecta a la organización, a la seguridad del trabajo diario y a la posibilidad de evitar divergencias entre copias.

33. Referencia rápida de compilación

Las siguientes tablas resumen los comandos más habituales para compilar el ejemplo inibase. El nombre del programa se escribe siempre en minúsculas y los modificadores siempre en mayúsculas. Estas referencias sirven como guía rápida para los primeros pasos; la ayuda real del script debe consultarse ejecutándolo sin parámetros.

Windows

Paso	Objetivo	Comando	Notas
1	Entrar en Common	cd /d E:\Projects\GEN_Framework\Common	Punto de entrada recomendado.
2	Mostrar opciones del script	compile.bat	Consulta la ayuda y valores disponibles.
3	Compilar inibase con valores por defecto	compile.bat inibase	Primera prueba recomendada.
4	Configurar con CMake	compile.bat inibase INTEL64 DEBUG CMAKE	Solo ejecuta la fase de configuración.
5	Compilar en Debug para Intel 64 bits	compile.bat inibase INTEL64 DEBUG COMPILE	Compilación habitual durante desarrollo.
6	Ejecutar test	compile.bat inibase INTEL64 DEBUG TEST	Valida el ejemplo si hay tests disponibles.
7	Compilar en Release	compile.bat inibase INTEL64 RELEASE COMPILE	Versión optimizada.
8	Usar Clang si está disponible	compile.bat inibase INTEL64 DEBUG CLANG COMPILE	Requiere entorno preparado para Clang.
9	Desarrollo completo	compile.bat inibase INTEL64 DEBUG CLANG MEMCTRL TRACE FEEDBACK COMPILE	Clang, memoria, trazas y feedback.
10	Distribución completa	compile.bat inibase INTEL64 RELEASE NOTCLANG NOTMEMCTRL NOTTRACE NOTFEEDBACK COMPILE	Salida limpia para distribución.

Linux

Paso	Objetivo	Comando	Notas
1	Entrar en Common	cd ~/Projects/GEN_Framework/Common	Punto de entrada recomendado.
2	Dar permisos de ejecución	chmod +x compile.bash	Solo si el script no tiene permisos.
3	Mostrar opciones del script	./compile.bash	Consulta la ayuda y valores disponibles.
4	Compilar inibase con valores por defecto	./compile.bash inibase	Primera prueba recomendada.
5	Configurar con CMake	./compile.bash inibase INTEL64 DEBUG CMAKE	Solo ejecuta la fase de configuración.
6	Compilar en Debug para Intel 64 bits	./compile.bash inibase INTEL64 DEBUG COMPILE	Uso habitual en PC Linux o WSL2.
7	Ejecutar test	./compile.bash inibase INTEL64 DEBUG TEST	Valida el ejemplo si hay tests disponibles.
8	Compilar en Release	./compile.bash inibase INTEL64 RELEASE COMPILE	Versión optimizada.
9	Compilar para ARM o Raspberry Pi	./compile.bash inibase ARM64 DEBUG COMPILE ./compile.bash inibase RPI64 DEBUG COMPILE	Ejemplos de plataformas Linux.
10	Compilar usando Docker	./compile.bash inibase DOCKERS	Entorno reproducible o compilación cruzada.
11	Desarrollo con Docker	./compile.bash inibase INTEL64 DEBUG CLANG MEMCTRL TRACE FEEDBACK COMPILE DOCKERS	Clang, diagnóstico y contenedor.
12	Distribución completa	./compile.bash inibase INTEL64 RELEASE NOTCLANG NOTMEMCTRL NOTTRACE NOTFEEDBACK COMPILE	Salida limpia para distribución.

WSL2 con unidad Windows compartida

Paso	Objetivo	Comando o recurso	Notas
1	Usar una única fuente de verdad	D:\Projects\GEN_Framework\nD:\Projects\MiProyecto	Evita duplicar fuentes entre Windows y Linux.
2	Entrar en Common desde WSL2	cd /mnt/d/Projects/GEN_Framework/Common	Equivalente en Linux de la unidad D: de Windows.
3	Dar permisos de ejecución	chmod +x compile.bash	Solo si el script no tiene permisos.
4	Compilar inibase por defecto	./compile.bash inibase	Valida el árbol compartido.
5	Compilar en Debug	./compile.bash inibase INTEL64 DEBUG COMPILE	Uso habitual desde WSL2.
6	Compilar usando Docker	./compile.bash inibase INTEL64 DEBUG COMPILE DOCKERS	Aísla el entorno de compilación.
7	Reducir el coste de /mnt/d	Compiler Cache Dev Drive	Mitiga la pérdida de rendimiento de Plan9/9P.

Como regla general, se debe empezar por la compilación más sencilla y añadir modificadores solo cuando sean necesarios. Primero se valida que inibase compila; después se prueban plataforma, modo, compilador, trazas, control de memoria y Docker.

Capítulo III: Visión general y arquitectura del framework

1. Una visión global antes de entrar en el detalle

Después de preparar el entorno de desarrollo y compilar un primer ejemplo, el siguiente paso natural es entender cómo está organizado GEN Framework. Antes de utilizar clases concretas o estudiar un módulo en profundidad, conviene disponer de una visión general de su arquitectura. Un framework grande no se comprende únicamente por la lista de ficheros que contiene, sino por la forma en la que agrupa responsabilidades, aísla dependencias y permite que una aplicación se mantenga portable entre plataformas.

GEN Framework nace con una idea central: desarrollar aplicaciones en C/C++ manteniendo el rendimiento del código nativo, pero evitando que la lógica de la aplicación quede atada a una plataforma concreta. Para conseguirlo no basta con crear unas pocas funciones auxiliares. Es necesario definir una arquitectura modular, un conjunto de factorías, capas de abstracción y convenciones que separen con claridad qué pertenece al núcleo común, qué pertenece a cada módulo funcional y qué pertenece a una plataforma determinada.

Este capítulo presenta esa visión general. No pretende ser una referencia exhaustiva de cada clase ni una descripción completa de todos los ficheros del framework. Su propósito es ofrecer al lector un mapa mental que facilita la comprensión de los capítulos posteriores, dedicados a tipos básicos, utilidades, entrada y salida de datos, gráficos, interfaces de usuario, comunicaciones, scripting, bases de datos y otros subsistemas.

La arquitectura de GEN Framework puede entenderse como una combinación de tres ideas: modularidad, factorías y aislamiento de plataforma. La modularidad permite dividir el framework en áreas funcionales. Las factorías permiten crear objetos adecuados sin que la aplicación tenga que conocer la implementación concreta. El aislamiento de plataforma garantiza que las diferencias entre Windows, Linux, Android, Raspberry Pi u otros destinos queden confinadas en zonas específicas del código.

2. La esencia modular de GEN Framework

La modularidad es una de las propiedades más importantes de GEN Framework. Un framework multiplataforma no puede ser una masa de código monolítica donde todo dependa de todo. Si cada módulo conoce detalles internos de todos los demás, cualquier cambio se vuelve peligroso. Si cada clase llama directamente a APIs nativas del sistema operativo, la portabilidad desaparece. Si cada aplicación tiene que decidir qué implementación usar para cada plataforma, el framework deja de cumplir su función.

Por eso GEN Framework se organiza en grupos funcionales. Cada grupo reúne clases que comparten una responsabilidad. Algunas áreas proporcionan utilidades básicas, otras gestionan entrada y salida de datos, otras trabajan con gráficos, otras con dispositivos de entrada, sonido, compresión, cifrado, bases de datos, scripting o flujo de aplicación. Esta separación permite razonar mejor sobre el sistema.

La modularidad también facilita el crecimiento. Un framework vivo necesita incorporar nuevas plataformas, nuevos drivers, nuevos formatos, nuevos protocolos o nuevos sistemas de renderizado sin romper lo que ya existe. Si la arquitectura está bien separada, un nuevo módulo puede añadirse en el lugar adecuado y exponer su funcionalidad mediante las interfaces existentes. La aplicación que utiliza el framework no tiene por qué absorber todo ese detalle.

Otro aspecto importante es que la modularidad no significa aislamiento absoluto. Los módulos colaboran entre sí. Un sistema gráfico puede necesitar leer recursos desde el sistema de ficheros. Una interfaz de usuario puede apoyarse en gráficos, entrada de teclado y ratón, temporizadores y trazas. Un módulo de comunicaciones puede usar buffers, cadenas, hilos y control de errores. Lo importante es que esas colaboraciones se produzcan a través de capas claras y no mediante dependencias caóticas.

En GEN Framework podemos hablar de módulos principales, representados por factorías de alto nivel, y módulos adicionales o transversales que proporcionan servicios más específicos. Las factorías principales son XFACTORY, DIOFACTORY, GRPFACTORY, INPFACTORY y SNDFACTORY. A su alrededor aparecen grupos como Compression, Cipher, AppFlow, Scripts, UserInterface o Databases, que amplían el alcance del framework y permiten construir aplicaciones más completas.

3. El papel de las factorías

Uno de los conceptos esenciales de GEN Framework es el uso de factorías. Una factoría es un componente encargado de crear objetos. Esta idea puede parecer sencilla, pero tiene consecuencias arquitectónicas muy importantes. En lugar de que la aplicación cree directamente una clase concreta, solicita a la factoría el objeto que necesita. La factoría decide qué implementación corresponde.

En un framework multiplataforma esto resulta fundamental. Una aplicación puede necesitar abrir un fichero, crear una pantalla, iniciar un dispositivo de entrada, reproducir sonido o acceder a un recurso del sistema. La operación conceptual puede ser la misma en todas las plataformas, pero la implementación interna no lo es. Windows tiene sus propias APIs. Linux tiene otras. Android tiene su propio modelo. Raspberry Pi puede añadir particularidades. La aplicación no debería tener que conocer todo eso.

Las factorías actúan como frontera entre la lógica portable y la implementación concreta. El código de la aplicación pide una abstracción. El framework proporciona una instancia adecuada. De esta forma se reduce la necesidad de usar condiciones de compilación en el código de aplicación y se centraliza la decisión de plataforma dentro del framework.

Este patrón también mejora el mantenimiento. La incorporación de una nueva implementación para una plataforma concreta no obliga a cambiar la aplicación: la factoría crea el objeto adecuado cuando corresponde. Si se sustituye un backend gráfico, se modifica la parte interna de la factoría o sus clases asociadas, pero la interfaz pública puede mantenerse. El soporte para una nueva arquitectura se incorpora en la capa de plataforma sin contaminar el resto del código.

Por tanto, una factoría en GEN Framework no es solo una comodidad para crear objetos. Es una pieza clave del aislamiento de plataforma y de la coherencia del diseño. Cada una de las factorías principales se asocia a un dominio funcional: utilidades generales, entrada y salida de datos, gráficos, entrada de usuario y sonido.

Factoría	Ámbito principal	Responsabilidad general
XFACTORY	eXtender Utils	Utilidades base, tipos auxiliares, sistema, cadenas, buffers, tiempo, hilos, trazas y servicios comunes.
DIOFACTORY	Data Input/Output	Flujos de datos, comunicaciones, dispositivos, protocolos, ficheros, URLs, red y acceso a entradas/salidas externas.
GRPFACTORY	Graphic Factory	Pantallas, canvas, bitmaps, fuentes, renderizado 2D, backends gráficos y recursos visuales.
INPFACTORY	Input Factory	Teclado, ratón, táctil, eventos de entrada y abstracción de dispositivos de interacción.
SNDFACTORY	Sound Factory	Dispositivos de sonido, reproducción, buffers, streams de audio y abstracción de salida sonora.

4. Factorías y aislamiento de plataforma

El aislamiento de plataforma es uno de los objetivos más importantes del framework. La aplicación debe poder expresar lo que quiere hacer sin depender directamente de la API nativa del sistema operativo. Esto no significa que la plataforma desaparezca, sino que sus diferencias quedan contenidas en lugares concretos.

En GEN Framework, las factorías participan directamente en ese aislamiento. Cuando un módulo necesita una implementación que depende del sistema, no debería construirla directamente desde el código de aplicación. La creación se canaliza a través de una factoría. De ese modo, la selección de la clase concreta puede depender de la plataforma de compilación, de las opciones activadas, del backend disponible o de la configuración del framework.

Por ejemplo, una aplicación no debería tener que preguntarse si para crear una pantalla debe usar una clase específica de Windows, otra de Linux o una variante para Android. La aplicación debería trabajar con una abstracción gráfica común. Internamente, GRPFACTORY y las clases asociadas resuelven qué implementación corresponde. Lo mismo ocurre con entrada, sonido, comunicaciones o determinadas utilidades del sistema.

Esta separación aporta varias ventajas. La primera es claridad. El código de aplicación deja de estar lleno de condiciones de compilación. La segunda es mantenibilidad. Las diferencias se concentran en el framework, no en cada programa que lo utiliza. La tercera es portabilidad. Una aplicación que respeta las abstracciones del framework puede compilarse en varias plataformas con menos cambios. La cuarta es capacidad de evolución. El framework puede añadir plataformas o backends sin obligar a reescribir la aplicación.

La idea general puede resumirse así: la aplicación trabaja con interfaces y servicios comunes; las factorías crean objetos concretos; el código dependiente de plataforma queda confinado dentro del framework, especialmente en las carpetas y módulos de plataforma.

5. El confinamiento del código de plataforma

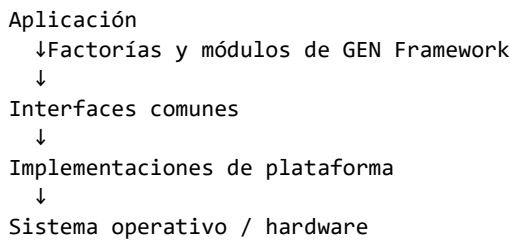
Un principio esencial de GEN Framework es que el código específico de plataforma debe estar confinado. Esto significa que las llamadas nativas a Windows, Linux, Android u otros sistemas no deben estar dispersas por todo el framework ni mucho menos por las aplicaciones que lo utilizan. Deben concentrarse en módulos claramente identificables, normalmente dentro de la zona de plataforma.

La carpeta o grupo Platform contiene las implementaciones que dependen de un sistema concreto. Allí es donde tienen sentido las clases que conocen detalles de Windows, Linux, Android, Raspberry Pi u otros destinos. Es también donde aparecen las diferencias inevitables: tipos nativos, llamadas del sistema, gestores de ventana, APIs gráficas, acceso a dispositivos, permisos, rutas, temporizadores o mecanismos de sincronización específicos.

Esta organización permite que el resto del framework permanezca lo más portable posible. Las capas superiores definen conceptos comunes. La parte de plataforma implementa esos conceptos usando las herramientas de cada sistema. Si el código específico estuviera mezclado con el resto, cada módulo tendría que conocer todos los sistemas y la arquitectura se volvería frágil.

Confinar la plataforma no significa ignorarla. Al contrario: significa reconocer que existe y darle un lugar adecuado. Un framework multiplataforma serio no puede basarse en la ilusión de que todos los sistemas son iguales. Windows, Linux y Android no son iguales. Lo que sí puede hacer el framework es ofrecer una interfaz coherente por encima de esas diferencias y limitar el impacto de cada sistema en la aplicación final.

El programador que usa GEN Framework debe aprender a respetar esa frontera. Si una aplicación introduce llamadas directas a APIs nativas sin necesidad, rompe parte del valor del framework. Puede haber casos donde sea inevitable acceder a una funcionalidad muy específica de una plataforma, pero la regla general debe ser clara: primero se busca la abstracción del framework; solo si no existe, se evalúa una extensión controlada.



6. XFACTORY: eXtender Utils

XFACTORY representa el área de utilidades base del framework, a menudo identificada con eXtender Utils. Es una de las partes más fundamentales porque proporciona servicios que acaban siendo utilizados por muchos otros módulos. En cualquier arquitectura C/C++ grande existe una capa de utilidades comunes: cadenas, buffers, fechas, tiempos, rutas, hilos, sincronización, trazas, configuración, tipos auxiliares y estructuras de soporte.

El papel de XFACTORY es actuar como punto de creación y acceso a objetos de esa capa común. Aunque el usuario no siempre sea consciente de ello, muchas funcionalidades de alto nivel dependen de estas utilidades. Un sistema de ficheros necesita rutas. Una comunicación necesita buffers. Un sistema gráfico necesita cargar recursos. Un módulo de trazas necesita cadenas, tiempo y salida de datos. Un sistema de configuración necesita serialización, ficheros y tipos básicos.

Dentro del ámbito de XFACTORY podemos situar funcionalidades como gestión de cadenas, buffers dinámicos, rutas, fechas y horas, temporizadores, hilos, mutexes, eventos, trazas, variantes de datos, listas, mapas o estructuras auxiliares. Algunas de estas clases pueden ser completamente portables. Otras pueden apoyarse en implementaciones de plataforma cuando necesitan acceder a servicios del sistema operativo.

Las utilidades de cadenas son especialmente importantes en un framework multiplataforma. Windows, Linux y Android pueden tener diferencias en codificación, rutas, separadores, Unicode y representación interna. Un framework que quiera evitar errores debe proporcionar clases y convenciones propias para tratar texto de forma coherente.

Los buffers también son una base común. Muchas áreas del framework trabajan con bloques de memoria: comunicaciones, compresión, cifrado, lectura de ficheros, imágenes, audio o serialización. Disponer de una abstracción uniforme para buffers reduce errores y facilita reutilizar código.

Las trazas son otro componente esencial. En aplicaciones multiplataforma, depurar únicamente con el IDE no siempre es suficiente. Hay escenarios donde la aplicación corre en un dispositivo remoto, en Android, en una Raspberry Pi, en un servicio o en un sistema sin interfaz gráfica. Un sistema de trazas integrado permite observar el comportamiento de la aplicación de forma uniforme.

En resumen, XFACTORY proporciona el suelo sobre el que se apoyan muchas otras partes del framework. No es necesariamente la parte más visible para el usuario final, pero sí una de las más importantes para la estabilidad, coherencia y portabilidad del conjunto.

7. DIOFACTORY: Data Input/Output

DIOFACTORY agrupa el dominio de entrada y salida de datos. En cualquier aplicación real, los datos entran y salen constantemente: se leen ficheros, se escriben logs, se abren comunicaciones TCP o UDP, se accede a puertos serie, se procesan URLs, se trabaja con protocolos, se reciben datos de dispositivos o se interactúa con servicios externos.

El nombre DIO puede entenderse como Data Input/Output. Su responsabilidad principal es proporcionar una forma coherente de trabajar con flujos de datos, comunicaciones y dispositivos. Esto incluye tanto operaciones

aparentemente sencillas, como abrir un fichero, como operaciones más complejas, como gestionar un stream de red, un protocolo industrial o una comunicación con un sensor.

Dentro de este grupo podemos encontrar abstracciones para streams, ficheros, sockets, comunicaciones TCP/UDP, puertos serie, protocolos, clientes o servidores, URLs, DNS, TLS, dispositivos de entrada/salida y drivers. La presencia de una factoría permite que la aplicación solicite una comunicación o un dispositivo sin depender directamente de la implementación de una plataforma concreta.

El área DIO es especialmente sensible al aislamiento de plataforma. Las APIs de red, los puertos serie, los permisos, las rutas de dispositivos y las llamadas del sistema cambian entre Windows, Linux y Android. Si una aplicación implementara directamente todas esas diferencias, el código se volvería muy difícil de mantener. DIOFACTORY concentra esa complejidad y ofrece una capa más uniforme.

También es importante entender que DIO no se limita a red. Entrada y salida incluye cualquier forma de comunicación con el exterior. Puede tratarse de un fichero local, de una conexión remota, de un dispositivo conectado por puerto serie, de una interfaz SPI o I2C en sistemas embebidos, de un protocolo específico o de un stream cifrado.

En aplicaciones industriales o embebidas, esta capa puede ser determinante. Muchos proyectos no son simplemente una interfaz gráfica bonita: necesitan hablar con hardware, sensores, dispositivos externos, buses de comunicación, servicios de red o protocolos propietarios. GEN Framework proporciona una base para que esas comunicaciones no contaminen la lógica principal de la aplicación.

8. GRPFACTORY: Graphic Factory

GRPFACTORY agrupa la parte gráfica del framework. El desarrollo gráfico multiplataforma es una de las áreas donde más claramente se ve la necesidad de abstracción. Crear una ventana, obtener un contexto gráfico, dibujar en pantalla, cargar una imagen, representar texto o gestionar un canvas no se hace de la misma forma en todas las plataformas.

El objetivo de GRPFACTORY es proporcionar un punto de creación para objetos gráficos y backends de renderizado. Dentro de este ámbito podemos situar pantallas, canvas, bitmaps, fuentes, colores, primitivas de dibujo, imágenes, renderizado 2D, soporte vectorial, integración con motores como AGG, OpenGL u OpenGL ES, y sistemas capaces de representar formatos gráficos como SVG o DXF cuando el framework los incorpora.

La parte gráfica tiene varias capas. Una capa básica puede representar colores, puntos, rectángulos, regiones, transformaciones o bitmaps. Una capa superior puede representar un canvas donde se dibujan líneas, textos, polígonos o imágenes. Otra capa puede conectar ese canvas con un backend concreto. Finalmente, una capa de plataforma se encarga de que todo eso aparezca en una pantalla real o en un buffer válido para el sistema.

Esta separación permite que una aplicación dibuje usando conceptos comunes, aunque internamente el backend sea diferente. En Windows puede existir una implementación concreta. En Linux puede haber otra. En Android puede utilizarse otro camino. Si el framework expone una API coherente, la aplicación no necesita conocer todos esos detalles.

Además, el área gráfica no solo sirve para interfaces de usuario. También puede ser útil para renderizar informes, generar imágenes, visualizar datos, representar gráficos vectoriales, cargar recursos, dibujar sobre memoria o preparar salidas para diferentes dispositivos.

GRPFACTORY es, por tanto, una de las factorías más visibles para el usuario, porque muchas aplicaciones muestran algo en pantalla. Pero su valor no está únicamente en dibujar. Su valor está en permitir que la lógica visual se mantenga portable y que el backend concreto quede encapsulado.

9. INPFACTORY: Input Factory

INPFACTORY agrupa la entrada de usuario y de dispositivos de interacción. En una aplicación gráfica o interactiva, no basta con dibujar. También hay que recibir eventos: teclado, ratón, pantalla táctil, punteros, botones, gestos o dispositivos específicos. Cada plataforma tiene su propio sistema de eventos y su forma de representar esa entrada.

El objetivo de INPFACTORY es abstraer esa diversidad. La aplicación no debería depender directamente de mensajes de ventana de Windows, eventos X11, eventos Android o APIs concretas de una plataforma. Debería trabajar con eventos comunes definidos por el framework.

Dentro de este grupo podemos situar teclado, ratón, entrada táctil, estados de botones, coordenadas de puntero, eventos de movimiento, pulsaciones, liberaciones, rueda, gestos básicos o adaptadores de entrada. En algunos entornos también puede extenderse hacia dispositivos físicos específicos, mandos, sensores o entradas propias de una plataforma.

La entrada es especialmente importante porque conecta al usuario con la aplicación. Si la abstracción es pobre, cada plataforma obliga a escribir código diferente. Si la abstracción es adecuada, la aplicación puede reaccionar ante acciones del usuario de forma uniforme.

INPFACTORY suele colaborar con la parte gráfica y con la interfaz de usuario. Un botón de una interfaz necesita saber cuándo se pulsa el ratón o cuándo se toca la pantalla. Un control de texto necesita recibir teclado. Una aplicación de visualización necesita interpretar movimientos. Esa colaboración debe hacerse sin romper la separación de responsabilidades: la entrada captura eventos, la interfaz los interpreta y el gráfico los representa.

10. SNDFACTORY: Sound Factory

SNDFACTORY agrupa el área de sonido. Aunque no todas las aplicaciones necesitan audio, disponer de una abstracción sonora puede ser importante en aplicaciones multimedia, interfaces con avisos, sistemas industriales con alarmas, juegos, herramientas interactivas o entornos donde el sonido forma parte de la experiencia de usuario.

Trabajar con sonido de forma multiplataforma puede ser complejo. Cada sistema tiene sus propias APIs, formatos preferidos, dispositivos de salida, buffers, latencias y modelos de reproducción. El objetivo de SNDFACTORY es proporcionar una capa común para crear objetos relacionados con sonido sin que la aplicación dependa de una API concreta.

Dentro de este grupo podemos encontrar dispositivos de sonido, streams de audio, buffers, reproducción, control de volumen, carga de formatos, salida sonora o integración con backends específicos. Como ocurre con el resto de factorías, la implementación concreta puede variar por plataforma.

El sonido también puede relacionarse con otros módulos. Puede cargar datos desde ficheros gestionados por DIO, utilizar buffers de XUtils, aplicar decodificación o compresión, generar eventos para AppFlow o integrarse con una interfaz de usuario. La modularidad permite que estas colaboraciones no se conviertan en dependencias desordenadas.

SNDFACTORY completa así el conjunto de factorías principales, cubriendo un área que no siempre aparece en los ejemplos iniciales, pero que forma parte de una arquitectura de aplicación completa.

11. Grupos adicionales y servicios transversales

Además de las factorías principales, GEN Framework incluye o puede integrar grupos funcionales adicionales que amplían su alcance. Estos grupos no siempre se presentan como factorías independientes de primer nivel, pero

proporcionan servicios importantes para construir aplicaciones reales. Entre ellos podemos destacar Compression, Cipher, AppFlow, Scripts, UserInterface y Databases.

Estos grupos deben entenderse como capas transversales. Algunos se apoyan en XUtils para cadenas, buffers o trazas. Otros se apoyan en DIO para leer o escribir datos. Otros interactúan con gráficos, entrada o sonido. Su objetivo es resolver necesidades frecuentes que aparecen cuando una aplicación deja de ser un ejemplo simple y se convierte en un producto completo.

Grupo	Función principal	Relación con la arquitectura
Compression	Compresión y descompresión de datos, ficheros o buffers.	Se apoya en buffers, streams y ficheros para reducir tamaño o empaquetar información.
Cipher	Cifrado, hashes, firmas o mecanismos criptográficos.	Protege datos y comunicaciones, normalmente junto con DIO y buffers.
AppFlow	Organización del flujo de aplicación, estados, procesos y control de ejecución.	Ayuda a estructurar aplicaciones más complejas que un main lineal.
Scripts	Integración de lenguajes embebidos o motores de script.	Permite ampliar comportamiento sin recompilar partes de la aplicación.
UserInterface	Controles, ventanas, diálogos y elementos de interfaz.	Se apoya en gráficos, entrada y recursos.
Databases	Acceso a bases de datos y persistencia estructurada.	Conecta la aplicación con almacenamiento y consultas.

12. Compression

El grupo de compresión proporciona funcionalidad para reducir el tamaño de datos, empaquetar información o trabajar con formatos comprimidos. En aplicaciones reales, la compresión aparece con frecuencia: recursos de aplicación, logs, ficheros de intercambio, comunicaciones, copias de seguridad o almacenamiento temporal.

La compresión suele trabajar con buffers y streams. Por eso se relaciona de forma natural con XUtils y DIO. Un fichero puede leerse mediante DIO, almacenarse en un buffer común, comprimirse y escribirse de nuevo. Una comunicación puede enviar datos comprimidos para reducir ancho de banda. Un paquete de recursos puede descomprimirse al iniciar la aplicación.

Desde el punto de vista de la arquitectura, Compression no debería obligar a la aplicación a conocer los detalles de una librería externa concreta. El framework puede encapsular la implementación y exponer operaciones coherentes. Así es posible sustituir o ampliar algoritmos sin cambiar la lógica de aplicación.

13. Cipher

El grupo Cipher agrupa funcionalidad relacionada con criptografía. En una aplicación moderna, la protección de datos puede ser esencial. Se pueden necesitar hashes para verificar integridad, cifrado simétrico para proteger información local, mecanismos de firma, generación de claves, codificaciones o apoyo a comunicaciones seguras.

La criptografía debe utilizarse con cuidado. No se trata de inventar algoritmos propios, sino de encapsular correctamente operaciones fiables y ofrecerlas de forma coherente al resto del framework. Cipher puede colaborar con DIO cuando se transmiten datos, con Compression cuando se empaquetan recursos, con Databases cuando se almacenan datos sensibles o con XUtils cuando se manipulan buffers y cadenas.

Desde el punto de vista del diseño, Cipher es un buen ejemplo de grupo transversal: no pertenece únicamente a red, ni a ficheros, ni a bases de datos. Puede ser utilizado por todos esos módulos. Por eso debe estar bien separado y exponer una interfaz clara.

14. AppFlow

AppFlow representa una idea importante: una aplicación no es solo una colección de funciones. Tiene un flujo. Inicializa recursos, configura subsistemas, cambia de estado, procesa eventos, ejecuta tareas, responde a errores y finaliza de forma controlada. En programas pequeños, todo esto puede estar dentro de main. En aplicaciones reales, esa organización se vuelve insuficiente.

El grupo AppFlow ayuda a estructurar el comportamiento de la aplicación. Puede incluir conceptos como estados, transiciones, procesos, tareas, máquinas de estado, control de ejecución o coordinación entre módulos. Su objetivo es que la aplicación tenga una arquitectura interna clara, no una secuencia improvisada de llamadas.

AppFlow se relaciona con muchas partes del framework. Puede iniciar gráficos, abrir comunicaciones, cargar configuración, activar scripts, responder a eventos de entrada o gestionar ciclos de vida. Su utilidad aumenta a medida que la aplicación crece.

En el contexto de GEN Framework, AppFlow refuerza la idea de arquitectura propia. No se trata únicamente de tener clases para abrir ficheros o dibujar. Se trata de disponer de una forma ordenada de construir aplicaciones completas.

15. Scripts

El grupo Scripts permite integrar lenguajes de scripting o motores embebidos dentro de una aplicación C/C++. Esta posibilidad es muy útil cuando queremos que parte del comportamiento sea configurable o extensible sin recompilar todo el programa.

Un motor de scripts puede servir para automatización, reglas de negocio, prototipado, configuración avanzada, pruebas, generación de contenidos o personalización por parte del usuario. En lugar de codificar todo en C++, algunas partes pueden delegarse en scripts que la aplicación carga y ejecuta en tiempo de ejecución.

El scripting también ayuda a separar niveles de responsabilidad. El núcleo de alto rendimiento puede permanecer en C/C++, mientras que ciertas reglas cambiantes pueden expresarse en un lenguaje más dinámico. Esta combinación puede ser muy potente si se diseña con cuidado.

Desde el punto de vista del framework, Scripts necesita comunicarse con otros módulos. Debe poder leer ficheros, acceder a datos, producir trazas, llamar a funciones del framework o recibir eventos. Por eso debe integrarse sin romper la arquitectura general.

16. UserInterface

UserInterface agrupa los elementos de interfaz de usuario. Si GRPFACTORY proporciona la base gráfica e INPFACTORY captura eventos de entrada, UserInterface construye sobre esas capas conceptos más cercanos al usuario: ventanas, controles, botones, listas, cuadros de texto, diálogos, menús, paneles, temas o estructuras de pantalla.

Una interfaz de usuario multiplataforma es uno de los retos más complejos de un framework. No basta con dibujar formas. Hay que gestionar eventos, estados visuales, foco, navegación, interacción, escalado, fuentes, recursos, temas y comportamiento. Además, cada plataforma tiene sus propias convenciones.

GEN Framework puede abordar esta complejidad creando una capa de interfaz propia apoyada en gráficos e input. De esta forma, una aplicación puede construir pantallas sin depender directamente de controles nativos de cada sistema. Esto no impide que existan integraciones específicas cuando sea necesario, pero permite que la lógica de interfaz tenga una base común.

UserInterface es un ejemplo claro de colaboración modular. Utiliza gráficos para representar, entrada para interactuar, recursos para cargar imágenes o estilos, trazas para diagnosticar, AppFlow para coordinar estados y posiblemente Scripts para personalización o comportamiento dinámico.

17. Databases

El grupo Databases proporciona funcionalidad relacionada con bases de datos y persistencia estructurada. Muchas aplicaciones necesitan almacenar información, consultar datos, mantener configuraciones complejas, registrar eventos o trabajar con modelos persistentes.

Una capa de bases de datos puede incluir conexiones, consultas, resultados, transacciones, adaptadores, motores concretos o abstracciones sobre SQL. Como siempre, la clave está en evitar que la aplicación dependa innecesariamente de detalles internos de una librería o motor específico cuando el framework puede ofrecer una interfaz común.

Databases se relaciona con DIO, porque una base de datos es una forma de entrada y salida de datos. También se relaciona con Cipher si se necesita proteger información, con Compression si se almacenan paquetes, con Scripts si se ejecutan reglas dinámicas y con AppFlow si la aplicación tiene procesos que dependen de datos persistentes.

No todas las aplicaciones necesitan bases de datos, pero cuando las necesitan, disponer de una capa integrada en el framework ayuda a mantener coherencia con el resto del sistema.

18. Relación entre módulos

Una forma útil de entender GEN Framework es imaginarlo como una serie de capas. En la base se encuentran utilidades comunes y abstracciones de plataforma. Sobre ellas aparecen los módulos de entrada/salida, gráficos, entrada, sonido y servicios transversales. Encima de todo se sitúa la aplicación.

La aplicación debe intentar depender de las capas superiores del framework, no directamente de la plataforma. Cuando necesita dibujar, usa la capa gráfica. Cuando necesita leer datos, usa DIO. Cuando necesita eventos de usuario, usa Input. Cuando necesita utilidades comunes, usa XUtils. Cuando necesita compresión, cifrado, scripts o bases de datos, acude al grupo correspondiente.

Esta organización no solo mejora la portabilidad. También mejora el diseño del propio programa. El desarrollador aprende a separar responsabilidades. Un módulo de negocio no debería abrir directamente una ventana. Una clase de interfaz no debería implementar un protocolo de comunicación. Un driver no debería conocer detalles de una pantalla. Cada parte debe estar en su lugar.

El framework no impide que el programador escriba mal código, pero proporciona una arquitectura que invita a organizarlo mejor. Esa es una de las razones por las que GEN Framework debe estudiarse como un sistema completo y no como una colección de funciones sueltas.

19. Cómo debe aproximarse el programador a esta arquitectura

Para un nuevo usuario, la cantidad de módulos puede parecer grande. La mejor forma de aprender GEN Framework no es intentar memorizar toda la arquitectura desde el primer día. Lo adecuado es construir una visión progresiva.

Primero conviene entender las utilidades base y la forma en que el framework representa cadenas, buffers, rutas, fechas, trazas y tipos comunes. Después se puede estudiar DIO, porque casi cualquier aplicación necesita leer o escribir datos. A continuación tiene sentido entrar en gráficos, entrada e interfaz de usuario si se van a crear aplicaciones visuales. Más tarde se pueden incorporar sonido, compresión, cifrado, scripts o bases de datos según las necesidades del proyecto.

También es importante aprender a reconocer cuándo una clase pertenece al mundo común y cuándo pertenece al mundo de plataforma. Si una clase está en Platform o depende de un sistema concreto, normalmente no debería ser usada directamente desde una aplicación salvo que exista una razón clara. La aplicación debería apoyarse en la abstracción superior.

Este criterio ayuda a mantener el objetivo principal: escribir una aplicación una vez y compilarla en varias plataformas sin tocar la lógica principal. Cada vez que una aplicación introduce una llamada directa a una API nativa, se reduce esa portabilidad. Cada vez que usa una factoría o una abstracción común, se refuerza.

20. Resumen de arquitectura

GEN Framework se organiza alrededor de una arquitectura modular diseñada para desarrollar aplicaciones C/C++ multiplataforma. Su núcleo no está en una única clase ni en una única librería, sino en la combinación de módulos funcionales, factorías y aislamiento de plataforma.

XFACTORY proporciona utilidades base. DIOFACTORY gestiona entrada y salida de datos. GRPFACTORY organiza la parte gráfica. INPFACTORY abstrae la entrada de usuario. SNDFACTORY cubre sonido. A su alrededor, grupos como Compression, Cipher, AppFlow, Scripts, UserInterface y Databases amplían las capacidades del framework para construir aplicaciones reales.

Las factorías no solo crean objetos. También permiten aislar la aplicación de las implementaciones concretas de cada plataforma. El código dependiente de sistema debe permanecer confinado en Platform, mientras que la aplicación trabaja con abstracciones comunes.

Esta arquitectura es la base del resto del libro. Cada área se estudia con más detalle, empezando por las utilidades fundamentales y avanzando hacia aplicaciones completas.

Capítulo IV: Primera aplicación con GEN Framework

1. El objetivo del capítulo

Después de preparar el entorno de desarrollo y conocer la arquitectura general del framework, el siguiente paso natural es construir una primera aplicación. En GEN Framework no se empieza escribiendo una función main de C++ ni invocando manualmente un bucle principal. Esa parte está embebida dentro del propio framework y forma parte de su infraestructura de arranque, ejecución y cierre.

Esta decisión es importante. Si el programador decide escribir su propia main y llamar desde ella a clases sueltas de GEN Framework, técnicamente puede hacerlo, pero en ese caso estaría utilizando GEN Framework como un conjunto de librerías, no como un framework. La diferencia no es solo terminológica. Cuando se usa GEN Framework como framework, la aplicación se integra en su ciclo de vida, en su sistema de rutas, en su configuración, en sus factorías, en su control de trazas, en su modelo de plataforma y en su manera uniforme de organizar recursos.

Por eso, este capítulo no parte de una main escrita por el usuario. Describe el esqueleto real que siguen los ejemplos del repositorio y la forma recomendada de crear una aplicación nueva. La referencia práctica principal es el ejemplo inibase, situado dentro de los ejemplos de consola, porque contiene una estructura suficientemente completa para entender inicialización, actualización, finalización, configuración y máquina de estados.

2. Las tres modalidades principales de aplicación

GEN Framework permite trabajar con distintos niveles de integración. Esta flexibilidad es importante porque no todas las aplicaciones necesitan la misma infraestructura. Una utilidad mínima, un firmware para un microcontrolador, una herramienta de consola, una aplicación gráfica de escritorio o una interfaz embebida sobre un pequeño display no tienen las mismas necesidades.

De forma práctica, podemos distinguir tres modalidades principales de desarrollo dentro de GEN Framework: uso base sin la modalidad completa de aplicación AppFlow, aplicación de consola con AppFlow y aplicación gráfica con AppFlow. Esta clasificación coincide con la organización de los ejemplos: el directorio Base agrupa ejemplos de uso básico del framework, Console agrupa aplicaciones de consola basadas en AppFlow, y Graphics agrupa aplicaciones gráficas basadas en AppFlowGraphics.

Modalidad	Directorio de ejemplos	Clase / enfoque habitual	Uso típico
Uso base del framework	Examples/Base	Sin apoyarse en la modalidad AppFlow de consola o gráfica; uso más directo de servicios de GEN Framework.	Ejemplos mínimos, pruebas de módulos, integración en sistemas pequeños o entornos donde se desea controlar más el flujo.
Aplicación de consola	Examples/Console	APPFLOWCONSOLE y módulos AppFlow asociados.	Herramientas de consola, servicios, procesos en segundo plano, utilidades de red o aplicaciones sin ventana gráfica.
Aplicación gráfica	Examples/Graphics	APPFLOWGRAPHICS, entrada, pantalla, canvas, viewports y opcionalmente UserInterface.	Aplicaciones visuales, interfaces de usuario, ejemplos gráficos, pantallas embebidas y aplicaciones multiplataforma con salida gráfica.

Esta separación no impide combinaciones más específicas. Una aplicación sin AppFlow puede seguir siendo de consola o gráfica si el entorno lo requiere. Por ejemplo, en microprocesadores o sistemas embebidos puede interesar no usar la infraestructura completa de aplicación y trabajar directamente con módulos de bajo nivel, incluso si existe una salida gráfica hacia un pequeño display SPI o I2C. En ese caso se intenta reducir dependencias, memoria y complejidad.

La idea clave es que la modalidad elegida determina cuánto del ciclo de vida asume GEN Framework por nosotros. Cuanto más usamos AppFlow, más integrados estamos en el framework: configuración, trazas, rutas, recursos, consola o gráficos se organizan de forma homogénea. Cuanto menos usamos AppFlow, más responsabilidad queda en el código de la aplicación.

3. Por qué no escribimos la función main

En muchos libros de C++ la primera aplicación empieza con `int main()`. En GEN Framework no es así. La main real está encapsulada dentro de la infraestructura del framework. Esto permite que el arranque sea homogéneo en todas las plataformas soportadas, que las diferencias entre Windows, Linux, Android o sistemas embebidos se resuelvan internamente y que la aplicación solo defina la clase que representa su comportamiento.

El mecanismo habitual en las aplicaciones integradas en el framework es declarar una instancia de aplicación mediante una macro de creación. En inibase aparece el patrón:

```
APPLICATIONCREATEINSTANCE(INIBASE, inibase)
```

El usuario no implementa main, sino una clase de aplicación. El framework crea la instancia, llama a sus métodos de ciclo de vida y se encarga de coordinar el proceso principal. Esta forma de trabajo es una de las diferencias entre usar GEN Framework como framework y usarlo simplemente como conjunto de librerías.

En los ejemplos de uso base también se observa esta filosofía. Incluso cuando no se usa la modalidad completa AppFlow de consola o gráfica, el código se estructura alrededor de funciones de inicialización, actualización y cierre gestionadas por la infraestructura de GEN Framework. La aplicación define qué debe ocurrir, pero no necesita reescribir la entrada nativa de cada plataforma.

4. El ciclo de vida de una aplicación con GEN Framework

Una aplicación con GEN Framework se organiza alrededor de un ciclo de vida claro. Este ciclo aparece en los ejemplos y se convierte en una disciplina de diseño: inicializar recursos, ejecutar actualizaciones periódicas y liberar correctamente todo lo que se ha creado.

Fase	Método habitual	Responsabilidad
Inicialización	AppProc_Ini()	Crear rutas, preparar configuración, inicializar trazas, registrar estados, crear temporizadores, preparar servicios y dejar la aplicación lista para comenzar.
Primera actualización	AppProc_FirstUpdate()	Ejecutar tareas que deben hacerse una vez cuando el entorno ya está iniciado: obtener dispositivos, preparar entrada, mostrar cabeceras, cargar elementos dependientes del sistema.
Actualización	AppProc_Update()	Ejecutar el trabajo periódico de la aplicación: lógica principal, estado actual, entrada, render, comunicaciones,

		temporizadores y máquina de estados.
Última actualización	AppProc_LastUpdate()	Disponible en algunas modalidades para tareas inmediatamente anteriores al cierre o transición final.
Terminación	AppProc_End()	Cerrar servicios, liberar objetos creados con factorías, guardar estado, detener módulos extendidos y destruir singletons de configuración.

No todas las aplicaciones tienen que usar todas las fases con la misma intensidad. Una aplicación de consola puede concentrar casi toda su lógica en AppProc_Update. Una aplicación gráfica puede repartir el trabajo entre inicialización de pantalla, entrada, canvas, interfaz de usuario y dibujo. Una aplicación mínima puede usar un patrón más simple. Lo importante es mantener el criterio: todo lo que se crea debe tener un lugar claro donde inicializarse y un lugar claro donde terminarse.

5. Uso base sin la modalidad AppFlow de consola o gráfica

Los ejemplos del directorio Base sirven para entender el uso más directo de GEN Framework. En esta modalidad no estamos construyendo una aplicación de consola con APPFLOWCONSOLE ni una aplicación gráfica con APPFLOWGRAPHICS. Se utiliza la infraestructura base del framework y los módulos necesarios, pero el ciclo de aplicación es más ligero.

Esta modalidad es útil cuando se quiere probar una clase, validar una funcionalidad, crear una utilidad muy pequeña o integrar GEN Framework en un entorno donde no conviene cargar toda la infraestructura de AppFlow. También es habitual en microprocesadores o sistemas embebidos, donde el coste de memoria, el número de dependencias y el control del bucle principal pueden ser más importantes que la comodidad de una aplicación completa.

En estos escenarios puede existir salida por consola, o incluso salida gráfica hacia un display pequeño conectado por SPI o I2C, pero eso no implica necesariamente usar AppFlowGraphics. Se puede trabajar con módulos concretos del framework y con el mínimo necesario para la plataforma.

Esta modalidad debe utilizarse con criterio. Es más flexible, pero también deja más decisiones al programador. Cuando se desea una aplicación completa, con configuración principal, logs, trazas, rutas de recursos, eventos y ciclo homogéneo, lo normal es usar AppFlow.

6. Aplicación de consola con AppFlow

La modalidad de consola se apoya en APPFLOWCONSOLE. Es la opción adecuada para herramientas sin interfaz gráfica, servicios, procesos en segundo plano, pruebas de comunicación, utilidades de red, ejemplos de bases de datos o aplicaciones que interactúan mediante teclado y salida textual.

El ejemplo inibase pertenece a esta familia. Su clase principal hereda de APPFLOWCONSOLE y de XFSMACHINE. Esto indica dos cosas: por un lado, se integra en el ciclo de aplicación de consola de GEN Framework; por otro, organiza su lógica interna con una máquina de estados finitos.

```
class INIBASE : public APPFLOWCONSOLE, public XOBSERVER, public XFSMACHINE
{
public:
    bool AppProc_Ini();
    bool AppProc_FirstUpdate();
    bool AppProc_Update();
```

```

    bool AppProc_End();

    bool InitFSMACHINE();
};

```

La aplicación no define main. Define una clase, una instancia de aplicación y los métodos que GEN Framework llama en el momento adecuado. Esto permite que el flujo principal sea homogéneo y que las diferencias de plataforma queden dentro del framework.

7. Máquina de estados finitos con XFSMACHINE

Uno de los patrones más importantes del ejemplo inibase es el uso de XFSMACHINE. La máquina de estados finitos permite separar de forma clara en qué estado está la aplicación, qué eventos pueden provocar cambios de estado y qué lógica debe ejecutarse continuamente mientras se permanece en un estado concreto.

Para usar XFSMACHINE no basta con crear la clase. Primero hay que definir dos enumeraciones: una para eventos y otra para estados. Los eventos representan lo que puede ocurrir. Los estados representan las situaciones estables en las que puede encontrarse la aplicación.

```

enum INIBASEXFSMEVENTS
{
    INIBASE_XFSMEVENT_NONE = 0,
    INIBASE_XFSMEVENT_INI,
    INIBASE_XFSMEVENT_UPDATE,
    INIBASE_XFSMEVENT_END,

    INIBASE_LASTEVENT
};

enum INIBASEXFSMSTATES
{
    INIBASE_XFSMSTATE_NONE = 0,
    INIBASE_XFSMSTATE_INI,
    INIBASE_XFSMSTATE_UPDATE,
    INIBASE_XFSMSTATE_END,

    INIBASE_LASTSTATE
};

```

Después, al principio de la aplicación, normalmente durante AppProc_Ini, se relacionan estados y eventos mediante InitFSMACHINE. Cada llamada a AddState declara qué eventos acepta un estado y a qué nuevo estado conduce cada evento.

```

bool INIBASE::InitFSMACHINE()
{
    AddState(INIBASE_XFSMSTATE_NONE,
        INIBASE_XFSMEVENT_INI, INIBASE_XFSMSTATE_INI,
        INIBASE_XFSMEVENT_END, INIBASE_XFSMSTATE_END,
        XFSMACHINESTATE_EVENTDEFEND);

    AddState(INIBASE_XFSMSTATE_INI,
        INIBASE_XFSMEVENT_UPDATE, INIBASE_XFSMSTATE_UPDATE,
        INIBASE_XFSMEVENT_END, INIBASE_XFSMSTATE_END,
        XFSMACHINESTATE_EVENTDEFEND);

    AddState(INIBASE_XFSMSTATE_UPDATE,
        INIBASE_XFSMEVENT_END, INIBASE_XFSMSTATE_END,

```

```

        XFSMACHINESTATE_EVENTDEFEND);

    return true;
}

```

El punto más interesante aparece en `AppProc_Update`. El patrón usado por GEN Framework divide la lógica en dos partes. La primera parte se ejecuta cuando no hay un evento nuevo. En ese caso, la aplicación permanece en el estado actual y ejecuta la lógica continua correspondiente a ese estado. La segunda parte se ejecuta cuando existe un evento nuevo. En ese caso se comprueba la transición, se cambia de estado si procede y se ejecuta la lógica asociada al cambio de estado.

```

bool INIBASE::AppProc_Update()
{
    if(GetEvent() == INIBASE_XFSMEVENT_NONE)
    {
        switch(GetCurrentState())
        {
            case INIBASE_XFSMSTATE_UPDATE:
                // Lógica continua mientras la aplicación está en UPDATE.
                break;

            default:
                break;
        }
    }
    else
    {
        if(GetEvent() < INIBASE_LASTEVENT)
        {
            CheckTransition();

            switch(GetCurrentState())
            {
                case INIBASE_XFSMSTATE_INI:
                    SetEvent(INIBASE_XFSMEVENT_UPDATE);
                    break;

                default:
                    break;
            }
        }
    }

    return true;
}

```

Esta separación le da una gracia especial a la máquina de estados. No se mezcla el trabajo permanente de un estado con la lógica puntual que solo debe ejecutarse cuando se produce una transición. En un estado podemos leer teclado, actualizar comunicaciones, refrescar estado o comprobar temporizadores. En una transición podemos inicializar algo una sola vez, preparar el siguiente estado o decidir si el flujo debe terminar.

En `inibase`, por ejemplo, al iniciar se establece el evento `INI`. La máquina transita al estado de inicialización y después se lanza el evento `UPDATE`. A partir de ahí, el estado `UPDATE` contiene el trabajo periódico de la aplicación. Cuando el usuario solicita salir, se establece el tipo de salida y la aplicación termina de forma ordenada.

8. Estructura típica de una aplicación de consola

Una aplicación de consola basada en AppFlow suele organizarse con una estructura sencilla: una carpeta de fuentes, una carpeta de CMake y una carpeta assets para todos los recursos que no son el ejecutable.

```
MiAplicacion/
|
├── assets/
│   ├── miaplicacion.ini
│   ├── miaplicacion.log
│   └── ...
├── CMake/
│   ├── CMakeLists.txt
│   └── CMakePresets.json
└── Sources/
    ├── MiAplicacion.cpp
    ├── MiAplicacion.h
    ├── MiAplicacion_CFG.cpp
    ├── MiAplicacion_CFG.h
    └── APP_GEN_Defines.h
```

Todos los ficheros de recursos de la aplicación, excepto el ejecutable generado, deben colocarse bajo assets. Esto incluye configuración, logs, idiomas, gráficos, fuentes, layouts, bases de datos locales, paquetes Android u otros recursos que la aplicación necesite en tiempo de ejecución. Esta regla facilita que XPATHMANAGER pueda ordenar y localizar los recursos de forma homogénea.

9. XPATHMANAGER y la carpeta assets

XPATHMANAGER tiene como objetivo mantener ordenados y accesibles todos los ficheros de recursos de la aplicación. En lugar de dispersar rutas absolutas por el código, la aplicación trabaja con secciones de ruta. Cada sección representa un tipo de recurso: raíz, gráficos, fuentes, layouts de interfaz, bases de datos u otros grupos que la aplicación necesite.

El directorio assets actúa como raíz de los recursos. Dentro de assets se utilizan los directorios por defecto que define XPATHMANAGER y los módulos activos de la aplicación. Por ejemplo, en aplicaciones gráficas es habitual encontrar subdirectorios como graphics, fonts o ui_layouts. En aplicaciones con base de datos puede existir databases. En aplicaciones Android puede existir package. Además, el fichero INI principal y los logs suelen encontrarse directamente dentro de assets.

```
assets/
|
├── miaplicacion.ini
├── miaplicacion.log
├── miaplicacion.lng
├── graphics/
├── fonts/
├── ui_layouts/
├── databases/
└── package/
```

No todas las aplicaciones necesitan todos estos directorios. Una consola simple puede tener solo el INI y el log. Una aplicación gráfica con UserInterface necesita layouts, fuentes y recursos gráficos. Una aplicación de base de

datos puede añadir databases. Lo importante es que todas siguen el mismo criterio: los recursos se agrupan dentro de assets y se accede a ellos mediante XPATHMANAGER.

En la inicialización se ajusta la ruta raíz y se crean las secciones necesarias. En ejemplos como inibase se ve el uso de la raíz por defecto:

```
GEN_XPATHSMANAGER.AdjustRootPathDefault(APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

En aplicaciones gráficas, además, se añaden secciones específicas para recursos visuales:

```
GEN_XPATHSMANAGER.AdjustRootPathDefault(APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, APPFLOW_DEFAULT_DIRECTORY_GRAPHICS);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_FONTS, APPFLOW_DEFAULT_DIRECTORY_FONTS);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_UI_LAYOUTS, APPFLOW_DEFAULT_DIRECTORY_UI_LAYOUTS);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

Los nombres por defecto pueden cambiarse si la aplicación lo necesita. También se pueden crear nuevas secciones de ruta para recursos propios. Sin embargo, lo recomendable al empezar es respetar la organización por defecto porque hace que todas las aplicaciones con GEN Framework se comporten de forma parecida y sean más fáciles de mantener.

10. Configuración principal de la aplicación

Una aplicación real necesita configuración. GEN Framework resuelve esta necesidad mediante un fichero principal en formato INI y un módulo específico de configuración por aplicación. En lugar de leer el INI directamente desde cualquier parte del código, se crea una clase propia, normalmente llamada [APLICACION]_CFG, que hereda de APPFLOWCFG.

En inibase, por ejemplo, existen los ficheros IniBase_CFG.cpp e IniBase_CFG.h. En una aplicación propia llamada MiAplicacion, lo normal sería crear MiAplicacion_CFG.cpp y MiAplicacion_CFG.h. Este módulo representa la configuración principal de la aplicación y centraliza las entradas del fichero INI.

```
Sources/
├─ MiAplicacion.cpp
├─ MiAplicacion.h
├─ MiAplicacion_CFG.cpp
└─ MiAplicacion_CFG.h

assets/
└─ miaplicacion.ini
```

APPFLOWCFG se apoya en XFILECFG para gestionar el fichero de configuración. XFILECFG proporciona el soporte de fichero INI, secciones, claves, tipos de valores, valores por defecto y mantenimiento del fichero. APPFLOWCFG añade las secciones comunes del flujo de aplicación: configuración general, logs, trazas, servicios de Internet, recursos de hardware u otras áreas activadas por CMake.

La clase de configuración de la aplicación suele funcionar como singleton. Para ello se define una macro en el CMakeLists.txt que indica cuál es la instancia principal de configuración que usará APPFLOW. En inibase aparece un patrón como este:

```
set(APP_CFG_INIBASE "-DAPPFLOW_CFG=INIBASE_CFG::GetInstance()")
add_definitions(${APP_CFG_INIBASE})
```

Este define hace que el código común que usa APPFLOW_CFG apunte a la instancia concreta de configuración de la aplicación. De esta forma, los módulos genéricos de AppFlow pueden trabajar con una configuración común, pero cada programa mantiene su propia clase de configuración.

11. Mapeo de variables de configuración

La configuración principal no debe tratarse como un diccionario disperso de cadenas. La forma recomendada es mapear cada entrada del INI contra variables miembro de la clase [APLICACION]_CFG. Después, el resto de la aplicación accede a esos valores mediante getters y setters. Esto da tipado, centralización y control.

La clase de configuración suele implementar dos métodos principales: DoVariableMapping y DoDefault. DoVariableMapping declara qué entradas existen en el fichero de configuración y a qué variables internas se asocian. DoDefault asigna los valores iniciales o por defecto.

```
class MIAPLICACION_CFG : public APPFLOWCFG
{
public:
    static MIAPLICACION_CFG& GetInstance(bool ini = true);
    static bool DelInstance();

    bool DoVariableMapping();
    bool DoDefault();

    bool GetServerActive();
    void SetServerActive(bool value);

private:
    bool server_active;
};
```

El mapeo se realiza dentro de DoVariableMapping. Primero se llama al mapeo de APPFLOWCFG para conservar las entradas comunes del framework. Después se añaden las entradas propias de la aplicación.

```
bool MIAPLICACION_CFG::DoVariableMapping()
{
    if(!APPFLOWCFG::DoVariableMapping())
    {
        return false;
    }

    AddValue(XFILECFG_VALUETYPE_BOOLEAN,
        __L("general"),
        __L("server_active"),
        &server_active,
        __L("Activa el modo servidor"),
        APPFLOW_CFG_DEFAULT_REMARK_COLUMN);

    return true;
}
```

El método DoDefault carga los valores por defecto. También debe llamar primero a APPFLOWCFG::DoDefault para no perder los valores comunes del framework.

```
bool MIAPLICACION_CFG::DoDefault()
{
    if(!APPFLOWCFG::DoDefault())
    {
        return false;
    }

    server_active = false;
    return true;
}
```

```
}
```

Este sistema tiene una ventaja muy importante: el fichero de configuración se mantiene automáticamente actualizado. Si una entrada existe en el INI pero ya no está mapeada en DoVariableMapping, se considera obsoleta y puede eliminarse del fichero por falta de uso. De esta forma, la configuración no acumula claves antiguas, renombradas o abandonadas. El fichero principal se conserva limpio y sincronizado con la clase real de configuración.

En otras palabras: el INI no manda sobre el código. El contrato válido está en el mapeo de la clase de configuración. Lo que está mapeado se conserva, se actualiza y se escribe con sus valores. Lo que deja de estar mapeado deja de formar parte de la configuración activa.

12. Relación entre AppFlow, configuración y trazas

Durante AppProc_Ini la aplicación establece nombre, versión, rutas y trazas. Después se cargan o preparan los objetivos automáticos de traza definidos por la configuración y se inicia el módulo extendido de AppFlow. En inibase se observa una secuencia conceptual de este tipo:

```
Application_GetName()->AddFormat(__L("%s"), APPLICATION_NAMEAPP);

GEN_SET_VERSION(Application_GetName()->Get(),
    APPLICATION_NAMEFILE,
    APPLICATION_VERSION,
    APPLICATION_SUBVERSION,
    APPLICATION_SUBVERSIONERR,
    APPLICATION_OWNER,
    APPLICATION_YEAROFCREATION)

XTRACE_SETAPPLICATIONNAME((*Application_GetName()));
XTRACE_SETAPPLICATIONVERSION(APPLICATION_VERSION, APPLICATION_SUBVERSION, APPLICATION_SUBVERSIONERR);

APPFLOW_CFG_SETAUTOMATICTRACETARGETS
APPFLOW_EXTENDED.APPStart(&APPFLOW_CFG, this);
```

El objetivo es que la aplicación no tenga que reconstruir manualmente todos los servicios comunes. La configuración principal decide qué trazas se activan, qué secciones de log se usan, qué servicios se habilitan y cómo se comporta parte del entorno. El código de aplicación consume esa configuración a través de la clase [APLICACION]_CFG.

13. CMakeLists.txt mínimo para una aplicación de consola

Una aplicación con GEN Framework necesita un CMakeLists.txt que active las funcionalidades necesarias y añada las fuentes de la aplicación. El ejemplo inibase activa AppFlow, consola, trazas, memoria, DIO y otras funcionalidades. Para una aplicación mínima, el CMakeLists.txt tendría la misma filosofía: seleccionar opciones, definir la macro de configuración, incluir GEN_Main.cmake, añadir directorios y fuentes, y terminar con GEN_End.cmake.

```
cmake_minimum_required(VERSION 3.16.0)
project(miaplicacion)

set(GEN_DIRECTORY "../..../GEN_FramewOrk")

option(DEBUG_FEATURE          "Debug mode"          ON)
option(XMEMORY_CONTROL_FEATURE "XMemory Control"    ON)
option(XTRACE_FEATURE         "XTrace"               ON)
```

```

option(APPFLOW_FEATURE           "Application Flow"           ON)
option(APPFLOW_CONSOLE_FEATURE   "Application Flow Console"   ON)
option(APPFLOW_CFG_GENERAL_FEATURE "Application CFG General" ON)
option(APPFLOW_LOG_FEATURE       "Application Flow Log"       ON)

set(APP_CFG_MIAPLICACION "-DAPPFLOW_CFG=MIAPLICACION_CFG::GetInstance()")
add_definitions(${APP_CFG_MIAPLICACION})

include("${GEN_DIRECTORY}/Common/CMake/GEN_Main.cmake")

set(APP_DIRECTORY_APPLICATION "../Sources")
list(APPEND GEN_INCLUDES_DIR_LIST ${APP_DIRECTORY_APPLICATION})

list(APPEND GEN_SOURCES_MODULES_LIST "${APP_DIRECTORY_APPLICATION}/MiAplicacion.cpp")
list(APPEND GEN_SOURCES_MODULES_LIST "${APP_DIRECTORY_APPLICATION}/MiAplicacion_CFG.cpp")

include("${GEN_DIRECTORY}/Common/CMake/GEN_End.cmake")

```

El CMakeLists.txt real puede activar más módulos según las necesidades: DIO, Cipher, Compression, Databases, Scripts, gráficos o UserInterface. Lo importante en este capítulo es entender el patrón: la aplicación no se construye aislada, sino integrada en el sistema de compilación de GEN Framework.

14. Aplicación gráfica con AppFlowGraphics

La modalidad gráfica se apoya en APPFLOWGRAPHICS. Es la base para aplicaciones con pantalla, entrada, canvas, viewports, renderizado y, opcionalmente, UserInterface. Los ejemplos del directorio Graphics muestran este enfoque.

Una clase gráfica típica hereda de APPFLOWGRAPHICS y puede añadir XFSMACHINE si se quiere organizar su lógica mediante estados. El ejemplo UI_Message sigue este modelo.

```

class UI_MESSAGE : public APPFLOWGRAPHICS, public XFSMACHINE
{
public:
    bool AppProc_Ini();
    bool AppProc_FirstUpdate();
    bool AppProc_Update();
    bool AppProc_End();

private:
    bool UpdateInput();
    bool Ini_Graphics(GRPSCREEN* screen);
    bool Ini_Canvas(GRPSCREEN* screen);
    bool Ini_UserInterface(bool on);
    bool DrawFrame();
};

```

En AppProc_Ini se suelen establecer las opciones iniciales del flujo gráfico. Por ejemplo, crear pantalla principal, mostrarla y activar entrada:

```

SetInitOptions(APPFLOWGRAPHICS_INIOPTION_CREATEMAINSCREEN |
               APPFLOWGRAPHICS_INIOPTION_SHOWMAINSCREEN |
               APPFLOWGRAPHICS_INIOPTION_INPUT);

```

También se preparan las rutas de recursos gráficos mediante XPATHMANAGER. La raíz sigue estando en assets, pero se añaden secciones para gráficos, fuentes y layouts de interfaz.

```

GEN_XPATHSMANAGER.AdjustRootPathDefault(APPFLOW_DEFAULT_DIRECTORY_ROOT);

```

```

GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, APPFLOW_DEFAULT_DIRECTORY_GRAPHICS);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_FONTS, APPFLOW_DEFAULT_DIRECTORY_FONTS);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_UI_LAYOUTS, APPFLOW_DEFAULT_DIRECTORY_UI_LAYOUTS);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();

```

La aplicación gráfica reparte el trabajo de actualización entre entrada, dibujo y actualización de pantalla. En el estado principal de ejecución se leen dispositivos, se dibuja el frame y se actualizan viewports. La máquina de estados se usa del mismo modo que en consola: una parte continua cuando no hay evento y una parte puntual cuando se produce una transición.

15. Añadir UserInterface a una aplicación gráfica

UserInterface es una capa superior sobre los módulos gráficos. Permite cargar layouts, gestionar elementos visuales, recibir eventos de entrada y responder a acciones de usuario. No toda aplicación gráfica necesita UserInterface. Una aplicación puede dibujar directamente sobre un canvas. Pero cuando se quieren botones, mensajes, opciones, paneles o vistas más estructuradas, UserInterface aporta una organización más cómoda.

Los layouts de UserInterface se colocan normalmente dentro de assets/ui_layouts. En UI_Message, por ejemplo, se obtiene la ruta de esa sección y se carga un fichero de layout empaquetado:

```

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_UI_LAYOUTS, xpath);
xpath.Slash_Add();
xpath.Add(__L("message.zip"));

GEN_USERINTERFACE.Load(xpath, screen, 0);
GEN_USERINTERFACE.SubscribeInputEvents(true);
GEN_USERINTERFACE.SubscribeOutputEvents(true, this, &GEN_USERINTERFACE.GetInstance());

```

La inicialización de UserInterface debe tener también su cierre correspondiente. Al terminar la aplicación se desuscriben eventos, se destruye la instancia y se liberan los recursos asociados. Esto sigue la regla general del framework: todo recurso creado en la inicialización debe tener un cierre claro en AppProc_End.

```

GEN_USERINTERFACE.SubscribeOutputEvents(false, this, &GEN_USERINTERFACE.GetInstance());
GEN_USERINTERFACE.SubscribeInputEvents(false);
GEN_USERINTERFACE.DeleteInstance();

```

16. Estructura típica de una aplicación gráfica

Una aplicación gráfica tiene una estructura parecida a una aplicación de consola, pero su carpeta assets suele contener más subdirectorios. Además del INI y el log, aparecen gráficos, fuentes, layouts de interfaz y, si se compila para Android, paquete de recursos.

```

MiAplicacionGrafica/
|
├── assets/
|   ├── miaplicaciongrafica.ini
|   ├── miaplicaciongrafica.log
|   ├── miaplicaciongrafica.lng
|   ├── graphics/
|   ├── fonts/
|   ├── ui_layouts/
|   └── package/
|
├── CMake/
|   ├── CMakeLists.txt
|   └── CMakePresets.json
|

```

```
└─ Sources/  
  └─ MiAplicacionGrafica.cpp  
  └─ MiAplicacionGrafica.h  
  └─ MiAplicacionGrafica_CFG.cpp  
  └─ MiAplicacionGrafica_CFG.h  
  └─ APP_GEN_Defines.h
```

El CMakeLists.txt de una aplicación gráfica activa módulos adicionales: gráficos, entrada, AppFlowGraphics y, si corresponde, UserInterface. La idea no cambia: la aplicación declara qué necesita y GEN_Main.cmake / GEN_End.cmake integran esas necesidades dentro del sistema general de construcción.

17. Esqueleto recomendado de una primera aplicación

Si queremos crear una primera aplicación con GEN Framework, el camino recomendado es empezar desde un ejemplo existente. Para consola, inibase es un buen punto de partida. Para gráficos, UI_Message, UI_Options o Canvas2D permiten ver distintos niveles de complejidad. No conviene empezar escribiendo todos los ficheros desde cero sin mirar los ejemplos, porque los ejemplos ya contienen las decisiones correctas de integración: macro de instancia, configuración, rutas, CMake y ciclo de vida.

Elemento	Aplicación de consola	Aplicación gráfica
Clase base	APPFLOWCONSOLE	APPFLOWGRAPHICS
Estado interno	XFSMACHINE si se necesita flujo por estados	XFSMACHINE recomendado para separar inicialización, ejecución y cierre
Recursos	assets con INI, log y recursos necesarios	assets con INI, log, gráficos, fuentes, ui_layouts y package si procede
Configuración	[APLICACION]_CFG heredando de APPFLOWCFG	[APLICACION]_CFG heredando de APPFLOWCFG
Entrada	Teclado de consola mediante APPFLOWCONSOLE	INPFACTORY / GEN_INPMANAGER, teclado, ratón, touch u otros dispositivos
Salida	Texto, logs y trazas	Pantalla, canvas, viewports y opcionalmente UserInterface
Ejemplo de referencia	Examples/Console/IniBase	Examples/Graphics/UI_Message, UI_Options o Canvas2D

El orden de trabajo recomendado sería: copiar la estructura de un ejemplo cercano, renombrar la aplicación, ajustar APPLICATION_NAMEAPP y APPLICATION_NAMEFILE, crear la clase [APLICACION]_CFG, definir la macro APPFLOW_CFG en CMakeLists.txt, revisar assets, compilar con los scripts de Common y después empezar a modificar la lógica de AppProc_Ini, AppProc_Update y AppProc_End.

18. Por qué este esqueleto importa

El esqueleto de una aplicación con GEN Framework no es una formalidad. Define la forma en la que la aplicación se integra con el framework. Cuando respetamos esta estructura, obtenemos inicialización uniforme, configuración centralizada, recursos ordenados, trazas homogéneas, aislamiento de plataforma y una forma clara de mantener el código.

La alternativa de escribir una main propia y llamar a clases sueltas puede parecer más directa al principio, pero rompe parte de la filosofía del framework. Se pierde integración con el ciclo de vida, se dispersa la configuración, se duplican decisiones de plataforma y se obliga al programador a resolver manualmente problemas que GEN Framework ya tiene organizados.

Por eso la primera aplicación no debe verse como un simple programa de prueba. Debe verse como una plantilla mental. Una aplicación con GEN Framework se define por su clase principal, su ciclo de vida, su máquina de estados si procede, su configuración principal, sus recursos en assets y su integración con CMake. A partir de ese patrón se puede crecer hacia aplicaciones más complejas sin perder orden.

Los capítulos siguientes profundizan en cada parte: configuración, trazas, rutas, AppFlow, gráficos, entrada, UserInterface y módulos de servicio. Lo importante en este punto es tener claro el esqueleto: GEN Framework proporciona la infraestructura y la aplicación proporciona la lógica.

Capítulo V: XUtils, las utilidades extendidas del framework

1. El origen de XUtils

XUtils significa eXtended Utils. El nombre resume muy bien su papel dentro de GEN Framework: no es una colección accidental de funciones auxiliares, sino la capa de utilidades extendidas sobre la que empezó a formarse el framework. Antes de que GEN Framework tuviera la forma actual, antes de que existieran las factorías gráficas, de entrada, sonido, comunicaciones o AppFlow, ya existía la necesidad de tener una base común para trabajar con cadenas, buffers, ficheros, rutas, fechas, trazas, temporizadores, contenedores, configuración y pequeñas abstracciones del sistema.

Ese origen práctico es importante. Muchos programadores de C/C++ terminan creando una carpeta de utilidades que viaja de proyecto en proyecto. Al principio son funciones sueltas. Después aparecen clases para no repetir código. Más tarde esas clases necesitan una forma coherente de inicializarse, liberar recursos, trabajar con Unicode, registrar errores, acceder al sistema de ficheros o compilar en varias plataformas. XUtils nace exactamente en ese terreno: como la parte del framework que convierte esas utilidades recurrentes en una base estable, común y reutilizable.

Por eso XUtils debe entenderse como el germen de GEN Framework. No es el módulo más espectacular del framework, pero sí uno de los más importantes. Cualquier aplicación acaba necesitando cadenas, rutas, buffers, tiempo, ficheros, listas, mapas, trazas o configuración. Si cada aplicación resolviera esos problemas a su manera, el framework perdería homogeneidad. XUtils evita esa dispersión y proporciona una forma común de trabajar.

En la práctica, cuando se desarrolla una aplicación con GEN Framework, XUtils aparece continuamente. Una ruta se expresa mediante XPATH. Un texto se manipula con XSTRING. Un bloque binario se guarda en XBUFFER. Una fecha se obtiene con XDATETIME. Un temporizador se crea con XTIMER. Una lista de objetos puede organizarse con XVECTOR, XLIST, XMAP o XTREE. Una configuración se lee mediante XFILECFG o XFILEINI. Una traza se emite mediante XTRACE. Esta presencia constante convierte a XUtils en el vocabulario básico del framework.

2. XUtils como base común del framework

La función principal de XUtils es crear una capa de trabajo homogénea para el resto de módulos. DataIO no debería inventar su propio tipo de cadena. Graphic no debería usar una clase distinta para rutas. AppFlow no debería tener una forma diferente de medir tiempos. Cipher no debería definir otro tipo de buffer binario. Si cada grupo hiciera eso, GEN Framework acabaría siendo una suma de bibliotecas independientes. XUtils permite que todos los módulos hablen el mismo idioma.

Este idioma común se apoya en varios principios. El primero es la portabilidad: las clases deben poder compilar en distintos sistemas con el menor número posible de cambios. El segundo es la estabilidad de API: los módulos públicos usan tipos de GEN Framework como XCHAR, XSTRING, XBUFFER, XDWORD, XPATH o XVECTOR. El tercero es el control: se mantiene el estilo C/C++ del framework, sin ocultar por completo la gestión de recursos. El cuarto es la integración: las clases de XUtils se diseñan para ser usadas entre sí y por el resto de grupos.

XUtils no sustituye a la biblioteca estándar de C++. GEN Framework puede apoyarse internamente en mecanismos del lenguaje o del sistema, pero la interfaz visible del framework usa sus propios tipos y patrones. Esta decisión puede parecer conservadora, pero aporta una ventaja clara: una aplicación con GEN Framework mantiene un aspecto coherente, independientemente de si internamente una plataforma usa una API de Windows, una llamada POSIX, una función del lenguaje C, una librería externa o una implementación propia.

3. XFACTORY y el aislamiento de plataforma

Dentro de XUtils, XFACTORY es una pieza fundamental. Su objetivo es crear y destruir aquellas instancias de XUtils cuya implementación puede depender de la plataforma. El programador no debería decidir manualmente si necesita un temporizador de Windows, de Linux, de Android o de un microcontrolador. La aplicación pide un XTIMER a GEN_XFACTORY y el framework devuelve la implementación adecuada para la plataforma compilada.

Este patrón sigue la filosofía general de GEN Framework. La aplicación trabaja con una interfaz común. El código específico queda confinado en los módulos de plataforma. Cuando una clase necesita una implementación distinta en Windows, Linux, Android o un entorno embebido, la factoría actúa como frontera. El resultado es que el código de aplicación no se llena de `#ifdef` ni de llamadas nativas.

En XFACTORY encontramos la creación de objetos como XTIMER, XDATETIME, XRAND, XFILE, XDIR, XCONSOLE, XMUTEX y XTHREAD. Algunos de ellos parecen simples, pero no lo son cuando se quiere mantener comportamiento homogéneo entre plataformas. Dormir un hilo, medir tiempo, crear un mutex, abrir un fichero o enumerar un directorio no se implementa igual en todos los sistemas. XFACTORY evita que esa diferencia se propague hacia arriba.

```
XTIMER* xtimer = GEN_XFACTORY.CreateTimer();
if(!xtimer) return false;

xtimer->Reset();

// ... trabajo de la aplicación ...

if(xtimer->GetMeasureSeconds() >= 1)
{
    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, __L("one second"));
}

GEN_XFACTORY.DeleteTimer(xtimer);
xtimer = NULL;
```

La regla práctica es sencilla: cuando una clase de XUtils tiene una posible dependencia de plataforma y existe un método de creación en XFACTORY, se debe usar la factoría. Crear directamente una clase concreta puede funcionar en un caso local, pero rompe la arquitectura. La factoría no es una comodidad: es una barrera de aislamiento.

4. Mapa general de XUtils

Antes de revisar las clases, conviene mirar XUtils como un conjunto de grupos funcionales. No todas las clases tienen el mismo peso ni el mismo tipo de uso. Algunas son tipos básicos usados constantemente. Otras son plantillas de colección. Otras se encargan de ficheros y formatos. Otras proporcionan eventos, trazas, sincronización, serialización o soporte de plataforma. La siguiente tabla resume esa organización conceptual.

Elemento	Descripción, relación y ejemplo
Base y tipos fundamentales	XBase, XString, XFString, XBuffer, XBufferBits, XPath, XVariant, XRect, XUUUID. Son las piezas que aparecen en casi cualquier aplicación con GEN Framework.
Colecciones y estructuras	XVector, XVectorSTL, XMap, XPair, XList, XStack, XTree, XAVLTree, XBinaryMap, XDictionary, XOrderedList, XInheritEnum. Permiten organizar datos con el estilo del framework.
Sistema, tiempo y plataforma	XFactory, XTimer, XDateTime, XRand, XDir, XFile, XConsole, XSystem, XProcessManager, XSharedMemoryManager, XSleep, XThread y XMUTEX mediante la factoría.
Ficheros y formatos	XFile, XFileTXT, XFileINI, XFileCFG, XFileCSV, XFileJSON, XFileXML, XFileRIFF, XFileZIP, XFileHEX, XFileDFU, XFileDBF, XFileCobol, XFileHash y otros formatos auxiliares.

Eventos y modelo publisher/observer	XEvent, XSubject, XObserver, XPublisher y eventos específicos como XBER_XEVENT, XFILEZIP_XEVENT, XLICENSE_XEVENT, XSCHEDULER_XEVENT o XDRIVEIMAGEMANAGER_XEVENT.
Diagnóstico y control	XTrace, XLog, XMemory_Control, XFeedback_Control, XErrorMsg. Ayudan a diagnosticar, registrar, controlar memoria y comunicar estados.
Concurrencia y planificación	XThread, XThreadCollected, XThreadListNonPreemptive, XScheduler, XScheduler_XEvent y XSleep. Proporcionan una base para tareas, hilos y temporización.
Serialización y codificación	XSerializable, XSerializationMethod, XSerializationMethodBinary, XSerializationMethodJSON, XASN1, XBER, XMPInteger. Sirven para convertir objetos y datos a formatos transportables.
Idioma, traducción y licencias	XTranslation, XTranslation_GEN, XLanguage_ISO_639_3, XLicense, XLicense_XEvent. Cubren textos traducibles, códigos de idioma y control de licencia.
Módulos auxiliares	XSTL, XFactoryMacros, XFileID, XFileHashID, XEEPROMMemoryManager y XDriveImageManager. Son piezas de apoyo para integrar funcionalidades concretas.

5. Tipos básicos, cadenas, buffers y rutas

El primer grupo de XUtils contiene clases que se usan de forma transversal. Son tan habituales que muchas veces se dejan de percibir como módulos individuales. XSTRING aparece en configuración, logs, rutas, protocolos y mensajes. XBUFFER aparece en ficheros binarios, cifrado, comunicaciones y serialización. XPATH aparece en cualquier acceso a disco. XVARIANT permite transportar valores de tipo variable. Esta capa es el equivalente al “alfabeto” de GEN Framework.

```
XSTRING name;
XBUFFER buffer;
XPATH xpath;

name.Format(__L("%s %02d"), __L("test"), 1);
name.ConvertToASCII(buffer, XSTRINGASCII_CODEPAGE_437);

xpath.Set(__L("assets"));
xpath.Slash_Add();
xpath.Add(__L("config.ini"));
```

Elemento	Descripción, relación y ejemplo
XBASE	Define funciones y elementos básicos compartidos. Relación: casi todo XUtils depende directa o indirectamente de XBase. Ejemplo: usar los tipos y macros base del framework antes de construir módulos superiores.
XSTRING	Cadena Unicode principal de GEN Framework. Relación: XBUFFER, XPATH, XFILETXT, XFILEINI, XLOG, XTRACE. Ejemplo: <code>string.Format(__L("value %d"), value);</code>
XFSTRING	Cadena rápida orientada a escenarios donde importa reducir coste de manipulación textual. Relación: XBUFFER y operaciones de texto. Ejemplo: usarla para montar texto temporal intensivo sin pasar por rutas más pesadas.
XBUFFER	Buffer binario mutable. Relación: XSTRING, XFILE, Cipher, Compress, DataIO y serialización. Ejemplo: <code>buffer.Add((XBYTE*)data, size);</code>
XBUFFERBITS	Lectura de flujos de bits sobre un XBUFFER. Relación: parsers binarios, protocolos y formatos compactos. Ejemplo: leer campos que no están alineados a byte.
XPATH	Representa rutas de fichero con operaciones homogéneas. Relación: XFILE, XDIR, XPATHSMANAGER, APPFLOWCFG. Ejemplo:

	xpath.Slash_Add(); xpath.Add(__L("file.txt"));
XPATHSMANAGER	Singleton que ordena las rutas de recursos de la aplicación. Relación: AppFlow, assets, XPATH y XFILE. Ejemplo: GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, xpath);
XVARIANT	Contenedor de valor de tipo variable. Relación: XFileJSON, serialización, configuración y datos heterogéneos. Ejemplo: variant.Set((XDWORD)10);
XRECT	Rectángulo genérico. Relación: Graphic, UserInterface y cálculos geométricos. Ejemplo: XRECT rect(0, 0, width, height);
XUUID	Generación y manejo de identificadores únicos. Relación: XRand y sistemas de identificación. Ejemplo: uuid.Generate(); uuid.GetToString(string);

6. Colecciones y estructuras de datos

GEN Framework no se limita a usar contenedores estándar visibles en la API pública. XUtils ofrece sus propias estructuras para mantener coherencia con el resto del framework, controlar propiedad de objetos, usar el estilo de iteración existente y mantener compatibilidad con plataformas donde el uso directo de STL puede no ser deseable o no encajar con el estilo histórico del código.

La clase más habitual es XVECTOR. En muchos módulos del framework aparece como contenedor de punteros a objetos de GEN Framework. Cuando esos objetos son propiedad del vector, es frecuente usar DeleteContents() seguido de DeleteAll(). Esta forma expresa claramente dos operaciones distintas: eliminar los objetos y limpiar la lista.

```
XVECTOR<XSTRING*> names;

XSTRING* name = GEN_NEW XSTRING();
name->Set(__L("main"));
names.Add(name);

for(int c=0; c<names.GetSize(); c++)
{
    XSTRING* item = names.Get(c);
    if(item) XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, item->Get());
}

names.DeleteContents();
names.DeleteAll();
```

Elemento	Descripción, relación y ejemplo
XVECTOR	Vector genérico principal del framework. Relación: prácticamente todos los módulos que mantienen listas. Ejemplo: vector.Add(item); item = vector.Get(index);
XVECTORSTL	Adaptación basada en STL cuando el módulo necesita esa implementación. Relación: XSTL y escenarios concretos. Ejemplo: usarlo si el subsistema local ya trabaja con esa variante.
XMAP	Mapa clave/elemento con estilo de GEN Framework. Relación: configuraciones, tablas de relación y búsqueda por clave. Ejemplo: map.Add(key, element);
XPAIR	Par de valores. Relación: XMAP y estructuras auxiliares. Ejemplo: XPAIR<XDWORD, XSTRING*> pair;
XLIST	Lista doblemente enlazada. Relación: colas, recorridos y estructuras donde insertar/eliminar en posiciones intermedias importa. Ejemplo: list.Add(data);
XSTACK	Pila genérica. Relación: parsers, recorridos y algoritmos LIFO. Ejemplo:

	stack.Push(item); item = stack.Pop();
XTREE	Árbol genérico con nodos. Relación: XML, jerarquías de UI, estructuras de proyecto y datos anidados. Ejemplo: node->AddChild(child);
XAVLTREE	Árbol AVL equilibrado. Relación: estructuras que necesitan búsqueda ordenada eficiente. Ejemplo: insertar claves manteniendo equilibrio automático.
XBINARYMAP	Mapa binario orientado a búsquedas sobre claves ordenadas. Relación: tablas compactas y acceso rápido. Ejemplo: buscar un elemento por clave sin recorrer toda la colección.
XDICTIONARY	Diccionario de entradas. Relación: traducción, catálogos, tablas de textos y equivalencias. Ejemplo: dictionary.Add(__L("id"), value);
XORDEREDLIST	Lista ordenada mediante comparador. Relación: planificación, rankings, ordenación de objetos. Ejemplo: insertar objetos manteniendo el orden definido por XCOMPARATOR.
XINHERITENUM	Utilidad para trabajar con enumeraciones heredables o extendidas. Relación: APIs que necesitan conservar valores base y ampliarlos. Ejemplo: definir un rango de enum propio sobre un rango existente.

7. Tiempo, sistema y objetos dependientes de plataforma

Hay utilidades que parecen neutras hasta que se intenta hacerlas portables. Medir tiempo con precisión, dormir un hilo, obtener información de sistema, crear procesos o usar memoria compartida cambia según la plataforma. XUtils encapsula esas diferencias y, cuando corresponde, las canaliza por XFACTORY.

```

XDATETIME* datetime = GEN_XFACTORY.CreateDateTime();
if(datetime)
{
    XSTRING text;

    datetime->Read();
    datetime->GetDateTimeToString(XDATETIME_FORMAT_STANDARD, text);

    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, text.Get());

    GEN_XFACTORY.DeleteDateTime(datetime);
    datetime = NULL;
}

```

Elemento	Descripción, relación y ejemplo
XFACTORY	Factoría de XUtils. Crea instancias dependientes de plataforma como XTIMER, XDATETIME, XRAND, XFILE, XDIR, XCONSOLE, XMUTEX y XTHREAD. Ejemplo: GEN_XFACTORY.CreateTimer();
XFACTORYMACROS	Macros auxiliares para crear y destruir objetos desde factorías. Relación: patrón común de factorías de GEN Framework. Ejemplo: usar macros de creación cuando el módulo define un flujo estándar.
XTIMER	Medición de tiempo y temporización. Relación: XFactory, XDateTime, AppFlow y Scheduler. Ejemplo: timer->GetMeasureSeconds();
XDATETIME	Fecha y hora. Relación: XFactory, XLog, XFileZIP, Scheduler. Ejemplo: datetime->Read();
XRAND	Generación de números aleatorios. Relación: XFactory, UUID, pruebas y generación de datos. Ejemplo: rand->Max(100);
XSLEEP	Pausas o esperas controladas. Relación: hilos, temporizadores y plataforma. Ejemplo: GEN_XSLEEP.MilliSeconds(100);
XSYSTEM	Información y operaciones del sistema. Relación: plataformas, AppFlow, recursos hardware. Ejemplo: consultar sistema operativo o características de hardware.

XPROCESSMANAGER	Gestión de procesos externos. Relación: herramientas, scripts, utilidades y plataforma. Ejemplo: lanzar una herramienta auxiliar y comprobar su finalización.
XSHAREDMEMORYMANAGER	Memoria compartida entre procesos. Relación: IPC, procesos y sincronización. Ejemplo: crear un bloque común para intercambio de datos.
XCONSOLE	Abstracción de consola. Relación: AppFlowConsole, DevTestsConsole y entrada/salida de texto. Ejemplo: console->KbHit(); console->GetChar();

8. Ficheros, directorios y XPATHSMANAGER

La gestión de ficheros es una de las primeras necesidades que resuelve cualquier base de utilidades. GEN Framework separa varios niveles. XPATH representa rutas. XDIR trabaja con directorios. XFILE abstrae operaciones de fichero. Sobre XFILE aparecen clases especializadas para formatos concretos. XPATHSMANAGER, por su parte, ordena dónde deben estar los recursos de la aplicación.

La idea es que una aplicación no tenga rutas dispersas por el código. Los recursos se agrupan bajo assets y, dentro de esa carpeta, se usan secciones conocidas: logs, databases, web, scraperweb, fonts, graphics, ui_layouts, sounds, scripts, firmware, certificates y secciones genéricas. El nombre de esas carpetas puede cambiarse y pueden añadirse rutas nuevas, pero el objetivo es siempre el mismo: que la aplicación tenga una forma homogénea de encontrar sus recursos.

```
XPATH xpath;
```

```
GEN_XPATHSMANAGER.AdjustRootPathDefault(__L("assets"));
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, __L("graphics"));
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

```
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, xpath);
xpath.Slash_Add();
xpath.Add(__L("logo.png"));
```

Elemento	Descripción, relación y ejemplo
XFILE	Clase base de acceso a ficheros. Relación: todos los formatos derivados, XFactory, XPath, XBuffer. Ejemplo: file->Open(xpath); file->Read(buffer);
XDIR	Enumeración y gestión de directorios. Relación: XFactory, XPath y herramientas de recursos. Ejemplo: xdir->FirstSearch(xpath, __L("*.txt"));
XFILEID	Define identificadores de tipo de fichero. Relación: módulos que reconocen formatos por cabecera o extensión. Ejemplo: usar ID de fichero para clasificar contenido.
XFILEHASHID	Define identificadores asociados a ficheros con hash. Relación: XFileHash y validación de integridad. Ejemplo: seleccionar el tipo de hash esperado.
XFILEHASH	Fichero con hash incluido. Relación: integridad, XHash32 y validación de datos. Ejemplo: comprobar que el contenido no ha sido modificado.
XFILESENTENCES	Fichero de sentencias o textos hash. Relación: traducción, textos y recursos. Ejemplo: cargar frases identificadas por clave.
XFILEXDB	Formato de diálogo o frases. Relación: recursos de texto y aplicaciones con mensajes estructurados. Ejemplo: cargar un banco de frases o diálogos.

9. Formatos de fichero soportados en XUtils

Una parte importante de XUtils está en FormatFiles. Estas clases resuelven la lectura y escritura de formatos habituales o necesarios para proyectos concretos. De nuevo aparece la filosofía práctica de GEN Framework: no se añaden formatos por catálogo, sino porque alguna aplicación o herramienta necesitó trabajar con ellos.

```
XFILEINI ini;
XPATH    xpath;
XSTRING  value;

xpath.Set(__L("assets"));
xpath.Slash_Add();
xpath.Add(__L("application.ini"));

if(ini.Open(xpath, true))
{
    ini.ReadValue(__L("General"), __L("Name"), value);
    ini.Close();
}
```

Elemento	Descripción, relación y ejemplo
XFILETXT	Lectura y escritura de texto. Relación: XSTRING, XBUFFER y codificaciones. Ejemplo: filetxt.AddLine(__L("hello"));
XFILEINI	Ficheros INI con secciones, claves y comentarios. Relación: XFILECFG y APPFLOWCFG. Ejemplo: leer [General] Name=...
XFILECFG	Capa de configuración sobre INI. Relación: APPFLOWCFG, mapeo de variables y configuración principal de aplicación. Ejemplo: mapear una entrada para que se mantenga actualizada.
XFILECSV	Ficheros CSV. Relación: importación/exportación de tablas. Ejemplo: recorrer registros y campos de un fichero separado por comas.
XFILEJSON	Lectura/escritura JSON mediante objetos, arrays y valores. Relación: XVARIANT y serialización JSON. Ejemplo: json.GetRoot()->GetValue(__L("name"));
XFILEXML	Lectura/escritura XML mediante elementos y atributos. Relación: XTREE, XSTRING y configuración estructurada. Ejemplo: recorrer nodos hijos de un elemento raíz.
XFILERIFF	Formato RIFF. Relación: audio, vídeo y contenedores binarios. Ejemplo: localizar chunks dentro de un fichero WAV o AVI.
XFILEZIP	Compresión/descompresión ZIP. Relación: XFILEZIP_XEVENT, recursos empaquetados y distribución. Ejemplo: comprimir una carpeta assets en un ZIP.
XFILEZIP_XEVENT	Evento de progreso o estado para operaciones ZIP. Relación: XPublisher y XObserver. Ejemplo: informar porcentaje de compresión.
XFILEHEX	Formato Intel HEX u otros datos ASCII hexadecimales. Relación: firmware y microcontroladores. Ejemplo: leer registros de firmware.
XFILEDFU	Formato DFU de actualización de firmware. Relación: dispositivos y actualización embebida. Ejemplo: cargar imágenes de firmware para un dispositivo.
XFILEDBF	Formato DBF. Relación: bases de datos antiguas, importación y compatibilidad. Ejemplo: leer registros de una tabla DBF.
XFILECOBOL	Estructuras de fichero COBOL. Relación: integración con sistemas legacy. Ejemplo: interpretar campos definidos por PIC.
XFILEHASH	Formato de fichero con hash integrado. Relación: validación e integridad. Ejemplo: leer contenido y verificar su huella.
XFILESENTENCES	Formato de frases o sentencias. Relación: textos de aplicación y traducción. Ejemplo: obtener una frase por identificador.

XFILEXDB	Formato de frases de diálogo. Relación: recursos textuales estructurados. Ejemplo: cargar mensajes organizados por clave.
-----------------	---

10. Configuración con XFILECFG

XFILECFG merece una mención específica porque conecta XUtils con AppFlow. La configuración principal de una aplicación con GEN Framework se apoya en ficheros INI, pero no se reduce a leer valores de texto. Normalmente existe una clase de configuración propia de la aplicación, derivada de APPFLOWCFG, que declara variables internas, define un mapeo de entradas y proporciona getters y setters. XFILECFG es la base que permite mantener ese fichero ordenado, sincronizado y limpio.

Cuando una entrada de configuración está mapeada, la aplicación sabe qué significa, de qué tipo es y dónde debe guardarse. Cuando una entrada deja de estar mapeada, el sistema puede eliminarla porque ya no forma parte de la configuración vigente. Esto evita que los INI acumulen claves obsoletas durante años. En proyectos largos esta limpieza automática tiene mucho valor: el fichero de configuración refleja la configuración real de la aplicación, no una historia acumulada de versiones anteriores.

```
// Ejemplo conceptual dentro de una clase APLICACION_CFG

bool APLICACION_CFG::DoVariableMapping()
{
    AddValue(_L("General"), _L("Name"),          XFILECFG_VALUETYPE_STRING, &name);
    AddValue(_L("Trace"), _L("IsActive"),        XFILECFG_VALUETYPE_BOOLEAN, &istraceactive);
    AddValue(_L("Server"), _L("Port"),           XFILECFG_VALUETYPE_INTEGER, &port);

    return true;
}
```

La sintaxis exacta del mapeo depende de la implementación concreta de la clase de configuración, pero la idea que debe quedar clara es esta: el fichero INI no se trata como texto libre, sino como una proyección persistente de variables conocidas por la aplicación.

11. Eventos, observadores y publicación

XUtils incluye la base del sistema de eventos del framework. XEVENT representa el evento. XSUBJECT representa el objeto que puede ser observado. XOBSERVER representa el receptor. XPUBLISHER organiza la publicación. Esta estructura es importante porque GEN Framework usa eventos en varios módulos: sonido, comunicaciones, scheduler, licencias, ZIP, BER y otros subsistemas.

El modelo evita acoplar directamente productor y consumidor. Un módulo puede emitir un evento sin saber quién lo recibe. Una aplicación puede suscribirse a determinados eventos sin modificar la clase que los produce. Esta separación encaja muy bien con un framework modular.

```
class MYOBSERVER : public XOBSERVER
{
public:

    bool HandleEvent(XEVENT* xevent)
    {
        if(!xevent) return false;

        switch(xevent->GetEventType())
        {
            case XEVENT_TYPE_APPLICATION : // procesar evento
                break;
        }

        return true;
    }
};
```

Elemento	Descripción, relación y ejemplo
----------	---------------------------------

XEVENT	Clase base de evento. Relación: XSubject, XObserver y eventos específicos. Ejemplo: transportar un tipo de evento y datos asociados.
XSUBJECT	Sujeto observable. Relación: XObserver y XPublisher. Ejemplo: una clase que permite suscripciones.
XOBSERVER	Observador que recibe eventos. Relación: módulos que reaccionan a cambios. Ejemplo: una aplicación suscrita a eventos de sonido.
XPUBLISHER	Publicador y gestor de relaciones sujeto/observador. Relación: XSubject y XEvent. Ejemplo: publicar un evento a todos los observadores suscritos.
XBER_XEVENT	Evento asociado a procesamiento BER. Relación: XBER y XPUBLISHER. Ejemplo: notificar avance o datos en una codificación BER.
XSCHEDULER_XEVENT	Evento del planificador. Relación: XScheduler y tareas programadas. Ejemplo: avisar de una tarea que debe ejecutarse.
XLICENSE_XEVENT	Evento de licencia. Relación: XLicense. Ejemplo: notificar validación, error o cambio de estado de licencia.
XDRIVEIMAGEMANAGER_XEVENT	Evento de gestión de imagen de disco. Relación: XDriveImageManager. Ejemplo: informar progreso de copia o creación de imagen.
XFILEZIP_XEVENT	Evento de operaciones ZIP. Relación: XFileZIP. Ejemplo: informar progreso de compresión o descompresión.

12. Máquina de estados finitos

XFSMACHINE ya apareció en el capítulo dedicado a la primera aplicación, pero pertenece a XUtils y por tanto debe entenderse como una utilidad general del framework. Su función es controlar estados y eventos de una forma explícita. En lugar de tener variables booleanas dispersas, una aplicación define estados, define eventos y registra qué transiciones son válidas.

La gracia de XFSMACHINE no está solo en cambiar de un estado a otro. Su utilidad práctica está en separar la lógica continua de un estado de la lógica que solo debe ejecutarse cuando se produce una transición. En los ejemplos de GEN Framework, el Update suele tener dos partes: cuando no hay evento nuevo, se ejecuta la lógica propia del estado actual; cuando hay un evento, se comprueba la transición y se ejecuta la lógica de entrada o cambio de estado.

```
enum MYAPP_XFSMEVENTS
{
    MYAPP_XFSMEVENT_NONE = 0 ,
    MYAPP_XFSMEVENT_INI    ,
    MYAPP_XFSMEVENT_UPDATE ,
    MYAPP_XFSMEVENT_END    ,
};

enum MYAPP_XFSMSTATES
{
    MYAPP_XFSMSTATE_NONE = 0 ,
    MYAPP_XFSMSTATE_INI    ,
    MYAPP_XFSMSTATE_UPDATE ,
    MYAPP_XFSMSTATE_END    ,
};

bool MYAPP::IniFSMachine()
{
    if(!AddState(MYAPP_XFSMSTATE_NONE,
        MYAPP_XFSMEVENT_INI, MYAPP_XFSMSTATE_INI,
        MYAPP_XFSMEVENT_END, MYAPP_XFSMSTATE_END,
        XFSMACHINESTATE_EVENTDEFEND)) return false;

    return true;
}
```

El uso correcto de XFSMACHINE ayuda a que una aplicación sea más fácil de depurar. Podemos ver qué estado está activo, qué evento ha provocado el cambio y qué transiciones están permitidas. Esto resulta especialmente útil en aplicaciones con ciclo de vida largo, comunicaciones, pantallas gráficas, protocolos o modos de funcionamiento.

13. Trazas, logs, memoria y feedback

Un framework utilizado en aplicaciones reales necesita diagnóstico. Las trazas permiten saber qué está ocurriendo en ejecución. Los logs permiten conservar información. El control de memoria ayuda a encontrar fugas o ciclos de vida incorrectos. El feedback permite comunicar estados o progreso. XUtils proporciona estas piezas para que el diagnóstico no sea improvisado en cada aplicación.

```
XTRACE_SETAPPLICATIONNAME ((*Application_GetName()));
XTRACE_SETAPPLICATIONVERSION (APPLICATION_VERSION, APPLICATION_SUBVERSION,
APPLICATION_SUBVERSIONERR);

XTRACE_PRINTCOLOR(XTRACE_COLOR_GREEN, __L("Application initialized"));
```

Elemento	Descripción, relación y ejemplo
XTRACE	Sistema de trazas. Relación: AppFlow, DataIO y diagnóstico. Ejemplo: XTRACE_PRINTCOLOR(...);
XLOG	Registro persistente de mensajes. Relación: XDateTime, XFileTXT, XMutex y configuración. Ejemplo: escribir un mensaje de nivel error en un fichero de log.
XMEMORY_CONTROL	Control de memoria del framework. Relación: GEN_NEW, detección de fugas y compilaciones DEBUG. Ejemplo: activar MEMCTRL durante desarrollo.
XFEEDBACK_CONTROL	Control de feedback y códigos de estado. Relación: procesos largos, UI y trazabilidad. Ejemplo: comunicar avance de una operación.
XERRORMSG	Gestión de mensajes de error. Relación: clases que devuelven códigos o estados. Ejemplo: traducir un código a texto de error legible.

14. Concurrencia, hilos y planificación

La concurrencia es otro terreno donde GEN Framework prefiere una capa propia. XTHREAD y XMUTEX no son simples envoltorios por estética: permiten que la aplicación use una forma común de crear hilos, agruparlos, destruirlos y sincronizar recursos aunque la implementación cambie en cada plataforma. El acceso se realiza mediante XFACTORY cuando corresponde.

```
static void ThreadFunction(void* param)
{
    MYAPP* app = (MYAPP*)param;
    if(!app) return;

    // trabajo del hilo
}

XTHREAD* thread = GEN_XFACTORY.CreateThread(XTHREADGROUPID_APPOWNER, __L("worker"),
ThreadFunction, this);
if(!thread) return false;
```

Elemento	Descripción, relación y ejemplo
XTHREAD	Clase base de hilo. Relación: XFactory, XMutex, AppFlow y tareas. Ejemplo: GEN_XFACTORY.CreateThread(...);
XTHREADCOLLECTED	Hilo gestionado dentro de una colección. Relación: XThreadCollected manager y ciclos de vida controlados. Ejemplo: hilos agrupados por tipo de trabajo.

XTHREADLISTNONPREEMPTIVE	Lista de hilos no preemptiva. Relación: entornos donde se controla manualmente la ejecución. Ejemplo: microcontroladores o bucles cooperativos.
XSCHEDULER	Planificador de tareas. Relación: XTimer, XDateTime, XThreadCollected y eventos. Ejemplo: ejecutar una tarea cada cierto intervalo.
XSCHEDULER_XEVENT	Evento del planificador. Relación: XScheduler y XPublisher. Ejemplo: notificar ejecución de una tarea programada.
XSLEEP	Esperas temporizadas. Relación: hilos, polling y temporizadores. Ejemplo: XSLEEP::Milliseconds(50).

15. Serialización, codificación y estructuras binarias

Cuando una aplicación necesita guardar objetos, enviarlos por red o convertirlos a un formato estable, aparece la serialización. XUtils separa el objeto serializable de los métodos de serialización. XSerializable define la idea de objeto que puede exportar o importar su estado. XSerializationMethod y sus variantes definen cómo se representa ese estado: binario, JSON u otros métodos.

```
class MYDATA : public XSERIALIZABLE
{
public:

    bool Serialize()
    {
        // Añadir campos al método de serialización activo
        return true;
    }
};
```

Elemento	Descripción, relación y ejemplo
XSERIALIZABLE	Base para objetos serializables. Relación: XSerializationMethod y clases de datos. Ejemplo: una clase que guarda sus campos en JSON o binario.
XSERIALIZATIONMETHOD	Método abstracto de serialización. Relación: XBuffer, XVariant y formatos concretos. Ejemplo: escribir campos usando una interfaz común.
XSERIALIZATIONMETHODBINARY	Serialización binaria. Relación: XBuffer y datos compactos. Ejemplo: guardar estructuras en un buffer para fichero o red.
XSERIALIZATIONMETHODJSON	Serialización en JSON. Relación: XFileJSON, XVariant y configuración. Ejemplo: exportar un objeto a una estructura legible.
XASN1	Lectura y escritura ASN.1. Relación: BER, certificados y estructuras criptográficas. Ejemplo: interpretar un bloque ASN.1.
XBER	Basic Encoding Rules. Relación: ASN.1, certificados y estructuras binarias. Ejemplo: decodificar tags y longitudes.
XBER_XEVENT	Eventos del procesamiento BER. Relación: XBER y observadores. Ejemplo: informar durante una decodificación compleja.
XMPINTEGER	Enteros multiprecisión. Relación: criptografía y operaciones matemáticas grandes. Ejemplo: manejar números mayores que los tipos nativos.

16. Internacionalización, traducción y textos

GEN Framework incluye clases para trabajar con idiomas y traducciones. Este soporte no debe verse como un añadido estético, sino como parte de la construcción de aplicaciones reales. Una aplicación que se despliega en varios entornos puede necesitar textos traducibles, mensajes comunes del framework, selección de idioma y códigos normalizados.

Elemento	Descripción, relación y ejemplo
XLANGUAGE_ISO_639_3	Códigos de idioma ISO 639-3. Relación: XTranslation y selección de idioma. Ejemplo: GEN_XTRANSLATION.SetActual(XLANGUAGE_ISO_639_3_CODE_SPA);
XTRANSLATION	Gestor de traducciones de aplicación. Relación: ficheros de frases, XPATH y XSTRING. Ejemplo: cargar un fichero de idioma y obtener un texto por identificador.
XTRANSLATION_GEN	Traducciones internas del framework. Relación: mensajes comunes de GEN Framework. Ejemplo: obtener una frase estándar del propio framework.
XFILESENTENCES	Ficheros de sentencias. Relación: XTranslation. Ejemplo: almacenar textos traducibles por ID.
XFILEXDB	Ficheros de diálogo o frases. Relación: textos estructurados de aplicación. Ejemplo: cargar diálogos o mensajes de interfaz.

17. Licencias, firmware, almacenamiento y recursos especiales

Algunas clases de XUtils responden a necesidades muy concretas de proyectos reales: control de licencias, actualización de firmware, gestión de memoria EEPROM, imágenes de disco o recursos persistentes. Son buenos ejemplos de la filosofía de GEN Framework: cuando una funcionalidad aparece varias veces en proyectos distintos, se convierte en utilidad común.

Elemento	Descripción, relación y ejemplo
XLICENSE	Generación y control de licencias. Relación: XLicense_XEvent, XFile, Cipher y datos de identificación. Ejemplo: validar una licencia antes de activar módulos.
XLICENSE_XEVENT	Evento asociado al estado de licencia. Relación: XLicense y observadores. Ejemplo: avisar de licencia no válida o próxima a expirar.
XEEPROMMEMORYMANAGER	Gestión de memoria EEPROM. Relación: microcontroladores y almacenamiento persistente pequeño. Ejemplo: reservar zonas para parámetros.
XDRIVEIMAGEMANAGER	Gestión de imágenes de disco o unidad. Relación: XDriveImageManager_XEvent, ficheros y progreso. Ejemplo: crear o restaurar una imagen.
XDRIVEIMAGEMANAGER_XEVENT	Evento de progreso del gestor de imágenes. Relación: XDriveImageManager. Ejemplo: notificar porcentaje de copia.
XFILEDFU	Actualización de firmware DFU. Relación: firmware, dispositivos y assets/firmware. Ejemplo: cargar un paquete de actualización.
XFILEHEX	Ficheros HEX de firmware. Relación: microcontroladores. Ejemplo: leer registros de firmware en formato texto hexadecimal.

18. Módulos auxiliares y de integración

No todos los ficheros de XUtils representan una gran clase de uso directo. Algunos son piezas de integración, compatibilidad o definición. Su presencia es importante porque hacen que el framework sea consistente y evitan repetir pequeñas decisiones en varios módulos.

Elemento	Descripción, relación y ejemplo
XSTL	Definiciones e includes relacionados con STL cuando se necesita integración. Relación: XVectorSTL y código que debe convivir con STL. Ejemplo: encapsular dependencias estándar en un punto común.

XFACTORYMACROS	Macros de apoyo para factorías. Relación: XFactory, DIOFactory, GRPFactory y otros patrones de creación. Ejemplo: simplificar creación y registro de módulos.
XFILEID	Identificadores de tipos de fichero. Relación: módulos FormatFiles. Ejemplo: reconocer un tipo de recurso.
XFILEHASHID	Identificadores para formatos con hash. Relación: XFileHash. Ejemplo: seleccionar algoritmo o tipo de validación.
XPAIR	Par de valores usado por estructuras de datos. Relación: XMap y algoritmos auxiliares. Ejemplo: agrupar clave y valor.
XINHERITENUM	Soporte para extender enumeraciones. Relación: eventos y rangos de tipos. Ejemplo: reservar un rango de eventos para una aplicación.
XINTERPOLATOR	Interpolación de valores. Relación: Graphic, animaciones, UI y procesos temporizados. Ejemplo: interpolar entre dos valores en función de un tiempo.
XBINARYMAP	Mapa binario optimizado. Relación: búsquedas y tablas compactas. Ejemplo: consultar un elemento por clave binaria.

19. Cómo estudiar XUtils desde DevTestsConsole

DevTestsConsole es una herramienta muy útil para entender XUtils porque no se limita a compilar clases: contiene pruebas de uso real. En sus fuentes aparecen pruebas de XString, XBuffer, XRand, XTrace, XLog, XTimer, XVector, XTree, XDir, XVariant, DateTime, XSystem, SharedMemory, XFileJSON, XFileXML, XFileRIFF, XFileDFU, XLicense, XSerializable, XSchedular, Compress y XPath, entre otras. Esto convierte a DevTestsConsole en una especie de laboratorio de comportamiento del framework.

Cuando se quiera estudiar una clase de XUtils, el orden recomendado es leer primero la cabecera para entender su API pública, después revisar la documentación Doxygen para ver la lista de métodos y, finalmente, buscar en DevTestsConsole una prueba que la utilice. Esta combinación es más útil que leer únicamente la implementación, porque muestra la intención de uso.

Por ejemplo, para aprender XSTRING conviene revisar Test_XString. Para entender XBUFFER, Test_XBuffer. Para ver cómo se gestionan árboles, Test_XTree. Para ver rutas y directorios, Test_XPath y Test_XDir. Para formatos, Test_XFileJSON, Test_XFileXML, Test_XFileRIFF o Test_XFileDFU. Para temporización, Test_XTimer. Para planificación, Test_Scheduler. Este enfoque ayuda a pasar de la teoría a la práctica.

20. Criterios de uso en una aplicación con GEN Framework

La pregunta importante no es solo qué clases existen, sino cuándo se deben usar. En una aplicación con GEN Framework conviene adoptar una regla sencilla: si existe una clase XUtils para resolver una necesidad común, debe preferirse esa clase antes que introducir una solución aislada. Esto no significa que esté prohibido usar C++ estándar o una librería externa, sino que la frontera pública de la aplicación debería mantenerse coherente con el framework.

Si necesitamos texto, usamos XSTRING. Si necesitamos datos binarios, XBUFFER. Si necesitamos una ruta, XPATH. Si necesitamos una lista de objetos del framework, XVECTOR. Si necesitamos un fichero INI de configuración, XFILECFG o XFILEINI. Si necesitamos un temporizador, XFACTORY y XTIMER. Si necesitamos trazas, XTRACE. Si necesitamos publicar eventos, XEVENT, XSUBJECT, XOBSERVER y XPUBLISHER. Esta disciplina reduce diferencias entre aplicaciones y acelera el mantenimiento.

También hay que respetar la propiedad de los objetos. Si un objeto se crea mediante una factoría, debe destruirse mediante la factoría correspondiente. Si un contenedor posee los objetos que almacena, debe liberar sus

contenidos. Si una clase tiene `Ini()`, `End()` y `Clean()`, debe usarse ese ciclo de vida. XUtils no es solo una lista de clases: es una forma de escribir código dentro de GEN Framework.

21. Resumen práctico

XUtils es la base práctica de GEN Framework. Su nombre, `eXtended Utils`, refleja su origen como conjunto de utilidades extendidas que con el tiempo se convirtió en una infraestructura común. Es el punto donde se concentran las piezas que cualquier aplicación necesita: cadenas, buffers, rutas, ficheros, fechas, temporizadores, colecciones, eventos, trazas, configuración, hilos, serialización y soporte de sistema.

Su importancia no está en que cada clase sea compleja por separado. Su importancia está en la suma. Cuando todas las aplicaciones usan las mismas clases para resolver problemas comunes, el framework gana coherencia. Cuando las instancias dependientes de plataforma se crean mediante `XFACTORY`, el código de aplicación queda más limpio. Cuando los recursos se localizan mediante `XPATHSMANAGER`, el proyecto mantiene orden. Cuando la configuración se gestiona con `XFILECFG`, el INI deja de ser texto suelto y se convierte en parte del modelo de la aplicación.

Por eso este capítulo debe leerse como una guía de vocabulario. Antes de entrar en módulos más específicos como `DataIO`, `Graphic`, `Input`, `Sound`, `Cipher`, `Databases` o `UserInterface`, conviene dominar XUtils. Quien entiende XUtils entiende gran parte de la forma de pensar de GEN Framework.

Capítulo VI: DataIO, entrada y salida de datos en GEN Framework

1. El origen de DataIO

DataIO significa Data Input Output. El nombre describe con bastante precisión su función dentro de GEN Framework: agrupar las clases que permiten a una aplicación comunicarse con el exterior, recibir datos, enviar datos, hablar con dispositivos, acceder a redes, trabajar con protocolos, consultar servicios web, manejar buses de hardware y resolver operaciones de entrada y salida que en una aplicación real aparecen continuamente.

Si XUtils fue el germen inicial de GEN Framework, DataIO fue el segundo gran bloque que empezó a darle forma de framework. Las utilidades extendidas resolvían las necesidades comunes: cadenas, buffers, rutas, fechas, ficheros, trazas, eventos o estructuras de datos. Pero una aplicación no vive aislada. Antes o después tiene que abrir un puerto serie, escuchar un socket, conectarse a un servidor, consultar una URL, enviar un correo, leer un sensor, escribir sobre un bus I2C, enumerar dispositivos USB, recibir una trama UDP, sincronizar la hora por NTP o dialogar con un protocolo industrial. Ahí aparece DataIO.

La importancia de DataIO dentro de GEN Framework no está solo en la cantidad de clases que contiene. Está en la idea que representa: una aplicación no debería estar llena de llamadas dispersas a sockets del sistema, APIs de Bluetooth, funciones específicas de Linux, detalles de WinSock, manejadores de puertos serie o implementaciones particulares de cada bus. Todo eso debe estar organizado detrás de una capa común, con una forma coherente de configurar, abrir, cerrar, leer, escribir y diagnosticar comunicaciones.

Como ocurre en el resto de GEN Framework, DataIO no nace como una abstracción teórica. Sus clases existen porque antes existió una necesidad concreta. Muchas de ellas responden a proyectos reales: comunicaciones TCP/IP, instrumentación industrial, sensores, dispositivos I2C y SPI, actualizaciones remotas, alertas, protocolos de red, servidores web embebidos, scraping de servicios, consultas DNS o integración con hardware. Esa procedencia práctica explica por qué DataIO mezcla elementos de muy bajo nivel con protocolos de alto nivel: ambos extremos son necesarios en aplicaciones reales.

2. DataIO como segundo bloque de GEN Framework

DataIO se apoya de manera constante en XUtils. No inventa un nuevo tipo de cadena, ni una nueva clase de buffer, ni un sistema paralelo de eventos. Usa XSTRING para texto, XBUFFER para datos binarios, XPATH para rutas, XEVENT para eventos, XTHREAD para concurrencia, XTIMER para tiempos de espera y las factorías de XUtils para objetos dependientes de plataforma. Esto es importante porque mantiene la homogeneidad del framework: DataIO no es una librería aislada pegada al proyecto, sino una extensión natural del vocabulario de GEN Framework.

El programador que ya entiende XUtils encuentra en DataIO una continuación lógica. Un DIOSTREAM lee y escribe XBUFFER. Un DIOWEBCLIENT trabaja con DIOURL y XBUFFER. Un protocolo puede emitir eventos de GEN Framework. Una configuración de stream usa clases propias de DataIO, pero mantiene el estilo de ciclo de vida Ini(), End(), Open(), Close(), Read() y Write(). Esta continuidad reduce el coste mental de pasar de una parte del framework a otra.

3. DIOFACTORY y el aislamiento de plataforma

La pieza que marca la arquitectura de DataIO es DIOFACTORY. Igual que XFACTORY oculta la creación de determinados objetos de XUtils dependientes de plataforma, DIOFACTORY centraliza la creación de objetos de entrada y salida cuya implementación real puede cambiar entre Windows, Linux, Android, Raspberry Pi o un entorno embebido.

Un stream TCP/IP no se implementa igual en todas las plataformas. Un enumerador de dispositivos Bluetooth tampoco. El acceso a un puerto serie puede depender de APIs distintas. Un objeto de captura PCap o un gestor Wi-Fi puede necesitar código específico del sistema. La aplicación no debe decidir directamente qué clase concreta de plataforma instanciar. Esa decisión pertenece a la factoría.

La versión común de DIOFACTORY define la interfaz. Las versiones de plataforma especializan esa interfaz creando objetos concretos cuando la plataforma los soporta. Si un tipo de stream o dispositivo no está disponible, la factoría puede devolver NULL. Este comportamiento es deliberado: obliga a la aplicación a comprobar la disponibilidad y evita asumir que todas las plataformas tienen todas las capacidades.

```
DIOSTREAMTCIPCONFIG diostreamcfg;
DIOSTREAM*          diostream = NULL;

diostreamcfg.GetRemoteURL()->Set(__L("endorasoft.com"));
diostreamcfg.SetMode(DIOSTREAMMODE_CLIENT);
diostreamcfg.SetRemotePort(25);

diostream = GEN_DIOFACTORY.CreateStreamIO(&diostreamcfg);
if(!diostream) return false;

if(diostream->Open())
{
    bool status = diostream->WaitToConnected(5);

    if(status)
    {
        XBUFFER buffer;

        diostream->WaitToFilledReadingBuffer(30, 5);
        diostream->Read(buffer);
    }

    diostream->Close();
}

GEN_DIOFACTORY.DeleteStreamIO(diostream);
```

Este ejemplo resume la filosofía de DataIO: la aplicación configura lo que necesita, pide a la factoría un objeto capaz de hacerlo y trabaja con una interfaz común. La plataforma concreta queda fuera de la lógica de aplicación.

4. Mapa general de DataIO

DataIO es un bloque amplio. Para estudiarlo conviene agrupar las clases por responsabilidad. No es útil leer los ficheros en orden alfabético, porque eso mezcla conceptos muy diferentes. Es mejor distinguir entre utilidades de red, streams, protocolos, web, scraping, nodos, drivers, cámaras, notificaciones y módulos auxiliares.

Grupo	Responsabilidad principal	Clases representativas
Núcleo y utilidades	Direcciones IP, MAC, URL, email, DNS, conectividad, Wake on LAN, PCap, alertas, actualización y objetos generales.	DIOFACTORY, DIOIP, DIOMAC, DIOURL, DIODNSRESOLVER, DIOWAKEONLAN
Streams	Capa común de comunicaciones: abrir, cerrar, leer, escribir, esperar conexión, enumerar dispositivos y configurar el transporte.	DIOSTREAM, DIOSTREAMCONFIG, DIOSTREAMTCIP, DIOSTREAMUDP, DIOSTREAMUART, DIOSTREAMUSB, DIOSTREAMI2C, DIOSTREAMSPI
TLS	Capa de transporte seguro y estructuras internas de mensajes TLS.	DIOSTREAMTLS, DIOSTREAMTLSCONFIG, DIOSTREAMTSLMESSAGES
Protocolos	Protocolos contruidos sobre streams o web: NTP, SMTP, SNMP, ModBus, DNS, Ping, AT, IEC 60870-5, DynDNS.	DIONTP, DIOSMTP, DIOSNMP, DIOMODBUS_CLIENT, DIOPING, DIOATCMDS
Web	Cliente HTTP, servidor web, cabeceras, API REST, plugins y generación de páginas.	DIOWEBCLIENT, DIOWEBSERVER, DIOWEBHEADER, DIOWEBSERVER_APIREST
ScraperWeb	Obtención estructurada de datos desde servicios web concretos o páginas externas.	DIOSCRAPERWEB, DIOSCRAPERWEBPUBLICIP, DIOSCRAPERWEBWEATHER

Nodes	Modelo de nodos, items, valores, unidades, handlers y eventos para sistemas de control.	DIONODE, DIONODEITEM, DIONODEITEMVALUE, DIONODEMANAGER
Drivers	Dispositivos concretos de hardware y buses I2C/SPI.	DIOI2CGPIOMCP2317, DIOSPITFTDISPLAYST7789, DIOLEDNEOPIXELWS2812B

5. Direcciones, identificadores y utilidades de red

El primer grupo de DataIO contiene clases que parecen pequeñas, pero que evitan que la aplicación trabaje con texto libre para conceptos de red. DIOIP representa una dirección IP. DIOMAC representa una dirección MAC. DIOURL representa una URL con sus partes. DIOEMAILADDRESS encapsula una dirección de correo. DIODNSRESOLVER resuelve nombres. DIOPUBLICINTERNETIP obtiene la IP pública mediante mecanismos externos. DIOWAKEONLAN construye el paquete mágico para despertar un equipo.

```
DIOURL url;
DIOIP ip;

url.Set(__L("www.genframework.com"));

if(GEN_DIODNSRESOLVER.ResolveURL(url.Get(), ip))
{
    XSTRING iptext;

    ip.GetXString(iptext);
    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, iptext.Get());
}
```

Estas clases tienen relación directa con streams y protocolos. Un DIOSTREAMTCPIPCONFIG puede usar una URL o una IP remota. Un protocolo NTP usa DIOURL para indicar el servidor horario. Wake on LAN necesita DIOMAC y DIOIP para construir el envío. La ventaja está en que los conceptos de red pasan a ser objetos del framework, no cadenas sueltas dispersas por el código.

6. Streams: la abstracción principal de comunicación

DIOSTREAM es una de las clases más importantes de DataIO. Representa la idea de canal de comunicación. Un stream puede ser TCP/IP, UDP, UART, USB, Bluetooth, I2C, SPI, ICMP o TLS, pero la aplicación intenta trabajar siempre con la misma idea: configurar, abrir, esperar estado, leer, escribir, vaciar buffers y cerrar.

La configuración se separa en clases específicas. DIOSTREAMCONFIG contiene la base común.

DIOSTREAMTCPIPCONFIG añade URL, puertos y modo cliente/servidor. DIOSTREAMUDPConfig, DIOSTREAMUARTCONFIG, DIOSTREAMUSBConfig, DIOSTREAMI2CConfig y DIOSTREAMSPICConfig adaptan esa idea al transporte concreto. Esta separación es importante porque evita constructores enormes llenos de parámetros y permite pasar la configuración a DIOFACTORY para que cree el stream adecuado.

```
DIOSTREAMUDPConfig udpcfg;
DIOSTREAM*          udpstream = NULL;

udpcfg.SetMode(DIOSTREAMMODE_CLIENT);
udpcfg.GetRemoteURL()->Set(__L("192.168.1.50"));
udpcfg.SetRemotePort(5000);

udpstream = GEN_DIOFACTORY.CreateStreamIO(&udpcfg);
if(udpstream && udpstream->Open())
{
    udpstream->WriteStr(__L("GEN UDP test"));
    udpstream->WaitToFlushOutXBuffer();
    udpstream->Close();
}

GEN_DIOFACTORY.DeleteStreamIO(udpstream);
```

El uso de DIOSTREAM también permite que protocolos superiores no dependan directamente del transporte. Un protocolo puede recibir un DIOSTREAM* y hablar por él sin saber si debajo hay TCP/IP, UART o USB, siempre que las operaciones necesarias estén soportadas.

7. Enumeración de dispositivos y descriptores

En comunicaciones reales no basta con abrir un puerto conocido. Muchas veces hay que descubrir qué dispositivos existen. Para eso DataIO incluye clases de enumeración y descriptores. DIOSTREAMENUMDEVICES es la base de los enumeradores. A partir de ella aparecen enumeradores de dispositivos USB, UART, Bluetooth, Bluetooth LE, Wi-Fi o servidores UDP. Los objetos DIOSTREAMDEVICE y sus derivados representan la información encontrada: nombre, identificador, dirección, tipo de conexión o datos específicos de cada tecnología.

```
DIOSTREAMENUMDEVICES* enumdevices = NULL;

enumdevices = GEN_DIOFACTORY.CreateStreamEnumDevices(DIOSTREAMENUMTYPE_BLUETOOTH_REMOTE);
if(enumdevices)
{
    enumdevices->Search();

    for(int c=0; c<enumdevices->GetDevices()->GetSize(); c++)
    {
        DIOSTREAMDEVICE* device = enumdevices->GetDeviceByIndex(c);
        if(device) XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, device->GetName()->Get());
    }

    GEN_DIOFACTORY.DeleteStreamEnumDevices(enumdevices);
}
```

De nuevo aparece la factoría: la forma de enumerar dispositivos Bluetooth en Windows no tiene por qué coincidir con Linux. La aplicación pide un enumerador por tipo; la plataforma decide si puede ofrecerlo y cómo lo implementa.

8. TLS y streams seguros

La carpeta TLS añade una capa importante: transporte seguro. DIOSTREAMTLS y DIOSTREAMTLSCONFIG permiten trabajar con comunicaciones cifradas sobre la infraestructura de streams. Las clases DIOSTREAMTLSMESSAGES, DIOSTREAMTLSMESSAGESEXTENSION, DIOSTREAMTLSMESSAGESHANDSHAKECLIENTHELLO y DIOSTREAMTLSMESSAGESHANDSHAKESERVERHELLO representan partes internas del protocolo TLS y ayudan a analizar o construir mensajes de negociación.

Este grupo es especialmente interesante porque muestra hasta dónde llega DataIO: no se limita a llamar a una biblioteca externa, sino que estructura conceptos del protocolo dentro del estilo de GEN Framework. Esto permite depurar, inspeccionar mensajes, trabajar con extensiones y mantener una visión controlada de lo que ocurre durante el handshake.

```
DIOSTREAMTLSCONFIG tlsconfig;
DIOSTREAM*         tlsstream = NULL;

tlsconfig.SetMode(DIOSTREAMMODE_CLIENT);
tlsconfig.GetRemoteURL()->Set(__L("www.genframework.com"));
tlsconfig.SetRemotePort(443);

tlsstream = GEN_DIOFACTORY.CreateStreamIO(&tlsconfig);
if(tlsstream && tlsstream->Open())
{
    if(tlsstream->WaitToConnected(10))
    {
        {
            tlsstream->WriteStr(__L("GET / HTTP/1.1\r\nHost: www.genframework.com\r\n\r\n"));
        }

        tlsstream->Close();
    }
}
```

```
GEN_DIOFACTORY.DeleteStreamIO(tlsstream);
```

9. Protocolos de alto nivel

Encima de los streams aparecen los protocolos. DatalO incluye protocolos de red generales, protocolos industriales y protocolos de integración. Algunos son sencillos, como DIONTP para obtener la hora UTC o DIOPING para comprobar alcance. Otros son más específicos, como DIOMODBUS_CLIENT, DIOMODBUSELECTRICMETER o DIOIEC60870_5. Otros resuelven necesidades de comunicación cotidiana, como DIOSMTP para correo, DIOSNMP para gestión de dispositivos, DIODYNDNS para actualización de DNS dinámico o DIOATCMDS y DIOATCMDGSM para módems GSM.

```
DIONTP*    ntp = GEN_NEW DIONTP();
DIOURL     url;
XDATETIME* datetime = GEN_XFACTORY.CreateDateTime();

url = __L("1.es.pool.ntp.org");

if(ntp && datetime)
{
    ntp->GetTime(url, DIONTP_DEFAULTTIMEOUT, GEN_XSYSTEM.HardwareUseLittleEndian(), (*datetime));
}

GEN_DELETE ntp;
GEN_XFACTORY.DeleteDateTime(datetime);
```

Los protocolos son un buen ejemplo de la filosofía práctica de GEN Framework. No todos tienen el mismo nivel de abstracción ni todos resuelven problemas genéricos. Algunos existen porque un proyecto necesitaba hablar con un medidor eléctrico, enviar un comando AT a un módem o sincronizar un dispositivo sin depender de servicios externos del sistema. Ese es el valor de DatalO: convertir necesidades concretas en piezas reutilizables.

10. DIOCoreProtocol y DIOProtocol

Dentro de Protocols hay dos familias que merecen una explicación separada. DIOCoreProtocol representa un protocolo propio de núcleo, con cabecera, mensajes, configuración, registro de datos, conexiones y eventos. DIOProtocol, dentro de Protocols/Protocol, representa una base de protocolo binario genérico, con datos de aplicación, gestor de conexiones y eventos asociados.

Estas clases son útiles cuando una aplicación no solo consume protocolos existentes, sino que necesita definir su propio protocolo de comunicación. En lugar de dispersar cabeceras, estados, conexiones y mensajes por distintas partes del programa, el framework ofrece una base para organizar esas piezas con el estilo DatalO.

```
// Esquema conceptual de un protocolo sobre un stream

DIOSTREAM* stream = GEN_DIOFACTORY.CreateStreamIO(&cfg);
if(stream && stream->Open())
{
    DIOPROTOCOL* protocol = GEN_NEW DIOPROTOCOL();

    // El protocolo organiza tramas, datos de aplicación y eventos.
    // La aplicación trabaja con mensajes de mayor nivel, no con bytes sueltos.

    GEN_DELETE protocol;
    stream->Close();
}

GEN_DIOFACTORY.DeleteStreamIO(stream);
```

11. Web, cliente HTTP y servidor embebido

La carpeta Web agrupa clases para trabajar con HTTP desde dos puntos de vista. DIOWEBCLIENT permite realizar operaciones GET, PUT y POST sobre una URL y almacenar la respuesta en un XBUFFER o en un fichero.

DIOWEBHEADER representa cabeceras HTTP. DIOWEBSERVER implementa un servidor web. A su alrededor aparecen API REST, plugins, CGI, PHP, resolución de endpoints, cadenas de consulta y eventos.

```
DIOWEBCLIENT webclient;
DIOURL        url;
XBUFFER       response;

url.Set(__L("https://www.genframework.com"));

if(webclient.Get(url, response))
{
    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, __L("Respuesta HTTP recibida"));
}
```

El servidor web es especialmente útil cuando una aplicación necesita exponer una interfaz de control, diagnóstico o configuración sin depender de un servidor externo. En aplicaciones industriales, embebidas o de automatización, un pequeño servidor HTTP integrado puede ser una forma muy práctica de consultar estado, lanzar comandos o integrar la aplicación con otros sistemas.

12. ScraperWeb y obtención de información externa

ScraperWeb es un bloque de DataIO orientado a obtener información desde recursos web concretos.

DIOSCRAPERWEB actúa como base. DIOSCRAPERWEBCACHE añade caché. Las especializaciones resuelven necesidades concretas: geolocalización por IP, fabricante de una MAC, IP pública, detección de sexo por nombre, traducción, identificación de user agent o información meteorológica.

Este bloque debe entenderse con prudencia. No sustituye a una API estable cuando existe una API oficial, pero puede ser muy útil cuando un proyecto necesita obtener un dato externo de forma automatizada y controlada. La caché es importante porque evita repetir consultas innecesarias y reduce dependencia de la red.

```
DIOSCRAPERWEBPUBLICIP publicip;
XSTRING                ip;

if(publicip.Get(ip))
{
    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, ip.Get());
}
```

13. Nodes: nodos, items, valores y handlers

El grupo Nodes organiza una idea habitual en sistemas de control: representar nodos que contienen items, valores, unidades y manejadores. Un nodo puede representar un dispositivo o una entidad lógica. Un item puede ser una entrada, una salida, un sensor o un actuador. DIONODEITEMVALUE almacena el valor;

DIONODEITEMVALUEUNITFORMAT ayuda a representar unidades; DIONODEITEMHANDLER y sus derivados conectan el item con una fuente real, como un GPIO digital o un sensor AM2315.

Este modelo es útil cuando una aplicación debe tratar de forma homogénea muchos puntos de entrada y salida. En vez de escribir lógica específica para cada sensor o actuador, se puede organizar un árbol de nodos, items y valores, y dejar que los handlers adapten cada caso concreto.

```
DIONODE    node;
DIONODEITEM item;
DIONODEITEMVALUE value;

item.GetName()->Set(__L("temperature"));
value.Set(23.5f);
```

```
// El nodo agrupa items; cada item puede tener valor, formato y handler asociado.
```

14. Drivers, I2C, SPI y dispositivos físicos

DataIO incluye drivers para hardware concreto. Algunos son genéricos o de apoyo, como DIOBUSPIRATE, DIOMPSSE, DIOGPIO, DIOLEDNEOPIXELWS2812B, DIOPIFACE o DIOTEMHUMSENSORAM2301. Otros se agrupan bajo I2C y SPI y representan integrados o dispositivos concretos: expansores GPIO, pantallas OLED, pantallas TFT, sensores de luz, sensores de temperatura y humedad, EEPROM, controladores PWM, cargadores de batería, conversores AD/DA o controladores táctiles.

Estos drivers muestran una de las facetas más importantes de GEN Framework: el framework no está pensado solo para aplicaciones de escritorio. También sirve para aplicaciones cercanas al hardware. Un programa puede usar la misma arquitectura general de GEN Framework y, al mismo tiempo, hablar con un display por SPI, un sensor por I2C o una cadena de LEDs WS2812B.

```
DIOI2CGPIOMCP2317* mcp2317 = GEN_NEW DIOI2CGPIOMCP2317();

if(mcp2317 && mcp2317->Ini(1, 0x23, 10))
{
    XBYTE value = 0;

    mcp2317->Configure();
    mcp2317->ReadInputs(DIOI2CGPIOMCP2317PORT_A, value);
    mcp2317->WriteOutputs(DIOI2CGPIOMCP2317PORT_B, 0xFF);

    mcp2317->End();
}

GEN_DELETE mcp2317;
```

El ejemplo anterior, basado en las pruebas de DevTestsConsole, ilustra la forma de trabajo habitual: crear el driver, inicializarlo con bus, dirección y timeout, configurar el dispositivo, leer o escribir registros, finalizar y liberar. El detalle exacto cambia para cada integrado, pero el patrón de ciclo de vida es coherente.

15. Cámaras, notificaciones, alertas y actualización

Otros módulos de DataIO resuelven necesidades de aplicación. DIOCAMERA abstrae cámaras y emite eventos mediante DIOCAMERA_XEVENT. DIONOTIFICATIONSMANAGER agrupa notificaciones del sistema. DIOALERTS permite enviar alertas por distintos canales, y DIOALERTSUDPSERVER puede recibir alertas por UDP. DIOAPPLICATIONUPDATE se orienta a la actualización de aplicaciones y utiliza eventos para informar de su progreso. DIORemoteFileCFG permite trabajar con configuración remota.

Estos módulos conectan DataIO con AppFlow. Una aplicación real no solo abre comunicaciones; también necesita avisar, actualizarse, publicar incidencias, recibir señales externas o exponer estado. DataIO proporciona las piezas de entrada y salida; AppFlow las integra en el ciclo de vida de la aplicación.

```
DIONOTIFICATIONSMANAGER* notifications = GEN_DIOFACTORY.CreateNotificationsManager();

if(notifications)
{
    DIONOTIFICATION notification;

    notification.GetTitle()->Set(__L("GEN"));
    notification.GetMessage()->Set(__L("Proceso terminado"));

    notifications->Send(notification);

    GEN_DIOFACTORY.DeleteNotificationsManager(notifications);
}
```

16. Diagnóstico, conectividad y seguridad operativa

DataIO incluye clases auxiliares para diagnosticar el estado de la comunicación. DIOCHECKINTERNETCONNECTION comprueba conectividad general. DIOCHECKTCPIPCONNECTIONS revisa conexiones TCP/IP. DIOPCAP y DIOPCAP_FILTERS permiten trabajar con captura de paquetes cuando la plataforma y las dependencias lo soportan. DIOTRACEINTERSTREAMS conecta el sistema de trazas con streams de DataIO. DIOPIPELINE permite canalizar entrada y salida de procesos del sistema operativo.

También existen clases de integración más especializadas, como DIOSSHREVERSE, DIOBACKDOOR o DIONATIVEMSGBROWSEREXTENSION. Estas piezas no son de uso cotidiano en todas las aplicaciones, pero muestran la amplitud de DataIO: comunicación, diagnóstico, integración con navegador, ejecución externa y conexión inversa forman parte del mismo ecosistema de entrada y salida.

17. Relación con XUtils, AppFlow y las demás factorías

DataIO no debe verse como un bloque aislado. XUtils le proporciona tipos, buffers, rutas, eventos, hilos, temporizadores y trazas. AppFlow lo usa para integrar comunicaciones dentro de aplicaciones de consola o gráficas. UserInterface puede utilizar DataIO para mostrar estado recibido desde sensores, protocolos o servicios web. Graphic puede renderizar datos obtenidos por DataIO. Cipher puede cifrar información antes de enviarla o integrarse con streams seguros. La arquitectura modular de GEN Framework se basa precisamente en estas relaciones controladas.

La clave es que DataIO proporciona la frontera con el exterior. Todo lo que entra o sale de la aplicación suele terminar pasando por este bloque o por una clase relacionada con él. Por eso conviene entender bien sus patrones: configuración separada, creación mediante DIOFACTORY cuando hay plataforma, lectura y escritura con XBUFFER, eventos para notificar cambios, y liberación simétrica mediante la misma factoría o el mismo ciclo de vida que creó el objeto.

18. Catálogo de clases y módulos de DataIO

La siguiente referencia resume las clases encontradas en el bloque DataIO. No sustituye a la documentación Doxygen ni a la lectura de los ficheros .h, pero sirve como mapa práctico. En la columna de relación se indica con qué otras piezas suele convivir la clase o qué patrón de uso resulta natural.

Núcleo, utilidades y servicios generales

Clase / módulo	Qué aporta	Relación y uso típico
DIOALERTS	Sistema de alertas de DataIO.	Se integra con XUtils y con el resto de módulos DataIO.
DIOALERTSUDPSERVER	Servidor UDP para recibir o distribuir alertas.	Se integra con XUtils y con el resto de módulos DataIO.
DIOALERTS_CONDITIONS	Definición de condiciones usadas por el sistema de alertas.	Se integra con XUtils y con el resto de módulos DataIO.
DIOAPPLICATIONUPDATE	Gestión de actualización de aplicación.	Se integra con XUtils y con el resto de módulos DataIO.
DIOAPPLICATIONUPDATE_XEVENT	Evento extendido asociado a la actualización de aplicación.	Transporta eventos hacia observadores de GEN Framework.
DIOBACKDOOR	Soporte para conexiones SSH inversas en Linux.	Se integra con XUtils y con el resto de módulos DataIO.
DIOCHECKINTERNETCONNECTION	Comprobación de conectividad general a Internet.	Se integra con XUtils y con el resto de módulos DataIO.
DIOCHECKTCPIPCONNECTIONS	Comprobación de conexiones TCP/IP abiertas o disponibles.	Se integra con XUtils y con el resto de módulos DataIO.
DIODNSRESOLVER	Resolución de nombres DNS a direcciones IP.	Se integra con XUtils y con el resto de módulos DataIO.

DIODEVICE	Dispositivo genérico de DataIO.	Se integra con XUtils y con el resto de módulos DataIO.
DIODEVICEI2C	Dispositivo genérico conectado por I2C.	Se integra con XUtils y con el resto de módulos DataIO.
DIODEVICESPI	Dispositivo genérico conectado por SPI.	Se integra con XUtils y con el resto de módulos DataIO.
DIODISPLAYDEVICE	Dispositivo de pantalla genérico.	Se integra con XUtils y con el resto de módulos DataIO.
DIODYDNS_MANAGER	Gestor de servicios DynDNS.	Se integra con XUtils y con el resto de módulos DataIO.
DIOEMAILADDRESS	Dirección de correo electrónico.	Tipo de apoyo usado por streams, protocolos y configuración.
DIOFACTORY	Factoría de creación de objetos DataIO dependientes de plataforma.	Entrada principal para crear objetos dependientes de plataforma.
DIOGPIO	Acceso GPIO de propósito general.	Se integra con XUtils y con el resto de módulos DataIO.
DIOIP	Representación de una dirección IP.	Tipo de apoyo usado por streams, protocolos y configuración.
DIOMAC	Representación de una dirección MAC.	Tipo de apoyo usado por streams, protocolos y configuración.
DIOMACMANUFACTURED	Información del fabricante asociado a una MAC.	Se integra con XUtils y con el resto de módulos DataIO.
DIONATIVEMSGBROWSEREXTENSION	Canal de mensajería nativa con extensiones de navegador.	Se integra con XUtils y con el resto de módulos DataIO.
DIONOTIFICATIONSMANAGER	Gestor de notificaciones del sistema o de aplicación.	Se integra con XUtils y con el resto de módulos DataIO.
DIOPIPELINE	Canalización de entrada/salida con procesos del sistema operativo.	Se integra con XUtils y con el resto de módulos DataIO.
DIOPCAP	Integración con captura de paquetes PCap.	Se integra con XUtils y con el resto de módulos DataIO.
DIOPCAP_FILTERS	Definición y gestión de filtros para PCap.	Se integra con XUtils y con el resto de módulos DataIO.
DIOPUBLICINTERNETIP	Obtención de la IP pública de salida a Internet.	Se integra con XUtils y con el resto de módulos DataIO.
DIOREMOTEFILECFG	Configuración remota basada en fichero.	Se integra con XUtils y con el resto de módulos DataIO.
DIOSSHREVERSE	Gestión de túnel o conexión SSH inversa.	Se integra con XUtils y con el resto de módulos DataIO.
DIOURL	Representación y manipulación de una URL.	Tipo de apoyo usado por streams, protocolos y configuración.
DIOWAKEONLAN	Construcción y envío de paquetes Wake on LAN.	Se integra con XUtils y con el resto de módulos DataIO.
DIOWIFIMANAGERMODE	Gestión de modos Wi-Fi cuando la plataforma lo soporta.	Se integra con XUtils y con el resto de módulos DataIO.
DIOXTRACEINTERSTREAMS	Interconexión del sistema de trazas mediante streams.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.

Streams, configuración y enumeración de dispositivos

Clase / módulo	Qué aporta	Relación y uso típico
DIOSTREAM	Stream base de DataIO.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMBLUETOOTH	Implementación de stream BLUETOOTH.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMBLUETOOTHCONFIG	Configuración del stream BLUETOOTH.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMBLUETOOTHLEREMOTEENUMDEVICES	Enumerador remoto de dispositivos BLUETOOTHLE.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMBLUETOOTHLEREMOTEENUMDEVICES_X_EVENT	Evento extendido asociado al módulo correspondiente.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMBLUETOOTHLOCALENUMDEVICES	Enumerador local de dispositivos BLUETOOTH.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.

DISTREAMBLUETOOTHREMOTEENUMDEVICES	Enumerador remoto de dispositivos BLUETOOTH.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMBLUETOOTH_XEVENTS	Evento extendido asociado al módulo correspondiente.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMCIPHER	Stream con capa de cifrado.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMCONFIG	Configuración base de streams.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMCONFIGSTRING	Conversión de configuración de stream a cadena y viceversa.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMDEVICE	Descriptor base de dispositivo de stream.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMDEVICEBLUETOOTH	Descriptor de dispositivo de stream BLUETOOTH.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMDEVICEBLUETOOTHLE	Descriptor de dispositivo de stream BLUETOOTHLE.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMDEVICEIP	Descriptor de dispositivo de stream IP.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMDEVICEUSB	Descriptor de dispositivo de stream USB.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMDEVICEWIFI	Descriptor de dispositivo de stream WIFI.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMENUMBLUETOOTHDEVICES	Enumerador de dispositivos o servidores asociados a streams.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMENUMDEVICES	Enumerador de dispositivos o servidores asociados a streams.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMENUMSERVERS	Enumerador de dispositivos o servidores asociados a streams.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMI2C	Implementación de stream I2C.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMI2CONFIG	Configuración del stream I2C.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMICMP	Implementación de stream ICMP.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMICMPCONFIG	Configuración del stream ICMP.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMIPLOCALENUMDEVICES	Enumerador local de dispositivos IP.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMSPI	Implementación de stream SPI.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMSPICONFIG	Configuración del stream SPI.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMTCPIP	Implementación de stream TCP/IP.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMTCPIPCONFIG	Configuración del stream TCP/IP.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMTCPIPSEVER	Servidor TCP/IP multi-socket.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUART	Implementación de stream UART.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUARTCONFIG	Configuración del stream UART.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUARTLOCALENUMDEVICES	Enumerador local de dispositivos UART.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUDP	Implementación de stream UDP.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUDPACKNOWLEDGE	Stream UDP con confirmación de recepción.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUDPCONFIG	Configuración del stream UDP.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUDPLOCALENUMSERVERS	Enumerador de dispositivos o servidores asociados a streams.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUSB	Implementación de stream USB.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.
DISTREAMUSBCONFIG	Configuración del stream USB.	Se apoya en DISTREAMCONFIG, DISTREAMDEVICE y DIOFACTORY.

		y DIOFACTORY.
DIOSTREAMUSBLOCALENUMDEVICES	Enumerador local de dispositivos USB.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMWIFIREMOTEENUMDEVICES	Enumerador remoto de dispositivos WIFI.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAM_XEVENT	Evento extendido asociado al módulo correspondiente.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.

TLS y mensajes de transporte seguro

Clase / módulo	Qué aporta	Relación y uso típico
DIOSTREAMTLS	Stream TLS para transporte seguro.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMTLSCONFIG	Configuración específica de un stream TLS.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMTLSMESSAGES	Estructura auxiliar para mensajes y extensiones TLS.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMTLSMESSAGE	Estructura auxiliar para mensajes y extensiones TLS.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMTLSMESSAGESHANDSHAKECLIENTHELLO	Mensaje TLS ClientHello usado durante el handshake.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.
DIOSTREAMTLSMESSAGESHANDSHAKESERVERHELLO	Mensaje TLS ServerHello usado durante el handshake.	Se apoya en DIOSTREAMCONFIG, DIOSTREAMDEVICE y DIOFACTORY.

Protocolos específicos y de alto nivel

Clase / módulo	Qué aporta	Relación y uso típico
DIOCLIPROTOCOL	Protocolo orientado a línea de comandos.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCLIPROTOCOLBUS	Bus para protocolo CLI.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL	Protocolo núcleo propio de DataIO.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL_CFG	Configuración del protocolo núcleo.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL_CONNECTION	Conexión individual del protocolo núcleo.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL_CONNECTIONSMANAGER	Gestor de conexiones del protocolo núcleo.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL_CONNECTIONSMANAGER_XEVENT	Evento extendido asociado al módulo correspondiente.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL_HEADER	Cabecera del protocolo núcleo.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL_MESSAGES	Gestión de mensajes del protocolo núcleo.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL_REGISTERDATA	Registro de datos del protocolo núcleo.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOCOREPROTOCOL_XEVENT	Evento extendido asociado al módulo correspondiente.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOATCMDGSM	Especialización de comandos AT para módulos GSM.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOATCMD	Base para protocolos basados en comandos AT.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIODNSPROTOCOL	Estructuras del protocolo DNS.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIODNSPROTOCOL_CLIENT	Cliente del protocolo DNS.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIODNSPROTOCOL_MITM_SERVER	Servidor DNS MitM para interceptación o filtrado.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIODNSPROTOCOL_MITM_SERVER_XEVENT	Evento extendido asociado al módulo	Normalmente trabaja encima de DIOSTREAM o

	correspondiente.	DIOWEBCLIENT.
DIODYDNS	Protocolo de actualización DynDNS.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOIEC60870_5	Protocolo IEC 60870-5 para entornos eléctricos e industriales.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOMODBUSELECTRICMETER	Acceso ModBus especializado para contador eléctrico.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOMODBUS_CLIENT	Cliente del protocolo ModBus.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIONTP	Cliente NTP para obtención de hora UTC.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOOBEXPUSH	Protocolo OBEX Push.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOPING	Comprobación de alcance mediante ping.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOPING_XEVENT	Evento extendido asociado al módulo correspondiente.	Transporta eventos hacia observadores de GEN Framework.
DIOSMTP	Cliente SMTP para envío de correo.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOSNMP	Soporte SNMP para gestión de dispositivos.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.

Base de protocolo binario genérico

Clase / módulo	Qué aporta	Relación y uso típico
DIOPROTOCOL	Base de protocolo binario genérico.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOPROTOCOLCONNECTIONSINUSEDEFINITIONS	Definiciones de conexiones en uso para DIOProtocol.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOPROTOCOL_APPLICATIONDATA	Datos de aplicación asociados a DIOProtocol.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOPROTOCOL_CONNECTIONSMANAGER	Gestor de conexiones de DIOProtocol.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOPROTOCOL_CONNECTIONSMANAGER_XEVENT	Evento extendido asociado al módulo correspondiente.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.
DIOPROTOCOL_XEVENT	Evento extendido asociado al módulo correspondiente.	Normalmente trabaja encima de DIOSTREAM o DIOWEBCLIENT.

Cliente, servidor web y componentes HTTP

Clase / módulo	Qué aporta	Relación y uso típico
DIOWEBCLIENT	Cliente HTTP para operaciones GET, PUT y POST.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBCLIENT_XEVENT	Evento extendido asociado al módulo correspondiente.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBHEADER	Cabecera HTTP común.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBPAGEHTMLCREATOR	Generador de páginas HTML.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBPAGEHTMLCREATOR_XEVENT	Evento extendido asociado al módulo correspondiente.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBSERVER	Servidor web integrado.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBSERVER_APIREST	Soporte de API REST sobre DIOWebServer.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBSERVER_PLUGIN	Base de plugin para el servidor web.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBSERVER_PLUGIN_CGI	Plugin CGI para el servidor web.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBSERVER_PLUGIN_PHP	Plugin PHP para el servidor web.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.

		TCP/IP.
DIOWEBSEVER_PLUGINSMANAGER	Gestor de plugins del servidor web.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBSEVER_QUERYSTRINGS	Gestión de query strings HTTP.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBSEVER_RESOLVEENDPOINT	Resolución de endpoints del servidor web.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.
DIOWEBSEVER_XEVENT	Evento extendido asociado al módulo correspondiente.	Trabaja con DIOURL, DIOWEBHEADER y streams TCP/IP.

ScraperWeb y servicios externos

Clase / módulo	Qué aporta	Relación y uso típico
DIOSCRAPERWEB	Base para extracción de información desde recursos web.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.
DIOSCRAPERWEBCACHE	Caché para resultados de ScraperWeb.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.
DIOSCRAPERWEBGEOLOCATIONIP	Scraper web especializado en geolocationip.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.
DIOSCRAPERWEBMACMANUFACTURER	Scraper web especializado en macmanufacturer.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.
DIOSCRAPERWEBPUBLICIP	Scraper web especializado en publicip.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.
DIOSCRAPERWEBSEXNAME	Scraper web especializado en sexname.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.
DIOSCRAPERWEBTRANSLATION	Scraper web especializado en translation.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.
DIOSCRAPERWEBUSERAGENTID	Scraper web especializado en useragentid.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.
DIOSCRAPERWEBWEATHER	Scraper web especializado en weather.	Especialización sobre DIOSCRAPERWEB y DIOWEBCLIENT.

Modelo de nodos y valores

Clase / módulo	Qué aporta	Relación y uso típico
DIONODE	Nodo base para sistemas de control.	Forma parte del modelo de nodos, items, valores y eventos.
DIONODEITEM	Item asociado a un nodo.	Forma parte del modelo de nodos, items, valores y eventos.
DIONODEITEMHANDLER	Handler base para enlazar un item con una fuente real.	Forma parte del modelo de nodos, items, valores y eventos.
DIONODEITEMHANDLER_GPIODIGITAL	Handler de item para GPIO digital.	Forma parte del modelo de nodos, items, valores y eventos.
DIONODEITEMHANDLER_SENSORAM2315	Handler de item para sensor AM2315.	Forma parte del modelo de nodos, items, valores y eventos.
DIONODEITEMVALUE	Valor asociado a un item de nodo.	Forma parte del modelo de nodos, items, valores y eventos.
DIONODEITEMVALUEUNITFORMAT	Formato y unidad de un valor de nodo.	Forma parte del modelo de nodos, items, valores y eventos.
DIONODEMANAGER	Gestor de nodos.	Forma parte del modelo de nodos, items, valores y eventos.
DIONODE_XEVENT	Evento extendido asociado al módulo correspondiente.	Forma parte del modelo de nodos, items, valores y eventos.

Drivers generales de hardware

Clase / módulo	Qué aporta	Relación y uso típico
DIOAIOBIMAINBOARD	AIOBI Main board.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un

		driver físico equivalente.
DIOBUSPIRATE	interface Bus Pirate 4.0.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOLEDNEOPIXELWS2812B	Led Neopixel WS2812B.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOLIGHTSENSORLDRANALOG	IO Sensor Light LDR (Light Dependent Resistor) Analog in port digital.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOMPSE	Multi-Protocol Synchronous Serial Engine (MPSE).	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOPIFACE	Raspberry Pi Face.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOTEMHUMSENSORAM2301	AOSONG AM2301 Sensor (Temperature and humidity module).	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.

Drivers I2C

Clase / módulo	Qué aporta	Relación y uso típico
DIOI2C6AXISTRACKINGBMI270	Driver I2C para 6AXISTRACKINGBMI270.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2C6AXISTRACKINGLSM303DLHC	Driver I2C para 6AXISTRACKINGLSM303DLHC.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2C9AXISTRACKINGMPU9150	Driver I2C para 9AXISTRACKINGMPU9150.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CADDACONVERTERPCF8591	Driver I2C para ADDACONVERTERPCF8591.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CAIRQUALITYCCS811	Driver I2C para AIRQUALITYCCS811.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CBATTERYCHARGERBQ24295	Driver I2C para BATTERYCHARGERBQ24295.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CEEPPROM24XXX	Driver I2C para EEPROM24XXX.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CGPIOMCP2317	Driver I2C para GPIOMCP2317.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CGPIOPCF8574	Driver I2C para GPIOPCF8574.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CLIGHTSENSORSL2561	Driver I2C para LIGHTSENSORSL2561.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CMONITORGAUGELTC2942	Driver I2C para MONITORGAUGELTC2942.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2COLEDDISPLAYSSD1306	Driver I2C para OLEDDISPLAYSSD1306.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CPWMCONTROLLERPCA9685	Driver I2C para PWMCONTROLLERPCA9685.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CTEMHUMSENSORAM2315	Driver I2C para TEMHUMSENSORAM2315.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CTEMHUMSENSORSHT20	Driver I2C para TEMHUMSENSORSHT20.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2CTOUCHSENSORAT42QT1060	Driver I2C para TOUCHSENSORAT42QT1060.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOI2C_DEVICES	Driver I2C para _DEVICES.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.

Drivers SPI

Clase / módulo	Qué aporta	Relación y uso típico
DIOSPIGPIOMCP23S17	Driver SPI para GPIOMCP23S17.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOSPILCDDISPLAYPCF8833	Driver SPI para LCDDISPLAYPCF8833.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOSPIOLEDDISPLAYSSD1306	Driver SPI para OLEDDISPLAYSSD1306.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.

		driver físico equivalente.
DIOSPIOLEDDISPLAYSSD1331	Driver SPI para OLEDDISPLAYSSD1331.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOSPI TFTDISPLAYILI9341	Driver SPI para TFTDISPLAYILI9341.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOSPI TFTDISPLAYST7789	Driver SPI para TFTDISPLAYST7789.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOSPI TOUCHSCREENSTMPE610	Driver SPI para TOUCHSCREENSTMPE610.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.
DIOSPI_DEVICES	Driver SPI para _DEVICES.	Se utiliza sobre DIOSTREAMI2C, DIOSTREAMSPI o un driver físico equivalente.

Cámaras y eventos de cámara

Clase / módulo	Qué aporta	Relación y uso típico
DIOCAMERA	Abstracción de cámara.	Se integra con XUtils y con el resto de módulos DataIO.
DIOCAMERA_XEVENT	Evento extendido emitido por una cámara.	Transporta eventos hacia observadores de GEN Framework.

Localización y direcciones

Clase / módulo	Qué aporta	Relación y uso típico
DIOLOCATIONADDRESS	Contenedor de dirección y datos de localización.	Tipo de apoyo usado por streams, protocolos y configuración.

19. Cómo estudiar DataIO desde DevTestsConsole

DevTestsConsole es el mejor complemento para estudiar DataIO porque contiene pruebas reales de uso. En sus fuentes aparecen ejemplos de DIOSTREAMTCPIP, DIOSTREAMTLS, DIOSStreamUSB, enumeración de Bluetooth, DIONTP, DIODNSResolver, DIOWakeOnLAN, DIOWifiManagerMode, DIONotificationsManager, DIOATCMDGSM, DIOSNMP, drivers I2C y drivers SPI. No todas las pruebas estarán activadas en todas las plataformas, pero el código muestra el patrón que debe aprenderse.

El orden recomendado para estudiar una clase de DataIO es el mismo que en otros módulos de GEN Framework. Primero se revisa la cabecera para ver qué API ofrece. Después se consulta la documentación Doxygen para entender herencia, métodos y relaciones. Por último, se busca una prueba en DevTestsConsole o en los ejemplos oficiales para ver cómo se crea, inicializa y libera el objeto en una aplicación real.

En DataIO es especialmente importante revisar también las macros de compilación. Muchos módulos están condicionados por flags como DIO_STREAMTCPIP_ACTIVE, DIO_STREAMUDP_ACTIVE, DIO_STREAMI2C_ACTIVE, DIO_STREAMSPI_ACTIVE, DIO_PCAP_ACTIVE o equivalentes. Que una clase exista en el código fuente no significa que esté activa en todas las compilaciones ni que la plataforma pueda crearla.

20. Criterios de uso en una aplicación con GEN Framework

Cuando una aplicación necesita comunicarse con el exterior, la regla general es no saltarse DataIO. Si hay que abrir una conexión TCP/IP, se debe usar DIOSTREAMTCPIP y DIOFACTORY. Si hay que enviar UDP, se usa DIOSTREAMUDP. Si hay que hablar con un puerto serie, se usa DIOSTREAMUART. Si hay que consultar una URL, se usa DIOWEBCLIENT. Si hay que obtener hora de red, se usa DIONTP. Si hay que tratar una IP, una MAC o una URL, se usan DIOIP, DIOMAC y DIOURL.

Esta disciplina evita que cada aplicación invente su propia forma de resolver los mismos problemas. Además, permite que el código pueda moverse de una plataforma a otra con menos cambios. Si una aplicación llama directamente a una API nativa, se acopla a esa plataforma. Si usa DataIO, la dependencia queda confinada en la capa de plataforma y en la factoría correspondiente.

También hay que recordar la simetría de propiedad. Lo que se crea con GEN_DIOFACTORY debe destruirse con GEN_DIOFACTORY. Lo que se crea con GEN_NEW debe liberarse con GEN_DELETE o con el patrón de la clase correspondiente. Un stream debe cerrarse antes de destruirse. Un driver debe ejecutar End() si fue inicializado. Este orden no es solo estilo: en comunicaciones y hardware evita dejar recursos abiertos, sockets colgados, buses ocupados o hilos sin finalizar.

21. Resumen práctico

DataIO es el segundo gran bloque que dio forma a GEN Framework. Su nombre, Data Input Output, resume su responsabilidad: organizar la entrada y salida de datos de una aplicación de forma portable, coherente y reutilizable.

Su centro arquitectónico es DIOFACTORY, que permite aislar las diferencias de plataforma en la creación de streams, enumeradores, gestores, PCap, Wi-Fi, notificaciones y otros objetos dependientes del sistema. Encima de esa base aparecen streams, protocolos, web, scraping, nodos, drivers y servicios auxiliares.

La aplicación que usa DataIO correctamente gana orden. En lugar de mezclar llamadas nativas, sockets, buffers, timeouts y protocolos en cualquier parte del código, concentra la comunicación en clases del framework. Esto hace que el código sea más fácil de leer, más fácil de portar y más fácil de diagnosticar.

Después de XUtils, DataIO es el bloque que más claramente demuestra la vocación práctica de GEN Framework: resolver problemas reales de comunicación, red, hardware y servicios externos sin romper la filosofía común del framework.

Capítulo VII: Graphic, el sistema gráfico de GEN Framework

1. El tercer bloque que dio forma a GEN Framework

Graphic es el tercer gran bloque que terminó de convertir GEN Framework en algo más amplio que una colección de utilidades y clases de entrada/salida. XUtils resolvía las piezas básicas que cualquier programador de C/C++ acaba necesitando. DataIO añadía la comunicación con el exterior, los streams, los protocolos, los dispositivos y los servicios de red. Graphic incorporó la capacidad de construir aplicaciones visuales, dibujar en pantalla, trabajar con imágenes, manejar formatos gráficos, crear ventanas, usar canvas 2D y generar representaciones vectoriales o estadísticas.

En una aplicación real, la parte gráfica no es solo estética. Una pantalla es una frontera de comunicación con el usuario. Un canvas puede representar una máquina, un dispositivo, una línea de producción, un panel industrial, una interfaz de diagnóstico, un visor de datos, una pantalla embebida o una herramienta de configuración. Por eso el módulo Graphic no debe verse únicamente como un conjunto de funciones para pintar líneas y rectángulos. Es una capa completa para convertir datos y estados internos en representación visual.

Como ocurre en el resto de GEN Framework, Graphic no nace como una abstracción académica. Sus clases responden a problemas concretos: crear pantallas en distintas plataformas, disponer de un canvas 2D común, cargar bitmaps, renderizar SVG, interpretar DXF, generar gráficas, controlar viewports, medir fotogramas, manejar secuencias de imágenes, leer vídeo o presentar un buffer mediante OpenGL ES. Cada funcionalidad existe porque en algún proyecto fue necesario resolver esa necesidad de forma portable y reutilizable.

El resultado es un bloque gráfico práctico, muy ligado al resto del framework. No pretende sustituir a todos los motores gráficos existentes ni competir con herramientas de propósito muy diferente. Su objetivo es dar a las aplicaciones con GEN Framework una base gráfica coherente, portable y suficiente para desarrollar interfaces, visores, ejemplos, herramientas de control, dashboards, pantallas embebidas y aplicaciones gráficas multiplataforma sin llenar el código de llamadas nativas a cada sistema operativo.

2. GRPFACTORY y el aislamiento de plataforma

La pieza central de la arquitectura gráfica es GRPFACTORY. Igual que XFACTORY y DIOFACTORY aíslan objetos dependientes de plataforma en sus respectivos bloques, GRPFACTORY centraliza la creación de componentes gráficos cuya implementación puede depender del sistema operativo, del backend gráfico o del modo de píxel.

El caso más evidente es la pantalla. Crear una ventana en Windows, una superficie en Linux, una actividad gráfica en Android o una salida sobre un display embebido no se resuelve con las mismas APIs. La aplicación no debe decidir qué clase concreta de pantalla instanciar. Esa responsabilidad pertenece a la factoría y a las clases de plataforma confinadas en Platform.

En el código común de Graphic se define la interfaz y parte de la lógica compartida. Las implementaciones concretas de plataforma especializan esa factoría para crear pantallas, gestores de escritorio o integraciones nativas. El mismo patrón se aplica a otros objetos cuando la plataforma interviene. En cambio, hay objetos que pueden crearse de forma común, como ciertos bitmaps o canvas basados en AGG, porque su funcionamiento depende más del formato de píxel que de la ventana nativa.

Esta distinción es importante: GRPFACTORY no es solo una fábrica de ventanas. Es el punto donde la aplicación deja de conocer el detalle de la plataforma. Si una funcionalidad gráfica depende del sistema, debe vivir detrás de la factoría o dentro del árbol Platform. Si una funcionalidad puede ser común, se mantiene dentro del bloque Graphic y se reutiliza en todas las plataformas.

```
GRPPROPERTIES properties;
GRP2DCANVAS* canvas = NULL;
```

```

properties.SetWidth(800);
properties.SetHeight(480);
properties.SetMode(GRPPROPERTYMODE_32_RGBA_8888);

canvas = GEN_GRPFACTORY.CreateCanvas(&properties);
if(canvas)
{
    GRP2DCOLOR_RGBA8 color(255, 255, 255);

    canvas->Buffer_Create();
    canvas->SetFillColor(&color);
    canvas->Rect(0.0f, 0.0f, 800.0f, 480.0f, true);

    GEN_GRPFACTORY.DeleteCanvas(canvas);
}

```

El ejemplo muestra el patrón típico de la factoría: se prepara una configuración o unas propiedades, se solicita el objeto, se comprueba si existe, se trabaja con él y se libera mediante la misma factoría. En una aplicación gráfica completa, AppFlowGraphics suele encargarse de crear la pantalla principal y emitir los eventos correspondientes, pero la filosofía de propiedad y creación es la misma.

3. Graphic dentro de una aplicación AppFlowGraphics

Los ejemplos del directorio Graphics muestran el uso práctico de este bloque. Canvas2D se centra en el dibujo directo sobre un canvas. StatisticsCharts muestra la generación de gráficas estadísticas. UI_Message y UI_Options combinan Graphic con UserInterface. WindowChromes trabaja con la configuración de marcos personalizados de ventana. Todos ellos se apoyan en AppFlowGraphics, que encapsula la entrada al programa, el ciclo de vida y la recepción de eventos gráficos.

Un detalle importante, ya comentado en el capítulo de la primera aplicación, es que el programador no escribe una función main propia. En una aplicación con GEN Framework, la entrada está integrada en el framework. La aplicación recibe puntos de extensión y eventos: inicialización, actualización, finalización, creación de pantalla, creación de canvas, entrada de usuario y otros mensajes. Esto permite que la parte gráfica sea portable y que la plataforma quede confinada en el framework.

El patrón habitual es recibir un GRPXEVENT de creación de pantalla, configurar el objeto GRPSCREEN, crear uno o varios viewports y cargar recursos desde assets usando XPATHSMANAGER. El ejemplo Canvas2D sigue precisamente ese modelo.

```

void CANVAS2D::HandleEvent_Graphics(GRPXEVENT* event)
{
    switch(event->GetEventType())
    {
        case GRPXEVENT_TYPE_SCREEN_CREATING : { GRPSCREEN* screen = event->GetScreen();
                                                if(!screen) break;

                                                Ini_Graphics(screen);
                                                }
                                                break;
    }
}

```

El evento no contiene lógica de negocio compleja. Sirve para conectar el momento exacto en que el framework está creando la pantalla con el código de configuración gráfica de la aplicación. Esa separación evita que la aplicación dependa de la función main o de llamadas nativas de la plataforma.

4. Pantallas, viewports y propiedades gráficas

GRPSCREEN representa la pantalla o ventana principal desde el punto de vista del framework. Permite establecer ancho, alto, posición, título, estilos, pantalla de escritorio seleccionada, rotación y viewports. Internamente, la

implementación real puede cambiar según la plataforma, pero la aplicación trabaja con la misma interfaz conceptual.

GRPVIEWPORT define una zona de renderizado dentro de la pantalla. Esta clase es fundamental cuando se quiere separar el tamaño físico de la ventana de las coordenadas lógicas de dibujo. Una pantalla puede tener un tamaño concreto en píxeles y, al mismo tiempo, un viewport que represente un espacio lógico distinto. Esto resulta útil para escalar fondos, adaptar interfaces, dibujar mapas, renderizar escenas o aislar regiones de trabajo.

GRPPROPERTIES agrupa propiedades comunes como dimensiones y modo de píxel. Muchas clases gráficas heredan o copian estas propiedades porque un bitmap, un canvas o una pantalla necesitan compartir información sobre formato y tamaño. GRPRECT permite expresar áreas rectangulares y se usa en recortes, regiones de refresco y operaciones geométricas.

```
bool CANVAS2D::Ini_Graphics(GRPSCREEN* screen)
{
    if(!screen) return false;

    screen->SetPosition(GRPPROPERTYMODE_SCREEN_CENTER, GRPPROPERTYMODE_SCREEN_CENTER);
    screen->SetWidth(1024);
    screen->SetHeight(768);
    screen->GetTitle()->Set(__L("Canvas 2D"));
    screen->SetDesktopScreenSelected(GRPSCREENTYPE_DESKTOP_SCREEN1);

    return screen->CreateViewport(APPLICATION_GUI_MAIN_VIEWPORT_ID,
                                0.0f, 0.0f,
                                (float)screen->GetWidth(), (float)screen->GetHeight(),
                                0, 0,
                                screen->GetWidth(), screen->GetHeight());
}
```

Este esqueleto refleja la idea central: la aplicación configura la pantalla mediante propiedades de GEN Framework, no mediante APIs nativas. El viewport principal queda asociado a un identificador y puede recuperarse posteriormente para obtener el canvas y dibujar.

5. Canvas 2D, colores, paths y áreas de reconstrucción

GRP2DCANVAS es la superficie de dibujo 2D del framework. Sobre él se dibujan primitivas, píxeles, líneas, rectángulos, elipses, textos, bitmaps y paths. La aplicación no debería preocuparse por cómo está almacenado internamente el buffer ni por qué backend se utiliza. Trabaja con una interfaz común de dibujo.

GRP2DCANVASAGG es la implementación basada en AGG. AGG aporta rasterización 2D de alta calidad y permite trabajar con distintos formatos de píxel. La factoría puede crear canvas específicos para RGB565, RGB888, BGRA8888 u otros modos. Esto tiene mucho sentido en un framework que pretende llegar tanto a escritorio como a sistemas embebidos: no todas las pantallas tienen el mismo formato de color ni el mismo coste de memoria.

GRP2DCOLOR y sus variantes permiten expresar colores. GRP2DPATH representa caminos vectoriales.

GRP2DVERTEXTS agrupa vértices. GRP2DREBUILDAREAS ayuda a gestionar zonas que necesitan redibujarse. En interfaces gráficas o displays pequeños, no siempre interesa repintar toda la pantalla; poder identificar áreas modificadas ayuda a optimizar el refresco.

```
GRP2DCOLOR_RGBA8 colorwhite(255, 255, 255);
GRP2DCOLOR_RGBA8 colorgray ( 10,  10,  10, 150);

GRPVIEWPORT* viewport = GetMainScreen()->GetViewport(APPLICATION_GUI_MAIN_VIEWPORT_ID);
if(viewport)
{
    GRP2DCANVAS* canvas = viewport->GetCanvas();
    if(canvas)
    {
        canvas->SetFillColor(&colorgray);
        canvas->Ellipse(75.0f, 130.0f, 40.0f, 8.0f, true);
    }
}
```

```

        canvas->SetFillColor(&colorwhite);
        canvas->Rect(10.0f, 10.0f, 200.0f, 80.0f, true);
    }
}

```

La llamada al canvas expresa intención gráfica, no detalle de plataforma. La misma lógica puede ejecutarse sobre una ventana de escritorio, una superficie Android o un backend adaptado a un display concreto, siempre que la plataforma proporcione la implementación necesaria.

6. Bitmaps, secuencias y formatos raster

GRPBITMAP representa una imagen raster en memoria. A diferencia de un simple buffer, conoce sus dimensiones, su modo de píxel y operaciones propias de una imagen: obtener o escribir píxeles, mezclar con alfa, clonar, copiar, convertir de modo, voltear, rotar, escalar, recortar, comparar o buscar un sub-bitmap. Esto lo convierte en una pieza central para interfaces 2D, sprites, fondos, capturas y manipulación de imágenes.

GRPBITMAPSEQUENCE permite agrupar varios bitmaps como una secuencia animada. En el ejemplo Canvas2D se utiliza para cargar una animación frame a frame y reproducirla. Este patrón es útil cuando se quiere animar un personaje, un icono, un indicador o una transición sin utilizar un sistema de animación más complejo.

GRPBITMAPFILE es la clase base para cargar y guardar bitmaps. Las especializaciones GRPBITMAPFILEBMP, GRPBITMAPFILEPNG, GRPBITMAPFILEJPG y GRPBITMAPFILETGA encapsulan formatos concretos. De esta forma, la aplicación no necesita conocer cada librería o parser de imagen. Pide cargar un fichero y recibe un GRPBITMAP adaptado al modo de la pantalla.

```

XPATH      xpath;
GRPBITMAPFILE* bitmapfile = GEN_NEW GRPBITMAPFILE();
GRPBITMAP*  background = NULL;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, xpath);
xpath.Slash_Add();
xpath.Add(_L("background.jpg"));

background = bitmapfile->Load(xpath, GetMainScreen()->GetMode());

if(background)
{
    // El bitmap ya puede dibujarse en un canvas o usarse como fondo.
}

delete bitmapfile;

```

El ejemplo muestra también la relación con XPATHSMANAGER. Los recursos gráficos de la aplicación deben estar bajo assets, normalmente en la sección Graphics. Esto mantiene ordenados los ficheros y hace que todas las aplicaciones usen una forma homogénea de localizar imágenes, fuentes, SVG, DXF u otros recursos.

7. Ficheros vectoriales: base común, SVG y DXF

El bloque FormatFiles/Vector es una de las partes más interesantes de Graphic. GRPVECTORFILE define la base común para documentos vectoriales. A partir de ella aparecen especializaciones para SVG y DXF. Esta separación permite trabajar con la idea general de fichero vectorial sin que la aplicación tenga que conocer todos los detalles del formato concreto desde el primer momento.

SVG es un formato muy útil para interfaces, iconos, escalado independiente de resolución y generación de gráficos. El parser SVG de GEN Framework organiza el documento como un árbol de objetos: rectángulos, círculos, elipses, líneas, paths, polígonos, polilíneas, textos, gradientes, estilos, transformaciones y referencias use. Cada objeto tiene una clase concreta, pero todas forman parte del mismo modelo de documento.

DXF, por su parte, conecta Graphic con el mundo CAD. Las clases DXF gestionan secciones, valores, entidades y objetos geométricos como líneas, círculos, arcos, elipses, polilíneas, textos, MText, puntos y 3DFace. Esto permite cargar planos, esquemas, geometría técnica o ficheros de prueba para renderizado vectorial.

```

XPATH          pathvf;
GRPVECTORFILE* vectorfile = NULL;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_GRAPHICS, pathvf);
pathvf.Slash_Add();
pathvf.Add(__L("logo.svg"));

vectorfile = GRPVECTORFILE::CreateInstance(pathvf);
if(vectorfile)
{
    if(vectorfile->Load(pathvf) == GRPVECTORFILERESULT_OK)
    {
        // El documento puede inspeccionarse o renderizarse sobre un canvas.
    }

    delete vectorfile;
}

```

La creación por ruta permite que el framework seleccione el tipo adecuado a partir del formato. También se puede crear por tipo explícito, como ocurre en StatiticsCharts cuando se genera un SVG en memoria para contener una gráfica.

8. Renderizado vectorial sobre canvas

Las clases GRP2DVECTORFILERENDERAGG, GRP2DVECTORFILESVGRENDERAGG y GRP2DVECTORFILEDXFRENDERAGG conectan el mundo de los documentos vectoriales con el canvas 2D. Un SVG o un DXF no son directamente píxeles; son estructuras geométricas, estilos, transformaciones y entidades. El renderizador recorre esa estructura y la traduce a operaciones de dibujo sobre un GRP2DCANVAS.

Esta separación es importante desde el punto de vista arquitectónico. El parser de SVG o DXF no debería depender de una pantalla concreta. El canvas no debería saber cómo se organiza internamente un DXF. El renderizador es la pieza intermedia: entiende el documento vectorial y sabe cómo convertirlo en primitivas 2D.

En versiones recientes del framework se ha trabajado precisamente sobre esta zona para centralizar el cache de renderizado vectorial y evitar duplicar lógica entre SVG y DXF. Esto encaja con la filosofía de GEN Framework: cuando aparece una parte común, debe subir a una clase común; cuando aparece una diferencia de formato, debe quedar confinada en la especialización correspondiente.

```

GRP2DVECTORFILERENDERAGG renderer;

if(vectorfile && canvas)
{
    renderer.SetCanvas(canvas);
    renderer.Render(vectorfile);
}

```

El fragmento anterior expresa el patrón conceptual. El nombre exacto de las funciones puede depender de la versión concreta, pero la idea es la misma: el documento vectorial se mantiene separado del canvas y el renderizador realiza la traducción entre ambos mundos.

9. OpenGL ES, EGL y presentación acelerada

Graphic también incluye una capa OpenGL. GRPEGLCONTEXT gestiona la creación y administración de un contexto EGL. GRPBLITGLES actúa como blitter OpenGL ES 3.0: toma un canvas o buffer 2D y lo presenta usando la GPU. Esta arquitectura es muy útil cuando el dibujo 2D se genera en memoria, pero la presentación final debe ser eficiente.

La idea no es obligar a toda aplicación gráfica a programar directamente en OpenGL. Al contrario: muchas aplicaciones pueden dibujar con GRP2DCANVAS y dejar que la plataforma o el backend se encarguen de presentar el resultado. OpenGL ES aparece como una capa de salida, no necesariamente como la API que debe usar toda la lógica gráfica de la aplicación.

Esta separación permite que el mismo código de dibujo pueda convivir con distintos mecanismos de presentación. En escritorio puede existir una ventana nativa. En Android puede existir una superficie gráfica. En un sistema embebido puede haber un framebuffer o un display con restricciones particulares. La aplicación no debería tener que reescribir su lógica por cada destino.

10. Vídeo y extracción de frames

GRPVIDEOFILE y GRPVIDEOFILEAVI amplían el bloque gráfico hacia la lectura de vídeo. No son el núcleo de la mayoría de las aplicaciones, pero permiten trabajar con secuencias de frames y propiedades de vídeo cuando el proyecto lo necesita. En los ejemplos aparecen usos de GRPVIDEOFILEAVI para abrir un fichero AVI, consultar propiedades y extraer frames que posteriormente pueden convertirse o dibujarse como bitmaps.

```
GRPVIDEOFILEAVI* avifile = GEN_NEW GRPVIDEOFILEAVI();

if(avifile)
{
    if(avifile->Open(xpath))
    {
        GRPVIDEOFILE_PROPERTYYS propertys;

        avifile->GetPropertyys(propertys);
        // A partir de aquí se pueden obtener frames y tratarlos como imágenes.

        avifile->Close();
    }

    delete avifile;
}
```

El patrón vuelve a ser el mismo: objeto especializado, apertura, consulta o lectura, cierre y liberación. Aunque el formato AVI sea concreto, el estilo de uso sigue siendo el de GEN Framework.

11. Gráficas estadísticas

El directorio StatisticsChars contiene un pequeño sistema de generación de gráficas. El nombre histórico del directorio no debe distraer de su función: crear representaciones estadísticas a partir de datos, configuración visual y un builder de salida. La clase base GRPSTATISTICSCHART define la lógica común.

GRPSTATISTICSCHARTDATA contiene categorías y series. GRPSTATISTICSCHARTCONFIG define título, colores, rejilla, leyenda, tamaños de fuente y márgenes. GRPSTATISTICSCHARTBUILDER abstrae la salida, y GRPSTATISTICSCHARTBUILDERSVG permite generar un SVG.

Encima de esa base aparecen tipos concretos: columnas, líneas, áreas, barras, columnas apiladas y sectores circulares, con versiones 2D y 3D. Esto permite generar dashboards, informes visuales o elementos gráficos sin depender de una librería externa de alto nivel.

```
GRPSTATISTICSCHART* chart = GEN_NEW GRPSTATISTICSCHARTCOLUMNS();

if(chart)
{
    chart->GetConfig()->SetTitle(__L("Sales by quarter"));
    chart->GetConfig()->SetShowValues(true);
    chart->GetConfig()->SetShowLegend(true);

    GRPSTATISTICSCHARTDATA* data = chart->GetData();
    data->AddCategory(__L("Q1"));
    data->AddCategory(__L("Q2"));
}
```

```

data->AddCategory(__L("Q3"));
data->AddCategory(__L("Q4"));

GRPSTATISTICSCHARTSERIE* serie = data->AddSerie(__L("2026"));
serie->AddValue(10.0);
serie->AddValue(18.0);
serie->AddValue(15.0);
serie->AddValue(22.0);

GRPSTATISTICSCHARTBUILDERSVG builder;
if(chart->Generate(builder, 420.0, 420.0) == GRPVECTORFILERESULT_OK)
{
    XSTRING svg;
    builder.GetResult(svg);
}

delete chart;
}

```

El ejemplo es representativo del diseño: la gráfica no escribe directamente en pantalla. Genera una salida a través de un builder. En este caso la salida es SVG, que puede almacenarse, renderizarse, cargarse como GRPVECTORFILE o integrarse en una interfaz.

12. Graphic y UserInterface

UserInterface se apoya de manera natural en Graphic. Una interfaz necesita pantalla, canvas, colores, bitmaps, fuentes, eventos gráficos y, en muchos casos, SVG o imágenes. Por eso los ejemplos UI_Message y UI_Options combinan ambos bloques. Graphic aporta la infraestructura de dibujo y presentación; UserInterface aporta widgets, layouts, interacción y composición de elementos de interfaz.

Esta separación permite utilizar Graphic sin UserInterface cuando se quiere dibujar directamente sobre un canvas, como en Canvas2D, o utilizar UserInterface cuando se desea trabajar con controles de más alto nivel. En aplicaciones pequeñas o pantallas embebidas puede bastar con dibujar directamente. En aplicaciones con más interacción, formularios, botones, mensajes o menús, UserInterface evita reinventar controles en cada proyecto.

El programador debe entender que UserInterface no sustituye a Graphic. Lo usa. Del mismo modo, Graphic no sustituye a AppFlowGraphics. Se integra con él para recibir el ciclo de vida y los eventos. Esta composición por capas es una de las claves de GEN Framework.

13. Mapa general de Graphic

El bloque Graphic es amplio. Para estudiarlo conviene agruparlo por responsabilidad, no por orden alfabético. La siguiente tabla resume sus grandes áreas.

Grupo	Responsabilidad principal	Clases representativas
Núcleo gráfico	Pantallas, propiedades, viewports, eventos, frame rate y factoría gráfica.	GRPFACORY, GRPSCREEN, GRPVIEWPORT, GRPPROPERTIES, GRPXEVENT
Canvas 2D	Dibujo raster 2D, colores, paths, vértices y áreas de repintado.	GRP2DCANVAS, GRP2DCANVASAGG, GRP2DPATH, GRP2DCOLOR, GRP2DREBUILDAREAS
Bitmaps	Imágenes en memoria, secuencias animadas y carga/guardado de formatos raster.	GRPBITMAP, GRPBITMAPSEQUENCE, GRPBITMAPFILE, GRPBITMAPFILEPNG, GRPBITMAPFILEJPG
Vectorial	Carga, análisis y renderizado de documentos SVG y DXF.	GRPVECTORFILE, GRPVECTORFILESVG, GRPVECTORFILEDXF, GRP2DVECTORFILERENDERAGG
OpenGL	Contexto EGL y presentación acelerada mediante OpenGL ES.	GRPEGLCONTEXT, GRPBLITGLS
Vídeo	Lectura de formatos de vídeo y extracción de información o frames.	GRPVIDEOFILE, GRPVIDEOFILEAVI
Gráficas estadísticas	Modelo de datos, configuración y generación de gráficas 2D/3D, especialmente hacia SVG.	GRPSTATISTICSCHART, GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTBUILDERSVG

14. Catálogo de clases y módulos de Graphic

La siguiente referencia resume las clases encontradas en el bloque Graphic. Debe leerse como mapa de orientación. La documentación Doxygen y los ejemplos del directorio Graphics completan el detalle de métodos, parámetros y dependencias exactas.

Núcleo

Clase / módulo	Qué hace	Relación y uso típico
GRPBITMAP	Imagen raster en memoria. Proporciona buffer, acceso a píxel, copia, escala, rotación, recorte y comparación.	Lo crean GRPFACTORY o los lectores de bitmap.
GRPBITMAPSEQUENCE	Secuencia de bitmaps usada para animaciones frame a frame.	Se integra con XUtils, AppFlowGraphics y el resto del bloque Graphic.
GRPDESKTOPMANAGER	Gestor de escritorios y monitores cuando la plataforma lo soporta.	Se usa desde AppFlowGraphics y emite eventos XEVENT_TYPE_GRAPHICS.
GRPDISPLAYDEVICE	Descripción genérica de un dispositivo de visualización físico o lógico.	Se usa desde AppFlowGraphics y emite eventos XEVENT_TYPE_GRAPHICS.
GRPFACTORY	Factoría gráfica. Crea pantallas, canvas, bitmaps y gestores dependientes de la plataforma cuando corresponde.	Es el punto de entrada para objetos gráficos creados por plataforma.
GRPFramerate	Medición y control del ritmo de fotogramas.	Se integra con XUtils, AppFlowGraphics y el resto del bloque Graphic.
GRPPROPERTIES	Propiedades comunes de objetos gráficos: ancho, alto, modo de píxel y otros atributos compartidos.	Se integra con XUtils, AppFlowGraphics y el resto del bloque Graphic.
GRPRECT	Plantilla para rectángulos y áreas. Se usa en recortes, zonas de refresco y operaciones geométricas.	Se integra con XUtils, AppFlowGraphics y el resto del bloque Graphic.
GRPSCREEN	Abstracción de pantalla o ventana gráfica. Define tamaño, posición, estilos, rotación, viewports y eventos de creación.	Se usa desde AppFlowGraphics y emite eventos XEVENT_TYPE_GRAPHICS.
GRPSCREENCFGCHROMES	Configuración de chomes personalizados de ventana: título, botones, bordes y apariencia de marco.	Se usa desde AppFlowGraphics y emite eventos XEVENT_TYPE_GRAPHICS.
GRPVIEWPORT	Zona lógica de renderizado dentro de una pantalla. Permite mapear coordenadas de pantalla y coordenadas del contenido.	Se usa desde AppFlowGraphics y emite eventos XEVENT_TYPE_GRAPHICS.
GRPXEVENT	Evento gráfico emitido por pantallas, canvas y subsistemas gráficos.	Se usa desde AppFlowGraphics y emite eventos XEVENT_TYPE_GRAPHICS.

2D y canvas

Clase / módulo	Qué hace	Relación y uso típico
GRP2DCANVAS	Superficie de dibujo 2D. Ofrece primitivas de dibujo, texto, bitmaps, trazos, rellenos y operaciones sobre el buffer.	Trabaja con GRPSCREEN, GRPVIEWPORT y GRPBITMAP.
GRP2DCANVASAGG	Implementación de canvas 2D basada en AGG para modos de píxel concretos.	Trabaja con GRPSCREEN, GRPVIEWPORT y GRPBITMAP.
GRP2DCOLOR	Representación de color 2D en formatos RGBA u otros formatos compatibles.	Trabaja con GRPSCREEN, GRPVIEWPORT y GRPBITMAP.
GRP2DPATH	Camino vectorial 2D. Agrupa movimientos, líneas, curvas y geometría reutilizable.	Trabaja con GRPSCREEN, GRPVIEWPORT y GRPBITMAP.
GRP2DREBUILDAREAS	Gestión de áreas que deben redibujarse o reconstruirse.	Trabaja con GRPSCREEN, GRPVIEWPORT y GRPBITMAP.
GRP2DVECTORFILE DXF RENDERAGG	Renderizador DXF sobre canvas AGG.	Se integra con XUtils, AppFlowGraphics y el resto del bloque Graphic.
GRP2DVECTORFILE RENDERAGG	Renderizador común de ficheros vectoriales sobre canvas AGG.	Se integra con XUtils, AppFlowGraphics y el resto del bloque Graphic.
GRP2DVECTORFILE SVG RENDERAGG	Renderizador SVG sobre canvas AGG.	Se integra con XUtils, AppFlowGraphics y el resto del bloque Graphic.
GRP2DVERTEXES	Contenedor de vértices 2D.	Trabaja con GRPSCREEN, GRPVIEWPORT y GRPBITMAP.

Formatos bitmap

Clase / módulo	Qué hace	Relación y uso típico
GRPBITMAPFILE	Clase base para carga y guardado de ficheros bitmap.	Carga o guarda GRPBITMAP desde assets/Graphics.
GRPBITMAPFILE BMP	Soporte para ficheros BMP.	Carga o guarda GRPBITMAP desde assets/Graphics.
GRPBITMAPFILE JPG	Soporte para ficheros JPEG.	Carga o guarda GRPBITMAP desde assets/Graphics.

GRPBITMAPFILE PNG	Soporte para ficheros PNG.	Carga o guarda GRPBITMAP desde assets/Graphics.
GRPBITMAPFILE TGA	Soporte para ficheros TGA/TARGA.	Carga o guarda GRPBITMAP desde assets/Graphics.

Vectorial común

Clase / módulo	Qué hace	Relación y uso típico
GRPVECTORFILE	Clase base de ficheros vectoriales. Permite crear instancias por tipo o ruta.	Base común para SVG, DXF y eventos de ficheros vectoriales.
GRPVECTORFILECONFIG	Configuración común de ficheros vectoriales.	Base común para SVG, DXF y eventos de ficheros vectoriales.
GRPVECTORFILE_XEVENT	Evento asociado a carga, procesado o renderizado de ficheros vectoriales.	Base común para SVG, DXF y eventos de ficheros vectoriales.

SVG

Clase / módulo	Qué hace	Relación y uso típico
GRPVECTORFILESVG	Documento SVG: carga, analiza y mantiene el árbol de objetos SVG.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG CSSSTYLESHEET	Hoja de estilos CSS aplicada a un SVG.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG CONFIG	Configuración específica de SVG.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG OBJ	Objeto base del árbol SVG y factoría de nodos SVG.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ CIRCLE	Nodo SVG de círculo.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ ELLIPSE	Nodo SVG de elipse.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ GRADIENT	Servidor de pintura para gradientes lineales o radiales.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ LINE	Nodo SVG de línea.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ PATH	Nodo SVG de path. Interpreta el atributo d.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ POLYLINE	Nodo SVG de polilínea.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ POLYGON	Nodo SVG de polígono.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ RECT	Nodo SVG de rectángulo.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ TEXT	Nodo SVG de texto y tspan.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVGOBJ USE	Nodo SVG use, referencia otro elemento por identificador.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG STYLE	Estilo SVG: relleno, trazo, opacidad, anchura y reglas de pintado.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.
GRPVECTORFILESVG TRANSFORM	Transformación afín SVG en matriz 2x3.	Se carga con GRPVECTORFILE y puede renderizarse con GRP2DVECTORFILESVGRENDERAGG.

DXF

Clase / módulo	Qué hace	Relación y uso típico
GRPVECTORFILEDXF	Documento DXF: carga, analiza secciones y entidades de un fichero CAD.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF CONFIG	Configuración específica del lector DXF.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF ENTITY	Entidad DXF base.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYDEF POINT	Definición auxiliar DXF: POINT.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYDEF STRINGVALUE	Definición auxiliar DXF: STRINGVALUE.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYDEF VERTEX	Definición auxiliar DXF: VERTEX.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF ENTITYOBJ	Entidad geométrica DXF base.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ	Entidad geométrica DXF especializada: 3DFACE.	Forma parte del parser DXF y puede

3DFACE		renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ ARC	Entidad geométrica DXF especializada: ARC.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ CIRCLE	Entidad geométrica DXF especializada: CIRCLE.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ ELLIPSE	Entidad geométrica DXF especializada: ELLIPSE.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ LWPOLYLINE	Entidad geométrica DXF especializada: LWPOLYLINE.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ LINE	Entidad geométrica DXF especializada: LINE.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ MTEXT	Entidad geométrica DXF especializada: MTEXT.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ POINT	Entidad geométrica DXF especializada: POINT.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ POLYLINE	Entidad geométrica DXF especializada: POLYLINE.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFENTITYOBJ TEXT	Entidad geométrica DXF especializada: TEXT.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF TEXTBLOCK	Estructura textual interna del parser DXF.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF TEXTPART	Estructura textual interna del parser DXF.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF TEXTSECTION	Sección textual DXF base.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFTEXTSECTION BLOCKS	Sección textual DXF especializada: BLOCKS.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFTEXTSECTION ENTITIES	Sección textual DXF especializada: ENTITIES.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXFTEXTSECTION HEADER	Sección textual DXF especializada: HEADER.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF VALUE	Valor DXF asociado a códigos de grupo.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.
GRPVECTORFILEDXF XDATACTRL	Control de datos extendidos XDATA de DXF.	Forma parte del parser DXF y puede renderizarse con GRP2DVECTORFILEDXFRENDERAGG.

Video

Clase / módulo	Qué hace	Relación y uso típico
GRPVIDEOFILE	Clase base para formatos de vídeo.	Lectura de vídeo y extracción de frames como bitmaps.
GRPVIDEOFILEAVI	Soporte de lectura de AVI.	Lectura de vídeo y extracción de frames como bitmaps.

OpenGL

Clase / módulo	Qué hace	Relación y uso típico
GRPBUTGLLES	Blitter OpenGL ES para llevar un canvas o buffer 2D a pantalla mediante GPU.	Capa de aceleración y presentación sobre OpenGL ES/EGL.
GRPEGLCONTEXT	Gestor de contexto EGL para OpenGL ES.	Capa de aceleración y presentación sobre OpenGL ES/EGL.

Gráficas estadísticas

Clase / módulo	Qué hace	Relación y uso típico
GRPSTATISTICSCHART	Base abstracta de gráficas estadísticas.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART AREA	Gráfica de área.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART AREA3D	Gráfica de área 3D.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART BARS	Gráfica de barras horizontales.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART BARS3D	Gráfica de barras 3D.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART BUILDER	Builder abstracto de salida para gráficas.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART BUILDERSVG	Builder que convierte la gráfica en SVG.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART COLUMNS	Gráfica de columnas agrupadas.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.

GRPSTATISTICSCHART COLUMNS3D	Gráfica de columnas 3D.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART CONFIG	Configuración visual de las gráficas: título, paleta, leyenda, rejilla y fuentes.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART DATA	Modelo de datos de gráficas: categorías, series y valores.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART LINES	Gráfica de líneas.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART LINES3D	Gráfica de líneas 3D.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART PIE	Gráfica circular o donut.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART PIE3D	Gráfica circular 3D.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART STACKEDCOLUMNS	Gráfica de columnas apiladas.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART STACKEDCOLUMNS3D	Gráfica de columnas apiladas 3D.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.
GRPSTATISTICSCHART COLOR	Tipos de estilo independientes del formato: color, relleno, trazo y texto.	Usa GRPSTATISTICSCHARTDATA, GRPSTATISTICSCHARTCONFIG y builders como SVG.

15. Cómo estudiar Graphic desde los ejemplos

Para estudiar Graphic no conviene empezar por las clases más internas. Es mejor seguir los ejemplos oficiales. Canvas2D muestra la relación entre AppFlowGraphics, GRPSCREEN, GRPVIEWPORT, GRP2DCANVAS, GRPBITMAPFILE, XPATHSMANAGER y eventos gráficos. Es el ejemplo adecuado para entender el dibujo directo.

StatisticsCharts es el ejemplo adecuado para estudiar las clases de gráficas estadísticas y el flujo de generación hacia SVG. Permite ver cómo se crea un chart, cómo se alimentan categorías y series, cómo se configura la leyenda y cómo un builder produce un resultado vectorial.

UI_Message y UI_Options muestran la relación entre Graphic y UserInterface. En ellos se ve que una interfaz de usuario no vive fuera del sistema gráfico: necesita pantalla, canvas, eventos, recursos y configuración de ventana. WindowChromes muestra la personalización de la ventana y el uso de GRPSCREENCFGCHROMES.

El orden recomendado de estudio es: primero Canvas2D, después WindowChromes, luego UI_Message o UI_Options, y finalmente StatisticsCharts si se quiere profundizar en generación de gráficos vectoriales y estadísticas. Mientras se estudian esos ejemplos, conviene consultar Doxygen para ver herencias, enumeraciones y métodos disponibles.

16. Criterios de uso en una aplicación con GEN Framework

Cuando una aplicación necesita mostrar información gráfica, la regla general es no saltarse Graphic. Si hay que crear una pantalla, se trabaja mediante AppFlowGraphics y GRPSCREEN. Si hay que dibujar, se usa GRP2DCANVAS. Si hay que cargar una imagen, se usa GRPBITMAPFILE. Si hay que tratar una imagen en memoria, se usa GRPBITMAP. Si hay que cargar un SVG o DXF, se usa GRPVECTORFILE y sus especializaciones. Si hay que generar una gráfica estadística, se usa el bloque GRPSTATISTICSCHART.

Esta disciplina evita que cada aplicación invente su propia forma de dibujar o de cargar recursos. También permite que la lógica gráfica de la aplicación sobreviva mejor a los cambios de plataforma. Una llamada directa a una API de ventana nativa acopla el programa a esa plataforma. Una llamada a Graphic mantiene la dependencia dentro del framework.

También debe respetarse el orden de propiedad. Los objetos creados por GRPFACTORY deben destruirse por GRPFACTORY. Los objetos creados con GEN_NEW deben liberarse con GEN_DELETE o mediante el patrón que marque la clase. Los recursos gráficos deben cargarse desde assets y localizarse mediante XPATHSMANAGER. Las pantallas y canvas controlados por AppFlowGraphics no deben tratarse como objetos nativos propiedad de la aplicación si el framework los está gestionando.

17. Resumen práctico

Graphic es el tercer bloque que consolidó GEN Framework. Aporta la base visual: pantallas, canvas, bitmaps, formatos gráficos, SVG, DXF, vídeo, OpenGL ES y gráficas estadísticas. Su objetivo no es esconder la existencia de la plataforma, sino confinarla en las factorías y en las implementaciones específicas, ofreciendo al programador una forma homogénea de trabajar.

La clave de su arquitectura está en GRPFACTORY, GRPSCREEN, GRPVIEWPORT y GRP2DCANVAS. Sobre esa base se construyen los demás servicios: carga de imágenes, renderizado vectorial, generación de estadísticas, presentación acelerada y relación con UserInterface.

Una aplicación con GEN Framework que usa Graphic correctamente mantiene separada la lógica de aplicación de los detalles nativos de la ventana o del backend gráfico. Esa separación permite compilar en varias plataformas, reutilizar recursos, mantener el estilo del framework y desarrollar interfaces o visualizaciones sin perder el control propio de C/C++.

Capítulo VIII: Input, dispositivos de entrada y abstracción de interacción

1. El bloque Input dentro de GEN Framework

El módulo Input aparece como una consecuencia natural del bloque gráfico. En cuanto una aplicación deja de ser únicamente una herramienta de consola y empieza a tener pantalla, canvas, elementos visuales, ventanas o interacción directa con el usuario, aparece una necesidad inmediata: recibir acciones del exterior de forma uniforme. No basta con dibujar; hay que saber qué está haciendo el usuario y convertir esa información en decisiones de la aplicación.

El problema es que la entrada de usuario no es homogénea entre plataformas. Windows, Linux, Android, una Raspberry Pi con pantalla táctil, un sistema embebido con un pequeño display SPI, un teclado físico, un ratón, una superficie táctil o un controlador externo no ofrecen exactamente el mismo modelo de eventos. Cada sistema tiene sus códigos, sus mensajes, sus colas de eventos, sus coordenadas, sus convenciones y sus limitaciones.

Input nace para evitar que cada aplicación tenga que resolver de nuevo ese problema. Su objetivo es que el código de la aplicación no dependa directamente de si la acción procede de un ratón, una pantalla táctil, un teclado, un joystick, un mando o una implementación particular de una plataforma. La aplicación debe trabajar con conceptos estables del framework y dejar que la capa de plataforma traduzca los detalles concretos.

Por eso Input no debe verse como una simple colección de clases para teclado y ratón. Es una capa de normalización de interacción. Del mismo modo que Graphic intenta ofrecer una forma común de dibujar, Input intenta ofrecer una forma común de interpretar lo que el usuario hace sobre la aplicación.

2. Un bloque nacido después del sistema gráfico

Históricamente, este bloque aparece al realizar el bloque gráfico. Cuando GEN Framework empieza a manejar pantallas, canvas, bitmaps, interfaces y aplicaciones gráficas, se hace evidente que el gráfico por sí solo no es suficiente. Una aplicación gráfica necesita saber cuándo se pulsa una tecla, cuándo se mueve un cursor, cuándo se arrastra sobre una zona, cuándo se hace una selección o cuándo se produce una pulsación táctil.

El diseño de Input está muy ligado a los ejemplos gráficos del framework. En aplicaciones como Canvas2D, UI_Options, UI_Message, WindowChromes o StatisticsCharts se puede ver un patrón repetido: la aplicación obtiene los dispositivos desde el gestor de entrada, guarda punteros a los botones y cursores que le interesan, y en cada ciclo de actualización consulta su estado para transformar esa entrada en acciones de aplicación o eventos de interfaz.

Esta relación con Graphic no significa que Input solo sirva para aplicaciones gráficas grandes. Una aplicación sin AppFlow, ejecutándose en un microcontrolador, podría utilizar una idea parecida para manejar botones físicos, una cruceta, un encoder, una pantalla táctil resistiva o un pequeño display por SPI o I2C. La idea de fondo sigue siendo la misma: aislar la lógica de la aplicación del origen físico concreto de la entrada.

3. La idea clave: descomponer los dispositivos en cursores y botones

Una de las decisiones más importantes de esta colección de clases es descomponer todos los dispositivos de entrada en dos elementos fundamentales: cursores y botones. Esta idea es sencilla, pero tiene una fuerza práctica enorme.

Un botón representa una acción discreta: pulsado, liberado, mantenido, soltado con detección de flanco, tecla física, botón de ratón, pulsación táctil o botón de un mando. Un cursor representa una posición o un movimiento: coordenadas X/Y/Z, desplazamiento del ratón, punto de contacto en una pantalla táctil, arrastre o trayectoria.

Con esta descomposición, el código de la aplicación deja de preguntarse si el usuario tiene ratón o pantalla táctil. La aplicación pregunta por un cursor y por un botón de selección. En una plataforma ese cursor puede venir del ratón; en otra, de una pantalla táctil; en otra, de un dispositivo específico. El comportamiento de la aplicación puede ser el mismo sin tocar la lógica principal.

Esto es especialmente importante en GEN Framework porque el framework busca que la aplicación sea portable. Programar una pantalla de configuración, un menú o un canvas interactivo directamente contra eventos de ratón de Windows obliga a reescribir partes de la aplicación para Android o para una placa con pantalla táctil. Si la aplicación trabaja con INPCURSOR e INPBUTTON, el cambio de plataforma queda confinado en la capa de Input y en la factoría correspondiente.

4. INPFACTORY y el aislamiento de plataforma

Las instancias dependientes de plataforma se gestionan mediante INPFACTORY. Esta factoría cumple el mismo papel arquitectónico que otras factorías de GEN Framework: separar la petición conceptual de un objeto de su implementación concreta. La aplicación no debería construir directamente un dispositivo Windows, Linux o Android; debe pedir un dispositivo de entrada al framework y dejar que la factoría real de la plataforma cree la clase adecuada.

En el código común, INPFACTORY define operaciones como CreateDevice(), DeleteDevice(), CreateSimulator() y CreateCapture(). La implementación base no crea nada específico: devuelve NULL o false cuando no hay una implementación concreta disponible. La utilidad aparece cuando la plataforma instala su propia factoría especializada, por ejemplo una factoría de Windows, Linux o Android capaz de crear teclados, ratones, pantallas táctiles, capturas o simuladores adaptados al sistema real.

Este diseño evita que el código de aplicación tenga que incluir cabeceras como INPWINDOWSDEVICE_MOUSE, INPLINUXDEVICE_TOUCHSCREEN o INPANDROIDDEVICE_KEYBOARD. Esas clases pueden existir en la zona de plataforma, pero no deben contaminar el código común de la aplicación. La aplicación trabaja con INPDEVICE, INPBUTTON, INPCURSOR e INPMANAGER.

```
INPDEVICE* device;

device = GEN_INPFACTORY.CreateDevice(INPDEVICE_TYPE_MOUSE);
if(device)
{
    GEN_INPMANAGER.AddDevice(device);
}
```

El ejemplo anterior muestra la idea general. En una aplicación AppFlow normal, gran parte de esta creación puede estar integrada en el arranque del flujo gráfico cuando se activa la opción de Input. El código de la aplicación suele consumir los dispositivos ya registrados en INPMANAGER, pero la responsabilidad de creación sigue perteneciendo a la factoría.

5. INPMANAGER: el punto de acceso a los dispositivos

INPMANAGER es el gestor central de dispositivos de entrada. Su responsabilidad es mantener una colección ordenada de INPDEVICE, permitir localizar dispositivos por índice o por tipo, actualizarlos y liberar la colección cuando ya no sea necesaria. En los ejemplos gráficos aparece como el punto de entrada habitual para que la aplicación encuentre teclado, ratón o pantalla táctil.

El gestor utiliza un mapa interno protegido por mutex. Esto encaja con la filosofía de GEN Framework: no se expone una lista desordenada de objetos al azar, sino una estructura que permite registrar dispositivos, consultarlos y actualizar su estado de forma homogénea. La aplicación no tiene por qué saber cómo se leen los eventos nativos; solo necesita saber qué dispositivos están disponibles.

```
INPDEVICE* inpdevice;

inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_KEYBOARD);
if(inpdevice)
{
    button[MYAPP_BUTTON_ESC] = inpdevice->GetButton(INPBUTTON_ID_ESCAPE);
    button[MYAPP_BUTTON_SPACE] = inpdevice->GetButton(INPBUTTON_ID_SPACE);
}
```

Este patrón aparece de forma repetida en los ejemplos gráficos. Durante la primera actualización de la aplicación se localizan los dispositivos y se guardan punteros a los botones necesarios. Después, en el Update, el código comprueba esos punteros y consulta su estado. Esto evita buscar el dispositivo y el botón en cada ciclo cuando ya se conocen las referencias que interesan a la aplicación.

6. INPDEVICE: un dispositivo reducido a elementos comunes

INPDEVICE representa un dispositivo de entrada genérico. Su enumeración de tipo permite distinguir entre teclado, ratón, joystick, pantalla táctil o Wiimote, pero la clase no se limita a una de esas familias. Su valor está en que todos los dispositivos se pueden describir mediante dos colecciones: botones y cursores.

Un teclado tiene muchos botones y normalmente ningún cursor. Un ratón tiene botones y un cursor. Una pantalla táctil tiene al menos un botón conceptual de selección y uno o varios cursores táctiles. Un joystick puede exponer botones y, dependiendo de la implementación, cursores o ejes traducidos a una forma equivalente. La lógica de aplicación puede tratar todos esos dispositivos con las mismas operaciones básicas.

Las funciones GetButton(), GetButtonByCode(), GetButtonState(), IsPressButton(), ReleaseAllButtons(), GetCursor() e IsChangeCursor() convierten a INPDEVICE en el punto de consulta del estado normalizado de entrada. La función Update() es virtual porque la lectura real depende de la plataforma y del tipo concreto de dispositivo.

```
INPDEVICE* mouse = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_MOUSE);
if(mouse)
{
    INPBUTTON* select = mouse->GetButton(INPBUTTON_ID_MOUSE_LEFT);
    INPCURSOR* cursor = mouse->GetCursor(0);

    if(select && select->IsPressedWithRelease())
    {
        // Selección en la posición actual del cursor.
    }
}
```

7. INPBUTTON: pulsación, estado y transición

INPBUTTON es la abstracción de una entrada discreta. No representa únicamente una tecla de teclado. También puede representar un botón de ratón, un botón de mando, un botón de smartphone o la pulsación conceptual de una pantalla táctil. Por eso la enumeración INPBUTTON_ID incluye grupos de identificadores para teclado, ratón, smartphone, Wiimote y touchscreen.

La clase conserva el identificador lógico del botón, el código nativo o código de tecla, el símbolo asociado, el estado, la presión y el tiempo de pulsación. La presencia de un temporizador interno permite medir cuánto tiempo lleva pulsado un botón, algo útil para distinguir una pulsación corta de una pulsación mantenida o para aplicar repeticiones.

La diferencia entre `IsPressed()` e `IsPressedWithRelease()` es importante. `IsPressed()` permite saber si el botón está en estado pulsado. `IsPressedWithRelease()` se utiliza para detectar el gesto completo de pulsar y soltar, que suele ser lo adecuado para acciones de interfaz: aceptar, abrir, seleccionar, cambiar de opción o ejecutar una orden una sola vez.

```
if (button[MYAPP_BUTTON_ESC])
{
    if (button[MYAPP_BUTTON_ESC]->IsPressedWithRelease())
    {
        SetEvent(MYAPP_XFSMEVENT_END);
    }
}
```

Esta forma de trabajo evita acciones repetidas involuntarias. Si se usara solo `IsPressed()`, una tecla mantenida podría ejecutar la misma acción decenas de veces mientras dura la pulsación. Al usar `IsPressedWithRelease()`, el código responde a la transición completa.

8. INPCURSOR: posición, cambio y selección

`INPCURSOR` representa una posición de entrada. Hereda de `INPCURSORMOTIONPOINT`, de modo que un cursor es también un punto con coordenadas X, Y y Z. Esta elección hace que el cursor pueda trabajar directamente con posiciones y, al mismo tiempo, pueda integrarse con la clase de movimiento.

La clase permite consultar y modificar las coordenadas, saber si el cursor ha cambiado, comprobar si su posición cae dentro de un rectángulo y acceder al objeto `INPCURSORMOTION` asociado. En aplicaciones gráficas, esto permite transformar el movimiento físico del ratón o del dedo en eventos de interfaz o en acciones sobre un canvas.

```
if (cursor)
{
    int cursor_x = (int)cursor->GetX();
    int cursor_y = (int)cursor->GetY();

    if (cursor->IsPositionInRect(0, 0, width, height))
    {
        // El cursor está dentro del área activa.
    }
}
```

La coordenada Y puede requerir adaptación según el sistema gráfico utilizado. En algunos ejemplos se invierte el eje Y antes de enviar eventos a `UserInterface`, porque el origen de coordenadas de la entrada y el origen de coordenadas del canvas o de la interfaz pueden no coincidir.

9. INPCURSORMOTIONPOINT e INPCURSORMOTION

`INPCURSORMOTIONPOINT` es la unidad mínima de una trayectoria: un punto con coordenadas X, Y y Z.

`INPCURSORMOTION` es la colección y el analizador de esos puntos. Juntas permiten estudiar el movimiento del cursor, no solo su posición actual.

`INPCURSORMOTION` permite añadir puntos, borrar la trayectoria, construir trayectorias sintéticas desde una línea, un círculo o una elipse, calcular ángulo, módulo, dirección y tiempo transcurrido. También permite reducir puntos

mediante modos como BYSTEPS, NMAX o NMAXPERCENT, lo que resulta útil cuando una trayectoria contiene demasiadas muestras y solo se necesita una representación simplificada.

Esta clase es la base para reconocer movimientos simples. No pretende ser un motor complejo de gestos, pero proporciona lo necesario para interpretar una dirección de arrastre, un desplazamiento significativo o una trayectoria suficientemente estable.

```
if(cursor && cursor->GetMotion()->IsReadyToTest(50))
{
    INPCURSORMOTION_DIR dir;

    dir = cursor->GetMotion()->GetDirectionByAngle(true);
    switch(dir)
    {
        case INPCURSORMOTION_DIR_UP : // Acción hacia arriba
            break;

        case INPCURSORMOTION_DIR_DOWN : // Acción hacia abajo
            break;

        default : break;
    }

    cursor->GetMotion()->Reset();
}
```

En Canvas2D se utiliza este tipo de lógica para traducir un movimiento del cursor en acciones como caminar hacia una dirección. El código no necesita saber si el movimiento procede del ratón o de una pantalla táctil. Solo recibe una trayectoria normalizada.

10. El patrón de uso en las aplicaciones gráficas

Los ejemplos gráficos muestran un patrón muy claro. Primero se habilita Input en la configuración de la aplicación gráfica. Después, en AppProc_FirstUpdate(), se piden los dispositivos al gestor, se guardan los botones y cursores que interesan, y finalmente en cada actualización se consulta el estado de esos objetos.

```
option(INP_FEATURE                                     "Inputs"
ON )
```

En el arranque de una aplicación gráfica con AppFlow suele aparecer la opción APPFLOWGRAPHICS_INIOPTION_INPUT. Esa opción indica al flujo gráfico que debe preparar la entrada. A partir de ese momento, la aplicación puede utilizar GEN_INPMANAGER para localizar dispositivos.

```
inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_MOUSE);
if(inpdevice)
{
    button[MYAPP_BUTTON_MOUSE] = inpdevice->GetButton(INPBUTTON_ID_MOUSE_LEFT);
    cursor = inpdevice->GetCursor(0);
}

inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_TOUCHSCREEN);
if(inpdevice)
{
    button[MYAPP_BUTTON_TOUCHSCREEN] = inpdevice->GetButton(INPBUTTON_ID_TOUCHSCREEN);
    cursor = inpdevice->GetCursor(0);
}
```

Este fragmento ilustra perfectamente la filosofía del bloque. El mismo miembro cursor puede terminar apuntando al cursor del ratón o al cursor de la pantalla táctil. A partir de ese momento, el resto del código puede

comportarse igual. La aplicación ya no está organizada alrededor de “ratón” o “táctil”, sino alrededor de “cursor” y “botón de selección”.

11. Input y UserInterface

Cuando una aplicación utiliza UserInterface, Input no suele actuar directamente sobre los elementos visuales. Lo habitual es convertir el estado de los botones y cursores en eventos de interfaz. De este modo, UserInterface recibe eventos como movimiento de cursor, selección, pulsación o gesto, sin conocer el dispositivo físico que los produjo.

Este patrón es especialmente potente porque separa tres responsabilidades: la plataforma lee los eventos nativos, Input los normaliza como botones y cursores, y UserInterface los interpreta como acciones sobre elementos visuales. La aplicación actúa como puente cuando necesita adaptar coordenadas, filtrar acciones o traducir botones concretos a eventos de interfaz.

```
if(changed)
{
    GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_CURSOR_MOVE, cursor_x, cursor_y);
}

if(button[MYAPP_BUTTON_MOUSE]->IsPressedWithRelease())
{
    GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_SELECTION, cursor_x, cursor_y);
    cursor->GetMotion()->Reset();
}
```

La misma lógica puede funcionar con pantalla táctil. Si el botón conceptual INPBUTTON_ID_TOUCHSCREEN está pulsado, la aplicación puede enviar un evento de selección usando las coordenadas del cursor táctil. El código de UserInterface no necesita una rama especial para cada tipo de dispositivo.

12. INPCAPTURE e INPCAPTURE_XEVENT

INPCAPTURE está orientado a capturar entrada, especialmente secuencias de teclado, y a publicarlas mediante el sistema de eventos de GEN Framework. Hereda de XSUBJECT, lo que indica que puede actuar como fuente de eventos para observadores interesados. Mantiene un contador de teclas, un límite máximo, un XBUFFER y un XSTRING con el contenido capturado.

INPCAPTURE_XEVENT transporta la información concreta de un evento de captura: código virtual, scan code, flags, estado de bloqueo, número de teclas, límite, buffer y cadena. Este diseño separa el objeto que captura de la notificación enviada al resto del sistema.

La captura de entrada no tiene el mismo papel que la consulta de botones en una aplicación gráfica. Es más adecuada cuando interesa recibir texto, secuencias o eventos globales de teclado. Al estar gestionada también por la factoría, la implementación real puede depender de la plataforma.

```
#ifdef INP_CAPTURE_ACTIVE
INPCapture* capture = GEN_INPFACTORY.CreateCapture();
if(capture)
{
    capture->SetLimit(256);
    capture->Application_SetHandle(applicationhandle);

    // El objeto puede publicar INPCAPTURE_XEVENT cuando se producen teclas.

    GEN_INPFACTORY.DeleteCapture(capture);
}
```

```
#endif
```

13. INPSIMULATE: entrada generada por software

INPSIMULATE representa la cara opuesta de la captura. En lugar de leer entrada física, permite simularla desde software. Sus operaciones permiten pulsar, soltar o hacer clic sobre una tecla, enviar texto, mover el ratón y traducir literales a códigos de teclado teniendo en cuenta teclas alternativas como ALTGR o SHIFT.

Esta funcionalidad puede ser útil para pruebas, automatización, herramientas internas o integración con scripts. Como ocurre con la captura, la implementación real depende de la plataforma. Simular una tecla en Windows, Linux o Android no se hace de la misma manera, por lo que la factoría vuelve a ser el punto de aislamiento.

```
#ifdef INP_SIMULATE_ACTIVE
INPSIMULATE* simulate = GEN_INPFACTORY.CreateSimulator();
if(simulate)
{
    simulate->Key_ClickByLiteral(__L("A"));
    simulate->Key_ClickByText(__L("GEN Framework"), 50);
    simulate->Mouse_SetPos(100, 100);

    GEN_INPFACTORY.DeleteSimulator(simulate);
}
#endif
```

14. Clases de plataforma

El bloque común de Input define los conceptos estables. Las clases de plataforma especializan esos conceptos. En la documentación aparecen implementaciones para Windows, Linux y Android, como dispositivos de teclado, ratón, pantalla táctil, factorías específicas y clases de captura o simulación adaptadas al sistema.

Esta división es fundamental. INPDEVICE define que un dispositivo tiene botones, cursores y un Update() virtual. Una clase concreta de Windows o Linux implementa cómo se lee realmente el teclado o el ratón. INPFACTORY define que se puede pedir un dispositivo; la factoría concreta de la plataforma decide qué clase crear. INPMANAGER conserva los dispositivos ya creados y permite que la aplicación acceda a ellos sin conocer su clase real.

Gracias a este confinamiento, el código de aplicación se mantiene limpio. No debería haber bloques #ifdef WINDOWS o #ifdef LINUX para leer el teclado en una aplicación normal. Esos detalles deben vivir en la zona de plataforma y llegar al programa ya convertidos en botones y cursores.

Grupo	Clases principales	Función dentro de Input
Factoría y gestión	INPFACTORY, INPMANAGER	Creación dependiente de plataforma, registro de dispositivos y actualización global.
Dispositivo	INPDEVICE	Base común para teclado, ratón, joystick, pantalla táctil o mandos.
Botones	INPBUTTON	Representa teclas, botones físicos, pulsaciones táctiles y estados de presión.
Cursores	INPCURSOR, INPCURSORMOTIONPOINT	Representa posición y punto de entrada normalizado.
Movimiento	INPCURSORMOTION	Guarda trayectoria, calcula dirección, ángulo, módulo y gestos simples.
Captura	INPCAPTURE, INPCAPTURE_XEVENT	Captura secuencias o eventos de teclado y los publica como eventos de GEN Framework.
Simulación	INPSIMULATE	Genera entrada por software para pruebas, automatización o integración.

15. Relación entre clases

La relación entre las clases de Input se puede resumir de forma jerárquica. INPFACTORY crea objetos concretos cuando dependen de la plataforma. INPMANAGER conserva los dispositivos disponibles. Cada INPDEVICE contiene una colección de INPBUTTON y una colección de INPCURSOR. Cada INPCURSOR contiene un INPCURSORMOTION, que a su vez contiene una lista de INPCURSORMOTIONPOINT. INPCAPTURE e INPSIMULATE son servicios auxiliares que también pueden depender de la plataforma.

```
INPFACTORY
+-- CreateDevice()      -> INPDEVICE especializado de plataforma
+-- CreateCapture()     -> INPCAPTURE especializado de plataforma
+-- CreateSimulator()   -> INPSIMULATE especializado de plataforma

INPMANAGER
+-- INPDEVICE keyboard
|   +-- INPBUTTON escape, space, arrows, ...
+-- INPDEVICE mouse
|   +-- INPBUTTON left, right, middle
|   +-- INPCURSOR mouse cursor
+-- INPDEVICE touchscreen
|   +-- INPBUTTON touchscreen
|   +-- INPCURSOR touch cursor
```

Esta estructura explica por qué el código de aplicación es tan compacto. Una vez que el dispositivo existe y está registrado, el programa solo necesita pedir el botón o cursor que le interesa. El resto de la complejidad queda por debajo.

16. Ejemplo de esqueleto de entrada en una aplicación gráfica

Un esqueleto típico de una aplicación gráfica de GEN Framework declara punteros a los botones de interés y un cursor común. En la primera actualización se asignan esos punteros desde los dispositivos disponibles. En cada Update se revisan los estados y se traducen a acciones.

```
enum MYAPP_BUTTON
{
    MYAPP_BUTTON_UP = 0,
    MYAPP_BUTTON_DOWN,
    MYAPP_BUTTON_ESC,
    MYAPP_BUTTON_MOUSE,
    MYAPP_BUTTON_TOUCHSCREEN,
    MYAPP_BUTTON_MAX,
};

INPBUTTON* button[MYAPP_BUTTON_MAX];
INPCURSOR* cursor;
bool MYAPP::AppProc_FirstUpdate()
{
    INPDEVICE* inpdevice;

    inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_KEYBOARD);
    if(inpdevice)
    {
        button[MYAPP_BUTTON_UP]    = inpdevice->GetButton(INPBUTTON_ID_UP);
        button[MYAPP_BUTTON_DOWN]  = inpdevice->GetButton(INPBUTTON_ID_DOWN);
        button[MYAPP_BUTTON_ESC]   = inpdevice->GetButton(INPBUTTON_ID_ESCAPE);
    }
}
```

```

inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_MOUSE);
if(inpdevice)
{
    button[MYAPP_BUTTON_MOUSE] = inpdevice->GetButton(INPBUTTON_ID_MOUSE_LEFT);
    cursor = inpdevice->GetCursor(0);
}

inpdevice = GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_TOUCHSCREEN);
if(inpdevice)
{
    button[MYAPP_BUTTON_TOUCHSCREEN] = inpdevice->GetButton(INPBUTTON_ID_TOUCHSCREEN);
    cursor = inpdevice->GetCursor(0);
}

return true;
}

bool MYAPP::UpdateInput()
{
    if(button[MYAPP_BUTTON_ESC])
    {
        if(button[MYAPP_BUTTON_ESC]->IsPressedWithRelease())
        {
            SetEvent(MYAPP_XFSMEVENT_END);
        }
    }

    if(cursor && button[MYAPP_BUTTON_MOUSE])
    {
        if(button[MYAPP_BUTTON_MOUSE]->IsPressedWithRelease())
        {
            int x = (int)cursor->GetX();
            int y = (int)cursor->GetY();

            GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_SELECCTION, x, y);
        }
    }

    return true;
}

```

Este ejemplo no pretende sustituir al código real de los ejemplos del framework, pero muestra el esqueleto conceptual. La aplicación no procesa mensajes nativos de Windows, eventos X11 o callbacks Android. Trabaja con botones, cursores y eventos de alto nivel.

17. Input en aplicaciones sin AppFlow

Aunque Input aparece muy vinculado a las aplicaciones gráficas con AppFlow, su modelo también puede aplicarse en aplicaciones sin AppFlow. En un sistema embebido puede no existir una ventana clásica ni un bucle gráfico completo, pero sí existir botones físicos, sensores táctiles, un pequeño display o un mando conectado por GPIO, I2C o SPI.

En ese caso, la aplicación podría crear o recibir dispositivos de entrada de forma más directa y actualizarlos en su propio bucle. La ventaja de mantener la misma abstracción es que la lógica sigue hablando de botones y cursores, no de pines, interrupciones o registros concretos. La parte de plataforma o de driver traduce lo físico a la forma común de Input.

Esto encaja con la filosofía de GEN Framework: se puede usar el framework completo cuando existe AppFlow, o se pueden utilizar módulos concretos cuando el entorno es más limitado. Lo importante es no perder la separación entre lógica de aplicación e infraestructura de plataforma.

18. Errores habituales al usar Input

Un error habitual es escribir lógica específica para ratón y después añadir otra lógica distinta para pantalla táctil. Esa duplicación rompe la idea del bloque. Lo recomendable es localizar el cursor y el botón de selección disponibles y escribir una sola ruta de comportamiento.

Otro error es consultar dispositivos en cada ciclo de Update en lugar de guardar las referencias de botones y cursor durante la primera actualización. Aunque no siempre sea grave, ensucia el código y repite búsquedas innecesarias.

También conviene distinguir entre estado continuo y transición. Para mover un objeto mientras una tecla está pulsada puede tener sentido `IsPressed()`. Para aceptar una opción de menú suele ser mejor `IsPressedWithRelease()`. Elegir mal esta diferencia provoca acciones repetidas, selecciones dobles o sensación de falta de control en la interfaz.

Por último, hay que recordar que el eje de coordenadas puede cambiar según el origen gráfico. Si la entrada entrega Y desde abajo y el canvas o la interfaz espera Y desde arriba, la aplicación debe adaptar la coordenada antes de enviar el evento. En los ejemplos gráficos se ve esta transformación cuando se calcula la posición que finalmente se envía a `UserInterface`.

19. Catálogo resumido de clases del bloque Input

Clase	Descripción	Ejemplo de uso
INPFACTORY	Factoría de creación de dispositivos, simuladores y capturas dependientes de plataforma.	Pedir un dispositivo de tipo <code>INPDEVICE_TYPE_MOUSE</code> sin conocer su clase real.
INPMANAGER	Gestor singleton de dispositivos registrados.	<code>GEN_INPMANAGER.GetDevice(INPDEVICE_TYPE_KEYBOARD)</code> .
INPDEVICE	Base común de un dispositivo con botones y cursores.	Obtener <code>INPBUTTON_ID_ESCAPE</code> o el cursor principal.
INPBUTTON	Botón normalizado con estado, símbolo, presión y temporización.	Detectar <code>IsPressedWithRelease()</code> para aceptar una acción.
INPCURSOR	Cursor normalizado con posición y movimiento asociado.	Usar <code>GetX()</code> , <code>GetY()</code> e <code>IsPositionInRect()</code> .
INPCURSORMOTIONPOINT	Punto de trayectoria con coordenadas X/Y/Z.	Elemento individual dentro de una trayectoria.
INPCURSORMOTION	Colección y análisis de puntos de movimiento.	Detectar dirección con <code>GetDirectionByAngle()</code> .
INPCAPTURE	Captura de entrada publicada como eventos.	Capturar texto o secuencias de teclado.
INPCAPTURE_XEVENT	Evento con datos de captura.	Transportar <code>vkcode</code> , <code>scancode</code> , <code>flags</code> , <code>buffer</code> y <code>string</code> .
INPSIMULATE	Simulación de teclado y ratón.	Enviar <code>Key_ClickByText()</code> o <code>Mouse_SetPos()</code> .

20. Resumen práctico

Input es el bloque que permite que una aplicación con GEN Framework interprete la interacción del usuario de forma portable. Aparece como complemento natural del sistema gráfico, pero su diseño también resulta útil en entornos más pequeños o embebidos.

Su idea central es reducir todos los dispositivos a botones y cursores. Esta reducción permite que un ratón, una pantalla táctil o un dispositivo distinto puedan alimentar la misma lógica de aplicación. `INPFACTORY` aísla la creación dependiente de plataforma, `INPMANAGER` organiza los dispositivos, `INPDEVICE` agrupa botones y

cursores, `INPBUTTON` representa acciones discretas e `INPCURSOR` junto con `INPCURSORMOTION` representa posición y movimiento.

Al combinar `Input` con `UserInterface`, la aplicación transforma botones y cursores en eventos de interfaz. El resultado es un código más portable, limpio y mantenible.

Capítulo IX: Sound, sonido y reproducción en GEN Framework

1. El último bloque factory de GEN Framework

Después de estudiar XUtils, DataIO, Graphic e Input, el siguiente bloque natural dentro de la arquitectura de GEN Framework es Sound. Este módulo cierra, al menos por el momento, el conjunto de factorías principales del framework. Si XFACTORY proporciona la base común, DIOFACTORY permite comunicar la aplicación con el exterior, GRPFACTORY ofrece salida gráfica e INPFACTORY abstrae la entrada del usuario, SNDFACTORY completa el círculo proporcionando una abstracción de salida sonora.

El sonido no siempre aparece en las primeras aplicaciones. Muchas utilidades de consola no necesitan audio y muchas aplicaciones industriales pueden funcionar únicamente con trazas, ficheros o pantalla. Sin embargo, cuando una aplicación necesita avisos sonoros, música de fondo, efectos, respuesta a eventos, sincronización con animaciones o feedback acústico para el usuario, disponer de una capa integrada evita volver a resolver desde cero un problema que también depende de la plataforma.

El bloque Sound mantiene la misma filosofía que el resto de GEN Framework: la aplicación no debería conocer directamente la API nativa que reproduce el sonido. Debería crear elementos de sonido, configurar cómo se reproducen y pedir a la factoría que los ejecute. La implementación concreta queda por debajo. En la versión estudiada, el backend principal se apoya en OpenAL, pero la idea arquitectónica es más amplia: la aplicación trabaja contra SNDFACTORY y las clases comunes del módulo, no contra llamadas directas de OpenAL repartidas por el código de aplicación.

2. SNDFACTORY y el aislamiento de plataforma

SNDFACTORY es la factoría que centraliza la creación y el control de los elementos sonoros. Igual que ocurre con las demás factorías del framework, su papel no se limita a construir objetos. SNDFACTORY define una frontera. Por encima de esa frontera está el código portable de la aplicación. Por debajo están las implementaciones concretas, los dispositivos, los buffers, los contextos de audio, los hilos de reproducción y las APIs utilizadas para emitir sonido en cada plataforma.

La aplicación usa operaciones de alto nivel: crear un sonido desde un fichero, crear una nota sintética, reproducir, pausar, detener, modificar volumen, modificar pitch o esperar a que termine. La factoría se encarga de convertir esas operaciones en trabajo real. En una implementación basada en OpenAL, por ejemplo, esa conversión implica crear fuentes, buffers, asignar datos PCM, controlar estados de reproducción y mantener un hilo de actualización.

Esta separación es importante por tres razones. La primera es portabilidad: el código de aplicación no se ata a una API de sonido concreta. La segunda es coherencia: todos los sonidos se gestionan mediante los mismos tipos de GEN Framework, como XPATH, XBUFFER, XVECTOR, XEVENT o XFSMACHINE. La tercera es control: el framework puede emitir eventos de sonido, mantener estados y centralizar la destrucción de recursos.

```
// Activación típica del bloque Sound en CMake.
option(SND_FEATURE                               "Sound"
ON )
option(SND_FILE_WAV_FEATURE                       "Sound WAV file"
ON )
option(SND_FILE_OGG_FEATURE                       "Sound OGG file"
ON )
```

Estas opciones muestran la misma idea que vimos en capítulos anteriores: el sistema de compilación decide qué partes del framework se incorporan. Una aplicación que no necesita sonido no tiene por qué cargar esta funcionalidad. Una aplicación gráfica como Canvas2D sí puede activarla para reproducir música y efectos sincronizados con la animación.

3. Mapa general del bloque Sound

El bloque Sound es más pequeño que XUtils, DataIO o Graphic, pero está bastante bien estructurado. Sus clases pueden agruparse en cinco áreas: factoría y eventos, elementos sonoros, formatos de fichero, backend OpenAL y configuración de reproducción.

- Factoría y eventos: SNDFACTORY y SNDFACTORY_XEVENT definen el punto de acceso común y los eventos que notifican cambios de estado en la reproducción.
- Elementos sonoros: SNDITEM representa un sonido reproducible, ya proceda de un fichero o de una nota generada.
- Configuración de reproducción: SNDPLAYCFG encapsula volumen y pitch para aplicar parámetros concretos al reproducir.
- Formatos de fichero: SNDFILE, SNDFILEOGG y SNDFILEWAV cargan y decodifican datos sonoros desde disco.
- Backend OpenAL: SNDOPENALFACTORY, SNDOPENALPLAYITEM, SNDOPENALBUFFER y SNDOPENALSOURCE ejecutan la reproducción real usando OpenAL.

Esta organización permite que el usuario normal del framework se quede en el nivel alto. En la mayoría de aplicaciones basta con preparar rutas mediante XPATHMANAGER, crear elementos con GEN_SNDFACTORY.CreateItem, ajustar una configuración de reproducción y llamar a Sound_Play. Las clases OpenAL son importantes para entender cómo funciona internamente, pero no deberían convertirse en la API habitual de la aplicación.

4. SNDITEM: la unidad reproducible

SNDITEM es la clase que representa una unidad de sonido. Es el objeto que la aplicación conserva cuando quiere reproducir, pausar o detener un sonido. Un SNDITEM puede ser de tipo fichero o de tipo nota. Esta distinción aparece en el enum SNDITEM_TYPE, donde el elemento puede estar en estado desconocido, asociado a una nota o asociado a un fichero de sonido.

Además del tipo, SNDITEM mantiene un estado de reproducción. El estado permite saber si el sonido está sin iniciar, inicializado, reproduciéndose, en pausa, detenido o finalizado. Esto es esencial para evitar reproducir efectos de forma descontrolada. En Canvas2D, por ejemplo, antes de lanzar un sonido de pisada se comprueba si ya se está reproduciendo para no superponer varias copias del mismo efecto en el mismo instante.

La clase también conserva información de repetición: número de veces reproducido, número de veces que debe reproducirse, contador de reproducción, tiempo acumulado, tiempo actual y temporizador interno. De este modo el sistema puede gestionar tanto sonidos que se reproducen una sola vez como sonidos en bucle.

```
// Creación de un item de sonido desde una ruta.
XPATH xpath;
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SOUNDS, xpath);
xpath.Slash_Add();
xpath.Add(__L("efecto.ogg"));

SNDITEM* item = GEN_SNDFACTORY.CreateItem(xpath);
if(item)
{
    GEN_SNDFACTORY.Sound_Play(item, NULL, 1);
}
```

El detalle importante es que la aplicación no crea directamente un SNDFILEOGG ni un buffer OpenAL. La aplicación crea un SNDITEM a través de la factoría. Internamente, el sistema deduce el tipo de fichero por extensión, carga los datos y prepara el elemento para que el backend pueda reproducirlo.

5. SNDPLAYCFG: volumen y pitch por reproducción

SNDPLAYCFG encapsula parámetros de reproducción. En la versión analizada contiene volumen y pitch. El volumen permite ajustar la intensidad de un sonido concreto. El pitch permite alterar la velocidad o tono de reproducción, lo que puede ser útil para variaciones de efectos, avisos o recursos de interfaz.

Conviene distinguir entre la configuración global y la configuración de un sonido concreto. SNDFACTORY dispone de master volume como volumen general, mientras que SNDPLAYCFG permite aplicar parámetros a un ítem específico. Esta separación permite, por ejemplo, bajar la música de fondo sin cambiar el volumen de efectos importantes, o reproducir el mismo recurso con ajustes distintos según el contexto.

```
SNDPLAYCFG playCFG;
playCFG.SetVolume(25);
playCFG.SetPitch(1.0f);

SNDITEM* background = GEN_SNDFACTORY.CreateItem(xpath);
if(background)
{
    GEN_SNDFACTORY.Sound_Play(background, &playCFG, SNDFACTORY_INLOOP);
}
```

El valor SNDFACTORY_INLOOP indica que el sonido debe reproducirse en bucle. Esta constante es muy práctica para música de fondo o sonidos ambientales. Para efectos puntuales, como una pisada, se utiliza normalmente una repetición única.

6. Sonidos desde fichero: SNDFILE, OGG y WAV

El sistema de ficheros de sonido se agrupa en FormatFiles. La clase base es SNDFILE. Su responsabilidad es representar un fichero sonoro ya cargado y convertido a datos que el backend puede entregar al dispositivo de audio. Mantiene la ruta, número de canales, número de muestras, frecuencia de muestreo, duración y el buffer con los datos decodificados.

SNDFILE actúa como punto de creación. A partir de la extensión del fichero decide qué clase concreta debe instanciar. En el código revisado se contemplan ficheros OGG y WAV. Los OGG se gestionan mediante SNDFILEOGG y los WAV mediante SNDFILEWAV. Esta separación evita que la aplicación tenga que conocer el decodificador concreto.

SNDFILEOGG carga el fichero, lee su contenido en un XBUFFER y decodifica Vorbis usando stb_vorbis. Al final obtiene canales, frecuencia de muestreo, número de muestras, duración y datos PCM intercalados. SNDFILEWAV, por su parte, se apoya en el lector RIFF del framework para localizar los chunks fmt y data, leer el formato y obtener los datos de audio.

```
// El tipo concreto se decide por extensión.
SNDFILE* soundfile = SNDFILE::Create(__L("assets/sounds/greensleeves.ogg"));
if(soundfile)
{
    XDWORD duration = soundfile->GetDuration();
    XDWORD samplerate = soundfile->GetSampleRate();
    XWORD channels = soundfile->GetChannels();
}
```

Aunque una aplicación normal no necesita crear SNDFILE directamente, entender esta capa ayuda a comprender el flujo completo: fichero en assets/sounds, carga mediante XPATH, decodificación a XBUFFER, asignación a buffer OpenAL y reproducción mediante fuente de audio.

7. SNDNOTE: sonidos generados por frecuencia y duración

No todos los sonidos tienen que venir de un fichero. GEN Framework también permite crear un item a partir de una frecuencia y una duración. En ese caso se utiliza SNDNOTE. Esta clase es mínima: contiene frecuencia y duración. Su utilidad aparece cuando se quieren generar tonos, avisos sencillos o señales acústicas sin depender de un recurso externo.

Internamente, el backend OpenAL puede generar un buffer a partir de esa nota. SNDOPENALBUFFER incluye métodos GenerateNote que rellenan un XBUFFER con muestras calculadas, y después Assign entrega esos datos a OpenAL. Es un ejemplo pequeño pero interesante: el framework no solo reproduce recursos, también puede sintetizar una señal básica.

```
// Crear y reproducir un tono simple de 440 Hz durante 250 ms.
SNDITEM* beep = GEN_SNDFACTORY.CreateItem(440, 250);
if(beep)
{
    SNDPLAYCFG cfg;
    cfg.SetVolume(50);
    GEN_SNDFACTORY.Sound_Play(beep, &cfg, 1);
}
```

Este tipo de funcionalidad es especialmente útil en herramientas de diagnóstico, interfaces simples, pequeños dispositivos o aplicaciones donde se quiere emitir una señal sonora sin incluir ficheros de audio.

8. Eventos de sonido

SNDFACTORY hereda de XSUBJECT. Esto significa que el sistema de sonido puede publicar eventos dentro del modelo de observadores del framework. La clase SNDFACTORY_XEVENT encapsula esos eventos y permite asociarlos al SNDITEM implicado. Los eventos principales son inicialización, reproducción, pausa, parada y finalización.

Esta integración con XEVENT es muy importante porque permite que una aplicación observe el comportamiento del audio sin bloquear el flujo principal. Una aplicación puede saber cuándo empezó un sonido, cuándo terminó o cuándo se detuvo, y reaccionar en consecuencia. En una interfaz de usuario se podría activar un indicador visual; en un juego o demo gráfica se podría coordinar una animación; en una herramienta industrial se podría registrar una alarma sonora.

```
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_INI    , &GEN_SNDFACTORY.GetInstance());
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_PLAY   , &GEN_SNDFACTORY.GetInstance());
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_PAUSE  , &GEN_SNDFACTORY.GetInstance());
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_STOP   , &GEN_SNDFACTORY.GetInstance());
SubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_END    , &GEN_SNDFACTORY.GetInstance());
```

El patrón es el mismo que en otros módulos de GEN Framework: el productor del evento es la factoría, y la aplicación o clase observadora se suscribe a los tipos de evento que le interesan.

9. SNDOPENALFACTORY: implementación de reproducción

SNDOPENALFACTORY es la implementación concreta basada en OpenAL. Hereda de SNDFACTORY y proporciona la funcionalidad real que la clase base solo define como contrato. En Ini abre el dispositivo de audio, crea el contexto OpenAL, lo hace actual, crea un mutex, prepara un hilo de reproducción y marca el sistema como activo.

La existencia de un hilo y un mutex no es accidental. La reproducción sonora no debería depender de que la aplicación se quede bloqueada esperando a que acabe un efecto. El sistema debe poder actualizar el estado de los sonidos, comprobar tiempos, detectar finales, reiniciar bucles y publicar eventos mientras la aplicación continúa

su ciclo normal. Por eso el backend mantiene una lista de SNDOPENALPLAYITEM y ejecuta Update de forma periódica.

SNDOPENALFACTORY también limita el número de canales simultáneos mediante maxchannels. En el código estudiado se inicializa a 16. Este límite evita crear fuentes de forma ilimitada y permite controlar mejor el coste de reproducción. Cuando se lanza Sound_Play, se crea una fuente OpenAL, un play item, se aplica volumen y pitch, se configura si va en bucle y se lanza la máquina de estados.

```
bool SNDOPENALFACTORY::Ini()
{
    device = alcOpenDevice(NULL);
    context = alcCreateContext(device, NULL);

    alcMakeContextCurrent(context);

    playmutex = GEN_XFACTORY.Create_Mutex();
    playthread = GEN_XFACTORY.CreateThread(XTHREADGROUPID_SOUND,
        __L("SNDOPENALFACTORY::SNDOPENALFACTORY"),
        SNDOPENALFACTORY::ThreadPlay,
        this);

    soundactive = true;
    return true;
}
```

El fragmento anterior resume el concepto, no pretende sustituir al código real. Lo que importa para el lector es entender que SNDFACTORY ofrece la interfaz y SNDOPENALFACTORY se ocupa del dispositivo, contexto, buffers, fuentes, hilos y estados internos.

10. Buffer, source y play item

En OpenAL hay tres conceptos internos que aparecen reflejados en las clases de GEN Framework.

SNDOPENALBUFFER representa el buffer de audio, es decir, el bloque de muestras que se entrega al dispositivo. SNDOPENALSOURCE representa la fuente que reproduce ese buffer. SNDOPENALPLAYITEM une un SNDITEM con una fuente y añade una máquina de estados para controlar su vida.

SNDOPENALBUFFER puede crearse, destruirse, recibir datos y asignarlos a OpenAL. Si el sonido procede de un fichero, el buffer se llena con los datos decodificados de SNDFILE. Si procede de una nota, se genera una onda a partir de la frecuencia y duración. SNDOPENALSOURCE controla operaciones como Play, Stop, Pause, UnPause, volumen, pitch, bucle y estado real de la fuente.

SNDOPENALPLAYITEM es especialmente interesante porque no se limita a ser una estructura de unión. Hereda de XFSMACHINE. Esto permite modelar el proceso de reproducción como estados: NONE, INI, PLAY, PAUSE, STOP y END. Esta idea encaja con lo visto en el capítulo IV: una máquina de estados permite separar lógica continua y transiciones puntuales.

```
// Relación conceptual interna.
SNDITEM
    -> describe qué sonido se quiere reproducir.
SNDOPENALPLAYITEM
    -> asocia el item con una fuente y controla su estado.
SNDOPENALSOURCE
    -> ejecuta Play, Pause, Stop y controla volumen/pitch.
SNDOPENALBUFFER
    -> contiene las muestras que se entregan a OpenAL.
```

11. La máquina de estados de reproducción

El sonido también utiliza una máquina de estados finitos. Esto es coherente con el resto de GEN Framework y permite que la reproducción sea controlable. Cuando se solicita `Sound_Play`, el item no se reproduce de cualquier manera. Se crea un play item, se prepara su fuente, se asignan parámetros y se establece el evento de inicialización. A partir de ahí, `Update` procesa la transición.

Durante el estado `PLAY`, el sistema mide el tiempo transcurrido, actualiza el tiempo de reproducción y decide si el sonido ha terminado. Si el item está en bucle, comprueba si la fuente dejó de sonar para relanzarla o mantener la reproducción. Si se trata de un sonido con un número concreto de repeticiones, decrementa el contador y reinicia la fuente cuando corresponde. Cuando ya no quedan repeticiones, transita a `STOP` o `END` y publica el evento correspondiente.

Este comportamiento es importante porque permite reproducir efectos sin bloquear la aplicación. La aplicación lanza el sonido y sigue dibujando, leyendo entrada o procesando comunicaciones. El sistema de sonido mantiene internamente el estado.

12. Sound dentro de una aplicación gráfica

El ejemplo `Canvas2D` muestra bien por qué el módulo `Sound` aparece después de `Graphic` e `Input`. Una aplicación gráfica puede dibujar una escena, mover un personaje, leer entrada del usuario y, además, sincronizar sonidos con la animación. Esa sincronización no debería obligar a escribir código de audio de bajo nivel dentro del render.

`Canvas2D` activa el módulo de sonido en `CMake` y coloca los recursos sonoros en `assets/sounds`. Durante la inicialización obtiene la ruta de la sección de sonidos mediante `XPATHMANAGER`, carga una música de fondo y la reproduce en bucle. Después carga dos sonidos de pisada, que se reproducen en momentos concretos de la animación del personaje.

```

XPATH xpathsounds;
XPATH xpath;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SOUNDS, xpathsounds);
xpathsounds.Slash_Add();

xpath = xpathsounds;
xpath.Add(__L("greensleeves.ogg"));

backgroundsound = GEN_SNDFACTORY.CreateItem(xpath);
if(backgroundsound)
{
    GEN_SNDFACTORY.Sound_Play(backgroundsound, &playCFGsound, SNDFACTORY_INLOOP);
    GEN_SNDFACTORY.Sound_SetVolume(backgroundsound, 25);
}

```

Este fragmento muestra el patrón recomendado. Primero se obtiene la ruta correcta mediante `XPATHMANAGER`. Después se construye el path al recurso. Finalmente, la factoría crea el item y lo reproduce. No hay rutas absolutas ni dependencia directa de `OpenAL` en la lógica de la aplicación.

13. Sincronización de pisadas en Canvas2D

La demostración de `Canvas2D` es especialmente didáctica porque sincroniza los sonidos de las pisadas con fotogramas concretos del personaje. El personaje tiene una secuencia de animación. Cuando el frame actual alcanza determinados índices, la aplicación reproduce un sonido de armadura caminando. Esto crea la sensación de que el audio forma parte de la animación, no de que suena de forma independiente.

El código comprueba el frame de la secuencia y solo reproduce el efecto si el SNDITEM correspondiente no está ya en estado PLAY. Este detalle evita saturar la salida sonora con múltiples reproducciones solapadas del mismo efecto. Es una técnica simple, pero refleja muy bien cómo debe integrarse Sound en el ciclo gráfico.

```
if(charactersequence[indexcharactersequence]->GetActualFrameIndex() == 4)
{
    if(armorwalkingsounds[0]->GetStatus() != SNDITEM_STATUS_PLAY)
    {
        playCFGsound.SetVolume(10);
        GEN_SNDFACTORY.Sound_Play(armorwalkingsounds[0], &playCFGsound, 1);
    }
}

if(charactersequence[indexcharactersequence]->GetActualFrameIndex() == 17)
{
    if(armorwalkingsounds[1]->GetStatus() != SNDITEM_STATUS_PLAY)
    {
        playCFGsound.SetVolume(10);
        GEN_SNDFACTORY.Sound_Play(armorwalkingsounds[1], &playCFGsound, 1);
    }
}
```

La lógica es clara: la animación decide el momento, pero la reproducción la gestiona SNDFACTORY. Esto permite mantener separado el comportamiento visual del detalle técnico de audio. El programador piensa en términos de “cuando el pie toca el suelo, lanzo una pisada”, no en términos de buffers, fuentes o estados de OpenAL.

14. assets/sounds y organización de recursos

Como vimos en el capítulo IV, los recursos de una aplicación con GEN Framework se colocan bajo assets. El sonido sigue la misma regla. Los ficheros de audio deben estar dentro de la sección de sonidos, normalmente assets/sounds. De esta forma XPATHMANAGER puede proporcionar una ruta homogénea y la aplicación no depende de una ubicación absoluta.

```
Canvas2D/
├── assets/
│   ├── sounds/
│   │   ├── greensleeves.ogg
│   │   ├── armour-walking1.ogg
│   │   └── armour-walking2.ogg
│   ├── graphics/
│   └── fonts/
└── Sources/
```

Esta estructura facilita el empaquetado y la portabilidad. Una aplicación de escritorio, una compilación Linux, una aplicación Android o una versión embebida pueden mantener el mismo criterio lógico aunque el despliegue final cambie. El código no debería asumir rutas fijas del sistema operativo, sino pedir la sección correspondiente a XPATHMANAGER.

15. Patrón de cierre y limpieza

El cierre del sonido es tan importante como la inicialización. Si una aplicación se suscribe a eventos, debe desuscribirse. Si creó sonidos, debe detenerlos y liberar los items. Si la factoría creó hilos, fuentes, buffers o

contextos, estos deben finalizar de forma ordenada. GEN Framework insiste mucho en esta disciplina: Ini crea, End destruye y Clean deja los miembros en un estado conocido.

```
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_INI    , &GEN_SNDFACTORY.GetInstance());
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_PLAY   , &GEN_SNDFACTORY.GetInstance());
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_PAUSE  , &GEN_SNDFACTORY.GetInstance());
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_STOP   , &GEN_SNDFACTORY.GetInstance());
UnSubscribeEvent(SNDFACTORY_XEVENT_TYPE_SOUND_END    , &GEN_SNDFACTORY.GetInstance());

GEN_SNDFACTORY.Sound_StopAll();
GEN_SNDFACTORY.DeleteAllItems();
```

En una aplicación real, dejar sonidos activos o fuentes sin liberar puede producir errores difíciles de detectar: bloqueos al cerrar, recursos de audio ocupados, hilos vivos o referencias a items ya destruidos. Por eso el patrón de cierre no debe considerarse opcional.

16. Relación con otros módulos del framework

Sound no trabaja aislado. Se apoya en muchas piezas ya estudiadas. Usa XUtils para rutas, cadenas, buffers, vectores, temporizadores, hilos, mutexes, eventos y máquinas de estado. Se relaciona con AppFlow porque la inicialización y el cierre se integran en el ciclo de vida de la aplicación. Se relaciona con Graphic cuando el sonido acompaña a una animación. Se relaciona con Input cuando una acción de usuario dispara un efecto. Y se relaciona con XPATHMANAGER para localizar recursos de forma homogénea.

También tiene una relación natural con DIO, aunque en el código de alto nivel no siempre se vea. Los ficheros de sonido son datos que se leen desde disco, y esa lectura termina apoyándose en clases de fichero, buffers y formatos. El módulo WAV utiliza RIFF, y OGG utiliza decodificación Vorbis sobre un buffer. Por tanto, Sound se beneficia de la infraestructura común sin obligar al programador de aplicación a manejar todos esos detalles.

17. Catálogo resumido de clases del bloque Sound

SNDFACTORY es la clase principal del módulo. Define el singleton, registra eventos, mantiene la lista de SNDITEM, gestiona el volumen maestro, crea items desde fichero o nota, y ofrece operaciones de reproducción, pausa, parada, espera, volumen y pitch. La aplicación debería acceder al sonido a través de esta factoría.

SNDFACTORY_XEVENT representa los eventos del sistema de sonido. Transporta el tipo de evento y el SNDITEM asociado. Permite integrar el sonido en el sistema de observadores del framework.

SNDITEM representa un sonido reproducible. Puede estar asociado a un SNDFILE o a un SNDNOTE. Guarda ID, tipo, estado, contadores de reproducción, tiempos, temporizador y configuración de reproducción.

SNDPLAYCFG encapsula los parámetros de reproducción. En la versión revisada contiene volumen y pitch, y permite copiar configuración entre instancias.

SNDNOTE representa un sonido generado por frecuencia y duración. Es útil para avisos simples o señales acústicas que no dependen de ficheros externos.

SNDFILE es la base común para ficheros de sonido. Mantiene ruta, canales, muestras, frecuencia, duración y buffer de datos decodificados. Su método de creación selecciona la clase concreta a partir de la extensión.

SNDFILEOGG carga y decodifica ficheros OGG/Vorbis. Usa stb_vorbis para convertir el contenido del fichero en datos PCM que después puede usar el backend de audio.

SNDFILEWAV carga ficheros WAV. Se apoya en la estructura RIFF para leer formato y datos, calcular tamaño, frecuencia, canales y duración.

`SNDOOPENALFACTORY` implementa la reproducción sobre OpenAL. Abre dispositivo, crea contexto, gestiona hilos, mutex, lista de reproducciones activas y publica eventos de estado.

`SNDOOPENALPLAYITEM` une un `SNDITEM` con una `SNDOOPENALSOURCE` y controla su ciclo mediante `XFSMACHINE`. Es la pieza que permite tratar cada reproducción como una máquina de estados independiente.

`SNDOOPENALSOURCE` representa una fuente OpenAL. Controla Play, Stop, Pause, volumen, pitch, bucle, offsets, cola y estado real de reproducción.

`SNDOOPENALBUFFER` representa el buffer OpenAL. Crea y destruye buffers, conserva un `XBUFFER` auxiliar, genera notas y asigna datos PCM mono o estéreo a OpenAL.

18. Criterios de uso en una aplicación con GEN Framework

El primer criterio es usar siempre la factoría. Una aplicación con GEN Framework no debería crear directamente fuentes OpenAL ni buffers OpenAL. Ese detalle pertenece al backend. El código de aplicación debe trabajar con `SNDITEM`, `SNDPLAYCFG` y `GEN_SNDFACTORY`.

El segundo criterio es organizar los recursos en `assets/sounds` y acceder mediante `XPATHMANAGER`. Esto mantiene la misma disciplina que gráficos, fuentes, layouts o bases de datos.

El tercer criterio es no reproducir efectos sin control. Antes de lanzar sonidos muy frecuentes conviene comprobar el estado del item, como hace `Canvas2D` con las pisadas. Esto evita solapamientos no deseados y reduce carga.

El cuarto criterio es suscribirse a eventos cuando el estado del sonido sea relevante. Si a la aplicación no le importa cuándo termina un sonido, no necesita eventos. Si debe reaccionar al final de una alarma, música o efecto, `SNDFACTORY_XEVENT` es el camino natural.

El quinto criterio es cerrar de forma ordenada. `Sound_StopAll`, `DeleteAllItems` y la desuscripción de eventos deben formar parte del cierre de la aplicación cuando se ha utilizado el módulo de sonido.

19. Resumen práctico

El bloque `Sound` completa el conjunto de factorías principales de GEN Framework. Su objetivo es abstraer la salida sonora de la misma forma que `Graphic` abstrae la salida visual e `Input` abstrae la entrada del usuario. La aplicación crea items de sonido, configura su reproducción y solicita acciones a `SNDFACTORY`. El backend concreto se encarga de los detalles técnicos.

El módulo permite trabajar con sonidos desde fichero, especialmente OGG y WAV en la versión revisada, y también con notas generadas mediante frecuencia y duración. `SNDPLAYCFG` permite ajustar volumen y pitch. `SNDFACTORY_XEVENT` integra el sonido en el sistema de eventos del framework.

La implementación OpenAL muestra cómo GEN Framework encapsula una API concreta bajo una interfaz común. `SNDOOPENALFACTORY` gestiona dispositivo, contexto, hilos, mutex, buffers, fuentes y máquinas de estado de reproducción. Para el programador de aplicación, sin embargo, el patrón sigue siendo simple: localizar el recurso con `XPATHMANAGER`, crear el `SNDITEM`, reproducirlo con `GEN_SNDFACTORY` y cerrarlo correctamente.

`Canvas2D` es un buen ejemplo de integración real. La música de fondo se reproduce en bucle y las pisadas del personaje se sincronizan con frames concretos de la animación. Esa es la filosofía de GEN Framework aplicada al sonido: integrar funcionalidad práctica en la aplicación sin romper la arquitectura ni contaminar el código con detalles de plataforma.

Capítulo X: Cipher, encriptación y hashes en GEN Framework

1. El objetivo del capítulo

Después de revisar XUtils, DataIO, Graphic, Input y Sound, el siguiente bloque que conviene estudiar es Cipher. En GEN Framework, Cipher agrupa las clases relacionadas con criptografía práctica: cálculo de hashes, checksums, cifrado simétrico, cifrado asimétrico, intercambio de claves, manejo de claves, certificados X.509 y lectura de contenedores de claves en formatos propios o estándar.

Este bloque no debe verse como una colección aislada de algoritmos. En una aplicación real, la seguridad aparece mezclada con muchas otras partes del framework. Un hash puede utilizarse para comprobar la integridad de un fichero leído con XFile, un cifrado simétrico puede proteger un XBUFFER antes de enviarlo por un DIOStream, una clave RSA puede cargarse desde assets/certificates mediante XPATHSMANAGER y un certificado X.509 puede formar parte de una conexión segura o de un proceso de validación.

El nombre del bloque, Cipher, se utiliza en un sentido amplio. Incluye cifrado, descifrado, generación de firmas, obtención de resúmenes, checksums y manejo de claves. No todo lo que está dentro de Cipher cifra datos. Un hash, por ejemplo, no cifra. Produce una huella o resumen que permite identificar datos, comprobar integridad o alimentar otros procesos criptográficos. Esta diferencia es importante y conviene tenerla clara desde el principio.

Como en otros módulos de GEN Framework, el diseño está orientado a ser práctico. Las clases trabajan con los tipos habituales del framework: XBUFFER, XSTRING, XPATH, XFILE, XMPINTEGER, XVECTOR y los mecanismos de memoria y ciclo de vida propios de GEN Framework. Esto permite que el bloque Cipher encaje con el resto del framework sin introducir una forma externa de programar.

2. Hash, checksum, cifrado y firma no son lo mismo

Una aplicación puede necesitar seguridad o integridad de muchas maneras distintas. A veces basta con saber si un bloque de datos ha cambiado. En ese caso puede servir un checksum como CRC16 o CRC32. Otras veces se necesita una huella criptográfica para identificar un contenido de forma mucho más robusta; ahí aparecen MD5, SHA1, SHA2 o Whirlpool. En otros escenarios hay que ocultar el contenido mediante cifrado simétrico. Y cuando intervienen identidad, intercambio de claves o firma, entran en juego algoritmos asimétricos como RSA o mecanismos de curva elíptica como X25519.

GEN Framework separa estas necesidades en clases diferentes. HASH y sus derivadas calculan resúmenes. CIPHER y sus derivadas transforman datos con una clave para cifrar o descifrar. Las clases CIPHERKEY representan claves. Las clases CIPHERKEYSFILEGKF y CIPHERKEYSFILEPEM se encargan de leer o mantener claves y certificados en ficheros. Las clases de certificado X.509 almacenan y procesan información propia de certificados.

Esta separación ayuda a evitar confusiones. No se debe usar un hash como si fuera cifrado, porque no permite recuperar el dato original. No se debe usar un checksum simple como si fuera una prueba criptográfica fuerte, porque su objetivo principal es detectar errores o cambios accidentales. Tampoco se debe inventar un protocolo de seguridad combinando piezas al azar. El bloque Cipher proporciona herramientas, pero la aplicación debe utilizarlas con criterio.

3. Mapa general del bloque Cipher

Las clases de Cipher pueden agruparse por responsabilidad. Esta agrupación facilita entender el bloque antes de entrar clase por clase.

Grupo	Clases principales	Responsabilidad
-------	--------------------	-----------------

Base de hash	HASH	Interfaz común para calcular resúmenes sobre memoria, XBUFFER, XFILE o rutas XPATH.
Hashes y checksums	HASHCKS16, HASHCRC16, HASHCRC32, HASHMD5, HASHSHA1, HASHSHA2, HASHWHIRLPOOL, HASHCOMP128V1	Implementaciones concretas de checksum, hash criptográfico o algoritmo especializado.
Base de cifrado	CIPHER	Interfaz común para cifrar, descifrar, definir modo de encadenamiento, padding, vector de inicialización, claves y resultado.
Cifrado simétrico	CIPHERDES, CIPHER3DES, CIPHERAES, CIPHERBLOWFISH	Algoritmos de cifrado por clave compartida sobre bloques o buffers.
Claves	CIPHERKEY, CIPHERKEYSYMMETRICAL, CIPHERKEYPUBLICRSA, CIPHERKEYPRIVATERSA, CIPHERKEYECDSA	Representación común de claves simétricas, RSA y ECDSA.
Ficheros de claves	CIPHERKEYSFILEGKF, CIPHERKEYSFILEPEM	Lectura, mantenimiento, exportación o decodificación de claves y certificados.
Cifrado asimétrico	CIPHERRSA, CIPHERRSA_CONTEXT	Cifrado, descifrado, firma y generación de claves RSA mediante enteros multiprecisión.
Intercambio de claves	CIPHERECDSAX25519	Generación de clave privada, clave pública y clave compartida usando Curve25519/X25519.
Certificados	CIPHERCERTIFICATEX509, CIPHERCERTIFICATEX509_ID, CIPHERTRUSTEDROOTCERTIFICATESX509	Representación de certificados X.509, identidad del certificado y raíces de confianza.

4. La clase HASH

HASH es la interfaz común para las clases de resumen. Su valor principal es que permite tratar de forma homogénea algoritmos muy distintos. La aplicación puede llamar a Do sobre un bloque de memoria, sobre un XBUFFER, sobre una ruta XPATH o sobre un XFILE abierto. Después puede recuperar el resultado como XBUFFER, como puntero a bytes o como cadena hexadecimal mediante GetResultString.

Esta clase se apoya en XBuffer para almacenar el resultado y en XFile/XPath para operar directamente sobre ficheros. Esa integración con XUtils es importante: el cálculo de hashes no queda limitado a arrays de bytes, sino que puede incorporarse al flujo habitual de una aplicación con GEN Framework.

```

XBUFFER input;
XSTRING text;
XSTRING result;

text = __L("The quick brown fox jumps over the lazy dog");
input.Add((XBYTE*)text.Get(), text.GetSize());

HASH* hash = GEN_NEW HASHSHA2(HASHSHA2TYPE_256);
if(hash)
{
    hash->Do(input);
    hash->GetResultString(result);

    XTRACE_PRINTCOLOR(XTRACE_COLOR_BLUE, __L("SHA2-256: %s"), result.Get());

    GEN_DELETE hash;
}

```

El mismo patrón puede aplicarse a un fichero completo. La aplicación obtiene la ruta mediante XPATHMANAGER y deja que HASH lea el contenido por bloques internos.

```

XPATH   xpath;
XSTRING result;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpath);
xpath.Slash_Add();
xpath.Add(__L("data.bin"));

HASHCRC32 crc32;
crc32.Do(xpath);
crc32.GetResultString(result);

```

5. Checksums y hashes rápidos

HASHCKS16, HASHCRC16 y HASHCRC32 cubren necesidades de comprobación rápida. No deben confundirse con hashes criptográficos fuertes. Su utilidad típica está en detectar errores accidentales, validar bloques en protocolos sencillos, comprobar integridad básica de ficheros o comparar cambios en datos donde no hay un atacante intentando generar colisiones.

HASHCKS16 representa un checksum de 16 bits. HASHCRC16 y HASHCRC32 implementan CRC de 16 y 32 bits. Estas clases heredan o se apoyan en la interfaz HASH, pero además exponen métodos específicos como GetResultCRC16, GetResultCRC32 o Update para cálculo incremental.

Clase	Uso típico	Ejemplo de resultado
HASHCKS16	Checksum simple de 16 bits para validación ligera.	XWORD mediante GetResultCKS16().
HASHCRC16	CRC de 16 bits en protocolos, tramas o dispositivos.	XWORD mediante GetResultCRC16().
HASHCRC32	CRC de 32 bits para ficheros, buffers y comprobaciones rápidas.	XDWORD mediante GetResultCRC32().

```

XBUFFER   frame;
HASHCRC16 crc16;

frame.Add(data, size);
crc16.Do(frame);

XWORD value = crc16.GetResultCRC16();

```

6. Hashes criptográficos generales

HASHMD5, HASHSHA1, HASHSHA2 y HASHWHIRLPOOL representan hashes criptográficos de propósito general. La clase común HASH permite utilizarlos con el mismo flujo: crear el objeto, llamar a Do, obtener el resultado y liberar la instancia si se creó dinámicamente.

MD5 y SHA1 siguen apareciendo en muchas bases de código por compatibilidad, identificación histórica de ficheros o integración con sistemas antiguos, pero no deberían ser la elección principal para nuevas funciones de

seguridad. Para nuevos desarrollos, el bloque SHA2 suele ser la opción más razonable dentro de estas clases, especialmente SHA-256 o SHA-512, según la necesidad de tamaño de salida y compatibilidad.

HASHSHA2 permite seleccionar el tamaño mediante HASHSHA2TYPE. En DevTestsConsole se prueban SHA2-224, SHA2-256, SHA2-384 y SHA2-512 usando el mismo patrón de uso.

Clase	Tipo	Uso recomendado dentro de una aplicación con GEN Framework
HASHMD5	Hash de 128 bits	Compatibilidad, huellas históricas, no como base nueva de seguridad.
HASHSHA1	Hash de 160 bits	Compatibilidad con sistemas antiguos, certificados o protocolos heredados.
HASHSHA2	SHA-224, SHA-256, SHA-384, SHA-512	Opción principal para nuevas huellas criptográficas dentro del bloque.
HASHWHIRLPOOL	Hash de 512 bits	Huella robusta de tamaño grande, útil cuando se desea una salida amplia.

```

XBUFFER input;
XSTRING result;

input.Add((XBYTE*)buffer, size);

HASHSHA2 sha512(HASHSHA2TYPE_512);
sha512.Do(input);
sha512.GetResultString(result);

```

7. HASHCOMP128V1

HASHCOMP128V1 es una clase más especializada. No responde al patrón típico de hash de fichero o buffer genérico, sino a un algoritmo asociado a autenticación GSM clásica. Trabaja con RAND y Ki, y genera dos resultados diferenciados: SRES y Kc. Por esa razón expone métodos específicos como SetInput, GetSRES y GetKc.

En una aplicación con GEN Framework normal no suele ser la primera clase de hash que se utiliza. Su presencia muestra que el bloque Cipher no solo contiene algoritmos genéricos, sino también piezas concretas que aparecieron por necesidades reales de integración o comunicación.

```

XBYTE rand[HASHCOMP128V1_RAND_SIZE];
XBYTE ki[HASHCOMP128V1_KI_SIZE];
XBUFFER input;
XBUFFER sres;
XBUFFER kc;

HASHCOMP128V1 comp128;
comp128.SetInput(rand, ki, input);
comp128.Do(input);
comp128.GetSRES(sres);
comp128.GetKc(kc);

```

8. La clase CIPHER

CIPHER es la base común para las clases que transforman datos mediante una clave. Su responsabilidad es mayor que la de una simple función de cifrado. Mantiene el tipo de cifrado, el modo de encadenamiento, el tipo de padding, el ajuste de padding, las claves, el vector de inicialización y el buffer de resultado.

La interfaz pública ofrece Cipher y Uncipher sobre punteros de bytes o sobre XBUFFER. Después de ejecutar la operación, el resultado se obtiene con GetResult. Este diseño encaja con la forma de trabajar del framework: los datos entran en un XBUFFER, el algoritmo produce otro XBUFFER, y el resultado se puede pasar a DIO, Compression, Databases, un fichero o una capa de red.

CIPHER también define los modos de encadenamiento CIPHERCHAININGMODE_ECB, CBC, CFB y CTR. No todos los algoritmos implementan todos los modos de la misma manera, pero la interfaz permite configurar el comportamiento desde un punto común. En los tests de DevTestsConsole se usa CBC con padding de ceros y un vector de inicialización.

Concepto	Métodos relacionados	Función
Tipo de cifrado	GetType(), SetType()	Identifica el algoritmo concreto.
Modo de encadenamiento	GetChainingMode(), SetChainingMode()	Selecciona ECB, CBC, CFB o CTR.
Padding	GetPaddingType(), SetPaddingType()	Ajusta datos a tamaño de bloque cuando es necesario.
Clave	GetKey(), SetKey()	Asocia la clave correspondiente al algoritmo.
Vector de inicialización	GetInitVector(), SetInitVector()	Aporta el IV para modos que lo requieren.
Resultado	GetResult(), GetResultString()	Devuelve el buffer generado por Cipher o Uncipher.

9. Cifrado simétrico

El cifrado simétrico utiliza una misma clave para cifrar y descifrar. Es el mecanismo habitual para proteger grandes volúmenes de datos porque suele ser mucho más rápido que el cifrado asimétrico. En GEN Framework, las clases principales de esta familia son CIPHERDES, CIPHER3DES, CIPHERAES y CIPHERBLOWFISH, además de la implementación base CIPHER que actúa como XOR sencillo.

La clase CIPHERKEYSYMMETRICAL almacena la clave como un XBUFFER. La misma clave puede asociarse a un CIPHER concreto mediante SetKey. El algoritmo se configura con modo de encadenamiento, padding y vector de inicialización. El patrón de uso se repite entre DES, 3DES, AES y Blowfish.

Clase	Descripción	Comentario práctico
CIPHER	Implementación base tipo XOR.	Útil como base o prueba, no como cifrado fuerte.
CIPHERDES	DES de bloque de 64 bits.	Algoritmo heredado; útil para compatibilidad.
CIPHER3DES	Triple DES basado en DES.	Compatibilidad con sistemas antiguos que aún lo exigen.
CIPHERAES	AES con bloques de 128 bits.	Opción simétrica principal en nuevos desarrollos.
CIPHERBLOWFISH	Blowfish con clave variable.	Alternativa simétrica presente en el bloque.

```
XBYTE keydata[] = { 0x60, 0x3d, 0xeb, 0x10, 0x15, 0xca, 0x71, 0xbe,
                    0x2b, 0x73, 0xae, 0xf0, 0x85, 0x7d, 0x77, 0x81,
```

```

        0x1f, 0x35, 0x2c, 0x07, 0x3b, 0x61, 0x08, 0xd7,
        0x2d, 0x98, 0x10, 0xa3, 0x09, 0x14, 0xdf, 0xf4 };

XBYTE inivector[] = { 0x00, 0x01, 0x02, 0x03,
                      0x04, 0x05, 0x06, 0x07,
                      0x08, 0x09, 0x0A, 0x0B,
                      0x0C, 0x0D, 0x0E, 0x0F };

XBUFFER input;
XBUFFER ciphered;
CIPHERKEYSYMMETRICAL key;
CIPHERAES cipher;

key.Set(keydata, sizeof(keydata));
input.Add(data, size);

cipher.SetChainingMode(CIPHERCHAININGMODE_CBC);
cipher.SetPaddingType(XBUFFER_PADDINGTYPE_ZEROS);
cipher.SetInitVector(inivector, sizeof(inivector));
cipher.SetKey(&key);

if(cipher.Cipher(input))
{
    ciphered.CopyFrom((*cipher.GetResult()));
}

```

Para descifrar se crea o reutiliza una instancia configurada con la misma clave, el mismo modo, el mismo padding y el mismo vector de inicialización.

```

XBUFFER plain;

if(cipher.Uncipher(ciphered))
{
    plain.CopyFrom((*cipher.GetResult()));
}

```

10. Claves y representación interna

CIPHERKEY es la base común de las claves. Mantiene el tipo de clave, permite consultar su tamaño y ofrece CopyFrom para copiar una clave compatible. Las clases derivadas adaptan esa base a familias concretas: CIPHERKEYSYMMETRICAL para claves simétricas, CIPHERKEYPUBLICRSA para claves públicas RSA, CIPHERKEYPRIVATERSA para claves privadas RSA y CIPHERKEYECDSA para claves ECDSA.

La decisión de representar las claves como clases propias tiene una ventaja clara: el algoritmo no trabaja con datos sueltos sin contexto. Una clave conoce su tipo, su tamaño y su estructura. Esto facilita que CIPHER, CIPHERRSA o los ficheros de claves puedan validar mejor lo que reciben.

Clase	Representa	Relación
CIPHERKEY	Clave genérica.	Base para todas las claves del bloque.
CIPHERKEYSYMMETRICAL	Clave binaria compartida.	Usada por DES, 3DES, AES, Blowfish y otros cifrados simétricos.
CIPHERKEYPUBLICRSA	Módulo y exponente público RSA.	Usada por CIPHERRSA para operaciones con

		clave pública.
CIPHERKEYPRIVATERSA	Factores primos y exponente privado RSA.	Usada por CIPHERRSA para operaciones privadas y optimización CRT.
CIPHERKEYECDSA	Clave ECDSA.	Deriva de la clave simétrica como almacenamiento binario especializado.

11. Ficheros de claves GKF y PEM

Una aplicación no puede depender únicamente de claves creadas en memoria. En la práctica, las claves se guardan, se cargan, se exportan o se distribuyen. GEN Framework incluye dos clases importantes para ese trabajo: CIPHERKEYSFILEGKF y CIPHERKEYSFILEPEM.

CIPHERKEYSFILEGKF trabaja con el formato propio .gkf, fichero de claves de GEN Framework (GKF). Ese formato se basa en XML y permite almacenar metadatos de creador, fecha de creación, fecha de última modificación y distintas claves con atributos como tipo, tamaño, módulo, exponente o factores primos. DevTestsConsole utiliza un fichero keys.gkf en assets/certificates para cargar claves públicas y privadas RSA.

CIPHERKEYSFILEPEM trabaja con entradas PEM, claves y certificados. Se apoya en XASN1, XBER y el patrón de observador para decodificar estructuras codificadas. También colabora con CIPHERCERTIFICATEX509 cuando el contenido es un certificado.

```

XPATH xpath;
XPATH xpathgeneric;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpathgeneric);
xpath.Create(3, xpathgeneric.Get(), __L("keys"), CIPHERKEYSFILEGKF_EXT);

CIPHERKEYSFILEGKF filekeys(xpath);

CIPHERKEYPUBLICRSA* publickey = (CIPHERKEYPUBLICRSA*)filekeys.GetKey(CIPHERKEYTYPE_RSA_PUBLIC);
CIPHERKEYPRIVATERSA* privatekey = (CIPHERKEYPRIVATERSA*)filekeys.GetKey(CIPHERKEYTYPE_RSA_PRIVATE);

```

12. RSA y CIPHERRSA

CIPHERRSA implementa el cifrado asimétrico RSA dentro del estilo de GEN Framework. Hereda de CIPHER, pero añade métodos propios porque RSA necesita trabajar con claves públicas y privadas, tamaños de bloque, padding PKCS#1, enteros multiprecisión y, en algunos casos, firma.

La clase se apoya en XMPINTEGER para las operaciones de enteros grandes. CIPHERRSA_CONTEXT mantiene el módulo público, exponentes, factores primos, valores CRT y datos auxiliares. La clase CIPHERRSA puede cargar claves mediante SetKey, cifrar con una clave, descifrar con la otra y generar nuevas claves mediante GenerateKeys.

En DevTestsConsole se carga una pareja de claves desde keys.gkf, se cifra un texto largo usando la clave privada y se descifra usando la clave pública, comprobando después que el resultado coincide con el original. Ese test muestra dos ideas: primero, que el algoritmo trabaja con XBUFFER; segundo, que CIPHERRSA divide internamente los datos en bloques adecuados al tamaño de clave y padding.

```

CIPHERRSA cipher;
XBUFFER input;
XBUFFER output;

```

```

input.Add((XBYTE*)text, size);

if(cipher.SetKey(publickey))
{
    if(cipher.SetKey(privatekey))
    {
        if(cipher.Cipher(input, CIPHERKEYTYPE_RSA_PRIVATE))
        {
            output.CopyFrom(*cipher.GetResult());
            cipher.Uncipher(output, CIPHERKEYTYPE_RSA_PUBLIC);
        }
    }
}

bool same = input.Compare(cipher.GetResult());

```

RSA también ofrece Sign, que combina el dato con una clase HASH. Este punto es importante: para firmar no se debe firmar una cadena de forma improvisada, sino construir un flujo claro donde el hash elegido forma parte de la operación.

```

CIPHERRSA rsa;
HASHSHA2 hash(HASHSHA2TYPE_256);

rsa.SetKey(privatekey);
rsa.Sign(input, CIPHERKEYTYPE_RSA_PRIVATE, &hash);

```

13. Curve25519 / X25519

CIPHERECDSAX25519 proporciona una pieza orientada al intercambio de claves usando Curve25519/X25519. Aunque el nombre histórico de la clase incluye ECDSA, el uso que aparece en DevTestsConsole es el de generación de clave privada, creación de clave pública y cálculo de clave compartida entre dos extremos.

Este tipo de mecanismo no se usa para cifrar directamente un fichero grande. Su utilidad es permitir que dos participantes generen una clave compartida a partir de sus claves privadas y las claves públicas del otro extremo. Esa clave compartida puede alimentar después un cifrado simétrico, por ejemplo AES, dentro de un protocolo diseñado correctamente.

El test de DevTestsConsole crea dos objetos, genera dos claves privadas, crea las claves públicas correspondientes, intercambia las públicas y genera dos claves compartidas. El test final compara byte a byte que ambas claves compartidas coinciden.

```

CIPHERECDSAX25519 peer1;
CIPHERECDSAX25519 peer2;

peer1.GenerateRandomPrivateKey();
peer2.GenerateRandomPrivateKey();

peer1.CreatePublicKey();
peer2.CreatePublicKey();

peer1.CreateSharedKey(peer2.GetKey(CIPHERECDSAX25519_TYPEKEY_PUBLIC));
peer2.CreateSharedKey(peer1.GetKey(CIPHERECDSAX25519_TYPEKEY_PUBLIC));

```

```
XBYTE* shared1 = peer1.GetKey(CIPHERECDSAX25519_TYPEKEY_SHARED);
XBYTE* shared2 = peer2.GetKey(CIPHERECDSAX25519_TYPEKEY_SHARED);
```

14. Certificados X.509 y raíces de confianza

Cipher también incluye clases para trabajar con certificados. CIPHERCERTIFICATEX509 representa un certificado X.509: versión, número de serie, algoritmo, fechas de validez, identidad del emisor o sujeto, clave pública asociada, hash y datos del certificado. CIPHERCERTIFICATEX509_ID agrupa campos de identidad como país, organización, unidad organizativa y nombre común.

CIPHERTRUSTEDROOTCERTIFICATESX509 permite leer un fichero de certificados raíz y mantener sus líneas en memoria. También puede generar una cabecera embebida a partir de un fichero de origen. En DevTestsConsole se observa el uso de assets/certificates/cacert.pem como origen para decodificar certificados mediante CIPHERKEYSFILEPEM.

Estas clases se relacionan con DIO y con TLS, pero pertenecen a Cipher porque el certificado es una pieza criptográfica: contiene identidad, periodos de validez, algoritmo, clave pública y datos que deben poder validarse.

```
CIPHERTRUSTEDROOTCERTIFICATESX509 trustedrootcertificates;
CIPHERKEYSFILEPEM                filekeys;
XPATH                             xpath;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_CERTIFICATES, xpath);
xpath.Slash_Add();
xpath.Add(__L("cacert.pem"));

if(trustedrootcertificates.ReadFromFile(&xpath))
{
    filekeys.DecodeCertificates(trustedrootcertificates.GetLines());
}
```

15. Relación con assets y XPATHMANAGER

El bloque Cipher no debe trabajar con rutas absolutas. Las claves, certificados y ficheros de prueba deben colocarse dentro de assets, normalmente en una sección certificates o equivalente. Este criterio mantiene la coherencia con lo explicado en el capítulo de la primera aplicación: todos los recursos de la aplicación, excepto el ejecutable, viven bajo assets y se accede a ellos mediante XPATHMANAGER.

DevTestsConsole sigue este criterio con assets/certificates. Allí aparecen cacert.pem, cacert.h, certificate.crt, certificate.key, id_rsa, id_rsa.pub, root.pem y keys.gkf. Una aplicación propia puede usar una estructura similar cuando necesite cargar claves o certificados.

```
assets/
|
+-- certificates/
|   +-- cacert.pem
|   +-- keys.gkf
|   +-- certificate.crt
|   +-- certificate.key
|
+-- miaplicacion.ini
```

16. Activación del bloque en CMake

Como ocurre con otros bloques del framework, Cipher se activa por funcionalidades. Esto permite compilar solo lo que la aplicación necesita. DevTestsConsole activa hashes, cifrado simétrico, ficheros de claves, RSA y X25519 mediante opciones CMake específicas.

Una aplicación que solo necesita SHA2 no tiene por qué activar todo el bloque asimétrico. Una aplicación que usa TLS o certificados normalmente requiere más piezas. Esta granularidad ayuda a controlar dependencias, tiempos de compilación y tamaño final.

```
option(HASH_FEATURE "Hash" ON )
option(HASH_CRC32_FEATURE "Hash CRC32" ON )
option(HASH_MD5_FEATURE "Hash MD5" ON )
option(HASH_SHA1_FEATURE "Hash SHA1" ON )
option(HASH_SHA2_FEATURE "Hash SHA2" ON )
option(HASH_WHIRLPOOL_FEATURE "Hash Whirlpool" ON )

option(CIPHER_SYMMETRIC_FEATURE "Cipher Symetric" ON )
option(CIPHER_SYMMETRIC_DES_FEATURE "Cipher Symetric DES" ON )
option(CIPHER_SYMMETRIC_AES_FEATURE "Cipher Symetric AES" ON )
option(CIPHER_SYMMETRIC_BLOWFISH_FEATURE "Cipher Symetric BlowFish" ON )

option(CIPHER_ASYMMETRIC_FEATURE "Cipher Asymetric" ON )
option(CIPHER_ASYMMETRIC_FILEKEY_GFK "Cipher Asymetric File Key (.Gen Key)" ON )
option(CIPHER_ASYMMETRIC_FILEKEY_PEM "Cipher Asymetric File Key (.PEM)" ON )
option(CIPHER_ASYMMETRIC_RSA_FEATURE "Cipher Asymetric RSA" ON )
option(CIPHER_ASYMMETRIC_X25519_FEATURE "Cipher Asimetric ECDSA X25519" ON )
```

17. Catálogo de clases del bloque Cipher

Clase	Qué hace	Clases relacionadas / ejemplo de uso
HASH	Interfaz común de hash sobre memoria, XBUFFER, XPATH y XFILE.	Base de HASHCRC32, HASHMD5, HASHSHA1, HASHSHA2 y otras.
HASHCKS16	Checksum de 16 bits.	Validación ligera de datos o tramas.
HASHCRC16	CRC de 16 bits.	Protocolos y dispositivos que esperan CRC16.
HASHCRC32	CRC de 32 bits.	Huella rápida de ficheros y buffers.
HASHMD5	Hash MD5.	Compatibilidad con sistemas antiguos.
HASHSHA1	Hash SHA1.	Compatibilidad y protocolos heredados.
HASHSHA2	Familia SHA2.	SHA-224, SHA-256, SHA-384 y SHA-512.
HASHWHIRLPOOL	Hash Whirlpool de 512 bits.	Huella grande de datos.
HASHCOMP128V1	Algoritmo COMP128v1.	RAND, Ki, SRES y Kc.
CIPHER	Base de cifrado.	Gestiona clave, modo, padding, IV y resultado.
CIPHERDES	DES.	Cifrado simétrico heredado.
CIPHER3DES	Triple DES.	Compatibilidad con entornos antiguos.
CIPHERAES	AES.	Cifrado simétrico principal para nuevos usos.
CIPHERBLOWFISH	Blowfish.	Cifrado simétrico alternativo.
CIPHERKEY	Base de claves.	Tipo, tamaño y copia de clave.

CIPHERKEYSYMMETRICAL	Clave simétrica.	Usada por AES, DES, 3DES y Blowfish.
CIPHERKEYPUBLICRSA	Clave pública RSA.	Módulo y exponente.
CIPHERKEYPRIVATERSA	Clave privada RSA.	Factores primos y exponente.
CIPHERKEYECDSA	Clave ECDSA.	Representación específica de clave ECDSA.
CIPHERKEYSFILEGKF	Fichero de claves de GEN Framework (GKF).	Carga y actualiza claves .gkf.
CIPHERKEYSFILEPEM	Lectura de PEM y certificados.	Decodifica claves/certificados con XASN1/XBER.
CIPHERRSA_CONTEXT	Contexto interno RSA.	N, E, D, P, Q, valores CRT y datos auxiliares.
CIPHERRSA	RSA.	Cifrar, descifrar, firmar y generar claves.
CIPHERECDSAX25519	Curve25519/X25519.	Claves privada, pública y compartida.
CIPHERCERTIFICATEX509_ID	Identidad X.509.	País, organización, unidad organizativa y nombre común.
CIPHERCERTIFICATEX509	Certificado X.509.	Versión, serie, algoritmo, fechas, clave pública y hash.
CIPHERTRUSTEDROOTCERTIFICATESX509	Raíces de confianza.	Lee certificados raíz y puede generar cabecera embebida.

18. Cómo estudiar Cipher desde DevTestsConsole

DevTestsConsole es el mejor punto de entrada para ver el uso práctico del bloque Cipher. En sus pruebas aparecen cinco bloques muy útiles: Test_Hash, Test_Cipher_Simetric, Test_CipherFileKeys, Test_CipherRSA y Test_CipherECDSAX25519.

Test_Hash muestra cómo crear diferentes clases HASH y obtener sus resultados. Test_Cipher_Simetric muestra el patrón común de cifrado y descifrado con clave simétrica, modo CBC, padding e IV. Test_CipherFileKeys muestra la lectura de certificados y el uso de CIPHERKEYSFILEPEM. Test_CipherRSA muestra carga de claves desde keys.gkf y prueba de cifrado/descifrado RSA. Test_CipherECDSAX25519 muestra el intercambio de claves entre dos participantes.

Para aprender el módulo, conviene ejecutar una prueba cada vez y seguir el flujo con el depurador. El objetivo no es memorizar el algoritmo interno, sino entender qué clase representa cada concepto, qué objeto posee el resultado y dónde se libera cada recurso.

19. Criterios de uso en una aplicación con GEN Framework

El primer criterio es elegir la herramienta adecuada. Para comprobar si un fichero cambió accidentalmente, CRC32 puede ser suficiente. Para una huella criptográfica, SHA2 es una opción más adecuada. Para proteger datos grandes, lo normal es usar cifrado simétrico. Para intercambio de claves o identidad, entran RSA, X25519 y certificados.

El segundo criterio es no dispersar claves ni rutas. Las claves y certificados deben vivir bajo assets, en una sección controlada por XPATHMANAGER, y el código debe acceder a ellos mediante rutas gestionadas por el framework. Esto evita rutas absolutas y facilita mover la aplicación entre plataformas.

El tercer criterio es cerrar el ciclo de vida. Los objetos creados con GEN_NEW deben liberarse con GEN_DELETE o mediante el mecanismo previsto. Los buffers deben vaciarse cuando ya no se necesitan. Las claves no deberían copiarse sin necesidad ni dejarse en logs. Las trazas deben ser útiles, pero no deben imprimir secretos en producción.

El cuarto criterio es no inventar protocolos criptográficos. Cipher ofrece piezas de bajo nivel. La seguridad real depende de cómo se combinen esas piezas. Usar AES, RSA o SHA2 no convierte automáticamente una aplicación en segura si el diseño de claves, IVs, padding, autenticación o almacenamiento es incorrecto.

20. Resumen práctico

Cipher es el bloque de GEN Framework dedicado a hashes, checksums, cifrado, claves y certificados. Su valor principal está en integrar estas operaciones dentro del estilo del framework: XBUFFER para datos, XPATH y XFILE para ficheros, XSTRING para resultados legibles, XMPINTEGER para operaciones asimétricas y assets/XPATHMANAGER para recursos criptográficos.

HASH proporciona una interfaz común para resúmenes. CIPHER proporciona una interfaz común para cifrado. Las clases de clave separan la representación de cada tipo de clave. Los ficheros GKF y PEM permiten persistir o importar material criptográfico. RSA y X25519 cubren escenarios asimétricos. Las clases X.509 permiten manejar certificados y raíces de confianza.

En aplicaciones reales, Cipher suele trabajar junto con DataIO, Compression, AppFlow y XUtils. Los datos se leen, se comprimen, se cifran, se firman, se transmiten o se almacenan. El programador debe entender el papel de cada clase y usarla dentro de una arquitectura clara, sin convertir la criptografía en código improvisado disperso por la aplicación.

Capítulo XI: Compress, compresión de datos y ficheros en GEN Framework

1. El objetivo del capítulo

Después de estudiar los bloques principales del framework, los sistemas de entrada y salida, gráficos, entrada de usuario, sonido y cifrado, aparece una necesidad muy habitual en aplicaciones reales: reducir el tamaño de los datos. La compresión no es un elemento decorativo. En una aplicación con recursos, trazas, configuraciones, paquetes de actualización, comunicaciones, copias de seguridad o intercambio de ficheros, comprimir correctamente puede marcar la diferencia entre un sistema cómodo de desplegar y otro pesado, lento o difícil de mantener.

El bloque Compress de GEN Framework agrupa las clases dedicadas a la compresión y descompresión de datos. Su objetivo no es sustituir a todos los formatos existentes ni convertirse en una herramienta universal de empaquetado, sino ofrecer una capa coherente, con estilo de GEN Framework, para trabajar con algoritmos de compresión desde el propio framework. Como ocurre con otros bloques, la idea es que la aplicación no tenga que usar directamente APIs externas o estructuras de librerías de terceros cuando puede hacerlo a través de clases propias, tipos de GEN Framework y buffers comunes.

Este capítulo explica cómo está organizado el bloque Compress, qué papel cumplen sus clases, cuándo conviene usar cada algoritmo y cómo se relaciona con XUtils, DataIO, XPATHMANAGER y XFileZIP. También incluye ejemplos de uso inspirados en DevTestsConsole, donde se comprime y descomprime un buffer y se añade un fichero a un ZIP para validar el flujo completo.

2. El bloque Compress dentro de GEN Framework

Compress es un bloque transversal. No pertenece directamente al sistema de ficheros, aunque pueda comprimir ficheros. No pertenece a DataIO, aunque pueda utilizarse sobre datos que entran o salen por un stream. No pertenece a Cipher, aunque muchas veces compresión y cifrado aparecen juntos en un mismo flujo. Su lugar natural es el de un servicio auxiliar que transforma datos: recibe una secuencia de bytes y produce otra secuencia de bytes, normalmente más pequeña o empaquetada.

Esta separación es importante. La aplicación no debería mezclar la lógica de compresión con la lógica de negocio. Tampoco debería repetir código de zlib, LZW o algoritmos internos en cada módulo que necesite comprimir. Lo razonable es encapsular esa responsabilidad en clases concretas y usar una interfaz común. Esa interfaz común está representada por COMPRESSBASE y por los derivados que implementan algoritmos específicos.

En una aplicación con GEN Framework, Compress suele trabajar con XBUFFER. Esto encaja con la filosofía del framework: los datos binarios se manipulan mediante una clase común, en lugar de pasar de forma dispersa punteros, tamaños y memoria reservada manualmente. El uso de XBUFFER permite que la salida de una compresión pueda escribirse después a un fichero, enviarse por red, almacenarse en una base de datos, cifrarse o integrarse en un paquete de recursos.

3. Compresión de datos frente a compresión de ficheros

Una distinción práctica que conviene tener clara desde el principio es la diferencia entre comprimir datos y empaquetar ficheros. Comprimir datos significa tomar un bloque de bytes y generar otro bloque comprimido. El origen puede ser una cadena UTF-8, un buffer de memoria, una imagen ya cargada, una traza, una configuración o cualquier otro contenido binario. El resultado también es un buffer. Las clases COMPRESS_ZIP, COMPRESS_GZ, COMPRESS_LZW y COMPRESS_LZRW1KH trabajan principalmente en este nivel.

Empaquetar ficheros es una operación de nivel superior. En ese caso no solo interesa comprimir un bloque, sino conservar nombres de fichero, rutas internas, entradas dentro de un archivo, metadatos y estructura de paquete. Para ese escenario aparece XFILEZIP dentro del sistema de ficheros, que se relaciona con Compress pero tiene una responsabilidad diferente. DevTestsConsole muestra ambas ideas: por un lado usa XFILEZIP para añadir un fichero a un ZIP; por otro lado usa COMPRESSBASE para comprimir y descomprimir un buffer.

Esta diferencia ayuda a elegir la herramienta adecuada. Si la aplicación necesita reducir el tamaño de un XBUFFER antes de enviarlo por red, Compress es suficiente. Si necesita crear un paquete con varios ficheros dentro, lo natural es usar XFILEZIP o una capa de fichero equivalente. Si necesita comprimir un fichero suelto, puede leerlo a XBUFFER, comprimirlo y escribir el resultado; pero si necesita un contenedor con entradas internas, entonces se necesita una clase de formato de fichero.

4. Mapa general de clases

El bloque Compress es pequeño si se compara con XUtils, DataIO o Graphic, pero está muy claramente estructurado. Existe una clase base, un gestor de creación y varias implementaciones de algoritmo. Esta simplicidad es precisamente una de sus virtudes: el programador puede entender rápidamente qué clase debe utilizar.

Clase / módulo	Responsabilidad	Uso típico
COMPRESSBASE	Clase base común para operaciones de compresión y descompresión.	Interfaz común, clase nula o copia directa cuando no hay algoritmo específico.
COMPRESSMANAGER	Gestor/factoría ligera del bloque Compress.	Crear una instancia de algoritmo a partir de COMPRESSBASE_TYPE.
COMPRESS_ZIP	Compresión basada en zlib/deflate.	Buffers comprimidos en formato zlib, uso general.
COMPRESS_GZ	Compresión GZip basada en zlib con cabecera GZ.	Datos compatibles con flujos o ficheros GZip.
COMPRESS_LZW	Algoritmo LZW, con soporte interno para uso tipo GIF.	Recursos, datos repetitivos y compatibilidad con flujos LZW.
COMPRESS_LZRW1KH	Algoritmo LZRW1/KH orientado a velocidad y bajo coste.	Datos donde interesa compresión rápida y lógica ligera.
XFILEZIP	Formato de fichero ZIP dentro de XUtils/FormatFiles.	Crear paquetes ZIP, añadir ficheros y trabajar con entradas.

5. COMPRESSBASE: la interfaz común

COMPRESSBASE es la clase base del bloque. Define el contrato que deben cumplir las clases de compresión: un método Compress para transformar datos originales en datos comprimidos y un método Decompress para recuperar los datos originales. La clase trabaja con XBYTE como unidad de dato, XDWORD como tamaño y XBUFFER como contenedor de salida. De esta forma mantiene el estilo de tipos del framework y evita exponer tipos estándar o estructuras externas.

El enum COMPRESSBASE_TYPE identifica los algoritmos disponibles. En los fuentes aparecen COMPRESSBASE_TYPE_NONE, COMPRESSBASE_TYPE_LZW, COMPRESSBASE_TYPE_LZRW1KH, COMPRESSBASE_TYPE_ZIP y COMPRESSBASE_TYPE_GZ. Este enum permite que la aplicación solicite un algoritmo sin crear directamente la clase derivada. La selección puede quedar centralizada en COMPRESSMANAGER.

La implementación base actúa como una clase neutra. Si se llama a Compress o Decompress sobre COMPRESSBASE sin un algoritmo concreto, el comportamiento es copiar el contenido de origen al buffer de

destino. Esto puede parecer simple, pero tiene sentido como caso base: permite que una ruta de código funcione incluso cuando la compresión real no está activada o cuando se quiere mantener el mismo flujo sin modificar la lógica externa.

```
XSTRING source;
XBUFFER origin;
XBUFFER packed;

source = __L("Texto de prueba para compresión");
source.ConvertToUTF8(origin, false);

COMPRESSBASE compress;
compress.Compress(origin.Get(), origin.GetSize(), &packed);

// En la clase base el resultado es una copia del buffer original.
```

En aplicaciones reales normalmente no se instancia COMPRESSBASE directamente salvo como fallback. Lo habitual es usar COMPRESSMANAGER o crear una clase derivada concreta cuando se sabe exactamente qué algoritmo se necesita.

6. COMPRESSMANAGER: creación de algoritmos

COMPRESSMANAGER actúa como gestor de creación del bloque Compress. No es una factoría principal del framework como XFACTORY, DIOFACTORY, GRPFACTORY, INPFACTORY o SNDFACTORY, pero sigue la misma idea conceptual: centralizar la creación de objetos. El método Create recibe un COMPRESSBASE_TYPE y devuelve un puntero a COMPRESSBASE. Internamente, según las macros activas, crea COMPRESS_LZW, COMPRESS_LZR1K, COMPRESS_ZIP, COMPRESS_GZ o una instancia base.

Esta clase también dispone de un patrón de instancia estática mediante GetInstance y DeleteInstance. En código sencillo se puede crear un COMPRESSMANAGER local con GEN_NEW, como hace DevTestsConsole. En código más global puede usarse la instancia compartida. Lo importante es que la aplicación no necesita conocer todos los includes y clases concretas si trabaja a través del tipo de compresión.

```
COMPRESSMANAGER* manager = GEN_NEW COMPRESSMANAGER();
COMPRESSBASE* compressor = NULL;

if(manager)
{
    compressor = manager->Create(COMPRESSBASE_TYPE_ZIP);
}

// ... usar compressor ...

if(compressor) delete compressor;
if(manager) delete manager;
```

Este patrón refuerza la idea de modularidad. El código de aplicación puede elegir el tipo de compresión en configuración o por contexto, y el gestor se encarga de devolver la clase adecuada. Si un algoritmo no está compilado, el gestor no lo ofrece como implementación real y se debe tratar el caso NULL o fallback.

7. COMPRESS_ZIP: compresión deflate sobre buffers

COMPRESS_ZIP implementa compresión usando zlib con el flujo deflate clásico. Aunque el nombre puede sugerir un fichero .zip completo, en esta clase el trabajo principal es comprimir y descomprimir buffers. La creación de un archivo ZIP con entradas internas pertenece a XFILEZIP. Esta diferencia es importante para no confundir algoritmo de compresión con formato de contenedor.

La clase expone Compress y Decompress con la misma firma que COMPRESSBASE. En Compress se limpia el buffer de destino, se reserva memoria inicial, se ejecuta ZCompress y se ajusta el tamaño final del XBUFFER al tamaño comprimido. En Decompress, el tamaño del buffer de destino debe estar preparado para contener el resultado

descomprimido; por eso en el ejemplo de DevTestsConsole se redimensiona decmpbuffer al tamaño original antes de llamar a Decompress.

```
XSTRING sentence;
XBUFFER origin;
XBUFFER compressed;
XBUFFER decompressed;

sentence = __L("Datos repetitivos, datos repetitivos, datos repetitivos.");
sentence.ConvertToUTF8(origin, false);

decompressed.Resize(origin.GetSize());

COMPRESS_ZIP zip;
if(zip.Compress(origin.Get(), origin.GetSize(), &compressed))
{
    zip.Decompress(compressed.Get(), compressed.GetSize(), &decompressed);
}
```

COMPRESS_ZIP es una buena elección cuando se quiere una compresión general de datos en memoria y se dispone de zlib. Es apropiado para buffers de configuración, respuestas de red, datos temporales o cualquier bloque que después se almacena o transmite dentro de otro contenedor.

8. COMPRESS_GZ: compresión compatible con GZip

COMPRESS_GZ también se apoya en zlib, pero configura el flujo para generar o leer datos con cabecera GZip. En los fuentes se observa el uso de deflateInit2 con la combinación de ventana y codificación GZ, y de inflateInit2 con 16 + MAX_WBITS para reconocer el formato GZip. Esto diferencia a COMPRESS_GZ de COMPRESS_ZIP: ambos utilizan zlib, pero no generan exactamente el mismo envoltorio de datos.

El uso de GZip es habitual cuando se trabaja con datos que deben ser compatibles con herramientas externas, flujos web, respuestas HTTP comprimidas o ficheros .gz. Si la aplicación solo necesita comprimir internamente un buffer, ZIP/deflate puede ser suficiente. Si necesita interoperar con un formato GZip, entonces COMPRESS_GZ es más adecuado.

```
COMPRESS_GZ gz;
XBUFFER origin;
XBUFFER packed;
XBUFFER unpacked;

// origin ya contiene datos binarios.
unpacked.Resize(origin.GetSize());

if(gz.Compress(origin.Get(), origin.GetSize(), &packed))
{
    gz.Decompress(packed.Get(), packed.GetSize(), &unpacked);
}
```

Como en el resto de algoritmos, la aplicación debe conocer o conservar el tamaño esperado del resultado descomprimido si quiere reservar correctamente el buffer de salida. Esto puede hacerse guardando el tamaño original en una cabecera propia, en un fichero de metadatos, en una estructura de paquete o en el protocolo que transporte los datos.

9. COMPRESS_LZW: compresión LZW y soporte de uso tipo GIF

COMPRESS_LZW implementa el algoritmo LZW. LZW es un algoritmo clásico basado en diccionario, especialmente conocido por su presencia histórica en formatos de imagen como GIF. En GEN Framework, la clase no solo expone Compress y Decompress, sino también métodos internos orientados a codificación y decodificación de datos tipo GIF: LZW_GIF_Encode y LZW_GIF_Decode.

El código mantiene una tabla de cadenas, códigos de clear/end, bits de salida, tabla de hash y parámetros relacionados con anchura, altura, interlace y lectura de píxeles. Esto muestra que la clase no está pensada

únicamente como compresor genérico de texto, sino también como apoyo a formatos gráficos que necesitan LZW en su proceso interno.

En una aplicación normal no se suele llamar a todos los métodos internos. El uso habitual consiste en aplicar Compress y Decompress sobre XBUFFER. Los métodos LZW_GIF_Encode y LZW_GIF_Decode pertenecen a un nivel más especializado y tienen sentido cuando se implementa o mantiene soporte de un formato gráfico concreto.

```
COMPRESS_LZW lzw;
XBUFFER      origin;
XBUFFER      packed;
XBUFFER      unpacked;

// El buffer de salida descomprimido debe reservarse con el tamaño esperado.
unpacked.Resize(origin.GetSize());

if(lzw.Compress(origin.Get(), origin.GetSize(), &packed))
{
    lzw.Decompress(packed.Get(), packed.GetSize(), &unpacked);
}
```

LZW puede resultar útil en datos con repetición y en módulos que necesiten compatibilidad con formatos históricos. Como criterio práctico, no debe elegirse automáticamente para todo. En GEN Framework conviene elegir el algoritmo según el formato, la compatibilidad necesaria y el coste de CPU/memoria aceptable.

10. COMPRESS_LZRW1KH: compresión rápida y ligera

COMPRESS_LZRW1KH implementa una variante de LZRW1/KH. Este tipo de algoritmo está orientado a compresión rápida, con estructuras de hash relativamente pequeñas y una lógica de coincidencias sencilla. En los fuentes se observa una tabla hash de 4096 entradas, búsqueda de coincidencias y escritura de marcadores de datos copiados o comprimidos. Su orientación natural es reducir datos con bajo coste de proceso.

Una característica interesante de esta clase es que ofrece dos formas de trabajo. La primera usa XBUFFER como destino, igual que el resto de compresores. La segunda acepta un void* param1 y utiliza CompressResult para emitir el resultado byte a byte. Este diseño permite adaptar el algoritmo a salidas especiales sin forzar siempre la construcción de un XBUFFER intermedio.

```
COMPRESS_LZRW1KH lzw;
XBUFFER          origin;
XBUFFER          packed;
XBUFFER          unpacked;

packed.Resize(origin.GetSize() + 1024);
unpacked.Resize(origin.GetSize());

lzw.Compress(origin.Get(), origin.GetSize(), &packed);
lzw.Decompress(packed.Get(), packed.GetSize(), &unpacked);
```

La modalidad con callback mediante CompressResult puede ser útil en escenarios avanzados: escribir directamente en un stream, alimentar una cola, enviar datos por bloques o adaptar el resultado a un formato propietario. En esos casos, la clase derivada o el código de integración puede redefinir CompressResult y utilizar param1 como contexto.

11. XFILEZIP: paquetes ZIP y ficheros

Aunque XFILEZIP no pertenece estrictamente al bloque Compress como algoritmo, aparece de forma natural cuando se habla de compresión de ficheros. La razón es sencilla: un archivo ZIP no es solamente un buffer comprimido; es un contenedor con entradas, nombres internos, estructura y metadatos. Por eso GEN Framework separa el compresor de datos del formato de fichero.

DevTestsConsole muestra un ejemplo muy claro. Primero crea una ruta al fichero que se quiere añadir, otra ruta interna dentro del ZIP y una ruta al archivo .zip resultante. Después crea XFILEZIP, abre el ZIP, añade el fichero y

cierra el contenedor. Ese ejemplo sirve para entender que el empaquetado de ficheros se resuelve desde la capa de ficheros, aunque internamente pueda apoyarse en compresión.

```
#define NAMEFILECMPINZIP __L("testcmp.txt")

XPATH xpathcmpfile;
XPATH xpathcmpinzip;
XPATH xpathzipfile;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpathcmpfile);
xpathcmpfile.Add(NAMEFILECMPINZIP);

xpathcmpinzip.Add(NAMEFILECMPINZIP);

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpathzipfile);
xpathzipfile.Slash_Add();
xpathzipfile.Add(__L("test.zip"));

XFILEZIP* zipfile = GEN_NEW XFILEZIP();
if(zipfile)
{
    if(zipfile->Open(xpathzipfile))
    {
        zipfile->AddFile(xpathcmpfile, xpathcmpinzip);
        zipfile->Close();
    }

    delete zipfile;
}
```

Esta clase se utiliza cuando el objetivo es generar un paquete de ficheros, no solo comprimir un bloque de memoria. Resulta útil para copias de seguridad, exportación de datos, paquetes de recursos, envío de ficheros múltiples o creación de archivos temporales con estructura interna.

12. Relación con assets y XPATHMANAGER

En una aplicación con GEN Framework, los ficheros de recursos se colocan bajo assets y se accede a ellos mediante XPATHMANAGER. Esto también afecta a la compresión. Si se va a crear un ZIP de configuración, un paquete de logs, un fichero GZ o una copia comprimida de una base de datos local, lo razonable es obtener la ruta desde XPATHMANAGER en lugar de construir una ruta absoluta manualmente.

El patrón general consiste en pedir la ruta de la sección adecuada, añadir el nombre de fichero y trabajar con XFILE, XFILEZIP o XBUFFER. Esto mantiene la aplicación portable y evita rutas rígidas. En Windows, Linux o Android la ubicación física puede cambiar, pero la aplicación sigue usando las secciones lógicas definidas por el framework.

```
XPATH xpath;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_ROOT, xpath);
xpath.Slash_Add();
xpath.Add(__L("backup.zip"));

// xpath apunta al fichero dentro del árbol de recursos de la aplicación.
```

Si la aplicación necesita una sección específica, por ejemplo backups, exports o packages, se puede crear una nueva sección en XPATHMANAGER o reutilizar una sección existente. El criterio es el mismo que vimos en capítulos anteriores: todos los recursos deben estar ordenados y localizables de forma homogénea.

13. Ejemplo completo: comprimir y descomprimir un texto

El siguiente ejemplo resume el uso típico de Compress sobre un buffer. Se parte de un XSTRING, se convierte a UTF-8 en un XBUFFER, se crea un compresor ZIP mediante COMPRESSMANAGER, se comprime el buffer y después se descomprime para reconstruir el texto original. Es el mismo patrón conceptual que aparece en DevTestsConsole.

```

COMPRESSMANAGER* manager      = GEN_NEW COMPRESSMANAGER();
COMPRESSBASE*   compressor    = NULL;

XSTRING sentence;
XSTRING finalsentence;
XBUFFER buffer;
XBUFFER cmpbuffer;
XBUFFER decmpbuffer;

sentence = __L("Compress test from GEN Framework.");
sentence.ConvertToUTF8(buffer, false);

decmpbuffer.Resize(buffer.GetSize());

if(manager)
{
    compressor = manager->Create(COMPRESSBASE_TYPE_ZIP);
}

if(compressor)
{
    if(compressor->Compress(buffer.Get(), buffer.GetSize(), &cmpbuffer))
    {
        compressor->Decompress(cmpbuffer.Get(), cmpbuffer.GetSize(), &decmpbuffer);
        finalsentence.ConvertFromUTF8(decmpbuffer);
    }
}

if(compressor) delete compressor;
if(manager)    delete manager;

```

Hay dos detalles importantes. El primero es que el buffer descomprimido se reserva con el tamaño original. El segundo es que la verificación final no debe hacerse comparando tamaños comprimidos, sino reconstruyendo el dato original y comparándolo con el contenido inicial. En pruebas de integridad, esto puede reforzarse calculando un hash antes y después, usando el bloque Cipher/Hash estudiado en el capítulo anterior.

14. Compresión, cifrado y orden de operaciones

Compress se relaciona de forma natural con Cipher. Muchas aplicaciones necesitan guardar datos de forma compacta y además protegida. En ese caso hay que decidir el orden de operaciones. La regla práctica habitual es comprimir primero y cifrar después. La razón es que los datos cifrados deben parecer aleatorios; al intentar comprimir después de cifrar, normalmente no existen patrones aprovechables y el tamaño no se reduce de forma significativa.

Un flujo típico sería: serializar datos a XBUFFER, comprimir con Compress, cifrar con Cipher y escribir el resultado en un fichero o enviarlo por DataIO. Para recuperar la información se invierte el orden: leer, descifrar, descomprimir y deserializar. Separar cada paso en su módulo correspondiente hace que el código sea más mantenible y permite cambiar el algoritmo de compresión o cifrado sin alterar toda la aplicación.

```

// Flujo conceptual:
// 1. datos originales -> XBUFFER
// 2. COMPRESS_ZIP     -> buffer comprimido
// 3. CIPHERAES        -> buffer cifrado
// 4. XFILE/DIOSTREAM  -> almacenamiento o envío

// Para leer:
// 1. XFILE/DIOSTREAM  -> buffer cifrado
// 2. CIPHERAES        -> buffer comprimido
// 3. COMPRESS_ZIP     -> datos originales

```

Este flujo también muestra por qué GEN Framework organiza los bloques por responsabilidad. Compress no cifra. Cipher no comprime. DataIO no decide el algoritmo. Cada módulo aporta una pieza y la aplicación los combina de forma controlada.

15. Activación mediante macros y compilación

Los fuentes de Compress están preparados para compilar algoritmos concretos mediante macros de activación. En CompressManager.cpp aparecen condiciones como COMPRESS_LZW_ACTIVE, COMPRESS_LZRW1KH_ACTIVE, COMPRESS_ZIP_ACTIVE y COMPRESS_GZ_ACTIVE. Esto permite incluir o excluir algoritmos según las necesidades del proyecto, la plataforma o las dependencias disponibles.

Esta técnica encaja con el sistema de compilación de GEN Framework. Una aplicación pequeña o embebida puede no necesitar todos los algoritmos. Una aplicación de escritorio con exportación de paquetes puede activar ZIP. Una aplicación que trabaja con formatos compatibles con GZip puede activar GZ. Un módulo gráfico con formatos históricos puede necesitar LZW. El objetivo es no arrastrar dependencias innecesarias cuando no aportan valor.

```
// Ejemplo conceptual de selección por compilación:
#define COMPRESS_ZIP_ACTIVE
#define COMPRESS_GZ_ACTIVE

// CompressManager puede crear COMPRESS_ZIP y COMPRESS_GZ
// cuando esas opciones estén activadas en el proyecto.
```

La forma exacta de activar estas macros depende del CMakeLists.txt y de las opciones comunes del framework. Como criterio general, las aplicaciones con GEN Framework no deberían definir macros de forma dispersa en cada fuente, sino centralizar la selección de funcionalidades en CMake y en los ficheros de configuración de compilación.

16. Errores habituales al usar Compress

El error más común al usar Compress es no reservar correctamente el buffer de salida de la descompresión. La clase puede comprimir un buffer y ajustar el tamaño comprimido, pero para descomprimir necesita espacio suficiente para reconstruir el contenido original. Si la aplicación no conserva el tamaño original, tiene que almacenarlo en algún lugar: una cabecera, un campo de protocolo, un metadato de fichero o una estructura de paquete.

Otro error habitual es confundir COMPRESS_ZIP con XFILEZIP. El primero comprime datos usando deflate; el segundo trabaja con un archivo ZIP como contenedor de ficheros. Si se quiere añadir varios ficheros a un paquete, se necesita XFILEZIP. Si se quiere comprimir un bloque de memoria, se necesita un COMPRESSBASE derivado.

También hay que recordar que no todos los datos se comprimen bien. Un JPEG, un PNG, un MP3, un OGG o un fichero ya comprimido pueden no reducirse apenas. Incluso pueden crecer ligeramente por cabeceras o estructura interna. La compresión es especialmente útil en texto, configuraciones, logs, estructuras repetitivas, datos serializados sin comprimir o paquetes con muchos recursos pequeños.

Finalmente, no conviene mezclar memoria manual sin control. GEN Framework ofrece XBUFFER precisamente para ordenar este trabajo. Si la aplicación empieza a reservar memoria con new, malloc o buffers locales grandes sin una política clara, aumenta la probabilidad de errores. El estilo recomendado es usar XBUFFER y dejar que las clases del framework ajusten tamaños y contenidos.

17. Catálogo resumido del bloque Compress

Grupo	Clases	Comentario práctico
Base común	COMPRESSBASE	Define el contrato común y ofrece un comportamiento neutro de copia.
Gestión	COMPRESSMANAGER	Crea algoritmos por tipo y centraliza la selección de

		compresor.
Zlib/deflate	COMPRESS_ZIP	Compresión general de buffers usando deflate. No es un contenedor ZIP completo.
GZip	COMPRESS_GZ	Genera/lee datos GZip, útil para compatibilidad externa.
Diccionario clásico	COMPRESS_LZW	Algoritmo LZW, con soporte especializado para flujos tipo GIF.
Compresión rápida	COMPRESS_LZRW1KH	Algoritmo ligero con salida a buffer o emisión mediante CompressResult.
Formato de fichero	XFILEZIP	Contenedor ZIP para añadir y organizar ficheros dentro de un paquete.

18. Criterios de uso en una aplicación con GEN Framework

Para elegir una clase de Compress conviene empezar por la pregunta correcta: ¿se quiere comprimir un bloque de datos o crear un fichero contenedor? Si se quiere comprimir un XBUFFER, se elige un algoritmo de Compress. Si se quiere crear un ZIP con ficheros, se usa XFILEZIP. Después se decide el algoritmo según compatibilidad, velocidad, tamaño final y dependencias.

COMPRESS_ZIP es una buena opción general cuando se quiere compresión estándar basada en zlib.

COMPRESS_GZ es útil cuando se necesita compatibilidad con GZip. COMPRESS_LZW tiene sentido en formatos o datos donde LZW sea necesario o conveniente. COMPRESS_LZRW1KH puede interesar cuando la velocidad y la simplicidad del algoritmo son más importantes que obtener la máxima tasa de compresión.

En todos los casos, el resultado debe integrarse con el resto del framework: XPATHMANAGER para rutas, XBUFFER para memoria binaria, XFILE o DataIO para almacenamiento y transmisión, Cipher para protección y AppFlow para ubicar el proceso dentro del ciclo de vida de la aplicación. Compress no debe verse como una utilidad aislada, sino como una pieza más del flujo de datos de GEN Framework.

19. Resumen práctico

Compress proporciona a GEN Framework una capa coherente para comprimir y descomprimir datos. Su diseño es sencillo: una clase base común, un gestor de creación y varias implementaciones de algoritmo. Esa sencillez permite utilizarlo sin introducir dependencias directas ni lógica específica de compresión en cada aplicación.

COMPRESSBASE define el contrato. COMPRESSMANAGER crea el algoritmo adecuado. COMPRESS_ZIP y COMPRESS_GZ se apoyan en zlib para casos generales y compatibilidad GZip. COMPRESS_LZW cubre escenarios basados en diccionario y formatos tipo GIF. COMPRESS_LZRW1KH ofrece una alternativa rápida y ligera. XFILEZIP complementa el bloque cuando el objetivo no es comprimir un buffer, sino crear un paquete de ficheros.

La idea central es la misma que en el resto del framework: organizar una funcionalidad habitual de C/C++ dentro de una arquitectura homogénea. En lugar de repetir código de compresión en cada programa, GEN Framework ofrece una forma común de trabajar con buffers, rutas, ficheros y algoritmos, manteniendo el estilo del framework y facilitando la evolución de las aplicaciones.

Capítulo XII: Databases, bases de datos SQL en GEN Framework

1. El papel de Databases dentro del framework

El bloque Databases agrupa las clases que GEN Framework utiliza para acceder a bases de datos relacionales mediante SQL. Su objetivo no es convertir la aplicación en dependiente de un motor concreto, sino proporcionar una capa de acceso homogénea para trabajar con motores distintos desde una misma idea de conexión, consulta, resultado, fila, variante de dato y transacción.

En esta fase del framework, Databases está orientado principalmente a bases de datos SQL. Esto incluye SQLite para escenarios locales o embebidos, MySQL/MariaDB para servidores ampliamente utilizados y PostgreSQL para entornos donde se requiere un motor relacional avanzado. La arquitectura permite incorporar conectores documentales, como MongoDB u otros modelos NoSQL, aunque el bloque actual está organizado alrededor del acceso relacional y del lenguaje SQL.

La filosofía coincide con el resto de GEN Framework: si una aplicación puede expresar su intención de forma común, el detalle concreto debe quedar encapsulado en el módulo correspondiente. La aplicación no debería tener una rama distinta para cada motor de base de datos salvo en aquellos puntos donde la sintaxis SQL realmente difiere. El bloque Databases intenta reducir esas diferencias y agruparlas en conectores que comparten una interfaz de uso.

2. Por qué un conector común es importante

En C/C++ es habitual que cada motor de base de datos tenga su propia biblioteca cliente, sus propios tipos, su propia forma de abrir conexiones, preparar consultas, enlazar parámetros, recorrer resultados y gestionar errores. Si una aplicación utiliza directamente esas APIs, queda atada al motor elegido. Cambiar de SQLite a PostgreSQL, o de MySQL a SQLite para pruebas locales, puede convertirse en una reescritura parcial del acceso a datos.

GEN Framework intenta evitar esa dependencia directa. El programador trabaja con clases del bloque DB_SQL y selecciona el tipo de base de datos mediante un valor de enumeración. A partir de ahí, la factoría del bloque crea el conector adecuado. La aplicación sigue viendo una base de datos, una conexión, una consulta y un resultado. Internamente, el framework usa las clases concretas de SQLite, MySQL/MariaDB o PostgreSQL.

Esta capa común no elimina por completo las diferencias entre motores. SQL no es idéntico en todos ellos. Los tipos de datos, los placeholders de parámetros, las fechas, las transacciones, los savepoints o la creación de tablas pueden tener matices. Lo que hace GEN Framework es proporcionar un punto común de trabajo y dejar localizadas las diferencias inevitables.

Motor	Modelo	Uso habitual
SQLite	Base local en fichero	Pruebas, aplicaciones embebidas, configuración avanzada, herramientas de escritorio y despliegues sin servidor.
MySQL/MariaDB	Base SQL cliente/servidor	Aplicaciones conectadas a un servidor, soluciones web, herramientas internas o sistemas ya basados en MySQL/MariaDB.
PostgreSQL	Base SQL cliente/servidor avanzada	Aplicaciones que necesitan un motor relacional robusto, transacciones completas, tipos avanzados o despliegues de servidor.

3. DB_SQL_FACTORY y selección del motor

La creación de una base de datos SQL se realiza mediante DB_SQL_FACTORY. Esta clase concentra la selección del conector y evita que la aplicación tenga que construir directamente una clase MYSQL_DATABASE, SQLITE_DATABASE o POSTGRESQL_DATABASE. El tipo se expresa mediante DB_SQL_DATABASE_TYPE.

```
DB_SQL_DATABASE* database = NULL;

database = DB_SQL_FACTORY::Create(DB_SQL_DATABASE_TYPE_SQLITE);
if(!database) return false;
```

Los valores principales de DB_SQL_DATABASE_TYPE son DB_SQL_DATABASE_TYPE_SQLITE, DB_SQL_DATABASE_TYPE_MYSQL y DB_SQL_DATABASE_TYPE_POSTGRESQL. Cada valor se compila solo si el soporte correspondiente está activado mediante las opciones del proyecto. Esto permite generar aplicaciones con un subconjunto de conectores, evitando dependencias innecesarias.

En el ejemplo Databases, la pulsación de teclas permite cambiar dinámicamente el tipo de base de datos: M selecciona MySQL, L selecciona SQLite y P selecciona PostgreSQL. Después, la tecla T ejecuta una prueba común sobre el motor seleccionado. Este ejemplo muestra con claridad la intención del bloque: el flujo de prueba es prácticamente el mismo aunque el motor cambie.

```
case 'M' : DataBase_ChangeType(DB_SQL_DATABASE_TYPE_MYSQL);      break;
case 'L' : DataBase_ChangeType(DB_SQL_DATABASE_TYPE_SQLITE);      break;
case 'P' : DataBase_ChangeType(DB_SQL_DATABASE_TYPE_POSTGRESQL); break;
case 'T' : Database_DoTest();                                     break;
```

4. Organización del bloque SQL

El bloque SQL se divide en una capa común y varias capas de implementación. La capa común define el vocabulario: base de datos, conexión, consulta, resultado, fila, variante y error. Las implementaciones concretas heredan de esas clases y adaptan cada operación al cliente nativo de SQLite, MySQL/MariaDB o PostgreSQL.

Grupo	Clases principales	Responsabilidad
Capa común	DB_SQL_DATABASE, DB_SQL_CONNECTION, DB_SQL_QUERY, DB_SQL_RESULT, DB_SQL_VARIANT, DB_SQL_ROW, DB_SQL_ERROR, DB_SQL_FACTORY	Define la interfaz común de trabajo.
SQLite	SQLITE_DATABASE, SQLITE_CONNECTION, SQLITE_QUERY, SQLITE_RESULT	Adapta la interfaz común a una base SQLite en fichero.
MySQL/MariaDB	MYSQL_DATABASE, MYSQL_CONNECTION, MYSQL_QUERY, MYSQL_RESULT, MYSQL_VARIANT	Adapta la interfaz común al cliente MySQL compatible.
PostgreSQL	POSTGRESQL_DATABASE, POSTGRESQL_CONNECTION, POSTGRESQL_QUERY, POSTGRESQL_RESULT	Adapta la interfaz común al cliente PostgreSQL.

5. DB_SQL_DATABASE: la abstracción de base de datos

DB_SQL_DATABASE es la clase base que representa una base de datos SQL desde el punto de vista de la aplicación. No es una conexión ni una consulta concreta, sino el objeto que agrupa el motor, el estado de apertura, la conexión asociada, la lista de tablas, la pila de errores, la codificación y la lógica común de ejecución.

Entre sus responsabilidades están abrir y cerrar la base de datos, crear conexiones, crear consultas, ejecutar consultas ya construidas, gestionar transacciones, hacer commit o rollback, crear y liberar savepoints, listar tablas, crear o borrar tablas de forma auxiliar y consultar el número de registros de una tabla.

La clase también contiene una idea importante: la base de datos puede ser thread safe si dispone de un mutex interno. El método IsThreadSafe permite consultar esa condición, y Lock/Unlock permiten proteger operaciones cuando la implementación o el uso de la aplicación lo requieren.

```
if(database->Open())
{
    if(database->IsThreadSafe())
    {
        database->Lock();
    }

    // Operaciones sobre la base de datos.

    if(database->IsThreadSafe())
    {
        database->Unlock();
    }

    database->Close();
}
```

DB_SQL_DATABASE no debe verse como una clase auxiliar menor. Es el contrato principal del bloque Databases. Las clases MYSQL_DATABASE, SQLITE_DATABASE y POSTGRESQL_DATABASE especializan ese contrato para cada motor concreto.

6. DB_SQL_CONNECTION: opciones de conexión

DB_SQL_CONNECTION representa los datos necesarios para conectar con una base de datos. En motores cliente/servidor, las opciones habituales son URL, PORT, DATABASE, USER, PASSWORD y TIMEOUT. En SQLite, además, aparece PATH porque la base de datos se encuentra normalmente en un fichero dentro de la sección databases de assets.

La conexión no se configura mediante una estructura rígida distinta para cada motor, sino mediante pares clave/valor. Este diseño permite compartir una forma común de configurar conectores diferentes, manteniendo suficiente flexibilidad para opciones específicas.

```
DB_SQL_CONNECTION* connection = database->CreateConnection();
if(!connection) return false;

connection->SetOption(__L("URL")      , __L("127.0.0.1"));
connection->SetOption(__L("DATABASE"), __L("test"));
connection->SetOption(__L("USER")     , __L("sirgraham"));
connection->SetOption(__L("PASSWORD"), __L("sirgraham"));
connection->SetOption(__L("TIMEOUT") , __L("30"));
```

En el caso de SQLite, el ejemplo obtiene el directorio de bases de datos a través de XPATHMANAGER y lo pasa como opción PATH. Esta integración es coherente con el resto del libro: los recursos de aplicación viven bajo assets y se accede a ellos por secciones de ruta, no mediante rutas absolutas dispersas por el código.

```
XPATH path;

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_DATABASES, path);
connection->SetOption(__L("PATH"), path.Get());
```

7. Configuración de base de datos en la aplicación

El ejemplo Databases mantiene la configuración principal en Databases_CFG.cpp y Databases_CFG.h. Como vimos en el capítulo de primera aplicación, la configuración de GEN Framework no debe estar repartida por el programa. Se centraliza en una clase derivada de APPFLOWCFG, se mapea contra entradas del fichero INI y se accede posteriormente mediante getters específicos.

La sección database del INI contiene los campos url, port, databasename, user, password y timeoutconnection. Esto permite cambiar el servidor o la base de datos sin recompilar. En SQLite, el nombre de la base puede combinarse con la ruta de assets/databases. En MySQL/MariaDB y PostgreSQL, esos mismos valores permiten abrir una conexión de servidor.

```
[database]
url=127.0.0.1
port=0
databasename=test
user=sirgraham
password=sirgraham
timeoutconnection=30
```

El mapeo de variables mantiene el fichero actualizado y evita claves obsoletas. Las variables reales quedan dentro de DATABASES_CFG y el resto de la aplicación no necesita manipular directamente el fichero INI.

```
AddValue(XFILECFG_VALUETYPE_STRING, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_URL, &db_url);
AddValue(XFILECFG_VALUETYPE_INT, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_PORT, &db_port);
AddValue(XFILECFG_VALUETYPE_STRING, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_DATABASENAME, &db_databasename);
AddValue(XFILECFG_VALUETYPE_STRING, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_USER, &db_user);
AddValue(XFILECFG_VALUETYPE_STRING, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_PASSWORD, &db_password);
AddValue(XFILECFG_VALUETYPE_INT, DATABASESCFG_SECTION_DATABASE, DATABASESCFG_DATABASE_TIMEOUTCONNECTION, &db_timeoutconnection);
```

8. Integración con assets y XPATHMANAGER

El bloque Databases utiliza la sección XPATHSMANAGERSECTIONTYPE_DATABASES para mantener ordenados los ficheros de base de datos. En una aplicación con GEN Framework, el ejecutable no debe ser el lugar donde se colocan recursos, configuraciones o ficheros de datos. Esos recursos deben vivir dentro de assets.

El ejemplo activa la sección databases y crea las carpetas necesarias en disco. Este patrón permite que una aplicación local con SQLite tenga su fichero de base de datos en una ubicación homogénea, y que una aplicación cliente/servidor mantenga igualmente bajo assets sus configuraciones, logs y recursos auxiliares.

```
GEN_XPATHSMANAGER.AdjustRootPathDefault(APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.AddPathSection(XPATHSMANAGERSECTIONTYPE_DATABASES, APPFLOW_DEFAULT_DIRECTORY_DATABASES);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

9. DB_SQL_QUERY: construcción, binding y ejecución

DB_SQL_QUERY representa una consulta SQL. La clase permite asignar el texto de la consulta mediante Set, enlazar parámetros mediante Bind, ejecutar la consulta y obtener el resultado asociado. Esta separación es importante porque una consulta no debería construirse concatenando valores de usuario dentro de una cadena si se puede evitar. El binding de parámetros ayuda a separar el texto SQL de los datos.

GEN Framework soporta binding de enteros, cadenas XCHAR, cadenas char, flotantes y fechas XDATETIME. Cada implementación concreta se encarga de adaptar esos valores al mecanismo de parámetros del motor. En SQLite y MySQL se utiliza el marcador de posición «?». En PostgreSQL se utilizan parámetros numerados como \$1, \$2, \$3 y \$4.

```
DB_SQL_QUERY* query = database->CreateQuery();
if(!query) return false;

query->Set(__L("INSERT INTO test (id,name,value,date) VALUES (?,?,,?)"));

query->Bind(0, 1);
query->Bind(1, __L("Sir Graham"));
query->Bind(2, 11.5f);
query->Bind(3, (*datetime));

bool status = database->Execute(query);

query->UnbindAll();
delete query;
```

La diferencia de placeholders obliga a tener cierta prudencia al escribir SQL común. El ejemplo Databases muestra esta adaptación explícita: para MySQL y SQLite se genera una sentencia con ?, mientras que para PostgreSQL se genera una sentencia con \$1, \$2, \$3 y \$4. El resto del flujo sigue siendo común.

```
switch(database->GetType())
{
    case DB_SQL_DATABASE_TYPE_MYSQL :
    case DB_SQL_DATABASE_TYPE_SQLITE : querystring.Format(__L("INSERT INTO %s (id,name,value,date)
VALUES (?,?,,?)"), DATABASES_TABLE_TEST_NAME);
        break;

    case DB_SQL_DATABASE_TYPE_POSTGRESQL : querystring.Format(__L("INSERT INTO %s (id,name,value,date)
VALUES ($1,$2,$3,$4)"), DATABASES_TABLE_TEST_NAME);
        break;
}
```

10. Transacciones, commit, rollback y savepoints

El acceso a una base de datos no debe entenderse únicamente como ejecutar sentencias aisladas. En aplicaciones reales es frecuente necesitar que varias operaciones se comporten como una unidad lógica. Si todas se ejecutan correctamente, se confirma la transacción. Si una falla, se deshacen los cambios.

DB_SQL_DATABASE define Transaction, Commit, Rollback, Savepoint y ReleaseSavepoint. Cada motor implementa estos métodos según su propia API y sintaxis. El código de aplicación puede trabajar con la idea común de transacción y dejar que el conector concrete el detalle.

```
database->Transaction();

bool status = database->Execute(queryinsert1);
status      &= database->Execute(queryinsert2);
```

```

if(status)
    database->Commit();
else database->Rollback();

```

Los savepoints permiten crear puntos intermedios dentro de una transacción. No todas las aplicaciones los necesitan, pero resultan útiles cuando una operación grande puede dividirse en bloques y se desea retroceder solo una parte del trabajo.

11. Creación y gestión de tablas

Aunque Databases permite ejecutar SQL directamente, DB_SQL_DATABASE incluye métodos auxiliares para operaciones habituales con tablas: Table_IsThere, Table_Create, Table_Delete y Table_GetNRecords. Estos métodos no sustituyen al diseño completo de un esquema de base de datos, pero simplifican ejemplos, herramientas y casos sencillos.

El ejemplo Databases borra la tabla de prueba, comprueba si existe y la crea de nuevo con un conjunto de campos adaptado al motor. Esto muestra un punto importante: la interfaz común no impide que haya diferencias en los tipos SQL. SQLite, MySQL/MariaDB y PostgreSQL no definen exactamente igual los campos, claves y fechas.

```

XCHAR* fields[] = { __L("id int PRIMARY KEY"),
                    __L("name varchar(50)"),
                    __L("value float"),
                    __L("date DATETIME") };

bool isexist = false;

database->Table_IsThere(__L("test"), __L("id"), isexist);
if(!isexist)
{
    database->Table_Create(__L("test"), fields, 4);
}

```

Para aplicaciones reales conviene ser más cuidadoso: versionar el esquema, controlar migraciones, separar la creación inicial de tablas de la lógica normal de ejecución y registrar errores de forma clara. El bloque Databases proporciona la base, pero el diseño de datos sigue siendo responsabilidad de la aplicación.

12. DB_SQL_RESULT, DB_SQL_ROW y DB_SQL_ROWHEADER

Cuando una consulta devuelve datos, el resultado se representa mediante DB_SQL_RESULT. Esta clase proporciona una interfaz común para recorrer filas, consultar el número de columnas, moverse al primer o último registro, avanzar con Next, abortar el recorrido y procesar la fila actual.

Cada fila se representa mediante DB_SQL_ROW, que contiene una colección de DB_SQL_VARIANT. La cabecera se representa como DB_SQL_ROWHEADER, una colección de nombres de columna. De esta forma, el código puede recorrer columnas sin depender de los tipos nativos de cada cliente de base de datos.

```

DB_SQL_RESULT* result = query->GetResult();
if(result)
{
    result->ProcessRow();

    while(result->HasNext())
    {
        DB_SQL_ROW*      row      = result->GetRow();
        DB_SQL_ROWHEADER* header = result->GetHeader();
    }
}

```

```

        for(XDWORD c=0; c<result->GetNumColumns(); c++)
        {
            console->Printf(__L("%s = "), header->Get(c)->Get());
            // Interpretar row->Get(c) según su tipo.
        }

        result->Next();
    }
}

```

El patrón de recorrido es deliberadamente parecido entre motores. El resultado concreto lo implementan MYSQL_RESULT, SQLITE_RESULT o POSTGRESQL_RESULT, pero la aplicación trabaja con DB_SQL_RESULT.

13. DB_SQL_VARIANT: tipos de datos SQL en una forma común

DB_SQL_VARIANT deriva de XVARIANT y adapta la idea de valor genérico al contexto de base de datos. Un campo leído desde SQL puede ser nulo, entero, doble entero, flotante, cadena, fecha, hora o fecha-hora. La variante permite transportar ese valor sin que el código de aplicación dependa de un tipo nativo de MySQL, SQLite o PostgreSQL.

Además del tipo, DB_SQL_VARIANT mantiene flags asociados al campo: not null, primary key, unique, unsigned, zerofill, binary, auto increment, blob, timestamp y otros indicadores útiles para describir metadatos de columnas.

```

DB_SQL_VARIANT value = row->Get(c);

switch(value.GetType())
{
    case DB_SQL_VARIANT_TYPE_STRING : console->Printf(__L("%s"), (XCHAR*)value); break;
    case DB_SQL_VARIANT_TYPE_INTEGER : console->Printf(__L("%d"), (int)value); break;
    case DB_SQL_VARIANT_TYPE_FLOAT : console->Printf(__L("%f"), (float)value); break;

    case DB_SQL_VARIANT_TYPE_DATETIME : { DB_SQL_DATETIME datetime = value;
        XSTRING string;
        datetime.GetDateTimeToString(XDATETIME_FORMAT_STANDARD, string);
        console->Printf(__L("%s"), string.Get());
    }
    break;
}
}

```

MYSQL_VARIANT especializa algunos comportamientos para MySQL, especialmente en la conversión de fechas y tamaños de datos. En otros motores, el resultado se adapta dentro de las clases de consulta y resultado correspondientes.

14. DB_SQL_ERROR y gestión de errores

DB_SQL_ERROR representa un error producido durante operaciones de base de datos. La clase base DB_SQL_DATABASE mantiene una lista de errores y ofrece GetLastError, Error y ClearPreviousErrors. Esta arquitectura permite que las operaciones fallen devolviendo false, pero que además quede una descripción de lo ocurrido.

Los tipos de error incluyen error de memoria, error de conexión, error de sentencia y error de transacción. En una aplicación real es importante no limitarse a saber que una operación falló. Hay que registrar el contexto: qué consulta se ejecutaba, qué motor se estaba usando, si la base estaba abierta, qué conexión se había configurado y cuál fue la última descripción de error.

```

if(!database->Execute(query))
{
    XCHAR* error = database->GetLastError();
    if(error)
    {
        APPFLOW_LOG_ENTRY(XLOGLEVEL_ERROR, APPFLOW_CFG_LOG_SECTIONID_GENERIC, false, __L("Database error:
%s"), error);
    }
}

```

15. SQLite: base local en fichero

SQLite es el conector más sencillo de desplegar porque no requiere un servidor externo. La base de datos se almacena en un fichero, normalmente dentro de assets/databases. Esto lo convierte en una opción muy cómoda para ejemplos, herramientas locales, pruebas, aplicaciones embebidas o programas que necesitan persistencia sin instalar un servicio.

Las clases `SQLITE_DATABASE`, `SQLITE_CONNECTION`, `SQLITE_QUERY` y `SQLITE_RESULT` implementan la interfaz común. La conexión utiliza `PATH` para localizar el directorio y `DATABASE` para indicar el nombre de la base. La consulta adapta el binding a la API de SQLite y el resultado permite recorrer filas mediante los métodos comunes.

```

database = DB_SQL_FACTORY::Create(DB_SQL_DATABASE_TYPE_SQLITE);

DB_SQL_CONNECTION* connection = database->CreateConnection();
connection->SetOption(__L("DATABASE"), __L("test"));
connection->SetOption(__L("PATH"), path.Get());

database->Open();

```

SQLite es especialmente interesante en GEN Framework porque encaja muy bien con la idea de assets. La aplicación puede llevar su base local como un recurso de datos, crearla si no existe y mantenerla en una ubicación homogénea en todas las plataformas soportadas.

16. MySQL y MariaDB: conexión cliente/servidor

El conector MySQL permite trabajar con un servidor MySQL o compatible, como MariaDB cuando se utiliza una biblioteca cliente y un protocolo compatibles. A diferencia de SQLite, aquí la base de datos no está en un fichero local gestionado por la aplicación, sino en un servidor al que se accede mediante URL, puerto, usuario, contraseña y nombre de base.

`MYSQL_DATABASE` define el tipo, nombre, transacciones, creación de consultas, creación de conexión y codificación. `MYSQL_CONNECTION` abre y cierra la conexión real. `MYSQL_QUERY` prepara y ejecuta sentencias, enlaza parámetros y construye resultados. `MYSQL_RESULT` permite recorrer filas y columnas con la misma interfaz que el resto de motores.

```

database = DB_SQL_FACTORY::Create(DB_SQL_DATABASE_TYPE_MYSQL);

DB_SQL_CONNECTION* connection = database->CreateConnection();
connection->SetOption(__L("URL"), APPFLOW_CFG.Database_GetURL()->Get());
connection->SetOption(__L("PORT"), __L("3306"));
connection->SetOption(__L("DATABASE"), APPFLOW_CFG.Database_DatabaseName()->Get());
connection->SetOption(__L("USER"), APPFLOW_CFG.Database_GetUser()->Get());
connection->SetOption(__L("PASSWORD"), APPFLOW_CFG.Database_GetPassword()->Get());

database->Open();

```

En un entorno profesional, el uso de MySQL/MariaDB requiere cuidar la configuración de red, las credenciales, los permisos del usuario, la codificación, la reconexión y el tratamiento de errores. GEN Framework proporciona la capa de acceso; la seguridad y la administración del servidor siguen siendo responsabilidad del despliegue.

17. PostgreSQL: conector relacional avanzado

PostgreSQL se integra mediante `POSTGRESQL_DATABASE`, `POSTGRESQL_CONNECTION`, `POSTGRESQL_QUERY` y `POSTGRESQL_RESULT`. Conceptualmente se usa igual que el resto: se crea una base de datos mediante `DB_SQL_FACTORY`, se configura una conexión, se abre, se crean consultas y se recorren resultados.

Una diferencia visible en el ejemplo es la forma de parametrizar consultas. PostgreSQL utiliza parámetros numerados como \$1, \$2, \$3, mientras que SQLite y MySQL utilizan el marcador «?». Este detalle aparece localizado en el punto donde se construye la consulta, no repartido por toda la aplicación.

```
query->Set(__L("INSERT INTO test (id,name,value,date) VALUES ($1,$2,$3,$4);"));

query->Bind(0, 1);
query->Bind(1, __L("Sir Graham"));
query->Bind(2, 11.5f);
query->Bind(3, (*datetime));

database->Execute(query);
```

PostgreSQL suele ser una buena opción cuando la aplicación necesita un motor relacional sólido, transacciones robustas, tipos de datos avanzados, funciones del servidor o despliegues con mayor exigencia de consistencia. La capa de GEN Framework permite mantener un estilo de código semejante al de otros motores.

18. Ejemplo completo de flujo básico

El siguiente esqueleto resume el patrón de uso que aparece en el ejemplo Databases: crear el motor, crear la conexión, configurar opciones, abrir, crear consulta, ejecutar, recorrer resultados y cerrar. No pretende ser una aplicación completa, sino una guía de lectura para entender las piezas.

```
DB_SQL_DATABASE* database = DB_SQL_FACTORY::Create(DB_SQL_DATABASE_TYPE_SQLITE);
if(!database) return false;

DB_SQL_CONNECTION* connection = database->CreateConnection();
if(!connection)
{
    delete database;
    return false;
}

connection->SetOption(__L("DATABASE"), __L("test"));
connection->SetOption(__L("PATH"), path.Get());

if(database->Open())
{
    DB_SQL_QUERY* query = database->CreateQuery();
    if(query)
    {
        query->Set(__L("SELECT id,name,value,date FROM test;"));

        if(database->Execute(query))
        {
```

```

        DB_SQL_RESULT* result = query->GetResult();
        if(result)
        {
            result->ProcessRow();
            while(result->HasNext())
            {
                DB_SQL_ROW* row = result->GetRow();
                // Leer valores de la fila.
                result->Next();
            }
        }

        delete query;
    }

    database->Close();
}

delete connection;
database->SetConnection(NULL);
delete database;

```

La limpieza es tan importante como la apertura. Una consulta creada con `CreateQuery` debe destruirse cuando deja de usarse. La conexión debe liberarse y la base debe cerrarse antes de destruirse. Esta disciplina sigue el patrón general de GEN Framework: Ini/End, apertura/cierre, creación/destrucción y punteros a NULL cuando procede.

19. Activación mediante CMake

El bloque Databases se activa mediante opciones de CMake. El ejemplo Databases activa el soporte SQL general y, además, los conectores concretos para SQLite, MySQL y PostgreSQL. En una aplicación real se pueden activar solo los necesarios.

```

option(DATABASES_SQL_FEATURE          "DataBases SQL"          ON)
option(DATABASES_SQL_SQLITE_FEATURE    "DataBases SQLite"      ON)
option(DATABASES_SQL_MYSQL_FEATURE     "DataBases MySQL"       ON)
option(DATABASES_SQL_POSTGRESQL_FEATURE "DataBases PostgreSQL" ON)

```

Activar un conector implica que las librerías y cabeceras correspondientes deben estar disponibles para la plataforma de compilación. SQLite suele ser más sencillo porque puede integrarse de forma más local. MySQL/MariaDB y PostgreSQL pueden requerir librerías cliente concretas, rutas de include y librerías enlazadas correctamente.

El `CMakeLists.txt` del ejemplo también activa AppFlow, consola, log, configuración general y configuración de log. Esto encaja con la idea del ejemplo: no solo probar SQL, sino hacerlo dentro de una aplicación con GEN Framework completa, con configuración, trazas, estado y recursos.

20. Relación con AppFlow, DIO, XUtils y Cipher

Databases no vive aislado. Se apoya en XUtils para cadenas, rutas, fechas, variantes, vectores, mapas, mutexes y trazas. Se apoya en AppFlow para configuración, logs, ciclo de aplicación y rutas de assets. Puede relacionarse con DIO cuando la aplicación obtiene datos de red antes de persistirlos, o cuando una operación de base de datos forma parte de un servicio de comunicación.

Cipher también puede intervenir si la aplicación necesita proteger credenciales, cifrar campos sensibles antes de almacenarlos o calcular hashes de integridad. Es importante distinguir entre cifrar la comunicación con el servidor, que puede depender del motor y su cliente, y cifrar datos de aplicación antes de escribirlos en tablas. GEN Framework proporciona bloques para ambas necesidades, pero la decisión de arquitectura debe tomarse caso por caso.

21. Catálogo resumido del bloque Databases

Clase / grupo	Qué aporta	Relación
DB_SQL_Base	Define tipos base, alias y enumeración DB_SQL_DATABASE_TYPE.	Se incluye en el resto del bloque SQL.
DB_SQL_FACTORY	Crea la base concreta según DB_SQL_DATABASE_TYPE.	Punto de entrada para seleccionar SQLite, MySQL/MariaDB o PostgreSQL.
DB_SQL_DATABASE	Contrato principal de base de datos SQL.	Open, Close, Execute, transacciones, tablas, errores y mutex.
DB_SQL_CONNECTION	Opciones de conexión mediante clave/valor.	URL, PORT, DATABASE, PATH, USER, PASSWORD, TIMEOUT.
DB_SQL_QUERY	Consulta SQL y binding de parámetros.	Set, Bind, Execute, GetResult y UnbindAll.
DB_SQL_RESULT	Resultado de una consulta.	Recorrido por filas, columnas y cursor.
DB_SQL_VARIANT	Valor SQL genérico basado en XVARIANT.	Tipos de campo y flags de metadatos.
DB_SQL_ROW / DB_SQL_ROWHEADER	Fila y nombres de columnas.	Lectura homogénea de resultados.
DB_SQL_ERROR	Representación de errores SQL.	Conexión, statement, memoria y transacción.
SQLITE_*	Implementación SQLite.	Base local en fichero.
MYSQL_*	Implementación MySQL/MariaDB compatible.	Servidor SQL cliente/servidor.
POSTGRESQL_*	Implementación PostgreSQL.	Servidor SQL con parámetros numerados.

22. Criterios de uso en una aplicación con GEN Framework

Para elegir motor conviene pensar primero en el tipo de despliegue. Si la aplicación necesita una base local sin servidor, SQLite es la opción natural. Si debe compartir datos entre varios clientes o integrarse con infraestructura existente, MySQL/MariaDB o PostgreSQL tienen más sentido. Si el objetivo es desarrollar y probar rápido, SQLite puede servir incluso cuando el despliegue final sea servidor, siempre que la aplicación no abuse de sintaxis específica.

También conviene separar la lógica de negocio del SQL concreto. El código que decide qué significa una operación para la aplicación no debería estar mezclado con detalles de placeholders, creación de tablas o conversión de variantes. Una buena aplicación con GEN Framework puede encapsular su acceso a datos en clases propias y usar Databases como capa de conexión y ejecución.

Hay que evitar construir SQL concatenando datos externos sin control. Aunque el ejemplo utiliza cadenas para construir algunas consultas de estructura, los valores de datos deben enlazarse con Bind siempre que sea posible. Esto mejora claridad, reduce errores de conversión y evita muchos problemas típicos de SQL construido manualmente.

23. Errores habituales al usar Databases

- Activar un conector en CMake sin tener instalada o disponible la librería cliente necesaria.
- Olvidar configurar DATABASE, PATH, URL, PORT, USER, PASSWORD o TIMEOUT antes de llamar a Open.
- Usar rutas absolutas para SQLite en lugar de la sección databases de XPATHMANAGER.
- Escribir una misma sentencia SQL para todos los motores cuando la sintaxis de parámetros o tipos no es idéntica.
- No llamar a UnbindAll después de reutilizar una consulta con parámetros.
- No cerrar la base de datos antes de destruir la conexión o el objeto DB_SQL_DATABASE.
- No consultar GetLastError o la pila de errores cuando Execute devuelve false.
- Confundir persistencia local con servidor: SQLite y PostgreSQL/MySQL resuelven problemas distintos aunque compartan interfaz SQL.

24. Resumen práctico

Databases proporciona a GEN Framework una capa común para trabajar con bases de datos relacionales. Su diseño gira alrededor de una idea sencilla: la aplicación debe poder hablar de bases de datos, conexiones, consultas, resultados, filas y variantes sin quedar atada directamente a la API nativa de un motor concreto.

DB_SQL_FACTORY crea el motor. DB_SQL_DATABASE define el contrato general. DB_SQL_CONNECTION configura la conexión. DB_SQL_QUERY contiene la sentencia y sus parámetros. DB_SQL_RESULT permite recorrer datos. DB_SQL_VARIANT adapta los tipos SQL al mundo de GEN Framework. Las implementaciones SQLITE, MYSQL y POSTGRESQL concretan ese contrato para cada motor.

El bloque está orientado a SQL y bases de datos relacionales. Su arquitectura deja abierta la incorporación de conectores documentales u otros modelos, pero su valor actual ya es claro: permitir que una aplicación con GEN Framework acceda a SQLite, MySQL/MariaDB o PostgreSQL mediante una misma arquitectura, integrada con assets, XPATHMANAGER, AppFlow, XUtils y el estilo general del framework.

Capítulo XIII: Scripts, algoritmos dinámicos y plugins en tiempo real

1. El papel de Scripts dentro de GEN Framework

El bloque Scripts permite que una aplicación desarrollada con GEN Framework ejecute algoritmos externos escritos en lenguajes de script. Su objetivo no es sustituir al código C/C++ de la aplicación, sino complementarlo allí donde interesa ganar flexibilidad: tareas auxiliares, reglas modificables, automatizaciones, pruebas, acciones de mantenimiento, pequeños algoritmos de negocio o extensiones cargadas en tiempo de ejecución.

En capítulos anteriores hemos visto que GEN Framework intenta separar la lógica de aplicación de las dependencias de plataforma. Scripts añade una separación adicional: permite separar ciertas decisiones de comportamiento del binario compilado. Cuando una parte del algoritmo se escribe en un fichero de script, puede modificarse, copiarse, sustituirse o ampliarse sin recompilar la aplicación. Esto resulta especialmente útil en proyectos donde determinadas reglas cambian con frecuencia o donde se quiere dar al usuario avanzado una forma controlada de extender el comportamiento.

La idea debe entenderse en términos prácticos. Una aplicación industrial puede necesitar variar una fórmula de cálculo. Una herramienta de pruebas puede requerir nuevas secuencias de acciones. Una aplicación de diagnóstico puede cargar pequeños procedimientos de verificación. Una utilidad gráfica puede automatizar operaciones repetitivas. En todos estos casos, recompilar toda la aplicación por cada cambio pequeño puede ser incómodo. El bloque Scripts permite colocar esa parte variable en assets/scripts y ejecutarla desde GEN Framework mediante una interfaz común.

El módulo está diseñado para trabajar con tres lenguajes: JavaScript, Lua y .GEN, representado por los ficheros con extensión .g. Este último puede entenderse como un lenguaje propio, una variante simplificada del lenguaje C para ejecutar lógica externa dentro del entorno de GEN Framework. La aplicación no tiene que tratar cada lenguaje de forma completamente independiente. La clase base SCRIPT proporciona una capa común y cada lenguaje concreto implementa su forma de cargar, registrar funciones y ejecutar el código.

2. De algoritmos compilados a algoritmos cargables

En C/C++, la forma natural de añadir funcionalidad es escribir código, compilar y enlazar. Esa estrategia es excelente para el núcleo de una aplicación: ofrece rendimiento, control, tipos fuertes y acceso completo al framework. Sin embargo, no siempre es la mejor respuesta para todo. Hay partes de una aplicación que son más cambiantes que críticas: scripts de mantenimiento, reglas de validación, secuencias de test, integración con herramientas externas, cálculos parametrizables o automatizaciones de interfaz.

Cuando esas partes viven dentro del ejecutable, cada modificación obliga a recompilar, probar y redistribuir. Cuando viven en scripts, la aplicación puede cargar el fichero en ejecución y aplicar la nueva lógica directamente. Esto convierte el bloque Scripts en una especie de sistema de plugins ligero. No se cargan librerías binarias ni se modifica el ejecutable principal: se carga texto, se interpreta o evalúa mediante el motor correspondiente y se exponen al script las funciones que la aplicación decide permitir.

Esta flexibilidad no debe confundirse con falta de control. La aplicación sigue decidiendo qué scripts se cargan, desde qué directorio, con qué bibliotecas, con qué funciones disponibles y en qué momento se ejecutan. El script no debería tener acceso indiscriminado a todo el framework; solo a lo que se expone mediante las bibliotecas de script. Esa frontera es importante porque permite adaptar el nivel de potencia a cada proyecto.

3. Tres lenguajes soportados

El bloque Scripts reconoce el tipo de script por su extensión. La clase SCRIPT incluye SCRIPT_TYPE y el método GetTypeByExtension, que asocia los ficheros .g, .lua y .js a sus implementaciones concretas cuando las opciones de compilación correspondientes están activas.

Extensión	Tipo interno	Clase principal	Uso típico
.g	SCRIPT_TYPE_G	SCRIPT_LNG_G	Lenguaje propio de GEN Framework, cercano al lenguaje C, útil para scripts simples y controlados.
.lua	SCRIPT_TYPE_LUA	SCRIPT_LNG_LUA	Integración con Lua para scripts ligeros, embebibles y fáciles de modificar.
.js	SCRIPT_TYPE_JAVASCRIPT	SCRIPT_LNG_JAVASCRIPT	Integración con JavaScript mediante Duktape para algoritmos dinámicos y automatizaciones.

La existencia de tres lenguajes no significa que una aplicación deba utilizarlos todos. Cada proyecto puede activar solo los que necesite mediante CMake. Una aplicación pequeña puede trabajar únicamente con Lua. Una herramienta que ya tenga scripts de usuario en JavaScript puede activar JavaScript. Un entorno muy controlado puede usar solamente .g. La selección se realiza en compilación y permite ajustar dependencias, tamaño y funcionalidad.

4. La clase SCRIPT como punto común

SCRIPT es la clase base del sistema. Representa un script cargado, su tipo, su ruta, su nombre, su texto, sus bibliotecas disponibles, sus funciones exportadas, su temporizador, su posible hilo de ejecución y su código de error. Al heredar de XSUBJECT también puede publicar eventos, lo que permite avisar a la aplicación cuando aparece un error o una interrupción durante la ejecución.

Entre sus métodos más importantes están Create, Load, LoadAdd, Save, Run, RunWithThread, IsRunThread, CancelExecution, AddLibrary, AddLibraryFunction y LoadScriptAndRun. Create construye la implementación adecuada según el tipo o la extensión. Load lee un fichero de script. LoadAdd permite concatenar varios ficheros en una misma instancia. Run ejecuta el contenido cargado. RunWithThread permite lanzar la ejecución en un hilo. LoadScriptAndRun es especialmente práctico cuando la aplicación tiene una lista de scripts configurada y quiere cargarlos y ejecutarlos de forma secuencial.

```
SCRIPT* script = SCRIPT::Create(__L("test.lua"));
if(script)
{
    XPATH xpath;

    GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SCRIPTS, xpath);
    xpath.Slash_Add();
    xpath += __L("test.lua");

    if(script->Load(xpath))
    {
        int returnvalue = 0;
        int error       = script->Run(&returnvalue);
    }

    GEN_DELETE script;
}
```

El ejemplo anterior muestra el uso manual. En una aplicación real basada en AppFlow suele ser más cómodo utilizar la lista de scripts de configuración y delegar en LoadScriptAndRun. Esa función crea cada script, ajusta bibliotecas mediante un callback opcional, carga el contenido desde assets/scripts, usa la caché si está activa y ejecuta el script.

```
SCRIPT::LoadScriptAndRun (APPFLOW_CFG.Scripts_GetAll(), SCRIPTSEXAMPLE::AdjustLibraries);
```

5. Carga, rutas y assets/scripts

Como ocurre con otros bloques de GEN Framework, los scripts no deben localizarse mediante rutas absolutas dispersas por el código. La aplicación debe usar XPATHSMANAGER y la sección XPATHSMANAGERSECTIONTYPE_SCRIPTS. En el ejemplo ScriptsExample, durante la inicialización se ajusta la raíz de assets y se añade la sección de scripts para que el framework conozca dónde buscar los ficheros externos.

```
GEN_XPATHSMANAGER.AdjustRootPathDefault (APPFLOW_DEFAULT_DIRECTORY_ROOT);
GEN_XPATHSMANAGER.AddPathSection (XPATHSMANAGERSECTIONTYPE_SCRIPTS ,
APPFLOW_DEFAULT_DIRECTORY_SCRIPTS);
GEN_XPATHSMANAGER.AddPathSection (XPATHSMANAGERSECTIONTYPE_GRAPHICS,
APPFLOW_DEFAULT_DIRECTORY_GRAPHICS);
GEN_XPATHSMANAGER.CreateAllPathSectionOnDisk();
```

La estructura típica queda así:

```
assets/
├── scriptsexample.ini
├── scriptsexample.log
├── scripts/
│   ├── test.g
│   ├── test.js
│   ├── test.lua
│   ├── test1.lua
│   ├── test2.lua
│   └── inputsimul.js
└── graphics/
    └── inputsimul.png
```

Esta organización permite que el ejecutable permanezca separado de los algoritmos externos. El programa se compila una vez, pero los scripts pueden cambiarse dentro de assets/scripts. Si el proyecto necesita organizar scripts por familias, también puede crear subdirectorios o secciones adicionales, pero el punto de partida debe ser siempre el mismo: usar XPATHSMANAGER para mantener una forma homogénea de localizar recursos.

6. Configuración de la lista de scripts

El ejemplo ScriptsExample muestra una integración directa entre Scripts y AppFlowCFG. La configuración principal contiene una sección scriptslist donde se declaran los scripts que la aplicación debe cargar. La aplicación no necesita tener los nombres escritos de forma rígida en el código; los obtiene mediante APPFLOW_CFG.Scripts_GetAll().

```
[scriptslist]
scripts001=test1.lua
scripts002=test2.lua
scripts003=
```

Este mecanismo encaja con la filosofía general del fichero de configuración de GEN Framework. La aplicación puede cambiar la lista de scripts modificando el INI. Si se activa SCRIPT_CACHE, esos scripts pueden precargarse en memoria durante AppProc_FirstUpdate para acelerar ejecuciones posteriores.

```
#ifdef SCRIPT_CACHE_ACTIVE
```

```

GEN_SCRIPT_CACHE.Cache_DelAll();
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SCRIPTS, xpath);
GEN_SCRIPT_CACHE.Cache_AllDirectory(xpath);
GEN_SCRIPT_CACHE.Cache_AllList(APPFLOW_CFG.Scripts_GetAll());

#endif

```

7. SCRIPT_CACHE: cache de scripts

SCRIPT_CACHE es un singleton que mantiene en memoria los textos de los scripts. Su utilidad aparece cuando una aplicación ejecuta repetidamente los mismos ficheros o cuando quiere precargar todos los scripts de un directorio. En lugar de leer el fichero cada vez desde disco, se genera un identificador y se almacena el contenido asociado.

La clase proporciona GenerateID, Cache_Add, Cache_Get, Cache_Set, Cache_Del, Cache_DelAll, Cache_AllDirectory y Cache_AllList. El identificador se calcula a partir de una cadena o ruta normalizada. Load y LoadScriptAndRun pueden consultar la caché si SCRIPT_CACHE_ACTIVE está activado.

```

GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_SCRIPTS, xpath);
GEN_SCRIPT_CACHE.Cache_AllDirectory(xpath);

XDWORD ID = GEN_SCRIPT_CACHE.GenerateID(xpath);
XSTRING* scripttext = GEN_SCRIPT_CACHE.Cache_Get(ID);

```

La caché no cambia la semántica del script. Solo evita repetir accesos al sistema de ficheros. En herramientas de pruebas, automatización o ejecución frecuente de plugins ligeros puede ser una mejora apreciable. En desarrollo, conviene recordar que una caché activa puede hacer que los cambios en el fichero no se vean hasta recargarla o limpiarla.

8. Eventos y errores de script

Los scripts pueden fallar por muchas razones: fichero inexistente, tipo no soportado, error de sintaxis, parámetros insuficientes, problema interno del motor o fallo en una función expuesta desde C++. GEN Framework representa esta información mediante SCRIPT_ERRORCODE y SCRIPT_XEVENT.

SCRIPT_ERRORCODE define errores comunes como SCRIPT_ERRORCODE_NONE, SCRIPT_ERRORCODE_INTERNALERROR y SCRIPT_ERRORCODE_INSUF_PARAMS, dejando SCRIPT_ERRORCODE_OWN como punto de extensión para errores propios de cada lenguaje o implementación. SCRIPT_XEVENT permite transportar información hacia los observadores: script afectado, nombre, código de error, texto del error, línea y token actual.

En ScriptsExample, el manejador de eventos traduce esos eventos a trazas. Cuando llega SCRIPT_XEVENT_TYPE_ERROR se muestra el código de error, el texto, la línea y el token actual. Cuando llega SCRIPT_XEVENT_TYPE_BREAK se informa de la línea y del token donde se produjo la interrupción.

```

void SCRIPTSEXAMPLE::HandleEvent_Script(SCRIPTEVENT* event)
{
    switch(event->GetEventType())
    {
        case SCRIPT_XEVENT_TYPE_ERROR : XTRACE_PRINTCOLOR(4,
            __L("Script ERROR [%d]: %s line %d -> \"%s\""),
            event->GetError(),
            event->GetErrorText()->Get(),
            event->GetNLine(),
            event->GetCurrentToken()->Get());
    }
}

```

```

        break;
    }
}

```

9. Bibliotecas de funciones: SCRIPT_LIB y SCRIPT_LIB_FUNCTION

La verdadera potencia del bloque Scripts no está únicamente en ejecutar JavaScript, Lua o .g. Está en poder exponer funciones de GEN Framework o de una aplicación concreta a esos scripts. Para ello se utilizan SCRIPT_LIB y SCRIPT_LIB_FUNCTION.

SCRIPT_LIB representa una biblioteca de funciones disponible para un script. Cada biblioteca tiene un identificador y un método AddLibraryFunctions. SCRIPT_LIB_FUNCTION representa una función concreta: mantiene el puntero a la biblioteca, el nombre visible desde el lenguaje de script y el puntero a la función C/C++ que se llama cuando el script invoca ese nombre.

La firma común de una función de biblioteca es SCRFUNCTIONLIBRARY. Recibe la biblioteca, el script que realiza la llamada, una lista de parámetros en XVECTOR<XVARIANT*> y un XVARIANT de retorno. Esto permite convertir los valores de JavaScript, Lua o .g al mundo de GEN Framework usando una representación común.

```

typedef void (*SCRFUNCTIONLIBRARY) (SCRIPT_LIB* library,
                                    SCRIPT* script,
                                    XVECTOR<XVARIANT*>* params,
                                    XVARIANT* returnvalue);

```

Una biblioteca propia sigue siempre el mismo patrón: hereda de SCRIPT_LIB, registra sus funciones en AddLibraryFunctions y define funciones Call_... que convierten parámetros, ejecutan código C/C++ y escriben el resultado.

```

bool SCRIPT_LIB_MIAPP::AddLibraryFunctions(SCRIPT* script)
{
    if(!script) return false;

    script->AddLibraryFunction(this, __L("MiFuncion"), Call_MiFuncion);

    return true;
}

void Call_MiFuncion(SCRIPT_LIB* library, SCRIPT* script,
                   XVECTOR<XVARIANT*>* params, XVARIANT* returnvalue)
{
    if(!returnvalue) return;

    (*returnvalue) = true;
}

```

Con esta idea, un proyecto puede crear su propio puente entre scripts y C/C++. Por ejemplo, una aplicación industrial puede exponer LecturaSensor, EscribirSalida, EjecutarSecuencia o CalcularConsigna. Una herramienta de test puede exponer ConectarEquipo, EnviarComando o ComprobarRespuesta. El script queda limpio y el acceso real al hardware o al framework permanece en C++.

10. Bibliotecas internas estándar

Al construirse una instancia de SCRIPT, el sistema añade bibliotecas internas. Algunas son estándar y otras dependen de opciones de compilación. Las estándar proporcionan funciones básicas para matemáticas, números aleatorios, temporización, cadenas, rutas, directorios y trazas.

Biblioteca	Clase	Funciones principales	Uso
Math	SCRIPT_LIB_MATH	Abs	Operaciones matemáticas básicas disponibles desde scripts.
Random	SCRIPT_LIB_RAND	RandMax, RandBetween	Generación de números aleatorios para pruebas, simulación o lógica externa.
Timer	SCRIPT_LIB_TIMER	Sleep	Pausas controladas durante automatizaciones o secuencias de prueba.
String	SCRIPT_LIB_STRING	AddString, FindString, CompareString, ReplaceString, SPrintf	Manipulación de texto sin escribir lógica repetida en cada lenguaje.
Path	SCRIPT_LIB_PATH	GetPathScript, GetNameScript	Obtener la ruta y el nombre del script en ejecución.
Dir	SCRIPT_LIB_DIR	IsItExists, ChangeDir, RemoveDir, MakeDir	Operaciones básicas sobre directorios.
Trace	SCRIPT_LIB_TRACE	TraceClearScreen, TraceClearMsgsStatus, TracePrintColor	Salida de diagnóstico integrada con el sistema de trazas de GEN Framework.

Estas bibliotecas hacen que un script no sea un texto aislado. Se ejecuta dentro del ecosistema de GEN Framework y puede usar servicios comunes del framework. En los scripts de ejemplo se ve claramente: test.g, test.js y test.lua calculan un máximo, imprimen por consola, generan números aleatorios y escriben trazas de color.

```
function main()
{
    Console_Printf("JavaScript version %s\n", "0.1.0");
    Console_Printf("Números aleatorios del 1 al 100: \n");

    for(i=0; i<10; i++)
    {
        Console_Printf("%d ", RandBetween(1,100));
    }

    TracePrintColor(1, "Prueba de envío %.2f con cadena %s", 10.45, "hola radiola");
}
```

11. Bibliotecas opcionales e integración con módulos de GEN Framework

Además de las bibliotecas estándar, el bloque Scripts dispone de bibliotecas opcionales que se activan mediante defines o opciones CMake. Estas bibliotecas permiten que los scripts accedan a consola, configuración, logs, sistema, procesos, ventanas, simulación de entrada o funciones específicas de desarrollo.

Biblioteca	Clase	Funciones expuestas	Relación con GEN Framework
Console	SCRIPT_LIB_CONSOLE	Console_GetChar, Console_PutChar, Console_Printf	Integra scripts con la consola de APPFLOWCONSOLE.
CFG	SCRIPT_LIB_CFG	GetFileCFGValue	Permite leer valores del fichero de configuración mediante XFILECFG.
Log	SCRIPT_LIB_LOG	Log_Ini, Log_CFG_SetLimit, Log_CFG_SetFilters, Log_CFG_SetBackup, Log_AddEntry	Integra scripts con el sistema XLOG/APPFLOW_LOG.
System	SCRIPT_LIB_SYSTEM	System_GetType, System_Reboot, System_PowerOff, System_Logout,	Accede a información o acciones del sistema mediante XUtils.

		System_GetEnviromentVar	
Process	SCRIPT_LIB_PROCESS	OpenURL, ExecApplication, MakeCommand, TerminateAplication, TerminateAplicationWithWindow	Permite abrir URLs, lanzar y cerrar procesos.
Window	SCRIPT_LIB_WINDOW	Window_SetPosition, Window_Resize, Window_SetFocus, Window_GetPosX, Window_GetPosY, Window_SetBmpFindCFG	Integra scripts con una aplicación gráfica y búsqueda de elementos en pantalla.
InpSimulate	SCRIPT_LIB_INPUTSIMULATE	InpSim_Key_Press, InpSim_Key_ClickByText, InpSim_Mouse_SetPos, InpSim_Mouse_Click	Permite simular teclado y ratón desde scripts.
Developed Test	SCRIPT_LIB_DEVTEST	DevTest_Func1	Biblioteca de prueba para validar paso de parámetros y retorno de objetos.

El ejemplo inputsimul.js demuestra una combinación muy potente: abrir una aplicación externa, mover su ventana, buscar una posición mediante una máscara bitmap, simular clics de ratón, escribir texto y terminar la aplicación. Esa clase de prueba sería mucho más rígida si estuviera compilada dentro del ejecutable. Como script, se puede ajustar rápidamente.

12. SCRIPT_LNG_JAVASCRIPT

SCRIPT_LNG_JAVASCRIPT implementa la ejecución de JavaScript. Internamente utiliza un contexto Duktape, registra funciones C/C++ como funciones globales del motor y convierte los parámetros hacia X VARIANT. El script define una función main, que actúa como punto de entrada convencional del fichero.

Cuando se registra una función mediante AddLibraryFunction, la clase empuja una función del lenguaje C en el contexto de Duktape, asocia el puntero real de la función de GEN Framework y la publica con el nombre visible para el script. Cuando JavaScript llama a esa función, LibraryCallback recupera el puntero, extrae los argumentos, los transforma en X VARIANT y delega en la función de biblioteca.

Este diseño permite que JavaScript sea solo la capa de expresión del algoritmo. El acceso a consola, trazas, rutas, directorios, procesos o funcionalidades de la aplicación sigue estando implementado en C/C++. La frontera entre script y framework es clara: el script llama por nombre; la biblioteca registrada ejecuta código nativo.

```
SCRIPT* script = SCRIPT::Create(__L("test.js"));
SCRIPT_SET_LIB_CFG(script, (XFILECFG*) &APPFLOW_CFG);
script->Load(xpath);
script->Run();
```

13. SCRIPT_LNG_LUA

SCRIPT_LNG_LUA implementa la ejecución de scripts Lua. Al crearse, inicializa un estado Lua y abre sus bibliotecas estándar. Como en JavaScript, se registra una función de callback que permite enlazar nombres visibles en el script con funciones C/C++ de GEN Framework. El script puede devolver un valor al finalizar la ejecución.

Lua es especialmente adecuado para entornos embebidos y para lógica ligera. Su sintaxis es sencilla, el motor es pequeño y resulta cómodo para reglas que no justifican recompilar la aplicación. En el ejemplo test.lua se define una función max, se imprimen mensajes de consola, se generan números aleatorios y se retorna un valor.

```
function main()

    Console_Printf("Lua version %s\n", _VERSION);
```

```

Console_Printf("Números aleatorios del 1 al 100: \n");

for i=0,10,1 do
    Console_Printf("%d ", RandBetween(1,100));
end

return 25

end

return main()

```

14. SCRIPT_LNG_G: el lenguaje .GEN

SCRIPT_LNG_G implementa el lenguaje propio de GEN Framework para ficheros con extensión .g. El código fuente muestra que esta implementación no se limita a delegar en un motor externo. Define tokens, operadores dobles, tipos internos, comandos, variables, funciones, parámetros y códigos de error propios. Es, por tanto, una pieza específica del framework.

Las clases auxiliares SCRIPT_LNG_G_COMMAND, SCRIPT_LNG_G_FUNCTIONTYPE y SCRIPT_LNG_G_VAR representan partes internas del lenguaje. SCRIPT_LNG_G_COMMAND asocia texto de comando con tokens. SCRIPT_LNG_G_FUNCTIONTYPE mantiene el nombre, el tipo de retorno y la localización de una función. SCRIPT_LNG_G_VAR representa variables con nombre, tipo, valor y marcas como argumento o valor de retorno.

Clase	Responsabilidad
SCRIPT_LNG_G	Motor del lenguaje .g: tokenización, variables, funciones, llamada a comandos y ejecución.
SCRIPT_LNG_G_COMMAND	Relación entre comandos del lenguaje y tokens internos.
SCRIPT_LNG_G_FUNCTIONTYPE	Descripción de funciones definidas en el script: nombre, retorno y localización.
SCRIPT_LNG_G_VAR	Variable del lenguaje .g con nombre, tipo, valor y atributos de ejecución.

El ejemplo test.g muestra una sintaxis cercana al lenguaje C. Define una función max, declara variables enteras, usa if, for, return, llamadas a Console_Printf, RandBetween, TracePrintColor y Console_GetChar. Para un programador C/C++, esta forma resulta familiar y facilita escribir lógica sencilla sin saltar mentalmente a un lenguaje muy diferente.

```

int max(int num1, int num2)
{
    int result;

    if(num1 > num2)
    {
        result = num1;
    }
    else
    {
        result = num2;
    }

    return result;
}

```

15. Integración en ScriptsExample

ScriptsExample es una aplicación de consola basada en APPFLOWCONSOLE y XFSMACHINE. Su objetivo es demostrar cómo integrar el bloque Scripts dentro de una aplicación real del framework. Como en otros ejemplos de GEN Framework, no aparece main de C++; la aplicación define una clase, una instancia con APPLICATIONCREATEINSTANCE, inicialización, primera actualización, actualización periódica y cierre.

Durante AppProc_Ini se configuran nombre, versión, grupos de hilos, trazas, rutas de scripts y gráficos, máquina de estados, mutex, objetivos automáticos de traza y AppFlow Extended. Durante AppProc_FirstUpdate se precarga la caché si está activada y se crean temporizadores. En AppProc_Update se atiende la consola y se reacciona a teclas. La tecla R lanza la ejecución de los scripts configurados.

```
case 'R' : { if(xtimerscriptrun) xtimerscriptrun->Reset();

                SCRIPT::LoadScriptAndRun(APPFLOW_CFG.Scripts_GetAll(),
                                           SCRIPTSEXAMPLE::AdjustLibraries);

                XQWORD timereleased = xtimerscriptrun->GetMeasureMilliseconds();
                XTRACE_PRINTCOLOR(1, __L("Run exec in %d milliseconds."),
                                (XDWORD)timereleased);
            }
            break;
```

El callback AdjustLibraries permite ajustar bibliotecas antes de ejecutar el script. En el ejemplo se utiliza para pasar la configuración principal a SCRIPT_LIB_CFG, de modo que una función expuesta al script pueda consultar valores de XFILECFG.

```
void SCRIPTSEXAMPLE::AdjustLibraries(SCRIPT* script)
{
    #ifdef SCRIPT_LIB_CFG_ACTIVE

        SCRIPT_SET_LIB_CFG(script, (XFILECFG*)&APPFLOW_CFG);

    #endif
}
```

16. Activación mediante CMake

El bloque Scripts se activa mediante opciones CMake. ScriptsExample muestra una configuración amplia, con los tres lenguajes, caché y varias bibliotecas opcionales. En una aplicación real se deberían activar solo las piezas necesarias.

```
option(SCRIPT_FEATURE           "Activate Scripts"           ON)
option(SCRIPT_G_FEATURE         "Activate Script G Language" ON)
option(SCRIPT_LUA_FEATURE       "Activate Script LUA Language" ON)
option(SCRIPT_JAVASCRIPT_FEATURE "Activate Script Javascript" ON)
option(SCRIPT_CACHE_FEATURE     "Activate Script Cache"     ON)

option(SCRIPT_LIB_CONSOLE_FEATURE "Script Lib Console"      ON)
option(SCRIPT_LIB_CFG_FEATURE     "Script Lib Config File"   ON)
option(SCRIPT_LIB_LOG_FEATURE     "Script Lib Log"           ON)
option(SCRIPT_LIB_PROCESS_FEATURE "Script Lib Process"       ON)
option(SCRIPT_LIB_WINDOW_FEATURE  "Script Lib Window"        ON)
option(SCRIPT_LIB_INPUTSIMULATE_FEATURE "Script Lib Input Simulated" ON)
```

Esta granularidad es importante. Activar JavaScript implica incluir su motor. Activar Lua implica enlazar con Lua. Activar bibliotecas de proceso, ventana o simulación de entrada aumenta la potencia de los scripts, pero también amplía lo que un script puede hacer. En aplicaciones donde los scripts puedan ser modificados por usuarios externos, conviene limitar las bibliotecas disponibles a las estrictamente necesarias.

17. Ampliar el sistema con funciones propias

Una de las ideas más interesantes del bloque Scripts es que no está cerrado a las bibliotecas incluidas por el framework. Un proyecto puede crear nuevas bibliotecas específicas y exponer funciones relacionadas con su dominio. Esta es la forma correcta de convertir un script en plugin de una aplicación con GEN Framework: no dando acceso descontrolado al código interno, sino publicando una API pequeña, estable y pensada para el script.

El patrón recomendado es crear una clase `SCRIPT_LIB_...` para el proyecto, registrar sus funciones en `AddLibraryFunctions` y añadir la biblioteca al script antes de ejecutarlo. La función recibe parámetros como `XVARIANT`, valida tipos, llama a la parte C/C++ y devuelve otro `XVARIANT`.

```
class SCRIPT_LIB_CONTROL : public SCRIPT_LIB
{
public:
    bool AddLibraryFunctions(SCRIPT* script)
    {
        script->AddLibraryFunction(this, __L("Control_SetOutput"), Call_SetOutput);
        script->AddLibraryFunction(this, __L("Control_GetInput") , Call_GetInput);
        return true;
    }
};
```

A partir de ahí, un script podría escribir algo tan directo como `Control_SetOutput(3, true)` o `valor = Control_GetInput(1)`. El algoritmo queda en el script, pero el acceso real al dispositivo continúa en C/C++, usando las clases de GEN Framework y respetando el aislamiento de plataforma. Esa separación permite cambiar la regla sin recompilar, manteniendo el control técnico en el framework.

18. Ejemplos de uso

Los ejemplos incluidos en `ScriptsExample` cubren varios escenarios. `test.g`, `test.js` y `test.lua` muestran la misma idea implementada en tres lenguajes: una función `max`, generación de números aleatorios, salida por consola, trazas y lectura de una tecla. Esto demuestra que el bloque Scripts intenta ofrecer una superficie homogénea aunque el lenguaje subyacente cambie.

`test1.js` demuestra el intercambio de datos más complejo. Define un objeto JavaScript y llama a `DevTest_Func1`, una función expuesta desde C++ mediante `SCRIPT_LIB_DEVTEST`. El objetivo es validar que no solo se pueden pasar enteros o cadenas simples, sino también estructuras de datos que el motor de JavaScript representa como objetos.

`inputsimul.js` es el ejemplo más avanzado. Usa `Log_Ini` y `Log_AddEntry` para registrar actividad, `ExecApplication` para lanzar WordPad, `Window_SetPosition` y `Window_Resize` para manipular la ventana, `Window_GetPosX` y `Window_GetPosY` para localizar una imagen de referencia, `InpSim_Mouse_Click` para simular el ratón, `InpSim_Key_ClickByLiteral` o `InpSim_Key_ClickByText` para teclado y `TerminateApplicationWithWindow` para cerrar la aplicación. Es un ejemplo muy claro de automatización externa controlada desde un script.

19. Relación con otros módulos

Scripts es un bloque transversal. Se apoya en XUtils para cadenas, rutas, variantes, mapas, vectores, eventos, hilos, temporizadores y trazas. Se apoya en AppFlow para configuración, lista de scripts, ciclo de vida y logs. Puede relacionarse con DataIO si una biblioteca propia expone comunicaciones o acceso a ficheros. Puede relacionarse con Graphic, Input y UserInterface si se crean scripts para automatizar pantallas, simular entrada o adaptar comportamientos visuales. También puede trabajar con Cipher, Compress o Databases si la aplicación decide exponer funciones seguras y bien definidas.

Esta transversalidad es precisamente su valor. El script se convierte en una capa de decisión externa, mientras que GEN Framework mantiene los servicios nativos. La aplicación decide qué parte del framework aparece visible para el script. Así se evita que cada script tenga que reinventar rutas, logs, trazas, configuración o temporizadores.

20. Criterios de uso

- Usar scripts para lógica cambiante, reglas externas, automatizaciones, pruebas, tareas auxiliares o plugins ligeros.
- Mantener en C/C++ el núcleo crítico de rendimiento, la gestión de memoria, el acceso directo a hardware y la lógica que requiere máximo control.
- Exponer al script solo las funciones necesarias. Una biblioteca pequeña y clara es mejor que una superficie enorme difícil de controlar.
- Utilizar assets/scripts y XPATHMANAGER para localizar ficheros. No introducir rutas absolutas en el código de aplicación.
- Usar AppFlowCFG y la sección scriptslis cuando la aplicación deba ejecutar una lista configurable de scripts.
- Activar mediante CMake únicamente los lenguajes y bibliotecas necesarios para el proyecto.
- Gestionar errores mediante SCRIPT_XEVENT y trazas, especialmente cuando los scripts puedan modificarse sin recompilar.

21. Errores habituales al usar Scripts

- Tratar los scripts como sustitutos del diseño de la aplicación. Deben complementar el código C/C++, no convertir el proyecto en una mezcla desordenada.
- Activar todos los lenguajes y todas las bibliotecas por comodidad, aunque el proyecto solo necesite uno o dos elementos.
- No controlar qué funciones se exponen. Un script con acceso a procesos, ventanas o sistema puede realizar acciones de alto impacto.
- Guardar scripts fuera de assets/scripts y perder la homogeneidad de XPATHMANAGER.
- Olvidar que la caché puede conservar una versión anterior del script durante una sesión.
- No validar parámetros en las funciones C/C++ llamadas desde script. Todo callback debe comprobar número, tipo y validez de sus argumentos.
- No recoger ni mostrar SCRIPT_XEVENT_TYPE_ERROR, dejando al usuario sin información cuando falla un script.

22. Catálogo resumido del bloque Scripts

Grupo	Clases / módulos	Responsabilidad
Núcleo	SCRIPT, SCRIPT_ERRORCODE, SCRIPT_XEVENT	Carga, ejecución, eventos y errores comunes del sistema de scripts.
Caché	SCRIPT_CACHE	Precarga y reutilización del texto de scripts desde memoria.

Lenguaje .g	SCRIPT_LNG_G, SCRIPT_LNG_G_COMMAND, SCRIPT_LNG_G_FUNCTIONTYPE, SCRIPT_LNG_G_VAR	Motor propio para ficheros .g, con tokens, comandos, funciones y variables.
Lua	SCRIPT_LNG_LUA	Ejecución de Lua y registro de callbacks hacia funciones de GEN Framework.
JavaScript	SCRIPT_LNG_JAVASCRIPT	Ejecución mediante Duktape y conversión de argumentos/retornos con XVARIANT.
Bibliotecas	SCRIPT_LIB, SCRIPT_LIB_FUNCTION	Base para exponer funciones C/C++ a cualquier lenguaje de script.
Bibliotecas estándar	Math, Random, Timer, String, Path, Dir, Trace	Funciones comunes disponibles para scripts.
Bibliotecas opcionales	Console, CFG, Log, System, Process, Window, InputSimulate, DevTest	Integración con consola, configuración, logs, sistema, procesos, ventanas y simulación de entrada.

23. Resumen práctico

El bloque Scripts convierte a GEN Framework en una plataforma más flexible. Permite cargar algoritmos externos en JavaScript, Lua o .g, ejecutarlos desde una aplicación C/C++, registrar funciones propias y modificar comportamientos sin recompilar. Esto encaja muy bien con la filosofía de GEN Framework: mantener el rendimiento y el control del código nativo, pero reducir trabajo repetitivo y facilitar la adaptación de aplicaciones reales.

La clase SCRIPT proporciona la base común. SCRIPT_LNG_G, SCRIPT_LNG_LUA y SCRIPT_LNG_JAVASCRIPT implementan los lenguajes concretos. SCRIPT_LIB y SCRIPT_LIB_FUNCTION permiten exponer funciones de C/C++ a los scripts. SCRIPT_CACHE mejora la carga repetida. SCRIPT_XEVENT permite diagnosticar errores. ScriptsExample demuestra la integración completa con AppFlow, XPATHMANAGER, configuración, caché, consola, trazas y bibliotecas opcionales.

La idea más importante es que el script no debe verse como un añadido improvisado, sino como una extensión controlada de la aplicación. Cuando se diseña bien, permite cambiar algoritmos, añadir tareas y crear plugins en tiempo real, manteniendo la aplicación principal compilada, robusta y alineada con la arquitectura de GEN Framework.

Capítulo XIV: UserInterface, widgets y diseño de interfaces multiplataforma

UserInterface es el bloque de GEN Framework que convierte las capacidades gráficas y de entrada del framework en una interfaz de usuario organizada. Su objetivo no es limitarse a dibujar controles, sino mantener un árbol de elementos, resolver posiciones y tamaños, aplicar una apariencia común, recibir entrada de ratón, teclado o pantalla táctil, publicar eventos y redibujar únicamente las zonas que han cambiado.

El resultado es una capa de widgets propia, apoyada en Graphic e Input, que conserva la misma apariencia y el mismo comportamiento en plataformas diferentes. Menús, opciones, formularios, cuadros de texto, botones, teclados virtuales y barras de progreso se describen con los mismos conceptos aunque cambien el sistema operativo, la resolución o el dispositivo de entrada.

Este capítulo revisa todas las clases públicas incluidas en el bloque UserInterface suministrado. Las clases se agrupan por responsabilidad y no por orden alfabético, porque la forma más útil de entender el módulo es seguir el recorrido completo: configuración, carga, composición, interacción, dibujo y actualización.

1. El lugar de UserInterface dentro de GEN Framework

UserInterface se sitúa sobre Graphic e Input. Graphic proporciona la pantalla, el viewport, el canvas 2D, los bitmaps, las fuentes y las áreas de reconstrucción. Input abstrae ratón, teclado, pantalla táctil y movimiento del cursor. UserInterface une ambos mundos: transforma la entrada en operaciones sobre elementos y transforma el estado de esos elementos en dibujo.

La aplicación sigue conservando el control del ciclo principal. En UI_Options, AppFlowGraphics crea la pantalla y el canvas; después se carga el layout XML, se envían al gestor los movimientos o selecciones del usuario y, al final de cada frame, se reconstruyen las áreas y se llama a Update. Esta separación evita que la lógica de negocio dependa de una API nativa de ventanas.

- Graphic aporta GRPSCREEN, GRPVIEWPORT y GRP2DCANVAS, además de imágenes, fuentes y operaciones de dibujo.
- Input aporta INPCURSOR, INPCURSORMOTION e INPBUTTON, con independencia del dispositivo físico.
- UserInterface aporta layouts, elementos, propiedades, skins, eventos, recursos y widgets.
- AppFlowGraphics coordina la inicialización, el ciclo de actualización y el cierre de la aplicación.

2. Arquitectura general y flujo de ejecución

El centro del bloque es UI_MANAGER, expuesto habitualmente mediante GEN_USERINTERFACE. El gestor carga uno o varios layouts y conserva los elementos que contienen. Cada UI_LAYOUT tiene un UI_SKIN, un fondo y un árbol de UI_ELEMENT. Los elementos especializados añaden texto, imágenes, selección, edición, scroll o progreso. El skin calcula geometría y dibuja; UI_XEVENT comunica la entrada y las acciones de salida.

El flujo normal de una aplicación puede resumirse así:

- Registrar las rutas de gráficos, fuentes y layouts con XPATHMANAGER.
- Obtener la pantalla y el viewport creados por AppFlowGraphics.
- Cargar el XML o el paquete ZIP con UI_MANAGER::Load.
- Suscribirse a los eventos de entrada y salida.
- Enviar movimiento, selección o gesto mediante UI_MANAGER::SendEvent.
- Consultar o modificar widgets por nombre y tipo.
- Marcar los elementos modificados con Elements_SetToRedraw.

- Reconstruir las áreas necesarias y ejecutar Update en cada frame.

3. Carga de una interfaz y ciclo mínimo

UI_Options contiene un ejemplo completo de inicialización. El layout se encuentra bajo la sección UI_LAYOUTS administrada por XPATHMANAGER. Load recibe la ruta, la pantalla y el índice de viewport. Después se activan las suscripciones, se crea el teclado virtual, se coloca el fondo y se solicita el primer redibujado.

```
GRPSCREEN* screen = GetMainScreen();
if(!screen) return false;

XPATH xpath;
GEN_XPATHSMANAGER.GetPathOfSection(XPATHSMANAGERSECTIONTYPE_UI_LAYOUTS, xpath);
xpath.Slash_Add();
xpath.Add(__L("example/example.xml"));

if(!GEN_USERINTERFACE.Load(xpath, screen, 0)) return false;

GEN_USERINTERFACE.SubscribeInputEvents(true);
GEN_USERINTERFACE.SubscribeOutputEvents(true, this, &GEN_USERINTERFACE.GetInstance());
GEN_USERINTERFACE.CreaterVirtualKeyboard(GEN_USERINTERFACE.Layouts_Get(__L("example")), screen);
GEN_USERINTERFACE.Layout_PutBackground();
GEN_USERINTERFACE.Elements_SetToRedraw();
```

En el cierre se realiza la operación inversa: se cancelan las suscripciones y se elimina la instancia del gestor. Conviene hacerlo antes de destruir la pantalla o los objetos de aplicación que reciben eventos.

```
GEN_USERINTERFACE.SubscribeOutputEvents(false, this, &GEN_USERINTERFACE.GetInstance());
GEN_USERINTERFACE.SubscribeInputEvents(false);
GEN_USERINTERFACE.DelInstance();
```

4. UI_MANAGER y UI_LAYOUT: gestión global y composición

UI_MANAGER. Es el singleton que coordina todo el subsistema. Load y LoadLayout leen la configuración; también admite layouts empaquetados, mantiene la colección de layouts, busca elementos, gestiona el elemento modal, publica eventos, resuelve textos automáticos, administra la caché de animaciones y controla el teclado virtual. Sus métodos Elements_SetToRedraw y Elements_RebuildDrawAreas conectan el cambio de estado con el renderizado incremental.

UI_LAYOUT. Representa una pantalla lógica o una composición de widgets. Tiene identificador, skin, fondo y colección de elementos raíz. Elements_Add incorpora un elemento; Elements_Get permite recuperar la colección o buscar por nombre, tipo o rol de chrome; Update dibuja los elementos que lo necesitan. Un fichero puede contener varios layouts, incluido el layout COMMON para recursos compartidos.

La búsqueda por nombre y tipo evita que la aplicación conserve punteros durante toda la sesión. También permite que un mismo código trabaje con diferentes archivos de configuración, siempre que mantengan los identificadores acordados.

```
UI_LAYOUT* layout = GEN_USERINTERFACE.Layouts_Get(__L("example"));
if(layout)
{
    UI_ELEMENT* menu = layout->Elements_Get(__L("menu_horz"));
    if(menu)
    {
        menu->SetVisible(!menu->IsVisible());
        GEN_USERINTERFACE.Elements_SetToRedraw(menu, true);
    }
}
```

```
    }
}
```

5. UI_ELEMENT: base común de todos los widgets

UI_ELEMENT. Es la raíz de la jerarquía visual y hereda de XSUBJECT. Conserva nombre, tipo, rol de chrome, padre, color, fondo, posición, nivel Z, dirección, márgenes, rectángulo, visibilidad, actividad, estados de preselección y selección, parpadeo, transición de aparición y elementos compuestos. GetComposeElements forma el árbol de widgets; SetToRedraw propaga la necesidad de dibujar de nuevo.

Un elemento no es solo un rectángulo. La relación father/compose permite construir formularios, botones con texto e imagen, menús con opciones o indicadores con una etiqueta porcentual. ZLevel establece el orden, mientras que IsDetached permite separar un elemento del flujo normal del padre cuando la composición lo requiere.

UI_ELEMENT_CHROMEROLE añade roles de ventana como icono, minimizar, maximizar y cerrar. UI_Options demuestra que una selección puede resolverse por rol antes de atender el nombre concreto del widget.

```
switch(element->GetChromeRole())
{
    case UI_ELEMENT_CHROMEROLE_MINIMIZE : GetMainScreen()->Minimize(true); break;
    case UI_ELEMENT_CHROMEROLE_MAXIMIZE : GetMainScreen()->Maximize(true); break;
    case UI_ELEMENT_CHROMEROLE_CLOSE    : SetExitType(APPFLOWBASE_EXITTYPE_BY_USER); break;
    default                             : break;
}
```

6. Geometría, límites y adaptación de coordenadas

UI_BOUNDARYLINE. Define el rectángulo resuelto de un elemento. Puede copiarse, ampliarse alrededor de otro rectángulo, comprobar si un punto queda dentro y obtener los bordes izquierdo, derecho, superior e inferior. Es la pieza usada tanto para cálculo como para hit testing.

UI_CSSBOX y UI_CSSBox_Get/UI_CSSBox_Set. UI_CSSAdapter.h ofrece un puente aditivo entre el modelo interno de GEN Framework y el modelo de caja CSS. UI_CSSBOX expresa left, top, width y height con origen superior izquierdo. Los helpers leen o escriben esa caja sin obligar a cambiar el sistema interno, cuyo valor Y almacenado representa el borde inferior.

UI_ELEMENT_CONTROLTRANSITION. Es actualmente un punto de extensión. La clase incluida solo implementa construcción, destrucción y limpieza, sin estado ni operaciones públicas adicionales. No debe presentarse como un sistema de transiciones ya terminado; sirve como esqueleto para evolución futura.

El adaptador CSS resulta especialmente útil si se integra un motor de layout externo o se desea importar geometría web. El resto de UserInterface continúa usando sus coordenadas habituales.

```
UI_CSSBOX box;
box.left   = 24.0;
box.top    = 16.0;
box.width  = 320.0;
box.height = 80.0;

UI_CSSBox_Set(element, box);
UI_CSSBOX resolved = UI_CSSBox_Get(element);
```

7. Configuración XML y estilos declarativos

UI_STYLEPROPERTY. Es un par clave/valor empleado por el sistema de estilos. Mantiene la propiedad como texto para que pueda proceder directamente de un nodo XML.

UI_STYLE. Agrupa propiedades de estilo, permite definir las, comprobar su existencia, recuperarlas como texto o número y rellenarlas desde XFILEXMLELEMENT. Es la capa ligera que desacopla los atributos declarativos de la construcción concreta del widget.

La configuración de UI_Options comienza con skin, colores, textos y animaciones globales, y después declara un layout. Cada element especifica type, name, posición, tamaño y propiedades visuales. Los elementos anidados se convierten en elementos compuestos.

```
<layouts>
  <skin drawmode="canvas" vectorfont="Nunito-SemiBold.ttf"
    backgroundimg="seamless_pattern2.jpg" backgroundseamlesspattern="yes" />

  <color name="colormenu">black,50</color>
  <text name="button_text">#[TEST_TEXT]</text>

  <layout name="example">
    <element name="menu-btn" type="button" xpos="5" ypos="704">
      <element name="animation00" type="animation">
        <image>menu_button.png</image>
      </element>
    </element>
  </layout>
</layouts>
```

Separar definición y ejecución facilita cambiar colores, fuentes, posiciones y recursos sin recompilar. La aplicación conserva la responsabilidad de validar los identificadores que utiliza y de reaccionar a los eventos.

8. Colores, textos, animaciones y fondos compartidos

UI_COLOR. Representa un color RGBA y su validez. Puede construirse desde una cadena, modificar canales, copiarse o analizar expresiones como red, black,50 o 160,195,255. La transparencia forma parte del mismo objeto.

UI_COLORS. Es el registro singleton de colores con nombre. Add asocia un identificador y una expresión de color; Get recupera la expresión. El XML de UI_Options usa colormenuline y colormenu para reutilizar una paleta.

UI_TEXT. Almacena el nombre y el contenido de un recurso textual.

UI_TEXTS. Es el registro singleton de textos con nombre. Permite declarar una cadena una sola vez y referenciarla desde varios elementos. Las máscaras #[...] se resuelven más tarde por traducción o por evento de la aplicación.

UI_ANIMATION. Carga una secuencia de bitmaps a partir de un recurso, incluido un recurso vectorial adaptado a un canvas de referencia. Conserva nombre, patrón de recurso y frames; GetBitmap permite recuperar un frame.

UI_ANIMATIONS. Es la caché singleton de animaciones. Add, Get y DeleteAll evitan cargar repetidamente los mismos recursos. UI_MANAGER::GetOrAddAnimationCache encapsula el patrón habitual.

UI_BACKGROUND. Mantiene el color y los recursos de fondo de un layout: nombre de bitmap, bitmap normal y bitmap de patrón. UI_MANAGER puede pintar color, imagen escalada o patrón repetido.

Estos registros convierten el XML en un sistema reutilizable. El nombre lógico permanece estable y el recurso concreto puede cambiar entre temas, idiomas o densidades de pantalla.

9. Skins y renderizado sobre el canvas

UI_SKIN. Define el contrato de apariencia. Guarda nombre, tipo, modo de dibujo y fuentes raster/vectoriales. Calcula los límites de cada tipo de widget y despacha su dibujo mediante `Draw_Text`, `Draw_Form`, `Draw_Menu`, `Draw_ProgressBar` y los demás métodos especializados.

UI_SKIN_TEXTBOX_PART. Es una unidad de maquetación interna de un `TextBox`. Conserva línea, posición, color, dimensiones y el texto o imagen que se dibuja en esa parte.

UI_SKINCANVAS_REBUILDAREAS. Extiende `GRP2DREBUILDAREAS` para asociar zonas del canvas con layouts o elementos. Puede reconstruir todo, reconstruir un elemento, capturar/restaurar bitmaps y localizar el área correspondiente a un widget.

UI_SKINCANVAS. Implementa `UI_SKIN` sobre `GRP2DCANVAS` y añade el cálculo de posiciones relativas, el tamaño del padre, fuentes, clipping, scrollbars y dibujo de todos los widgets. También resuelve posiciones center, auto y max y aplica márgenes y desplazamientos de scroll.

UI_SKINCANVAS_FLAT. Especializa `UI_SKINCANVAS` con una apariencia plana. En el bloque suministrado sobrescribe principalmente `Draw_Form`, por lo que reutiliza el resto del pipeline del skin de canvas.

La separación entre modelo y skin es la razón por la que la interfaz puede mantener un comportamiento común y cambiar de aspecto. Los widgets no dibujan directamente: exponen estado, y el skin interpreta ese estado en el backend gráfico seleccionado.

10. Propiedades reutilizables: selección, edición y scroll

UI_PROPERTY_SELECTABLE. Añade a un elemento un estado de selección: activo, preseleccionado, seleccionado o desactivado, junto con un temporizador y el tiempo durante el que se mantiene la selección.

`SetSelectableStateFromString` enlaza directamente con atributos XML como `active`, `preselect`, `selected` o `deactive`.

UI_PROPERTY_EDITABLE. Añade posición de cursor y tamaño máximo al contenido editable.

`UI_ELEMENT_EDITTEXT` combina esta propiedad con texto y selección.

UI_PROPERTY_SCROLLEABLE_STATUS. Conserva el estado de un eje: actividad, visibilidad, modo auto/forzado, límite, desplazamiento, paso, overflow y apariencia de la barra. Es el bloque de datos interno para horizontal y vertical.

UI_PROPERTY_SCROLLEABLE. Expone la política completa de desplazamiento. Permite activar ejes, fijar pasos o fracciones, elegir overflow visible/hidden/scroll/auto, calcular límites, mostrar u ocultar la barra y ajustar anchura, colores y separaciones. `Scroll_ResolvePolicy` relaciona viewport y extensión de contenido.

Las propiedades se implementan como bases reutilizables. Así, `Form`, `TextBox`, `Menu` y `MultiOption` pueden compartir scroll sin duplicar la lógica; `Option`, `ListBox` y `EditText` comparten selección; `EditText` añade edición.

```
UI_PROPERTY_SCROLLEABLE* scroll = dynamic_cast<UI_PROPERTY_SCROLLEABLE*>(element_menu);
if(scroll)
{
    UI_COLOR thumb(__L("white,75"));
    UI_COLOR track(__L("white,50"));

    scroll->Scroll_SetOverflow(UI_PROPERTY_SCROLLEABLE_TYPE_VERTICAL, UI_OVERFLOW_AUTO);
    scroll->Scroll_SetBarWidth(UI_PROPERTY_SCROLLEABLE_TYPE_ALL, 16.0);
    scroll->Scroll_SetBarColors(UI_PROPERTY_SCROLLEABLE_TYPE_ALL, thumb, track);
    scroll->Scroll_SetBarSeparation(UI_PROPERTY_SCROLLEABLE_TYPE_ALL, 5.0, 0.0);
}
```

11. Widgets de contenido: texto, imágenes y animación

UI_ELEMENT_TEXT. Muestra una cadena con tamaño de fuente, texto efectivo, máscara y límite de longitud. Puede ser independiente o formar parte de un botón, una opción o un indicador de progreso.

UI_ELEMENT_TEXTBOX. Representa texto multilínea y hereda la capacidad de scroll. Controla interlineado, separación respecto a imágenes, alineación, corte por palabras, máscara y altura real del contenido. El skin divide el contenido en UI_SKIN_TEXTBOX_PART para maquetarlo.

UI_ELEMENT_IMAGE. Muestra un GRPBITMAP con nivel alpha. El recurso puede proceder de formatos raster o de una imagen creada a partir de un fichero vectorial.

UI_ELEMENT_ANIMATION. Reproduce los frames de una UI_ANIMATION. Ofrece Play, Pause y Stop, temporización, índice de imagen y alpha. Es el widget visible; UI_ANIMATION es el recurso compartido que contiene los bitmaps.

UI_ELEMENT_SCROLL. Es el elemento visual básico asociado al desplazamiento. La política y el estado residen en UI_PROPERTY_SCROLLABLE, mientras que el skin se ocupa de dibujar y probar las barras.

UI_Options combina estos componentes: el TextBox mezcla texto y una animación; los botones contienen una animación y un texto; el porcentaje de una barra es otro UI_ELEMENT_TEXT cuya máscara se resuelve automáticamente.

12. Widgets de selección: Option, Button y CheckBox

UI_ELEMENT_OPTION. Es la base de los controles seleccionables. Puede contener texto y animación, decidir dónde se coloca el texto y limitar qué subelementos son visibles según el estado active, preselect, select o deactive.

UI_ELEMENT_BUTTON. Es una especialización semántica de Option. Aprovecha la composición de texto e imagen y publica una selección cuando el usuario lo activa.

UI_ELEMENT_CHECKBOX. Añade un valor booleano a Option mediante GetStatus y SetStatus. Su representación visual continúa en manos del skin.

La opción es más importante de lo que parece: concentra la máquina visual de estados. Un botón puede mostrar colores, texto o animaciones distintas para reposo, preselección, selección y desactivación sin que la aplicación tenga que redibujar manualmente cada variante.

```
UI_ELEMENT_BUTTON* button = (UI_ELEMENT_BUTTON*)
    GEN_USERINTERFACE.Element_Get(__L("menu-btn"), UI_ELEMENT_TYPE_BUTTON);

if(button)
{
    button->SetActive(true);
    button->SetVisible(true);
    GEN_USERINTERFACE.Elements_SetToRedraw(button, true);
}
```

13. MultiOption y representación por estados

UI_ELEMENT_MULTIOPTION. Agrupa varias opciones que representan el mismo control en estados diferentes. Hereda scroll y selección y conserva cada variante en su colección de elementos compuestos.

El XML de UI_Options declara chromeall-btn como multioption y anida cuatro botones con el mismo nombre lógico, uno para cada selectable status. Cada variante cambia la posición o el color del texto y de la animación. El skin elige la representación que corresponde al estado actual.

```
<element name="chromeall-btn" type="multioption" xpos="80" ypos="20">
  <element name="chrome-btn" type="button" selectablestatus="active">...</element>
  <element name="chrome-btn" type="button" selectablestatus="preselect">...</element>
  <element name="chrome-btn" type="button" selectablestatus="selected">...</element>
  <element name="chrome-btn" type="button" selectablestatus="deactive">...</element>
</element>
```

Este patrón permite una apariencia muy rica sin condicionales de dibujo en la aplicación. También es la base que usa el teclado virtual para construir teclas con diferentes estados visuales.

14. Contenedores, formularios, menús y listas

UI_ELEMENT_FORM. Es un contenedor con color de línea, rectángulo visible y capacidad de scroll. Agrupa otros elementos y puede actuar como panel, diálogo, formulario o banda de navegación.

UI_ELEMENT_MENU_OPTION_CFG. Es la estructura de configuración utilizada para crear opciones de menú en tiempo de ejecución. Contiene índice, nombre, leyenda, color, fuente, márgenes y dimensiones.

UI_ELEMENT_MENU. Especializa Form y añade Option_AddText. Puede declararse vacío en XML y poblarse desde C++, como hace UI_Options con ocho opciones.

UI_ELEMENT_LISTBOX. Combina texto y selección con un UI_ELEMENT_MENU asociado. El texto muestra la elección actual y el menú ofrece las alternativas.

Un menú dinámico resulta útil cuando las opciones proceden de configuración, dispositivos detectados o datos externos. Tras añadir elementos se recalculan los límites del árbol y se ajusta el área visible.

```
UI_ELEMENT_MENU* menu = (UI_ELEMENT_MENU*)
    GEN_USERINTERFACE.Element_Get(__L("ListBoxMenuID"), UI_ELEMENT_TYPE_MENU);

if(menu)
{
    UI_ELEMENT_MENU_OPTION_CFG cfg;
    cfg.index      = 1;
    cfg.nameoption = __L("option01");
    cfg.leyend     = __L("Option 01");
    cfg.colorstr   = __L("white,255");
    cfg.sizefont   = 18;
    cfg.marginwidth = 10;
    cfg.marginheight = 15;
    cfg.width      = 250;
    cfg.height     = 35;

    UI_ELEMENT_OPTION* option = menu->Option_AddText(cfg);
    if(option) option->SetVisibleLimitType(UI_ELEMENT_OPTION_VISIBLE_LIMIT_PRESELECT |
                                           UI_ELEMENT_OPTION_VISIBLE_LIMIT_SELECT);

    GEN_USERINTERFACE.Element_GetLayout(menu)->GetSkin()->
        CalculeBoundaryLine_AllElements(menu, true);
    GEN_USERINTERFACE.Elements_SetToRedraw(menu, true);
}
```

15. Edición de texto y teclado virtual

UI_ELEMENT_EDITTEXT. Combina UI_ELEMENT_TEXT, UI_PROPERTY_SELECTABLE y UI_PROPERTY_EDITABLE. Añade cursor visible, temporizador de parpadeo y cambio de estado del cursor. El atributo maxsizetext limita la entrada.

UI_VIRTUALKEYBOARD_KEYINFO. Describe la geometría y apariencia de una tecla: color, tamaño de fuente, margen, posición, anchura y altura. Es utilizada internamente durante la construcción del teclado.

UI_VIRTUALKEYBOARD. Crea un teclado virtual dentro de un layout. Ini lo vincula con layout y pantalla; Show lo presenta para un elemento editable; SelectInput interpreta la tecla; End libera los elementos. Gestiona mayúsculas, minúsculas y símbolos y modifica el texto respetando posición de cursor y tamaño máximo.

UI_MANAGER::CreatorVirtualKeyboard construye el teclado asociado al layout. Cuando se selecciona un EditText, el gestor puede mostrarlo con ese elemento como destino. Al cerrar, DeleteVirtualKeyboard elimina la infraestructura.

```
UI_LAYOUT* layout = GEN_USERINTERFACE.Layouts_Get(__L("example"));
GEN_USERINTERFACE.CreatorVirtualKeyboard(layout, GetMainScreen());

UI_ELEMENT_EDITTEXT* edit = (UI_ELEMENT_EDITTEXT*)
    GEN_USERINTERFACE.Element_Get(__L("ID Edit"), UI_ELEMENT_TYPE_EDITTEXT);

// El gestor muestra el teclado cuando el EditText pasa a ser el destino editable.
if(edit) GEN_USERINTERFACE.Elements_SetToRedraw(edit, true);
```

En el XML de UI_Options, el EditText está dentro de un Form. Esto demuestra el caso normal: un formulario compone el campo, mientras el teclado es una capa adicional que se muestra solo cuando hace falta.

16. Indicadores de progreso lineales, radiales y por imagen

UI_ELEMENT_PROGRESSBAR. Es una barra lineal horizontal o vertical. Mantiene nivel, colores de línea/fondo/relleno, gradiente, extremos redondeados y rectángulo de progreso. ContinuousCycle activa un segmento animado para tareas de duración desconocida.

UI_ELEMENT_PROGRESS_RADIAL. Representa el nivel como arco. Controla ángulo inicial, barrido, grosor, extremos redondeados, línea, color y gradiente. Un barrido de 360 grados produce un anillo completo.

UI_ELEMENT_PROGRESS_IMAGE. Revela una imagen llena sobre otra vacía según el nivel. OffsetStart y OffsetEnd delimitan la parte útil del recurso; es apropiado para termómetros, depósitos, baterías o indicadores con forma.

Los tres heredan de Option, por lo que pueden incorporar texto o animación. La máscara #[PROGRESSBAR_PERCENT] se resuelve a partir del nivel del padre. UI_Options incrementa o decrementa simultáneamente varias barras con F1 y F2.

```
UI_ELEMENT_PROGRESSBAR* progress = (UI_ELEMENT_PROGRESSBAR*)
    GEN_USERINTERFACE.Element_Get(__L("progressbar0"), UI_ELEMENT_TYPE_PROGRESSBAR);

if(progress)
{
    float level = progress->GetLevel();
    progress->SetLevel(level + 1.0f);
    GEN_USERINTERFACE.Elements_SetToRedraw(progress);
}

<element type="progressradial" name="progressbar1"
    xpos="50" ypos="475" width="150" height="150"
    level="15" thickness="18" roundcap="yes"
```

```

        startangle="-90" sweepangle="360"
        bckgrdcolor="32,40,48" color="160,195,255"
        gradientcolor="20,80,240" gradientmode="track">
<element type="text" xpos="center" ypos="center" sizefont="35">
    #[PROGRESSBAR_PERCENT]
</element>
</element>

```

17. Eventos de entrada y salida

UI_XEVENT. Extiende XEVENT para transportar posición, movimiento de cursor, elemento, skin y valores de máscara de texto. Los tipos de entrada son movimiento, trayectoria y selección; los de salida notifican selección, petición de cambio de texto y cambio de contenido.

La aplicación traduce el dispositivo físico a eventos UserInterface. UI_Options invierte el eje Y del cursor cuando es necesario y envía movimiento o selección al gestor. De este modo, la UI no necesita saber si el punto procede de ratón o pantalla táctil.

```

GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_CURSOR_MOVE, cursor_x, cursor_y);

if(button_mouse->IsPressedWithRelease())
{
    GEN_USERINTERFACE.SendEvent(UI_XEVENT_TYPE_INPUT_SELECTION, cursor_x, cursor_y);
}

```

Los eventos de salida vuelven a la aplicación mediante el patrón observer/subject. El evento seleccionado entrega el UI_ELEMENT; TEXTTOCHANGE permite resolver máscaras propias como VERSION o TEST_TEXT; CHANGECONTENTS informa de ediciones.

```

void MYAPP::HandleEvent_UserInterface(UI_XEVENT* event)
{
    switch(event->GetEventType())
    {
        case UI_XEVENT_TYPE_OUTPUT_SELECTED:
            if(event->GetElement()) UserInterface_ElementSelected(event->GetElement());
            break;

        case UI_XEVENT_TYPE_OUTPUT_TEXTTOCHANGE:
            UserInterface_ChangeLiteralText((UI_ELEMENT_TEXT*)event->GetElement(),
                                            event->GetMaskTextValue(),
                                            event->GetMaskTextResolved());

            break;

        case UI_XEVENT_TYPE_OUTPUT_CHANGECONTENTS:
            break;
    }
}

```

18. Textos automáticos, traducción y datos dinámicos

Las máscaras #[...] permiten que un layout contenga datos que no conoce al cargarse. UI_MANAGER intenta resolver recursos y traducciones y, cuando la aplicación debe proporcionar el valor, emite UI_XEVENT_TYPE_OUTPUT_TEXTTOCHANGE. El observador escribe el resultado en GetMaskTextResolved.

```

if(!maskvalue->Compare(__L("VERSION"), true))
{

```

```

maskresolved->Format(__L("%d.%d.%d"),
                    APPLICATION_VERSION,
                    APPLICATION_SUBVERSION,
                    APPLICATION_SUBVERSIONERR);
}

if(!maskvalue->Compare(__L("TEST_TEXT"), true))
{
    maskresolved->Set(__L("Tests"));
}

```

Este mecanismo evita acoplar el XML con las variables internas de la aplicación. Es útil para estado de red, fecha, usuario, versión, contadores o cualquier dato que cambie durante la ejecución.

19. Interfaces adaptables a pantallas y displays diferentes

UserInterface no obliga a que todas las coordenadas sean absolutas. UI_SKINCANVAS interpreta posiciones relativas como center, tamaños auto o max, márgenes y dimensiones del padre. Los elementos anidados se calculan dentro de su contenedor y los rectángulos visibles controlan clipping y scroll.

La configuración responsable debe apoyarse en tres niveles:

- Geometría relativa: usar center, auto, max, márgenes y elementos padre en lugar de repetir coordenadas globales.
- Layouts alternativos: cargar archivos o layouts distintos según resolución, orientación, densidad o familia de dispositivo.
- Recursos y fuentes adecuados: mantener nombres lógicos estables y cambiar imágenes, tipografías o tamaños mediante configuración.
- Overflow y scroll: declarar qué contenido se recorta, se desplaza o permanece visible cuando el viewport resulta menor.
- CSS adapter: integrar, si es necesario, un motor que trabaje con cajas de origen superior izquierdo sin modificar el núcleo.

El ejemplo ya utiliza xpos="center" y ypos="center" en textos internos, formas como contenedores y un visiblirect para una barra horizontal. Para una aplicación real conviene evitar que el layout completo dependa de una única resolución y preparar variantes para los formatos principales.

```

<element name="dialog" type="form"
    xpos="center" ypos="center" width="max" height="auto"
    margin="24,24,24,24" overflow="auto">
    <element name="title" type="text"
        xpos="center" ypos="20" width="auto" height="auto">
        #[DIALOG_TITLE]
    </element>
</element>

```

Los nombres exactos de atributos deben contrastarse con la versión de UI_MANAGER que se compile. El principio de diseño es estable: expresar intención relativa, resolverla en el skin y seleccionar la configuración adecuada antes de dibujar.

20. Actualización, áreas de reconstrucción y rendimiento

Modificar un widget no implica redibujar toda la pantalla. SetLevel, SetText o SetVisible cambian el modelo; después Elements_SetToRedraw marca el elemento y, opcionalmente, sus hijos. Elements_RebuildDrawAreas prepara las zonas afectadas y Update delega el dibujo en el skin.

```
GEN_USERINTERFACE.Elements_SetToRedraw(element, true);
GEN_USERINTERFACE.Elements_RebuildDrawAreas();
GEN_USERINTERFACE.Update();
```

UI_Options ejecuta ese patrón al final del frame. También reconstruye toda el área del canvas cuando la aplicación lo necesita, pero la ruta incremental debe ser la habitual. Un error frecuente es cambiar una propiedad y olvidar marcar el elemento, lo que hace que el estado interno sea correcto pero la pantalla conserve la imagen anterior.

21. Ejemplo integrado basado en UI_Options

El ejemplo entregado combina casi todos los conceptos del bloque. AppFlowGraphics crea la pantalla; XPATHMANAGER registra graphics, fonts y ui_layouts; GEN_USERINTERFACE carga example.xml; el XML declara recursos, fondo, texto, TextBox, Menu, barras de progreso, Form, EditText, MultiOption, Button e Image.

Durante la inicialización se añaden opciones al menú desde C++, se recalcula su geometría y se personalizan las barras de scroll. Durante la entrada, ratón y pantalla táctil generan los mismos eventos. F1 y F2 modifican indicadores; las flechas desplazan el menú; F5 recarga la interfaz. Durante el dibujo se marcan los elementos principales, se reconstruyen áreas y se llama a Update.

La selección de menu-btn muestra u oculta menu_horz. Las selecciones de una lista pueden copiar el texto elegido al ListBox, ocultar el menú y retirar el modal. Las máscaras de texto se resuelven en el observador. Este ejemplo es especialmente valioso porque muestra la frontera correcta: XML para composición y apariencia; C++ para datos dinámicos, navegación y lógica de aplicación.

22. Catálogo completo del bloque UserInterface

El bloque revisado queda organizado de la siguiente forma:

- Núcleo y coordinación: UI_MANAGER, UI_LAYOUT, UI_ELEMENT y UI_XEVENT.
- Geometría y compatibilidad: UI_BOUNDARYLINE, UI_CSSBOX, UI_CSSBox_Get, UI_CSSBox_Set y UI_ELEMENT_CONTROLTRANSITION.
- Configuración y recursos: UI_STYLEPROPERTY, UI_STYLE, UI_COLOR, UI_COLORS, UI_TEXT, UI_TEXTS, UI_ANIMATION, UI_ANIMATIONS y UI_BACKGROUND.
- Apariencia y dibujo: UI_SKIN_TEXTBOX_PART, UI_SKIN, UI_SKINCANVAS_REBUILDAREAS, UI_SKINCANVAS y UI_SKINCANVAS_FLAT.
- Propiedades: UI_PROPERTY_SELECTABLE, UI_PROPERTY_EDITABLE, UI_PROPERTY_SCROLLEABLE_STATUS y UI_PROPERTY_SCROLLEABLE.
- Contenido visual: UI_ELEMENT_SCROLL, UI_ELEMENT_TEXT, UI_ELEMENT_TEXTBOX, UI_ELEMENT_IMAGE y UI_ELEMENT_ANIMATION.
- Selección y acción: UI_ELEMENT_OPTION, UI_ELEMENT_MULTIOPTION, UI_ELEMENT_BUTTON y UI_ELEMENT_CHECKBOX.
- Contenedores y listas: UI_ELEMENT_FORM, UI_ELEMENT_MENU_OPTION_CFG, UI_ELEMENT_MENU y UI_ELEMENT_LISTBOX.
- Edición: UI_ELEMENT_EDITTEXT, UI_VIRTUALKEYBOARD_KEYINFO y UI_VIRTUALKEYBOARD.
- Progreso: UI_ELEMENT_PROGRESSBAR, UI_ELEMENT_PROGRESS_RADIAL y UI_ELEMENT_PROGRESS_IMAGE.

No todas estas clases deben ser usadas directamente por una aplicación. UI_SKIN_TEXTBOX_PART, UI_PROPERTY_SCROLLEABLE_STATUS y UI_VIRTUALKEYBOARD_KEYINFO son principalmente piezas auxiliares; UI_MANAGER, UI_LAYOUT y los widgets son la superficie habitual. UI_ELEMENT_CONTROLTRANSITION debe tratarse como infraestructura incompleta en la versión revisada.

23. Errores habituales al trabajar con UserInterface

- Buscar un elemento con un tipo incorrecto y convertir el puntero sin comprobar el resultado.
- Cambiar el estado de un widget sin llamar a `Elements_SetToRedraw`.
- Enviar coordenadas de cursor con un eje Y incompatible con el canvas.
- Conservar punteros a elementos después de recargar el layout con F5 u otro mecanismo.
- Crear jerarquías sin establecer father, nivel Z o elementos compuestos de forma coherente.
- Confundir `UI_ANIMATION`, que es el recurso de frames, con `UI_ELEMENT_ANIMATION`, que es el widget reproductor.
- Usar posiciones absolutas para toda la pantalla y esperar adaptación automática a cualquier resolución.
- Forzar scroll sin calcular viewport, extensión de contenido y política overflow.
- Suscribirse a eventos y no cancelar la suscripción durante el cierre.
- Suponer que `UI_ELEMENT_CONTROLTRANSITION` ya implementa animaciones de transición completas.

24. Criterios de diseño para una aplicación con GEN Framework

- Definir nombres de elemento estables y tratarlos como contrato entre XML y C++.
- Reservar el XML para estructura, estilo y recursos; mantener la lógica de negocio en la aplicación.
- Construir controles complejos por composición antes de crear nuevas clases de widget.
- Reutilizar colores, textos y animaciones mediante sus registros globales.
- Elegir un skin común para asegurar la misma apariencia en todas las plataformas.
- Preparar layouts alternativos para familias de pantalla y usar alineación relativa dentro de cada uno.
- Traducir todos los dispositivos de entrada a `UI_XEVENT` en un punto único.
- Actualizar solo las áreas modificadas y medir el coste de fondos, transparencias y animaciones.
- Mantener siempre una ruta de teclado o foco, no solo interacción mediante puntero.

25. Resumen práctico

UserInterface convierte Graphic e Input en un sistema completo de interfaces. `UI_MANAGER` carga y coordina; `UI_LAYOUT` organiza; `UI_ELEMENT` forma el árbol; las propiedades añaden selección, edición y scroll; los widgets representan contenido y controles; `UI_SKIN` calcula y dibuja; `UI_XEVENT` comunica la interacción; el teclado virtual y los indicadores cubren necesidades habituales de aplicaciones táctiles, embebidas y de escritorio.

La principal fortaleza del bloque es la independencia entre apariencia, estructura y lógica. Una misma aplicación puede conservar el comportamiento C++, cambiar el layout o el skin mediante configuración y ejecutarse sobre distintos sistemas y displays. La adaptación no surge de escalar ciegamente una pantalla fija, sino de combinar jerarquía, posiciones relativas, variantes de layout, recursos adecuados y políticas de overflow.

`UI_Options` demuestra el patrón recomendado: carga declarativa, acceso seguro por nombre y tipo, eventos unificados, modificación del modelo, marcado de redibujado y actualización incremental. A partir de esa base pueden construirse menús, diálogos, formularios, teclados, barras de progreso y controles propios manteniendo la identidad visual de GEN Framework entre plataformas.

Capítulo XV: ThirdPartyLibraries, crecer sobre cimientos abiertos

Construir software grande exige apoyarse en trabajo ya probado. Un framework que pretendiera implementar por sí mismo tipografías, compresión, codecs, bases de datos, captura de red, motores de script, audio y toolchains completos consumiría sus recursos en repetir problemas resueltos. Usar bibliotecas de terceros no es una carencia: es una forma de concentrar el esfuerzo propio donde GEN Framework aporta valor, en la arquitectura común, las factorías y la abstracción multiplataforma.

GEN Framework aplica, no obstante, un criterio exigente. Siempre que resulta viable integra la dependencia como código fuente, la compila junto con el proyecto y conserva detrás de sus propias clases la frontera con esa biblioteca. Esto facilita cambiar de compilador o plataforma, evita depender de instalaciones globales y permite adaptar opciones de compilación. Cuando la dependencia es un driver, un SDK de fabricante o una biblioteca dinámica, la integración se limita deliberadamente a la capa de plataforma.

La selección no se basa solo en la funcionalidad. También importan la portabilidad, la madurez, el tamaño, la mantenibilidad, la posibilidad de auditar el código y la licencia. «Open source» no significa «sin obligaciones»: GPL, LGPL, BSD, MIT, dominio público y licencias específicas permiten usos distintos y exigen avisos o condiciones diferentes.

1. Criterios de selección e integración

- Funcionalidad probada: la biblioteca resuelve un problema complejo mejor que una implementación improvisada.
- Portabilidad: se favorecen proyectos en C/C++ que puedan compilarse en varias arquitecturas y sistemas.
- Integración controlada: GEN Framework expone su propia API y confina las llamadas externas en el módulo o la plataforma adecuados.
- Código fuente cuando es posible: facilita reproducibilidad, depuración, configuración del compilador y builds cruzadas.
- Dependencia opcional: una aplicación solo debe cargar o enlazar la biblioteca si activa la función correspondiente.
- Licencia compatible: se conserva el texto legal y se revisan redistribución, enlace, modificaciones y atribuciones.
- Versión fijada: el árbol ThirdPartyLibraries actúa como una fotografía reproducible; actualizar exige pruebas y una nueva auditoría.

2. Alcance de esta revisión

La revisión se ha realizado sobre el contenido de ThirdPartyLibraries disponible para este capítulo. Las versiones auditadas proceden de macros, ficheros VERSION, source.properties, cabeceras o licencias de las copias incluidas. El README general se usa como orientación, pero no como autoridad cuando contradice al código de la propia biblioteca.

El término «versión auditada» describe la copia incluida en ThirdPartyLibraries, no la versión más reciente disponible en Internet. Para snapshots sin etiqueta se conserva esa denominación. El inventario es independiente de una versión concreta de GEN Framework: documenta la finalidad de cada componente dentro del framework y las condiciones necesarias para integrarlo.

3. Inventario general

La tabla siguiente resume las dependencias presentes o documentadas. Después se explica por qué existe cada una y cómo encaja en la arquitectura.

Componente	Versión auditada	Licencia	Uso principal en GEN Framework	Integración
AGG	2.5	GPL-2.0-or-later	Rasterizado vectorial y canvas 2D de Graphic.	Fuente
Duktape	2.7.0	MIT	Motor JavaScript embebido del módulo Script.	Fuente
FreeType	2.12.1	FTL o GPL-2.0	Carga y rasterizado de tipografías para Graphic/AGG.	Fuente
GLEW	1.13.0	BSD modificada + avisos	Carga de extensiones OpenGL, especialmente en Windows.	Fuente
InpOut32/64	1.5.0.1	Permisiva tipo MIT	Acceso desde Windows a puertos de E/S mediante DLL y driver.	Binario/driver
IJG JPEG	9e	IJG	Lectura y escritura JPEG en Graphic.	Fuente
libpcap	1.10.1	BSD-3-Clause	Captura de paquetes en Linux y Android.	Fuente
Npcap SDK	1.15	Npcap License	Captura de paquetes en Windows.	SDK/binario
libpng	1.6.40	PNG Reference Library 2	Lectura y escritura PNG; depende de zlib.	Fuente
Lua	5.3.4	MIT	Motor Lua embebido del módulo Script.	Fuente
MySQL Connector/C	6.1.11	GPL-2.0 + FOSS Exception	Conector MySQL del módulo Databases.	Fuente/binario
nameof	0.10.2	MIT	Nombres de tipos/enum en compilación; utilidad opcional disponible.	Header-only
OpenAL Soft	1.23.1	LGPL-2.0-or-later	Backend de reproducción y espacialización de Sound.	Fuente/binario
OpenGL Registry	Snapshot	MIT por encabezado/avisos	Declaraciones de extensiones; GEN Framework usa glext.h.	Cabeceras
PostgreSQL/libpq	10.6	PostgreSQL License	Conector PostgreSQL del módulo Databases.	Fuente/binario
rpi_ws281x	Snapshot (test 1.1.0)	BSD-3-Clause; módulo RP1 GPL-2.0+	Control de LED WS2812/SK6812 en Raspberry Pi.	Fuente/plataforma
SQLite	3.23.1	Dominio público	Base SQL embebida, sin servidor, en Databases.	Amalgamación
StackWalker	v5	Licencia no incluida	Pilas de llamadas y diagnóstico en Windows.	Fuente
stb_vorbis	1.14	MIT o dominio público	Decodificación Ogg Vorbis en Sound.	Fuente única
WinToast	1.3.0	MIT	Notificaciones toast de Windows 8/10/11.	Fuente
zlib	1.2.12	zlib	Deflate, GZip, ZIP y dependencia de libpng.	Fuente
Android NDK	r26b (26.1.10909125)	Apache-2.0 + avisos	Toolchain Android y android_native_app_glue.	SDK
GoogleTest	1.12.1 y 1.10.0	BSD-3-Clause	Pruebas unitarias activadas con GOOGLETEST_ACTIVE.	Fuente
ESP-IDF	5.2.0	Apache-2.0 + licencias de componentes	SDK y HAL para las plataformas ESP32.	SDK

4. Gráficos, imágenes y tipografías

Graphic combina varias bibliotecas especializadas. La ventaja de esta composición es que GEN Framework conserva su propia API y puede sustituir la implementación interna sin propagar una dependencia externa por toda la aplicación.

Anti-Grain Geometry (AGG)

Versión auditada: 2.5

Uso y motivo: Graphic usa sus primitivas, scanlines, conversores de paths, colores y motores de fuente para el canvas 2D. Se eligió por su rasterizador C++ de alta calidad, independiente de una ventana concreta y apto para generar píxeles en memoria.

Integración y licencia: Se integra como código fuente. La copia incluye GPL v2 y el README declara GPL v2 o posterior; al enlazar o distribuir una obra derivada debe revisarse el efecto copyleft.

FreeType

Versión auditada: 2.12.1

Uso y motivo: Renderiza glifos de fuentes TrueType/OpenType y alimenta el adaptador agg_font_freetype. Evita que la gestión de tipografías dependa de una API exclusiva de Windows, Linux o Android.

Integración y licencia: Código fuente. Licencia dual: FreeType Project License (permisiva con atribución) o GPL v2, a elección del distribuidor.

GLEW

Versión auditada: 1.13.0

Uso y motivo: Inicializa y expone extensiones OpenGL en tiempo de ejecución. En GEN Framework aparece en el contexto gráfico de Windows mediante glewInit, lo que evita mantener manualmente punteros de función por extensión.

Integración y licencia: Código fuente/cabeceras. Su LICENSE reúne una licencia BSD modificada y avisos adicionales de Mesa y Khronos; deben conservarse todos los avisos aplicables.

OpenGL Registry

Versión auditada: Snapshot sin versión semántica

Uso y motivo: Aporta las declaraciones oficiales de extensiones OpenGL. El inventario de GEN Framework indica que se utiliza glext.h; es una dependencia de cabeceras, no un motor de renderizado.

Integración y licencia: La carpeta vendorizada solo contiene GL y no incorpora una licencia raíz. Los encabezados modernos de Khronos incluyen identificadores MIT u otros avisos propios; la licencia debe verificarse archivo a archivo al actualizar.

libpng

Versión auditada: 1.6.40

Uso y motivo: Implementa PNG en Graphic, con soporte del formato de referencia y compresión mediante zlib. Se utiliza porque PNG exige más que leer una cabecera: filtros, chunks, color, transparencia, CRC y compatibilidad histórica.

Integración y licencia: Código fuente bajo PNG Reference Library License version 2, permisiva. Hay que conservar los avisos y marcar versiones alteradas.

IJG JPEG

Versión auditada: 9e (16-01-2022)

Uso y motivo: Proporciona la codificación y decodificación JPEG empleada por GRPBitmapFileJPG. Es una implementación madura y portable que evita reescribir DCT, cuantización, Huffman y manejo de variantes JPEG.

Integración y licencia: Código fuente bajo la licencia IJG, permisiva pero con condiciones y avisos propios; no debe denominarse simplemente MIT.

5. Sonido y multimedia

Sound separa la API de alto nivel del backend y de los decodificadores de fichero. Así una aplicación puede trabajar con fuentes, buffers y reproducción sin acoplarse a la API nativa de cada sistema.

OpenAL Soft

Versión auditada: 1.23.1

Uso y motivo: Actúa como backend OpenAL multiplataforma para reproducción, streaming, captura y posicionamiento espacial. GEN Framework incluye AL/al.h y AL/alc.h en sus factorías de Windows, Linux y Android.

Integración y licencia: La copia contiene código/proyectos y puede requerir bibliotecas dinámicas según la plataforma. Está bajo LGPL 2 o posterior; la forma de enlace y la posibilidad de sustituir la biblioteca son relevantes para la distribución.

stb_vorbis

Versión auditada: 1.14

Uso y motivo: Decodifica audio Ogg Vorbis desde memoria en SNDFileOGG. Su implementación en un solo fichero reduce la complejidad de integración y encaja bien con compilaciones embebidas o pequeñas.

Integración y licencia: Se compila directamente desde stb_vorbis.c. El propio fichero permite elegir MIT o dominio público. El valor 2.31 del README de ThirdPartyLibraries no corresponde a esta copia.

FFmpeg

Versión auditada: 4.0 en el README; ausente del paquete

Uso y motivo: El inventario histórico lo presenta como framework multimedia para vídeo, codecs, mux/demux y streaming. Sin embargo, no existe una carpeta ffmpeg en el contenido de ThirdPartyLibraries revisado.

Integración y licencia: No debe tratarse como dependencia vendorizada en este paquete. Si un proyecto la activa externamente, la licencia depende de la configuración: normalmente LGPL, pero puede convertirse en GPL al habilitar componentes GPL.

6. Lenguajes de script y utilidades de C++

Los motores embebidos permiten modificar reglas y algoritmos sin recompilar toda la aplicación. GEN Framework mantiene el ciclo de vida y la conversión de datos dentro de Script y deja a la biblioteca la ejecución del lenguaje.

Duktape

Versión auditada: 2.7.0

Uso y motivo: Ejecuta JavaScript embebido con una huella contenida y una API del lenguaje C. Script_Language_Javascript incluye duktape.h, de modo que Duktape proporciona el intérprete mientras GEN Framework aporta integración, objetos y flujo de aplicación.

Integración y licencia: Código fuente bajo MIT, apropiado para integración estática y multiplataforma con conservación del aviso de copyright.

Lua

Versión auditada: 5.3.4

Uso y motivo: Ejecuta Lua mediante lua.hpp. Se eligió por ser un lenguaje pequeño, maduro y concebido para incrustarse en aplicaciones C/C++.

Integración y licencia: Código fuente bajo MIT. Es una de las dependencias más sencillas de compilar para plataformas diferentes.

nameof

Versión auditada: 0.10.2

Uso y motivo: Biblioteca header-only para obtener nombres legibles de tipos, enumeraciones y miembros en tiempo de compilación. Puede mejorar trazas, serialización o herramientas de depuración sin RTTI manual.

Integración y licencia: MIT. La copia está presente en ThirdPartyLibraries y se considera una utilidad opcional: solo se incorpora a la compilación de los módulos o proyectos que incluyan su cabecera.

7. Compresión y formatos contenedores

La compresión es un buen ejemplo de por qué no conviene reinventar una pieza ampliamente estandarizada. Una implementación compatible debe respetar formatos, checksums, streaming, límites y casos corruptos acumulados durante décadas.

zlib

Versión auditada: 1.2.12

Uso y motivo: Implementa Deflate y los contenedores zlib/gzip. GEN Framework la usa en CompressGZ, CompressZIP y XFileZIP; además es dependencia de libpng.

Integración y licencia: Se integra como fuente bajo la licencia zlib, permisiva. Las versiones modificadas deben marcarse y no se puede falsear la autoría.

8. Bases de datos SQL

Databases presenta interfaces comunes, pero cada motor mantiene su protocolo, formato y ciclo de conexión. Las bibliotecas cliente oficiales reducen incompatibilidades y conservan el conocimiento específico de cada servidor.

SQLite

Versión auditada: 3.23.1

Uso y motivo: Base SQL embebida y autocontenida. La amalgamación `sqlite3.c` permite incorporar persistencia local sin desplegar un servidor ni un daemon separado.

Integración y licencia: Dominio público. Es portable y sencilla de compilar, aunque la versión incluida es antigua y debe evaluarse antes de exponerla a entradas no confiables.

MySQL Connector/C

Versión auditada: 6.1.11

Uso y motivo: `libmysqlclient` implementa conexión, protocolo, resultados y tipos MySQL para las clases `MySQL_*` de GEN Framework.

Integración y licencia: La distribución incluye código y binarios. GPL v2 con MySQL FOSS License Exception; la excepción debe comprobarse contra la licencia concreta del proyecto que enlaza con el conector.

PostgreSQL/libpq

Versión auditada: 10.6

Uso y motivo: `libpq` proporciona la interfaz cliente nativa utilizada por las clases `PostgreSQL_*` de GEN Framework.

Integración y licencia: PostgreSQL License, permisiva. El README general indica 11.1, pero `pg_config.h` de la copia auditada declara PostgreSQL 10.6.

9. Captura de red y funciones específicas de Windows

Estas dependencias encapsulan capacidades que dependen del sistema operativo o incluso de un driver. GEN Framework las mantiene detrás de clases de plataforma para que el resto del código no conozca sus detalles.

libpcap

Versión auditada: 1.10.1

Uso y motivo: Interfaz de captura e inyección de paquetes usada por `DIOPCap` en Linux y Android. También lee y escribe capturas `pcap`.

Integración y licencia: Código fuente bajo BSD de tres cláusulas. La versión se confirma en el fichero `VERSION`; el 1.8.1 del detalle del README está obsoleto.

Npcap SDK

Versión auditada: 1.15

Uso y motivo: Equivalente de captura para Windows, con cabeceras, import libraries y dependencia del driver/servicio `Npcap` instalado.

Integración y licencia: `Npcap License`: el código es visible, pero la edición gratuita no es software open source y no se puede redistribuir con un producto sin licencia OEM. Es una excepción real a la política declarada de GEN Framework y debe resolverse antes de distribuir.

InpOut32/64

Versión auditada: 1.5.0.1

Uso y motivo: Permite a DIOWINDOWSGPIOPCParallel abrir inpout32.dll o inpout64.dll y acceder a puertos de hardware desde espacio de usuario.

Integración y licencia: DLL y driver solo para Windows. El license.txt concede permisos equivalentes a MIT, pero el despliegue de drivers exige además firma, arquitectura y compatibilidad con la versión de Windows.

StackWalker

Versión auditada: v5

Uso y motivo: Obtiene pilas de llamadas con DbgHelp en Windows y facilita diagnósticos cuando no hay un depurador conectado.

Integración y licencia: La carpeta incluida no contiene LICENSE ni texto de licencia. El origen remite a CodeProject; por tanto, la redistribución debe quedar bloqueada hasta adjuntar y verificar la licencia exacta de la copia vendorizada.

WinToast

Versión auditada: 1.3.0

Uso y motivo: Integra notificaciones toast modernas de Windows 8, 10 y 11 en DIOWINDOWSNotificationsManager.

Integración y licencia: Código fuente bajo MIT. La versión se toma de wintoastlib.h y LICENSE.txt; los valores 1.1 y 1.4.0 del README general no describen la copia actual.

10. Hardware embebido y SDK de plataforma

NDK y ESP-IDF son toolchains/SDK completos, no pequeñas bibliotecas. Su presencia permite compilar capas de plataforma y acceder a APIs nativas, pero también multiplica el tamaño, las licencias transitivas y el coste de mantenimiento.

Android NDK

Versión auditada: r26b (26.1.10909125)

Uso y motivo: Aporta compilador, sysroot, bibliotecas y android_native_app_glue, que GEN Framework incluye en su actividad nativa Android.

Integración y licencia: SDK con licencia principal Apache 2.0 y numerosos avisos/licencias de componentes. La copia real es r26b, no r17 como indica el README.

ESP-IDF

Versión auditada: 5.2.0

Uso y motivo: Entorno oficial de Espressif para compilar GEN Framework sobre ESP32: HAL, FreeRTOS, drivers, red y herramientas.

Integración y licencia: Apache 2.0 en el nivel superior, con componentes bajo licencias adicionales. Al redistribuir una imagen o SDK deben conservarse los avisos de los componentes utilizados.

rpi_ws281x

Versión auditada: Snapshot; herramienta de prueba 1.1.0

Uso y motivo: Controla tiras WS2812/SK6812 desde Raspberry Pi y es usado por DIOLINUXLedNeoPixelWS2812BRPi.

Integración y licencia: El núcleo contiene avisos BSD de tres cláusulas. La variante para Raspberry Pi 5 añade un módulo RP1 marcado GPL-2.0-or-later; esa parte debe tratarse por separado.

11. Pruebas unitarias

Una biblioteca de pruebas no forma parte del producto final en muchas configuraciones, pero sí de la calidad del framework. Mantenerla en ThirdPartyLibraries permite reproducir la misma infraestructura de pruebas en distintos equipos.

GoogleTest

Versión auditada: 1.12.1 y copia heredada 1.10.0

Uso y motivo: GEN Framework inicializa GoogleTest y ejecuta RUN_ALL_TESTS cuando GOOGLETEST_ACTIVE está definido, tanto en Windows como en Linux.

Integración y licencia: Código fuente bajo BSD de tres cláusulas. Conservar dos copias facilita proyectos antiguos, pero aumenta el riesgo de que distintos tests compilen contra versiones diferentes.

12. Discrepancias detectadas en el inventario

La revisión demuestra por qué la versión no debe copiarse únicamente de una tabla mantenida a mano. En el contenido revisado aparecen estas diferencias relevantes:

- FreeType: el resumen indica 2.12.1 y el detalle 2.10.1; la cabecera confirma 2.12.1.
- libpcap: el resumen indica 1.10.1 y el detalle 1.8.1; VERSION confirma 1.10.1.
- PostgreSQL/libpq: el README indica 11.1; pg_config.h confirma 10.6.
- WinToast: el README mezcla 1.1 y 1.4.0; cabecera y licencia confirman 1.3.0.
- zlib: el resumen indica 1.2.12 y el detalle 1.2.11; zlib.h confirma 1.2.12.
- Android NDK: el README indica r17; source.properties confirma r26b.
- ESP-IDF: el README indica 5.1; version.cmake confirma 5.2.0.
- stb: el README indica 2.31; el fichero realmente usado es stb_vorbis 1.14.
- Npcap: el README mezcla 1.13, 1.15 y 1.60. La carpeta corresponde al SDK 1.15, pero la versión del runtime se determina en la instalación.
- FFmpeg 4.0 está documentado, pero no existe en el paquete revisado. nameof 0.10.2 sí existe y no figura en la lista general.

13. Lectura práctica de las licencias

Las licencias permisivas —MIT, BSD, zlib, PostgreSQL, IJG, PNG Reference Library y FreeType License— suelen permitir uso comercial, modificación e integración estática, pero obligan a conservar avisos y descargos. Dominio

público, como SQLite, elimina gran parte de esas condiciones, aunque conviene mantener la procedencia por trazabilidad.

Las licencias copyleft necesitan más atención. AGG se distribuye en esta copia bajo GPL v2 o posterior; MySQL Connector/C usa GPL v2 con una excepción FOSS; OpenAL Soft usa LGPL. El resultado depende de si se distribuye, de cómo se enlaza, de si el usuario puede sustituir la biblioteca y de la licencia del resto del producto. La integración en fuente no elimina estas obligaciones.

Npcap constituye el hallazgo más importante: su código puede consultarse, pero la edición gratuita no es open source y su redistribución está restringida. Un producto que instale o distribuya Npcap necesita la modalidad OEM correspondiente o una estrategia de instalación separada compatible con sus términos. StackWalker requiere otra corrección documental: la copia incluida debe incorporar su licencia exacta antes de distribuirse.

Este capítulo describe el inventario técnico; no sustituye una revisión jurídica del producto final, de su método de enlace ni de los componentes que realmente se distribuyen.

14. Política recomendada para mantener ThirdPartyLibraries

- Conservar por biblioteca un fichero VERSION o un manifiesto con versión o revisión, URL de origen, fecha y hash del paquete.
- Mantener LICENSE/NOTICE junto al código y generar un THIRD_PARTY_NOTICES para cada producto distribuido.
- Registrar si la dependencia se compila de forma estática o dinámica, se integra como header-only, como driver o como SDK de build.
- Distinguir presente, usada, opcional, histórica y solo necesaria para tests; no son estados equivalentes.
- Automatizar la comparación entre el manifiesto y las macros de versión de las cabeceras.
- Ejecutar pruebas en todas las plataformas soportadas antes de actualizar una dependencia.
- Revisar CVE y soporte upstream, especialmente para SQLite 3.23.1, PostgreSQL 10.6, GLEW 1.13.0 y otros componentes antiguos.
- Bloquear la publicación si falta una licencia o si una licencia comercial/no abierta contradice la política del proyecto.

15. Resumen práctico

ThirdPartyLibraries no es una colección accidental de ficheros: es la capa de conocimiento especializado sobre la que GEN Framework construye gráficos, audio, scripts, compresión, bases de datos, captura de red, pruebas y plataformas embebidas. Integrar muchas de estas bibliotecas en fuente ayuda a mantener builds reproducibles y multiplataforma, mientras las factorías y módulos de GEN Framework impiden que la dependencia invada la aplicación.

La calidad de esta estrategia depende de mantener el inventario. Versión, uso, forma de integración y licencia deben viajar juntos. Si uno de esos datos queda obsoleto, la compilación puede seguir funcionando y, aun así, el proyecto perder reproducibilidad, seguridad o derecho de redistribución.

La conclusión no es evitar bibliotecas de terceros, sino seleccionarlas con criterio, aislarlas arquitectónicamente, fijar sus versiones, conservar sus licencias y poder reemplazarlas. Esa disciplina permite construir algo mayor sin renunciar al control técnico del framework.

16. Fuentes de la revisión

- Repositorio oficial ThirdPartyLibraries: <https://gitlab.genframework.com/gen-framework/thirdpartylibraries>

- Sitio oficial de GEN Framework: <https://www.genframework.com/>
- Licencia y guía de Npcap: <https://npcap.com/guide/>
- Registro OpenGL de Khronos: <https://registry.khronos.org/OpenGL/>

Anexo I: Guía de estilo de código C++ en GEN Framework

Propósito. Este apéndice resume y adapta la guía de estilo utilizada para escribir nuevos módulos en GEN Framework. Su objetivo es que el código nuevo parezca escrito dentro del propio framework, manteniendo las mismas reglas visibles de estructura, nombres, ciclos de vida, comentarios, factorías, memoria y aislamiento de plataforma.

Principio no negociable. Cuando una convención habitual de C++ moderno entre en conflicto con la forma de escribir código en GEN Framework, se debe seguir el estilo de GEN Framework. El objetivo no es aplicar una guía externa, sino respetar la coherencia del repositorio existente.

Resumen global del estilo de GEN Framework

Un módulo nuevo debe mantener una apariencia coherente con el resto del framework. La forma de escribir no es un detalle superficial: en GEN Framework, la estructura visual del código ayuda a reconocer rápidamente el subsistema, el ciclo de vida, el propietario de los recursos y la frontera entre código común y código específico de plataforma.

- Un fichero de cabecera .h y un fichero de implementación .cpp por clase principal.
- Banner de GEN Framework al inicio de todos los ficheros fuente.
- Uso de #pragma once en cabeceras.
- Separadores de sección visibles: INCLUDES, DEFINES & ENUMS, CLASS, CLASS MEMBERS, etc.
- Nombres de clases en mayúsculas, normalmente con prefijo de módulo: X, DIO, APPFLOW, GRP, UI, CIPHER, SCRIPT.
- Uso preferente de tipos de GEN Framework: XBYTE, XWORD, XDWORD, XQWORD, XCHAR, XSTRING, XPATH, XBUFFER.
- Métodos de ciclo de vida Ini(), End() y Clean().
- Indentación de dos espacios y llaves en línea propia.
- Declaraciones alineadas en columnas.
- Bloques Doxygen de GEN Framework en ficheros y funciones.
- Uso de GEN_NEW cuando el código debe participar en el control de memoria del framework.
- Código específico de plataforma confinado en los directorios Platform/Platforms y en las factorías correspondientes.

Nombres de ficheros y módulos

GEN Framework no obliga a que el nombre del fichero esté completamente en mayúsculas. Lo habitual es utilizar un nombre tipo Pascal con prefijo del subsistema. Lo importante es respetar el estilo de la carpeta que se está ampliando.

Tipo de módulo	Ejemplo de fichero	Clase esperada
XUtils	XString.h / XString.cpp	XSTRING
DatalO	DIOProtocol.h / DIOProtocol.cpp	DIOPROTOCOL
AppFlow	APPFlowAlerts.h / APPFlowAlerts.cpp	APPFLOWALERTS
Graphic	GRP2DCanvas.h / GRP2DCanvas.cpp	GRP2DCANVAS

UserInterface	UI_Manager.h / UI_Manager.cpp	UI_MANAGER
Eventos	APPFlow_XEvent.h	APPFLOW_XEVENT
Plataforma Windows	XWINDOWS..., DIOWINDOWS..., GRPWINDOVS...	Prefijo WINDOWS
Plataforma Linux	XLINUX..., DIOLINUX..., GRPLINUX...	Prefijo LINUX

Los eventos utilizan la forma _XEvent en el fichero y _XEVENT en la clase. No se deben introducir variantes como Event, Events, Xevent o xevent si el módulo pertenece al estilo habitual del framework.

```
APPFlow_XEvent.h      -> APPFLOW_XEVENT
DIOCamera_XEvent.h   -> DIOCAMERA_XEVENT
APPFlowUpdate_XEvent.h -> APPFLOWUPDATE_XEVENT
```

Estructura de una cabecera

La cabecera define la interfaz pública del módulo. Debe ser estable, legible y coherente con el resto del subsistema. Se recomienda usar declaraciones adelantadas siempre que sea posible para reducir dependencias innecesarias.

```
/**-----
 *
 * @file      MYModule.h
 *
 * @class     MYMODULE
 * @brief     My Module class
 * @ingroup   MYGROUP
 *
 * @copyright EndoraSoft. All rights reserved.
 * -----*/

#pragma once

/*---- INCLUDES -----
-----*/

#include "XBase.h"
#include "XString.h"

/*---- DEFINES & ENUMS -----
-----*/

#define MYMODULE_DEFAULT_VALUE      10

enum MYMODULE_STATUS
{
    MYMODULE_STATUS_UNKNOWN          = 0 ,
    MYMODULE_STATUS_ACTIVE           ,
    MYMODULE_STATUS_ERROR            ,
};

/*---- CLASS -----
-----*/

class XBUFFER;
class XTIMER;

class MYMODULE
```

```

{
    public:

        virtual                MYMODULE                ();
        virtual                ~MYMODULE                ();

        bool                   Ini                        ();
        bool                   End                        ();

        XSTRING*               GetName                    ();
        void                   SetName                    (XCHAR* name);

    private:

        void                   Clean                      ();

        XSTRING                name;
        bool                   isactive;
};

```

```

/*---- INLINE FUNCTIONS + PROTOTYPES -----*/
-----*/

```

Puntos clave de la cabecera:

- El banner inicial forma parte del estilo del framework y no debe omitirse.
- Después del banner se coloca #pragma once.
- Los separadores de sección son largos y muy visibles.
- Las clases se declaran después de DEFINES & ENUMS y de las declaraciones adelantadas.
- Incluso si no hay funciones inline, se conserva la sección INLINE FUNCTIONS + PROTOTYPES.

Estructura de un fichero .cpp

El fichero .cpp contiene la implementación y sigue una estructura muy marcada. En módulos que participan en el framework se incluye GEN_Defines.h al principio y GEN_Control.h cerca del final de la zona de includes.

```

/*---- PRECOMPILE INCLUDES -----*/
-----*/

#include "GEN_Defines.h"

/*---- INCLUDES -----*/
-----*/

#include "MYModule.h"

#include "XBuffer.h"
#include "XTrace.h"

/*---- PRECOMPILE INCLUDES -----*/
-----*/

#include "GEN_Control.h"

/*---- GENERAL VARIABLE -----*/
-----*/

```

```
MYMODULE* MYMODULE::instance = NULL;
```

```
/*---- CLASS MEMBERS -----*/
-----*/
```

- El .cpp incluye primero GEN_Defines.h cuando corresponde.
- Después se incluye la cabecera del propio módulo.
- Las dependencias de implementación se añaden a continuación.
- GEN_Control.h se coloca tras los includes normales.
- Las variables estáticas de singleton se colocan en GENERAL VARIABLE.
- Las funciones se implementan en el mismo orden en que se declaran en la cabecera.

Separadores de sección

Los separadores son una de las marcas visuales más reconocibles del estilo de GEN Framework. No deben sustituirse por comentarios genéricos o por estilos de otros proyectos.

```
/*---- PRECOMPILE INCLUDES -----*/
-----*/
/*---- INCLUDES -----*/
-----*/
/*---- DEFINES & ENUMS -----*/
-----*/
/*---- CLASS -----*/
-----*/
/*---- GENERAL VARIABLE -----*/
-----*/
/*---- CLASS MEMBERS -----*/
-----*/
/*---- INLINE FUNCTIONS + PROTOTYPES -----*/
-----*/
```

Indentación, llaves y espaciado

GEN Framework utiliza una indentación de dos espacios, no tabuladores. Las llaves de funciones y bloques de control multilinea se colocan en línea propia.

```
bool MYMODULE::Ini()
{
    Clean();

    if(isactive)
    {
        value = 1;
    }

    return true;
}
```

Las salidas tempranas compactas son habituales para validar parámetros o estados simples:

```
if(!cfg) return false;
if(!stream) return false;
```

La alineación de declaraciones forma parte del estilo visual. No se debe eliminar con formateadores automáticos externos.

```
class MYMODULE
{
public:

    bool                Ini                ();
    bool                End                ();

    XSTRING*            GetName            ();
```

```

        void                SetName                (XCHAR* name);

private:

        void                Clean                ();

        XSTRING            name;
        bool                isactive;
};

```

Convenciones de nombres

Elemento	Regla de GEN Framework	Ejemplo
Clases	Mayúsculas y prefijo de módulo	XSTRING, DIOSTREAM, APPFLOWCFG, GRP2DCANVAS
Métodos	PascalCase y ciclo de GEN Framework	Ini(), End(), Clean(), GetName(), SetName()
Variables miembro	Minúsculas compactas	isactive, description, applicationdata
Defines	Mayúsculas con prefijo	DIOPROTOCOL_TIMEOUT, APPFLOW_ALERTS_WEBALERTCMD
Enums	Tipo y valores en mayúsculas con prefijo	MYMODULE_STATUS_UNKNOWN
Eventos	Sufijo _XEVENT	APPFLOW_XEVENT

Los nombres deben llevar el prefijo del subsistema. Esto permite identificar rápidamente a qué parte del framework pertenece una clase o una función.

Área	Prefijos habituales
XUtils	X, XFILE, XSTRING, XBUFFER, XTIMER
DataIO	DIO, DIOSTREAM, DIOPROTOCOL
AppFlow	APPFLOW, APPFLOWCFG, APPFLOWALERTS
Graphic	GRP, GRP2DCANVAS, GRPBITMAP
UserInterface	UI, UI_MANAGER, UI_ELEMENT
Cipher	CIPHER, HASH, CIPHERAES
Script	SCRIPT, SCRIPT_LIB
Input	INP, INPDEVICE
Sound	SND, SOUND
Platform Windows	XWINDOWS, DIOWINDOWS, GRPWINDOWS
Platform Linux	XLINUX, DIOLINUX, GRPLINUX
Microcontroladores	XSTM32, XESP32, DIOSTM32, DIOESP32

Ciclo de vida de una clase

Las clases de GEN Framework suelen seguir un ciclo muy reconocible: constructor, destructor, Ini(), End() y Clean(). Este patrón no solo ordena el código; también establece cuándo se inician recursos, cuándo se liberan y cuándo se restablecen los miembros internos.

```

MYMODULE::MYMODULE()
{
    Clean();
}

```

```

MYMODULE::~MYMODULE()
{
    End();

    Clean();
}

bool MYMODULE::Ini()
{
    if(isinitialized) return true;

    isinitialized = true;

    return true;
}

bool MYMODULE::End()
{
    if(!isinitialized) return false;

    isinitialized = false;

    return true;
}

void MYMODULE::Clean()
{
    isinitialized = false;
    type          = MYMODULE_TYPE_UNKNOWN;
    stream        = NULL;
}

```

- Clean() no debe reservar recursos; solo debe dejar los miembros en valores conocidos.
- Ini() devuelve bool cuando la inicialización puede fallar.
- End() libera recursos y deja la clase en estado finalizado.
- El destructor llama a End() cuando existen recursos que liberar.
- Los punteros liberados se ponen siempre a NULL.

Patrón singleton en GEN Framework

Cuando una clase se usa como singleton, GEN Framework mantiene un patrón clásico basado en puntero estático, GetInstance(), DelInstance() y prevención privada de copia.

```

#define MYMODULE_MANAGER MYMODULEMANAGER::GetInstance()

class MYMODULEMANAGER
{
public:

    static bool          GetIsInstanced          ();
    static MYMODULEMANAGER& GetInstance          ();
    static bool          DelInstance             ();

    bool                Ini                      ();
    bool                End                      ();

private:

    MYMODULEMANAGER          ();
    MYMODULEMANAGER          (MYMODULEMANAGER const&); // Don't

implement
    virtual                ~MYMODULEMANAGER    ();

    void                    operator =          (MYMODULEMANAGER const&); // Don't
implement

```

```

        void                                Clean                                ();

        static MYMODULEMANAGER*            instance;
};
MYMODULEMANAGER* MYMODULEMANAGER::instance = NULL;

MYMODULEMANAGER& MYMODULEMANAGER::GetInstance()
{
    if(!instance) instance = GEN_NEW MYMODULEMANAGER();

    return (*instance);
}

bool MYMODULEMANAGER::DelInstance()
{
    if(instance)
    {
        delete instance;
        instance = NULL;

        return true;
    }

    return false;
}

```

Documentación Doxygen

GEN Framework documenta ficheros y funciones con bloques Doxygen de formato propio. En módulos nuevos, todas las funciones implementadas en el .cpp deberían tener su bloque, incluyendo getters y setters si el módulo sigue ese patrón.

```

/**-----
-----
 *
 * @fn          bool MYMODULE::Ini(XCHAR* name, XDWORD type)
 * @brief       Ini
 * @ingroup     MYGROUP
 *
 * @param[in]   name :
 * @param[in]   type :
 *
 * @return      bool : true if successful.
 * -----*/
bool MYMODULE::Ini(XCHAR* name, XDWORD type)
{
    if(!name) return false;

    this->name.Set(name);
    this->type = type;

    return true;
}

```

Campo	Uso
@file	Nombre del fichero actual.
@class	Clase principal documentada.
@brief	Descripción breve con vocabulario de GEN Framework.
@ingroup	Grupo Doxygen del subsistema.
@param[in/out]	Parámetros con dos puntos.
@return	Valor de retorno para funciones no void.

Tipos públicos y API

La API pública de un módulo de GEN Framework debe usar tipos del framework. Los tipos estándar de C++ pueden aparecer en fronteras con librerías externas, pero no deben convertirse en el estilo visible de los módulos de GEN Framework.

Usar	Evitar en API pública
XBYTE	uint8_t, unsigned char
XWORD	uint16_t, unsigned short
XDWORD	uint32_t, unsigned int
XQWORD	uint64_t, unsigned long long
XCHAR	char/wchar_t directos
XSTRING	std::string
XBUFFER	std::vector<unsigned char>
XVECTOR	std::vector en cabeceras públicas

Memoria y propiedad de objetos

GEN Framework utiliza un estilo explícito de propiedad. Cuando se reserva memoria con objetos del framework, se usa GEN_NEW si el módulo participa en el control de memoria. Al liberar, el puntero debe ponerse a NULL.

```
object = GEN_NEW MYOBJECT();

if(object)
{
    delete object;
    object = NULL;
}
```

Cuando un objeto ha sido creado por una factoría, debe eliminarse mediante la factoría correspondiente:

```
if(timer)
{
    GEN_XFACTORY.DeleteTimer(timer);
    timer = NULL;
}
```

- No introducir std::unique_ptr o std::make_unique en módulos que siguen el patrón de GEN Framework clásico.
- No usar excepciones para el flujo normal de errores.
- Preferir bool o enums de resultado para indicar fallo.
- Si la propiedad de un puntero es opcional, usar miembros y métodos de acceso IsOwn... / SetIsOwn....

Reglas de includes

Las cabeceras deben incluir lo mínimo necesario. Si un miembro se almacena por valor, se incluye su cabecera. Si se usa solo como puntero o referencia, se prefiere declaración adelantada.

```
#include "XString.h"

class DIOSTREAM;

class MYMODULE
{
```

```
private:
    XSTRING          name;
    DIOSTREAM*      stream;
};
```

Flujo de control

Las salidas tempranas son frecuentes y ayudan a reducir anidamiento. Los switch suelen alinear los case y usan valores de enum en mayúsculas.

```
if(!buffer) return false;
if(!size)   return false;

switch(type)
{
    case MYMODULE_TYPE_A : result = DoA();
                          break;

    case MYMODULE_TYPE_B : result = DoB();
                          break;

    default :             break;
}
```

Código de plataforma y factorías

El código específico de plataforma debe estar confinado en los directorios de plataforma y oculto detrás de factorías o clases de abstracción. Esta regla es esencial para que una aplicación con GEN Framework pueda compilarse en varios sistemas sin llenar el código de aplicación con #ifdef de plataforma.

- Usar símbolos de plataforma existentes: WINDOWS, LINUX, ANDROID, MICROCONTROLLER.
- Mantener implementaciones específicas bajo Platforms/<PlatformName>/ cuando sea posible.
- Crear objetos dependientes de plataforma mediante las factorías del framework.
- Añadir métodos Create... y Delete... de forma simétrica cuando se amplíe una factoría.
- No exponer detalles nativos de Windows, Linux o Android en APIs comunes salvo que sea inevitable.

```
#if defined(WINDOWS)
...
#endif

#if defined(LINUX) || defined(ANDROID)
...
#endif
```

Trazas y diagnóstico

Los módulos reutilizables no deben introducir std::cout, printf o trazas improvisadas. Se deben usar las utilidades del framework, respetando los canales y macros existentes.

```
GEN_XLOG
XTRACE_PRINTCOLOR(...)
```

Estilos que se deben evitar

Un módulo de GEN Framework no debe parecer código C++ moderno genérico si el subsistema vecino no utiliza ese estilo. La consistencia local es más importante que una modernización aislada.

```
namespace gen { ... }           // No es el estilo visible de GEN
class MyModule { ... };        // Casing incorrecto para clases GEN
std::string name;              // Usar XSTRING en API pública
```

```
std::vector<Item> items;           // Usar contenedores GEN en API pública
nullptr                          // El código GEN clásico usa NULL
auto value = ...;                 // Reduce la claridad visible de tipos
std::unique_ptr<T> ptr;           // Propiedad no coherente con Clean/End
throw std::runtime_error(...);    // GEN usa bool/enums de resultado
enum class Status                 // GEN usa enum clásico en estos módulos
if(condition) {                   // Llave en posición no GEN
    ...                           // Indentación incorrecta
}
```

Plantilla mínima de módulo

La siguiente plantilla resume la forma recomendada para comenzar un nuevo módulo no singleton.

```
class MYMODULE
{
public:
    MYMODULE(){};
    virtual ~MYMODULE(){};

    bool Ini(){};
    bool End(){};
    bool Reset(){};

    bool IsInitialized(){};
    void SetIsInitialized(bool isinitialized);

    XSTRING* GetName(){};
    bool SetName(XCHAR* name);

private:
    void Clean(){};

    bool isinitialized;
    XSTRING name;
};

MYMODULE::MYMODULE()
{
    Clean();
}

MYMODULE::~MYMODULE()
{
    End();

    Clean();
}

bool MYMODULE::Ini()
{
    if(isinitialized) return true;

    isinitialized = true;

    return true;
}

bool MYMODULE::End()
{
    if(!isinitialized) return false;

    isinitialized = false;

    return true;
}
```

```
void MYMODULE::Clean()
{
    isinitialized = false;
    name.Empty();
}
```

Lista de comprobación para nuevos módulos

Área	Comprobación
Ficheros	Existen .h y .cpp con nombres coherentes con el subsistema.
Banner	El banner de GEN Framework aparece al inicio del .h y del .cpp.
Cabecera	Usa #pragma once y separadores de sección de GEN Framework.
Includes	GEN_Defines.h va primero en .cpp y GEN_Control.h al final de includes.
Nombres	Clases, enums, defines y prefijos siguen el subsistema.
Formato	Dos espacios, llaves en línea propia y alineación en columnas.
Ciclo de vida	Constructor llama a Clean(); destructor llama a End() y Clean().
Memoria	GEN_NEW cuando corresponde; punteros liberados puestos a NULL.
API	Tipos de GEN Framework en interfaces públicas.
Doxygen	Funciones implementadas documentadas con bloque de GEN Framework.
Plataforma	Código nativo confinado en platform/factorías cuando sea posible.
Consistencia	El módulo parece escrito junto a los ficheros vecinos.

Regla final de estilo

Cuando se extiende una carpeta existente de GEN Framework, la regla más importante es la consistencia local. Antes de crear un módulo nuevo conviene abrir dos o tres pares .h/.cpp cercanos y copiar su estructura visible: banner, separadores, indentación, orden de métodos, Doxygen, nombres de grupos, includes, patrón de singleton, eventos o factoría si aplica.

Un módulo correcto de GEN Framework debe parecer escrito al mismo tiempo que los ficheros que lo rodean. Las guías externas de C++ son secundarias frente al estilo establecido por el repositorio.

Anexo II: Utilidades — eXtended Trace Monitor

eXtended Trace Monitor es la utilidad de GEN Framework destinada a recibir, organizar y visualizar las trazas emitidas por una o varias aplicaciones. Trabaja como punto de observación central: las aplicaciones actúan como productores y el monitor escucha los destinos configurados, identifica el origen y presenta la información sin obligar a detener ni depurar localmente cada proceso.

La utilidad es de código abierto, está incluida en GEN Framework y puede compilarse junto con el resto del framework. También se distribuye compilada para facilitar su uso inmediato durante desarrollo, pruebas, integración y diagnóstico en campo.

Su utilidad se aprecia especialmente cuando una aplicación se ejecuta en otra máquina, en un servicio sin interfaz, en un sistema embebido o en un dispositivo conectado por red o puerto serie. En lugar de consultar cada equipo por separado, el desarrollador puede concentrar las trazas en una sola ventana.

Qué es una traza y qué problema resuelve

Una traza es un mensaje de diagnóstico emitido durante la ejecución. Describe lo que está haciendo el programa: inicialización de módulos, cambios de estado, comunicaciones, avisos, errores, datos relevantes o cierre de servicios. Su objetivo principal es observar el comportamiento en tiempo real.

GEN Framework permite acompañar cada traza con información que ayuda a clasificarla: aplicación, versión, identificador, origen de red, secuencia, fecha y hora, nivel y color. eXtended Trace Monitor utiliza esos datos para separar máquinas y aplicaciones y para aplicar filtros sin modificar el programa que está enviando.

El monitor no es únicamente una consola de texto. El protocolo de XTRACE también contempla órdenes de limpieza y mensajes de estado, por lo que la misma conexión puede mantener un panel de variables importantes además del flujo cronológico de mensajes.

Trazas y logs no son lo mismo

En GEN Framework, una traza y un log son mecanismos distintos aunque puedan contener mensajes parecidos. La traza está orientada a observación inmediata y puede enviarse a varios destinos. El log está orientado a persistencia: XLOGBASE y XLOG administran un fichero, niveles INFO/WARNING/ERROR, secciones, filtros, límites, reducción, copias de seguridad y vaciado de entradas pendientes.

La conexión entre ambos sistemas es deliberada. XLOGBASE::Ini recibe el parámetro isdumpintrace, cuyo valor predeterminado es true. Cuando se activa, las entradas del log también se emiten como trazas y aparecen en eXtended Trace Monitor con su etiqueta de nivel y sección. El fichero de log continúa existiendo; el monitor recibe una copia para observación en directo.

```
XPATH xpathlog;
// Construir xpathlog dentro de la sección de logs de la aplicación.

GEN_XLOG.Ini(xpathlog, __L("Mini Web Server"), true); // true: enviar también como traza
GEN_XLOG.SetFilters(__L("Ini,Status,End"), XLOGLEVEL_ALL);

GEN_XLOG.AddEntry(XLOGLEVEL_INFO, __L("Ini"), false,
    __L("Inicializando Web Server ... : Ok.));
```

Esta diferencia evita confusiones: guardar trazas desde el monitor crea una captura de lo recibido; configurar XLOG crea el registro persistente propio de la aplicación, con sus políticas de tamaño, backup y filtrado.

Arquitectura del sistema de trazas de GEN Framework

La clase central es XTRACE. Mantiene los destinos, el límite de tamaño, los metadatos de la aplicación, las direcciones de origen y la secuencia de mensajes. XTRACE_TARGET representa cada destino y conserva su tipo, objetivo y número de envíos. En compilaciones normales pueden configurarse varios targets para emitir el mismo mensaje por más de una vía.

Los tipos generales definidos por XTRACE son SPECIAL, FILE y NET. SPECIAL delega en la implementación de plataforma; FILE escribe en un fichero y NET empaqueta la traza para enviarla por red. Las variantes XWINDOWSTRACE, XLINUXTRACE, XANDROIDTRACE, XESP32TRACE y XSTM32TRACE adaptan la salida especial a cada entorno.

En Windows, el destino especial busca una ventana por su título y utiliza WM_COPYDATA para entregar el paquete a eXtended Trace Monitor. Si la ventana no está disponible, la implementación puede recurrir a OutputDebugString en compilaciones Microsoft. La salida de red utiliza los streams de DataIO y el formato de paquete de XTRACE; DIOXTRACEINTERSTREAMS permite procesar trazas que atraviesan streams y, opcionalmente, relacionarlas con el sistema de log.

- **Productor:** aplicación con GEN Framework que configura XTRACE y emite mensajes.
- **Target:** destino concreto, por ejemplo, una ventana local, un fichero o un host y un puerto de red.
- **Transporte:** UDP/red, mecanismo especial de plataforma, pipeline o canal local, UART/serie u otro stream habilitado por la compilación.
- **Receptor:** eXtended Trace Monitor, que escucha, decodifica y agrupa los paquetes.
- **Presentación:** árbol de orígenes, variables de estado, lista de trazas y controles de filtrado/captura.

Configuración básica desde una aplicación con GEN Framework

La aplicación debe identificarse antes de emitir. El nombre, la versión y el ID permiten distinguir procesos diferentes aunque compartan máquina o dirección. Después se configura uno o varios targets.

```
XTRACE_SETAPPLICATIONNAME(__L("Mini Web Server"));
XTRACE_SETAPPLICATIONVERSION(0, 1, 0);
XTRACE_SETAPPLICATIONID(__L("MINIWEB-LOCAL-01"));

// Ventana local de eXtended Trace Monitor en Windows.
XTRACE_SETTARGET(0, XTRACE_TYPE_SPECIAL, __L("eXtended Trace Monitor"));

// Monitor remoto escuchando en UDP/red.
XTRACE_SETTARGET(1, XTRACE_TYPE_NET, __L("192.168.1.50:10001"));

// Copia adicional en fichero.
XTRACE_SETTARGET(2, XTRACE_TYPE_FILE, __L("miniweb.trace"));

XTRACE_SETSIZELIMIT(64 * 1024);
XTRACE_RESOLVEALLRESOURCES;
```

Common_DefaultXTraceCFG incluye destinos de red predefinidos y una escucha local en el puerto 10001. En una red propia es preferible configurar explícitamente la dirección del monitor o el nombre DNS administrado por la organización.

XTRACE_TARGET::GetNSendings permite conocer cuántos mensajes se han enviado por un target. El límite configurado con XTRACE_SETSIZELIMIT controla el tamaño máximo que admite una traza y evita que un mensaje de diagnóstico crezca sin control.

Emisión de mensajes, colores y bloques de datos

Las macros XTRACE_PRINT, XTRACE_PRINTLEVEL y XTRACE_PRINTCOLOR cubren los casos habituales. El nivel transporta indicadores de presentación y clasificación. GEN Framework define los colores negro, azul, verde, púrpura, rojo y gris; también ofrece alias INFO, WARNING y ERROR.

```
XTRACE_PRINT(__L("Servidor iniciado en el puerto %d"), port);
XTRACE_PRINTCOLOR(XTRACE_COLOR_INFO, __L("Configuración cargada correctamente"));
XTRACE_PRINTCOLOR(XTRACE_COLOR_WARNING, __L("Reintento de conexión %d"), retry);
XTRACE_PRINTCOLOR(XTRACE_COLOR_ERROR, __L("No se pudo abrir el socket"));
```

Para inspeccionar buffers binarios se utilizan `XTRACE_PRINTDATABLOCK` y sus variantes por nivel o color. El receptor puede mostrar offset y representación textual, lo que resulta útil con protocolos, ficheros, tramas serie y paquetes de red.

```
XBUFFER packet;  
// Rellenar packet...  
  
XTRACE_PRINTDATABLOCKCOLOR(XTRACE_COLOR_BLUE, packet, 0, 16, true, true);
```

La aplicación debe evitar una frecuencia excesiva. Las trazas dentro de bucles de alta velocidad pueden alterar tiempos, saturar un transporte o desplazar información relevante. Es mejor emitir en cambios de estado, errores, hitos de operación y muestras reguladas.

Lectura de la ventana principal

La captura siguiente muestra una sesión real con Mini Web Server y un dispositivo STM32. La interfaz está organizada para observar simultáneamente procedencia, estado, mensajes y configuración del receptor.

eXtended Trace Monitor

All

Local (no internet)

Home [88.14.211.100]

UART STM32 F0

Name	Value
Version APP	Mini Web Server 0.1.0 (Beta)
Version S.O.	Microsoft Windows 11 Pro Bui

```

[30/07/2026 18:16:39.808] Mini Web Server 0.1.0
[30/07/2026 18:16:39.811] Mini Web Server 0.1.0
[30/07/2026 18:16:39.812] Mini Web Server 0.1.0
[30/07/2026 18:16:39.812] Mini Web Server 0.1.0
[30/07/2026 18:16:39.862] Mini Web Server 0.1.0
[30/07/2026 18:16:40.130] Mini Web Server 0.1.0
[30/07/2026 18:16:40.131] Mini Web Server 0.1.0
[30/07/2026 18:16:40.375] Mini Web Server 0.1.0
[30/07/2026 18:16:40.436] Mini Web Server 0.1.0
[30/07/2026 18:16:44.501] Mini Web Server 0.1.0
[30/07/2026 18:16:48.573] Mini Web Server 0.1.0
[30/07/2026 18:17:01.362] Mini Web Server 0.1.0
[30/07/2026 18:17:01.377] Mini Web Server 0.1.0
[30/07/2026 18:17:02.232] Mini Web Server 0.1.0
[30/07/2026 18:17:03.071] Mini Web Server 0.1.0
[30/07/2026 18:17:03.239] Mini Web Server 0.1.0
[30/07/2026 18:17:03.258] Mini Web Server 0.1.0
[30/07/2026 18:17:03.258] Mini Web Server 0.1.0
[30/07/2026 18:17:03.259] Mini Web Server 0.1.0

```

```

Mini Web Server 0.1.0 (Beta) [Debug]
Camino raíz de application: E:/Projects/GEN_Framework/Examples/Console/MiniWebServer/assets/

[Log] INFO Ini -----
[Log] INFO Ini Version S.O.: Microsoft Windows 11 Pro Build(26200) 64 bits
[Log] INFO Ini Memoria total 32580 Kb, libre 18329 Kb (56%).
[Log] INFO Ini Activando sistema de LOG ... : Correcto.
[Log] INFO Ini Activando estado aplicacion ... : Correcto.
[Log] INFO Ini Activando estado Internet ... : Correcto.
[Log] INFO Ini Inicializando Web Server ... : Ok.
[Log] INFO Ini Inicializando WebSocket ... : Ok.
[Log] WARNING Status Key pressed: 0x1B [?]
[Log] INFO General Aplicacion cerrada por el usuario
[Log] INFO End Desactivando servidor web ... : Ok.
[Log] INFO End Desactivando WebSocket ... : Ok.
[Log] INFO End Desactivando estado Internet ... : Correcto.
[Log] INFO End Desactivando estado aplicacion ... : Correcto.
[Log] INFO End Desactivando sistema de LOG ... : Correcto.
[Log] INFO End -----

```

IP Filter

Enable text filter

GEN FRAMEWORK

Code Fast. Run Faster.

Version 7.6.2

Level Filter

0 - Black

1 - Blue

2 - Green

3 - Purple

4 - Red

5 - Gray

Msg Status

Add time mark

Text tracing

Show IPs

ON

Clear

Stop

DynDNS

Del Stat.

Save

Minimize

Exit

xtracegen.dyndns.org:10001

GEN Framework

COM2,115200,8,N,1,NONE

UART STM32 F0

UDP

UART

Deactivated filter.

UART STM32 F0 COM2,115200,8,N,1,NONE: Close

Clear status from All.

Clear screen from All.

Figura B.1. eXtended Trace Monitor recibiendo trazas de una aplicación y de un dispositivo STM32.

Árbol de orígenes. El panel izquierdo agrupa las fuentes bajo All. En la captura aparecen un origen local, una máquina remota Home identificada por IP y un canal UART STM32 F0. Esta jerarquía permite seleccionar todo, una máquina o una aplicación/dispositivo concreto.

Variables de estado. La tabla Name/Value muestra valores publicados mediante los mensajes de estado. En el ejemplo aparecen Version APP y Version S.O.; los valores se actualizan sin añadir una línea nueva al flujo principal.

Panel de trazas. La zona central muestra fecha y hora, aplicación, versión y texto. Los colores diferencian mensajes normales, informativos, avisos o errores. Las entradas [Log] hacen visible el volcado de XLOG dentro del canal de trazas.

Filtros. La parte inferior permite filtrar por IP, texto y niveles/colores 0–5. También controla la visualización de mensajes de estado, marca temporal, texto de traza e IPs.

Controles. Clear limpia la vista, Stop pausa la recepción, DynDNS gestiona la resolución del destino, Del Stat elimina estados, Save guarda la captura, Minimize reduce la ventana y Exit finaliza la utilidad.

Destinos y estado interno. Los selectores muestran el host/puerto de red, el perfil y la configuración serie. Los indicadores UDP/UART informan de los canales habilitados. La consola negra registra operaciones internas como apertura/cierre del puerto y limpieza de pantalla.

Organización por máquina y aplicación

Los paquetes de XTRACE incluyen información suficiente para que el monitor separe los mensajes. La dirección pública y local permiten construir el origen de red; el nombre, versión e ID identifican la aplicación. La secuencia y la fecha ayudan a ordenar y diagnosticar pérdidas o retrasos.

Esta organización permite tener abierto un único monitor durante una prueba distribuida. Por ejemplo, una interfaz gráfica, un servicio web, un proceso de comunicaciones y un microcontrolador pueden enviar al mismo servidor. El árbol permite concentrarse en uno sin detener los demás.

Cuando se trabaja sin acceso a Internet, el monitor puede usar el origen local. El ejemplo muestra explícitamente Local (no internet), lo que confirma que la observación no depende de un servicio externo para funcionar dentro de la misma máquina o red.

Filtros por IP, texto, nivel, color y tipo

El filtrado es una operación de visualización. Permite reducir el ruido sin cambiar temporalmente el programa productor. El filtro IP limita orígenes; el filtro de texto busca términos; Level Filter activa o desactiva colores/niveles; Msg Status decide si se muestra el panel de estados.

- Negro/INFO para flujo normal y mensajes generales.
- Azul o verde para confirmaciones, pasos correctos o categorías elegidas por la aplicación.
- Púrpura/WARNING para situaciones que requieren atención pero permiten continuar.
- Rojo/ERROR para fallos y operaciones no completadas.
- Gris para información secundaria cuando la versión del emisor y el monitor lo permiten.

Los colores son categorías de diagnóstico, no una política universal obligatoria. Conviene que cada proyecto documente su uso y mantenga la misma semántica en todas sus aplicaciones.

Filtrar en el monitor no reduce el tráfico ya emitido. Para limitar coste de CPU, red o serie debe reducirse la generación en la aplicación, desactivar targets, usar compilaciones sin XTRACE_ACTIVE o introducir una política de muestreo.

Ventana de variables importantes

XTRACE_PRINTMSGSTATUS publica un valor identificado por nombre. El monitor no agrega una entrada cronológica cada vez: actualiza la fila correspondiente. Esto lo convierte en un panel de telemetría ligera para variables que interesa vigilar continuamente.

XTRACE_STATUS_MSG admite booleanos, enteros, cadenas, XDWORD, float y colores RGB.

XTRACE_STATUS_MSGS mantiene la colección recibida y permite añadir, buscar o eliminar estados. Cada estado conserva además un temporizador desde su última actualización, útil para detectar valores que han dejado de llegar.

```
XTRACE_PRINTMSGSTATUS(__L("Version APP"), __L("Mini Web Server 0.1.0 (Beta)"));
XTRACE_PRINTMSGSTATUS(__L("Clientes conectados"), nclients);
XTRACE_PRINTMSGSTATUS(__L("Internet disponible"), isinternet);
XTRACE_PRINTMSGSTATUS(__L("Uso de memoria (KB)"), memory_kb);
XTRACE_PRINTMSGSTATUS(__L("Temperatura"), temperature);
```

El panel es apropiado para conexiones activas, memoria, colas, estado de sensores, versión, modo operativo, latencia o último error. No sustituye a una plataforma histórica de métricas: representa el estado actual y está pensado para diagnóstico interactivo.

```
// Limpiar los estados de la aplicación actual o de todos los orígenes.
XTRACE_CLEARMSGSTATUS;
// XTRACE_CLEARALLMSGSTATUS;
```

Pausa, limpieza y guardado de una captura

Stop permite congelar la recepción o presentación para analizar un intervalo sin que las líneas nuevas desplacen la información. Clear limpia la pantalla del monitor, y las macros XTRACE_CLEARSCREEN/XTRACE_CLEARALLSCREENS permiten solicitar la limpieza desde el productor según el alcance deseado.

```
XTRACE_CLEARSCREEN;
XTRACE_PRINTHEADER(0, __L("Inicio de prueba de comunicaciones"));
```

Save conserva las trazas adquiridas para analizarlas o adjuntarlas a una incidencia. Antes de guardar conviene seleccionar el alcance correcto en el árbol y revisar los filtros: una captura filtrada puede no contener información que sí llegó al monitor.

Las trazas pueden contener rutas, direcciones, identificadores, datos de protocolo o información introducida por usuarios. El fichero guardado debe tratarse como material de diagnóstico potencialmente sensible.

Transporte remoto, UDP, pipeline y UART

El monitor puede recibir por varias vías porque XTRACE separa el mensaje del target. En red, NET utiliza host y puerto y se apoya en DataIO. En Windows, SPECIAL usa mensajería de ventana para una entrega local. Según la plataforma y la compilación, pueden habilitarse pipelines o canales locales equivalentes, y los targets de microcontrolador pueden usar UART o streams serie.

La captura muestra simultáneamente configuración de red y COM2 a 115200 baudios, 8 bits, un bit de parada y sin paridad, además de indicadores UDP y UART. Esto permite observar una aplicación de PC y un STM32 desde la misma herramienta.

UDP es adecuado para diagnóstico por su bajo acoplamiento: el productor no necesita establecer una sesión larga. A cambio, no garantiza entrega ni orden. Una ausencia aislada de traza no demuestra que el código no se ejecutara; para auditoría o persistencia deben utilizarse logs u otro canal fiable.

Control del volumen y coste de las trazas

XTRACE registra el número de envíos por target y aplica un límite de tamaño. Estas funciones ayudan a impedir mensajes desmesurados, pero la estrategia principal debe estar en el diseño de la instrumentación.

- No emitir en cada iteración de un bucle rápido salvo durante una prueba acotada.
- Publicar cambios de estado y errores, no el mismo valor sin variación.
- Usar la ventana de estados para variables repetitivas en lugar de generar líneas continuas.
- Limitar los bloques binarios y mostrar solo el fragmento necesario.
- Desactivar targets que no se estén utilizando.
- Separar builds de desarrollo y producción mediante XTRACE_ACTIVE y la configuración del proyecto.

Cuando XTRACE_ACTIVE no está definido, las macros se convierten en operaciones vacías o valores neutros según el compilador. Esto permite conservar llamadas de instrumentación en el código sin mantener el coste completo en todas las configuraciones.

Uso recomendado durante una sesión de diagnóstico

- Iniciar eXtended Trace Monitor y habilitar los receptores necesarios.
- Configurar el target de la aplicación con la dirección, puerto o mecanismo local correcto.
- Asignar nombre, versión e ID antes de los primeros mensajes.
- Comprobar en el árbol que aparece la máquina y la aplicación esperadas.
- Activar marca temporal e IP si el problema depende de orden o procedencia.
- Aplicar primero un filtro de origen y después texto/nivel para no ocultar información prematuramente.
- Publicar variables estables en el panel de estado y reservar el panel principal para acontecimientos.
- Pausar cuando se reproduzca el fallo y guardar la captura con un nombre que incluya fecha y escenario.
- Conservar también los ficheros XLOG si se necesita un registro persistente y completo.

Seguridad y despliegue

Las trazas son una herramienta de desarrollo y operación, no un canal de datos de negocio. No deben incluir contraseñas, claves, tokens, datos personales ni contenido completo de comunicaciones sensibles. Los dumps binarios merecen especial atención porque pueden revelar más información de la prevista.

Los filtros de la interfaz no protegen el transporte: solo controlan lo que se muestra. Cuando el monitor recibe desde otras máquinas, la red debe estar controlada mediante segmentación, VPN, reglas de firewall o un canal de transporte adecuado al entorno. No se debe exponer un receptor de diagnóstico a Internet sin una arquitectura de seguridad explícita.

En producción conviene limitar targets, frecuencia y contenido, y definir cuándo se habilitan. En dispositivos embebidos debe considerarse además el ancho de banda de UART, el tiempo de CPU y el efecto sobre tareas de tiempo real.

Clases y componentes relacionados

- XTRACE: gestor central de destinos, metadatos, empaquetado y emisión.
- XTRACE_TARGET: tipo, objetivo, recursos de red y contador de envíos.
- XTRACE_STATUS_MSG y XTRACE_STATUS_MSGS: valores tipados y colección del panel de estado.
- XWINDOWSTRACE, XLINUXTRACE y XANDROIDTRACE: especializaciones de plataforma de escritorio/móvil.
- XESP32TRACE y XSTM32TRACE: especializaciones para microcontroladores.

- DIOXTRACEINTERSTREAMS: procesamiento de trazas a través de streams de DataIO.
- XLOGBASE y XLOG: registro persistente que puede duplicar sus entradas hacia XTRACE.
- Common_DefaultXTraceCFG: destinos de red predeterminados y configuración local de referencia.

Resumen práctico

eXtended Trace Monitor centraliza las trazas de GEN Framework procedentes de aplicaciones, máquinas y dispositivos diferentes. Puede recibir por red, mecanismos locales de plataforma, pipeline o serie según la compilación; clasifica los mensajes por origen; filtra por IP, texto, nivel y color; muestra variables de estado; pausa, limpia y guarda capturas.

XTRACE y XLOG cumplen funciones distintas. XTRACE ofrece observación inmediata y múltiples targets; XLOG ofrece persistencia, filtros, límites y backups. La opción `isdumpintrace` une ambos sin confundirlos: el log conserva su fichero y el monitor recibe una copia en tiempo real.

La utilidad forma parte del ecosistema abierto de GEN Framework y sirve como herramienta cotidiana para desarrollo, soporte y diagnóstico remoto. Usada con una política de volumen y seguridad adecuada, permite entender lo que ocurre en un sistema distribuido sin depender del depurador de una única máquina.

Anexo III: Preguntas frecuentes — Preguntas y respuestas

Este anexo reúne respuestas breves a preguntas que suelen aparecer al descubrir, evaluar o empezar a utilizar GEN Framework. No sustituye a los capítulos anteriores: sirve como mapa de consulta para localizar una idea, aclarar una decisión de diseño o recordar por dónde continuar.

Las respuestas describen el enfoque general del framework. Una función concreta puede depender del módulo activado, del compilador, de la plataforma o de las bibliotecas de terceros disponibles. Cuando exista una diferencia, la configuración del proyecto, los ejemplos y el código fuente son la referencia definitiva.

1. Conceptos fundamentales

P1. ¿Qué es exactamente GEN Framework?

GEN Framework es una arquitectura de desarrollo en C/C++ para construir aplicaciones nativas, modulares y multiplataforma. Reúne tipos básicos, utilidades, comunicaciones, gráficos, entrada, sonido, cifrado, compresión, bases de datos, scripts, interfaces de usuario y servicios de aplicación bajo convenciones comunes. Su valor no está solo en la cantidad de clases, sino en cómo esas clases colaboran y aíslan las diferencias de cada plataforma.

P2. ¿Por qué no debe entenderse como un conjunto de bibliotecas independientes?

Porque sus módulos comparten tipos, ciclos de vida, factorías, configuración, trazas, rutas y criterios de compilación. Una clase básica puede servir a otra más especializada y varios servicios pueden formar una aplicación completa. Esa relación deliberada entre piezas permite razonar sobre el sistema como un framework y no como una carpeta de utilidades inconexas.

P3. ¿Qué significa desarrollar en modo Lego?

Significa atomizar las funcionalidades en clases pequeñas, con una responsabilidad clara y dependencias controladas. Esas piezas pueden probarse, sustituirse y combinarse para crear componentes mayores. GEN Framework está construido con ese principio y permite aplicarlo también en los proyectos que se desarrollan sobre él.

P4. ¿Por qué se utiliza C/C++?

C/C++ ofrece compiladores para una gran variedad de procesadores, sistemas operativos y entornos embebidos. Permite generar código nativo eficiente, trabajar cerca del hardware cuando es necesario y, al mismo tiempo, utilizar abstracciones modernas. La existencia de varios compiladores y toolchains evita depender de una única empresa o máquina virtual y aporta una base sólida para la portabilidad.

P5. ¿Qué significa que está diseñado de programadores para programadores?

Significa que sus interfaces nacen de necesidades reales de desarrollo. Las utilidades intentan reducir trabajo repetitivo sin ocultar aquello que el programador necesita controlar: memoria, configuración, rutas, comunicaciones, estados, recursos y diagnóstico. El objetivo es ofrecer herramientas cómodas y efectivas, no reemplazar el criterio técnico de quien las utiliza.

P6. ¿Las clases existen solo porque parecían útiles sobre el papel?

No. La filosofía del proyecto es incorporar funcionalidades que han sido necesarias en proyectos concretos. Los tests ayudan a comprobarlas, pero su diseño también recoge la experiencia de compilaciones, despliegues, dispositivos y aplicaciones reales. Esa procedencia práctica no elimina la necesidad de validar cada uso nuevo, pero evita construir una arquitectura puramente académica.

2. Primeros pasos

P7. ¿Dónde se obtiene GEN Framework?

El punto de entrada es la web de GEN Framework. Su sección de descargas proporciona scripts para Windows y Linux capaces de obtener el conjunto de repositorios. También es posible clonar cada repositorio mediante Git. Solo es necesario utilizar el procedimiento correspondiente al sistema que servirá como base de compilación.

P8. ¿Por qué el proyecto está dividido en varios repositorios?

La separación permite mantener responsabilidades claras y descargar o actualizar cada bloque de forma independiente. El núcleo, Common, ThirdPartyLibraries, Examples, Tests, Utilities, Resources y Documents tienen ritmos y finalidades diferentes. Los scripts de descarga y construcción coordinan esa organización para que el usuario no tenga que reconstruir manualmente la relación entre repositorios.

P9. ¿Qué herramientas se necesitan para empezar?

Como base se necesita Git, CMake, un sistema de construcción como Ninja y un compilador de C/C++ adecuado para la plataforma. En Windows suele utilizarse Visual Studio con MSVC; en Linux son habituales GCC o Clang. Visual Studio Code puede actuar como editor común en ambos entornos, y Docker o WSL2 resultan útiles cuando el proyecto necesita entornos reproducibles o compilación cruzada.

P10. ¿Qué debería compilarse primero?

Conviene comenzar por un ejemplo oficial sencillo y utilizar los scripts de Common. Esto valida de una sola vez la ubicación de los repositorios, el compilador, CMake, las bibliotecas de terceros y la configuración elegida. Cuando el ejemplo compila y se ejecuta, ya existe una referencia conocida con la que comparar un proyecto propio.

P11. ¿Por qué los ejemplos no siempre definen directamente la función main?

En las modalidades AppFlow, el arranque está integrado en la infraestructura del framework. La aplicación implementa su inicialización, actualización y cierre dentro de un ciclo de vida común. Así puede compartir la misma lógica general entre plataformas cuyas funciones de entrada, ventanas o servicios de sistema son diferentes.

P12. ¿Dónde debe situarse un proyecto propio?

La organización recomendada coloca el proyecto al mismo nivel que la carpeta GEN_FrameWork dentro de Projects. De ese modo puede acceder mediante rutas relativas a los repositorios necesarios sin mezclar su código con el código del framework. Esta separación también facilita actualizar GEN Framework y mantener el proyecto en su propio repositorio.

P13. ¿Es necesario conocer todas las clases antes de crear una aplicación?

No. Es más eficaz aprender por capas. Primero se prepara el entorno y se compila un ejemplo; después se entiende AppFlow, la configuración y las trazas; finalmente se incorporan los módulos que resuelven necesidades concretas. XUtils y los ejemplos oficiales ofrecen un buen vocabulario inicial, pero no es necesario memorizar el catálogo completo.

3. Arquitectura, módulos y aplicaciones

P14. ¿Qué función cumplen las factorías?

Las factorías crean objetos cuya implementación puede cambiar según la plataforma. La aplicación solicita una pantalla, un stream, un dispositivo de entrada o un recurso de sonido mediante una interfaz común; la factoría selecciona la clase concreta. Esto confina el código dependiente de Windows, Linux, Android o un sistema embebido y evita dispersarlo por la lógica de negocio.

P15. ¿Es obligatorio utilizar AppFlow?

No. GEN Framework admite un uso base de clases y servicios, aplicaciones de consola con AppFlowConsole y aplicaciones gráficas con AppFlowGraphics. AppFlow resulta conveniente cuando se desea un ciclo de vida homogéneo, configuración integrada, trazas, rutas y estados. Un proyecto pequeño puede utilizar solo los módulos que necesite, siempre que gestione correctamente su inicialización y cierre.

P16. ¿Qué aporta XUtils?

XUtils proporciona la base común: cadenas, buffers, rutas, ficheros, fechas, temporizadores, colecciones, eventos, máquinas de estados, hilos, configuración, trazas y otros servicios generales. Muchos módulos utilizan sus tipos para mantener una interfaz coherente. Comprender XSTRING, XBUFFER, XPATH, XVECTOR, XEVENT y las factorías básicas facilita el aprendizaje del resto del framework.

P17. ¿Cuándo debe utilizarse DataIO?

DataIO concentra las comunicaciones y la entrada o salida de datos: streams, sockets, puertos serie, Bluetooth, USB, protocolos web, notificaciones, buses de hardware y servicios externos. Es adecuado cuando la aplicación necesita comunicarse sin quedar ligada directamente a una API concreta del sistema operativo o del dispositivo.

P18. ¿Cómo se relacionan Graphic, Input y UserInterface?

Graphic proporciona pantallas, canvas, bitmaps, fuentes y renderizado. Input abstrae botones, cursores y dispositivos de interacción. UserInterface combina ambos bloques para construir layouts, widgets, estilos y eventos de interfaz. Una aplicación puede usar Graphic sin UserInterface, pero una interfaz completa suele apoyarse en los tres niveles.

P19. ¿Todos los módulos se incluyen siempre en el ejecutable?

No. El sistema de construcción permite activar funcionalidades mediante opciones y macros. Un programa sin sonido, bases de datos o scripts no debería cargar esas dependencias. Esta selección reduce el tamaño, el tiempo de compilación y la superficie de mantenimiento, algo especialmente importante en sistemas embebidos.

P20. ¿Cómo se controla que un módulo no dependa de todo el framework?

La dependencia debe avanzar desde piezas básicas hacia servicios más especializados y expresarse en la configuración de construcción. Las interfaces comunes, las factorías y los tipos de XUtils ayudan a evitar referencias circulares. Si una clase necesita conocer detalles internos de muchos módulos, normalmente conviene revisar su responsabilidad y dividirla.

P21. ¿Qué papel tienen la configuración y la carpeta assets?

La configuración separa decisiones de despliegue de la lógica compilada: puertos, rutas, targets de trazas, bases de datos o listas de scripts pueden cambiar sin reescribir el núcleo. La carpeta assets agrupa recursos que acompañan a la aplicación, como imágenes, sonidos, layouts, certificados o ficheros auxiliares. XPATHMANAGER ayuda a localizarlos de forma coherente.

4. Plataformas, compilación y rendimiento

P22. ¿Qué entiende GEN Framework por plataforma?

Una plataforma es la combinación del hardware —microcontrolador, microprocesador, PC, Raspberry Pi u otro dispositivo— y el sistema operativo o entorno de ejecución. También puede existir una plataforma sin sistema operativo. El target de compilación identifica el destino concreto para el que deben seleccionarse las implementaciones y herramientas adecuadas.

P23. ¿Multiplataforma significa que un mismo binario funciona en todas partes?

No. Significa que una misma arquitectura y una parte muy alta del código fuente pueden compilarse para destinos diferentes. Cada plataforma genera su propio binario y puede requerir una toolchain, bibliotecas o permisos específicos. La portabilidad consiste en confinar esas diferencias, no en fingir que el hardware y los sistemas operativos son idénticos.

P24. ¿El código de aplicación compila siempre sin ningún cambio?

El objetivo es mantener sin cambios la lógica común, pero algunas capacidades no existen en todos los destinos. Una pantalla, una cámara, un sistema de ficheros o una conexión concreta pueden necesitar alternativas. El diseño portable detecta esas diferencias en la configuración y en las capas de plataforma, evitando que se propaguen innecesariamente al resto de la aplicación.

P25. ¿Qué compiladores pueden utilizarse?

GEN Framework está pensado para trabajar con varios compiladores de C/C++. GCC, Clang y MSVC son habituales, y existen entornos adicionales según la plataforma. La compatibilidad exacta depende del target, de las opciones activadas y de las bibliotecas externas. Probar con más de un compilador ayuda a descubrir supuestos no portables y mejora la robustez del código.

P26. ¿Cómo se añade soporte para una plataforma nueva?

Primero se identifica qué interfaces comunes necesita el target. Después se implementan las clases específicas dentro de la capa Platform y se conectan a sus factorías. El código de aplicación debería seguir usando las interfaces existentes. Los cambios en el núcleo común solo son necesarios cuando la nueva plataforma revela una capacidad general que todavía no estaba modelada.

P27. ¿Las abstracciones reducen el rendimiento?

Toda abstracción tiene un coste potencial, pero el diseño en C/C++ permite mantenerlo visible y medirlo. Muchas decisiones se resuelven durante la compilación o mediante capas muy finas. El rendimiento real depende del algoritmo, del uso de memoria, de la entrada y salida y de la plataforma; por eso conviene medir los puntos críticos en el target real en lugar de asumir que una abstracción es gratuita o costosa.

P28. ¿Puede utilizarse GEN Framework en microcontroladores?

Sí, cuando existe una adaptación de plataforma y se selecciona un conjunto de módulos acorde con los recursos disponibles. En un microcontrolador suelen desactivarse servicios innecesarios, reducirse buffers y evitarse dependencias pesadas. La misma arquitectura puede compartirse con aplicaciones mayores, pero la configuración debe respetar la memoria, el almacenamiento y los periféricos del dispositivo.

5. Calidad, diagnóstico y seguridad

P29. ¿Qué diferencia existe entre una traza y un log?

Una traza está orientada a observar el comportamiento inmediato de una aplicación y puede enviarse a distintos targets, incluido eXtended Trace Monitor. Un log está pensado para conservar un registro persistente, aplicar rotación y permitir análisis posterior. Pueden colaborar, pero no son equivalentes: la traza favorece la interacción durante el diagnóstico y el log favorece la persistencia.

P30. ¿Por dónde se empieza cuando falla una compilación?

Conviene reproducir el error con una configuración conocida, identificar el primer mensaje relevante y comprobar target, compilador, rutas y repositorios. Después se revisan las opciones activadas y la disponibilidad de ThirdPartyLibraries. Comparar con un ejemplo que sí compila suele revelar rápidamente si el problema pertenece al entorno, a una dependencia o al código propio.

P31. ¿Cómo ayuda el control de memoria?

Las opciones de control de memoria permiten registrar reservas y liberaciones realizadas mediante los mecanismos previstos por el framework. Son útiles para localizar fugas, dobles liberaciones o recursos que sobreviven al cierre. No sustituyen a las herramientas del compilador ni a los analizadores de plataforma, pero aportan una visión integrada con las clases y las trazas de GEN Framework.

P32. ¿Los tests son suficientes para confiar en una clase?

Los tests comprueban contratos y regresiones, pero deben complementarse con ejemplos, pruebas de integración y ejecución en plataformas reales. GEN Framework valora especialmente las funcionalidades que ya han resuelto necesidades de proyectos concretos. Aun así, cada aplicación debe validar sus propios límites, cargas, dispositivos y condiciones de fallo.

P33. ¿Todas las clases son seguras para utilizarlas desde varios hilos?

No debe suponerse. La seguridad entre hilos depende de la responsabilidad y del contrato de cada clase. Los contenedores, callbacks, factorías o recursos de plataforma pueden requerir sincronización externa. El programador debe revisar la documentación y proteger el estado compartido, manteniendo las secciones críticas pequeñas y evitando bloquear dentro de callbacks no controlados.

P34. ¿Utilizar Cipher convierte automáticamente una aplicación en segura?

No. Cipher proporciona hashes, cifrado, claves y certificados, pero la seguridad depende también del protocolo, la generación y custodia de claves, la validación de certificados, los permisos, las actualizaciones y el tratamiento de errores. Deben utilizarse algoritmos vigentes y diseños revisados; disponer de primitivas criptográficas no sustituye a un análisis de amenazas.

P35. ¿Conviene mantener las trazas activas en producción?

Depende del producto. Las trazas operativas pueden ser muy valiosas, pero deben limitar volumen, frecuencia y contenido. Nunca deberían exponer contraseñas, claves, tokens o datos personales. Es recomendable separar configuraciones de desarrollo y producción, controlar los targets habilitados y documentar durante cuánto tiempo se conservan los registros.

6. Bibliotecas de terceros, licencias y mantenimiento

P36. ¿Por qué son necesarias las bibliotecas de terceros?

Permiten construir sistemas mayores sobre soluciones probadas para tipografías, imágenes, audio, compresión, bases de datos, red, scripts o toolchains. Reimplementar cada estándar consumiría recursos y aumentaría el riesgo. La decisión importante no es evitar toda dependencia, sino elegirla por su funcionalidad, portabilidad, mantenimiento, integración y licencia.

P37. ¿Por qué se prefiere integrar el código fuente?

Cuando la licencia y la tecnología lo permiten, compilar la biblioteca junto al proyecto reduce instalaciones globales y facilita utilizar el mismo código con distintas toolchains. También permite fijar opciones, depurar la integración y reproducir una construcción. Los SDK de fabricante, drivers o bibliotecas dinámicas pueden requerir una estrategia diferente y quedar confinados a la plataforma correspondiente.

P38. ¿Una biblioteca de código abierto puede utilizarse sin revisar nada?

No. Cada licencia establece condiciones de atribución, redistribución, modificación o enlace. El proyecto debe conservar los avisos legales, identificar la versión incluida y comprobar la compatibilidad con la forma de distribución del producto. Las licencias permisivas, LGPL, GPL, dominio público y licencias específicas no imponen las mismas obligaciones.

P39. ¿Por qué el inventario de ThirdPartyLibraries no se vincula a una versión concreta de GEN Framework?

Porque documenta la copia auditada de cada dependencia, su función y su licencia como una fotografía reproducible del árbol ThirdPartyLibraries. Esa información sigue siendo útil aunque cambie la versión general del framework. Al actualizar una dependencia debe repetirse la revisión de código, compilación, compatibilidad y licencia, sin depender de referencias a commits concretos en el texto del libro.

P40. ¿Puede sustituirse una biblioteca externa por otra?

Sí, si se mantiene el contrato que GEN Framework expone y se valida el comportamiento en todas las plataformas afectadas. La capa propia alrededor de la dependencia existe precisamente para evitar que su API se extienda por la aplicación. La sustitución debe revisar formatos, errores, rendimiento, licencia, opciones de compilación y compatibilidad con los datos existentes.

P41. ¿Cómo se mantiene una dependencia de terceros?

Conviene registrar versión, origen, licencia, parches locales y opciones de compilación. Una actualización debe probarse primero de forma aislada y después con los módulos y ejemplos que la utilizan. También deben compararse símbolos, formatos, comportamiento y avisos legales. Actualizar por rutina, sin una necesidad o una validación clara, puede introducir más riesgo que beneficio.

P42. ¿Cómo se contribuye código manteniendo el estilo del proyecto?

El Anexo I resume las reglas de nombres, estructura de ficheros, includes, ciclo de vida, memoria, Doxygen, factorías y código de plataforma. Antes de crear un módulo conviene estudiar dos o tres clases cercanas del mismo subsistema. La consistencia local es más importante que imponer un estilo externo que haga parecer ajeno el código nuevo.

7. Adopción y criterios de uso

P43. ¿Puede incorporarse GEN Framework a un proyecto existente?

Sí, y normalmente es mejor hacerlo de forma gradual. Puede empezarse por tipos, configuración, trazas o un servicio aislado, crear una frontera clara y ampliar la integración cuando el equipo comprenda los ciclos de vida y dependencias. Una migración completa solo tiene sentido si aporta beneficios suficientes frente al riesgo y al coste de reescritura.

P44. ¿Tiene sentido utilizarlo si la aplicación solo tendrá una plataforma?

Puede tenerlo. La modularidad, las factorías, la configuración, las trazas y la reutilización aportan orden incluso en un único destino. Sin embargo, el beneficio debe compararse con el tamaño y la vida prevista del producto. Para una utilidad desechable de pocas líneas, una arquitectura extensa puede no ser necesaria.

P45. ¿Puede construirse una interfaz gráfica completa?

Sí. Graphic aporta renderizado y recursos visuales; Input proporciona interacción; UserInterface añade layouts, widgets, estilos y eventos; y AppFlowGraphics coordina el ciclo de la aplicación. El resultado puede adaptarse a pantallas y dispositivos diferentes, siempre que el diseño de la interfaz contemple resolución, densidad, entrada y rendimiento del target.

P46. ¿Es posible combinar comunicaciones, bases de datos y scripts?

Sí. Esa combinación es uno de los objetivos de una arquitectura integrada. DataIO puede recibir información, Databases almacenarla, Cipher protegerla y Scripts aplicar reglas configurables. AppFlow coordina el servicio y XUtils aporta tipos, eventos, rutas y diagnóstico. La aplicación debe conservar fronteras claras para que la integración no convierta todos los módulos en dependencias mutuas.

P47. ¿Cuándo encaja especialmente bien GEN Framework?

Encaja en productos nativos que deben vivir durante años, compartir código entre plataformas, comunicarse con dispositivos o servicios y mantener control sobre rendimiento y recursos. También resulta útil cuando una organización desarrolla varias aplicaciones relacionadas y quiere reutilizar una arquitectura, un estilo de diagnóstico y módulos ya probados.

P48. ¿Cuándo puede no ser la mejor elección?

Puede no serlo para prototipos efímeros, proyectos cuyo ecosistema depende por completo de otra plataforma de alto nivel o equipos sin experiencia ni capacidad de mantenimiento en C/C++. La decisión debe considerar requisitos, vida útil, herramientas, conocimientos, licencias y coste total, no solo el tiempo necesario para mostrar la primera pantalla.

P49. ¿Cuál es un recorrido recomendado de aprendizaje?

Primero conviene leer la introducción y preparar el entorno; después compilar un ejemplo y entender AppFlow, configuración y trazas. El siguiente paso es estudiar XUtils y el módulo más cercano al problema que se desea resolver. Finalmente pueden revisarse ThirdPartyLibraries, la guía de estilo y estas preguntas. Aprender mediante una aplicación pequeña y ejecutable suele ser más eficaz que recorrer el catálogo de clases de forma lineal.

Anexo IV: Glosario de términos

Este glosario reúne conceptos técnicos, abreviaturas, nombres de herramientas e identificadores que aparecen a lo largo del libro y que pueden resultar poco familiares fuera de su contexto habitual.

Las definiciones describen el sentido con el que cada término se utiliza en esta obra. Sirven como referencia rápida, pero no sustituyen la documentación de una API, una biblioteca, una licencia o una plataforma concreta.

1. A-C

AES: Advanced Encryption Standard. Algoritmo de cifrado simétrico por bloques utilizado para proteger datos con una misma clave en el cifrado y el descifrado.

AGG: Anti-Grain Geometry. Biblioteca de renderizado 2D que puede actuar como backend para convertir geometría vectorial en píxeles con suavizado de bordes.

Algoritmo: Secuencia definida de operaciones que transforma unos datos de entrada en un resultado. En el libro aparece en compresión, cifrado, gráficos, scripts y otras áreas.

Android: Plataforma formada por dispositivos compatibles y el sistema operativo Android. Requiere toolchains, bibliotecas y clases de plataforma específicas.

API: Application Programming Interface. Contrato público mediante el que un módulo ofrece tipos, funciones o clases sin exponer necesariamente su implementación interna.

AppFlow: Bloque de GEN Framework que organiza el ciclo de vida de una aplicación: inicialización, primeras actualizaciones, ejecución periódica y cierre ordenado.

AppFlowConsole: Modalidad de AppFlow orientada a programas de consola o servicios sin interfaz gráfica completa.

AppFlowGraphics: Modalidad de AppFlow que integra el ciclo de aplicación con pantalla, renderizado, entrada de usuario y actualización de una interfaz gráfica.

AppProc_Ini / AppProc_Update: Métodos habituales del ciclo AppFlow. El primero prepara la aplicación y el segundo atiende su actualización periódica durante la ejecución.

ARM / ARM64: Familias de arquitecturas de procesador muy extendidas en móviles, placas y sistemas embebidos. ARM64 designa la variante de 64 bits.

Backend: Implementación concreta situada detrás de una interfaz común. Un mismo servicio gráfico, de sonido o de comunicaciones puede tener backends distintos según la plataforma.

Base de datos: Sistema que almacena y consulta información estructurada. El módulo Databases abstrae motores como SQLite, MySQL, MariaDB o PostgreSQL.

Binding: Vinculación que permite utilizar desde un lenguaje o API las funciones ofrecidas por otro componente. Suele adaptar tipos, llamadas y gestión de errores.

Bitmap: Imagen representada como una matriz de píxeles. Su tamaño, formato de color y profundidad determinan cómo se almacena y se renderiza.

Bluetooth: Familia de tecnologías inalámbricas de corto alcance. DataIO la trata como un medio de comunicación dependiente de las capacidades de la plataforma.

Buffer: Zona de memoria que conserva temporalmente datos mientras se producen, transforman o consumen. Es habitual en ficheros, red, audio, compresión y serialización.

Builder: Entorno de desarrollo y compilador de Embarcadero mencionado entre las toolchains compatibles. No debe confundirse con el patrón de diseño del mismo nombre.

Callback: Función o método que un componente registra para que otro lo invoque cuando sucede un evento, termina una operación o llega información.

Canvas: Superficie lógica de dibujo 2D. Agrupa operaciones para representar líneas, formas, texto, bitmaps y otros elementos gráficos.

Capa: Nivel de una arquitectura que agrupa responsabilidades y limita dependencias. GEN Framework separa, entre otras, capas comunes, especializadas y de plataforma.

Checksum: Valor calculado a partir de unos datos para detectar cambios o daños accidentales. No equivale necesariamente a un hash criptográfico.

Cipher: Módulo de GEN Framework dedicado a hashes, cifrado, claves, certificados y operaciones criptográficas relacionadas.

Cifrado: Transformación de información legible en datos protegidos mediante un algoritmo y una o varias claves. Su seguridad depende también del protocolo y de la gestión de esas claves.

Cifrado asimétrico: Cifrado basado en una pareja de claves relacionadas: una pública y otra privada. RSA y X25519 aparecen en el libro dentro de este ámbito.

Cifrado simétrico: Cifrado en el que una misma clave secreta protege y recupera la información. AES es uno de los algoritmos citados.

Clang: Compilador para los lenguajes C y C++ basado en LLVM. Probar una aplicación con Clang y con otras toolchains ayuda a descubrir supuestos no portables.

Clase: Tipo que reúne estado y comportamiento. El diseño en modo Lego de GEN Framework favorece clases pequeñas, responsabilidades claras y composición.

CMake: Sistema de generación de proyectos de compilación. Lee la configuración del proyecto y produce archivos adecuados para Ninja, Visual Studio u otros entornos.

CMakeLists.txt: Fichero de configuración principal de CMake. Declara fuentes, opciones, dependencias, definiciones y targets de construcción.

Código fuente: Texto escrito por el programador antes de la compilación. Integrar una biblioteca en código fuente facilita adaptar su construcción a diferentes plataformas.

Código nativo: Código máquina generado para el procesador y el sistema de destino. Se ejecuta sin depender de una máquina virtual obligatoria.

Compilación: Proceso que traduce código fuente y recursos en objetos, bibliotecas o ejecutables. Incluye análisis, generación de código y, normalmente, enlace.

Compilación cruzada: Construcción realizada en una plataforma anfitriona para producir un ejecutable destinado a otra arquitectura o sistema.

Compilador: Herramienta que traduce un lenguaje de programación a una representación ejecutable o intermedia. GCC, Clang, MSVC y Builder son ejemplos citados.

Compress: Módulo de GEN Framework que unifica algoritmos y formatos de compresión, además de operaciones sobre ficheros y buffers comprimidos.

Configuración: Conjunto de valores que modifica el comportamiento sin reescribir la lógica principal. Puede residir en ficheros INI, XML, JSON o estructuras creadas por la aplicación.

Consola: Interfaz textual basada en entrada y salida estándar. Es apropiada para herramientas, pruebas, servicios y aplicaciones sin ventana gráfica.

Contenedor: Estructura que agrupa elementos, como un vector o una lista. En Docker, el mismo término designa además un entorno de ejecución aislado y reproducible.

CRC: Cyclic Redundancy Check. Familia de códigos de comprobación eficaces para detectar errores accidentales en bloques de datos, comunicaciones o ficheros.

2. D-F

DatalO: Módulo de GEN Framework que abstrae entrada, salida y comunicaciones: streams, sockets, puertos serie, buses de hardware, web y otros dispositivos.

DEBUG / RELEASE: Configuraciones de construcción con objetivos distintos. DEBUG prioriza diagnóstico e información de depuración; RELEASE prioriza la distribución y la optimización.

Dependencia: Relación por la que un componente necesita tipos o servicios de otro. Controlarlas evita acoplamientos circulares y facilita seleccionar módulos.

Docker: Tecnología de contenedores utilizada para preparar entornos de compilación reproducibles con herramientas y dependencias conocidas.

Dominio público: Situación jurídica en la que una obra no está protegida por derechos exclusivos aplicables o ha sido dedicada expresamente a ese régimen. Debe comprobarse para cada recurso.

Driver: Componente que permite al software comunicarse con un dispositivo o servicio concreto. Suele pertenecer a la capa dependiente de plataforma.

Doxygen: Herramienta que genera documentación a partir del código fuente y de comentarios estructurados asociados a clases, funciones y tipos.

DXF: Drawing Exchange Format. Formato vectorial empleado para intercambiar información de dibujo técnico y geometría.

EGL: Interfaz que conecta APIs gráficas como OpenGL ES con el sistema nativo de ventanas y superficies. Gestiona contextos, buffers y presentación.

Embebido: Sistema integrado en un dispositivo con una función concreta y recursos normalmente limitados. Puede funcionar con sistema operativo, con uno de tiempo real o sin él.

Enlace: Fase que combina objetos compilados y bibliotecas para resolver símbolos y producir un ejecutable o una biblioteca final.

Evento: Notificación de que ha ocurrido algo relevante. Permite desacoplar al productor de la información de los objetos que reaccionan a ella.

eXtended Trace Monitor: Aplicación externa que recibe, organiza, filtra y presenta las trazas emitidas por aplicaciones construidas con GEN Framework.

Factoría (Factory): Objeto o patrón encargado de crear instancias concretas detrás de una interfaz común. En GEN Framework permite seleccionar implementaciones según plataforma y configuración.

Feedback: Información que una operación devuelve sobre su estado, avance o resultado. Puede enviarse mediante eventos, trazas o mecanismos específicos del módulo.

Firmware: Software estrechamente ligado al hardware de un dispositivo. Suele compilarse para una arquitectura concreta y controlar periféricos directamente.

FreeType: Biblioteca de código abierto utilizada para cargar y rasterizar fuentes tipográficas en sistemas gráficos.

Framework: Infraestructura coherente que define arquitectura, ciclos de vida, contratos y formas de extensión. No es simplemente una colección de bibliotecas independientes.

3. G-I

GCC: GNU Compiler Collection. Conjunto de compiladores multiplataforma que incluye soporte para los lenguajes C y C++.

GEN_DELETE / GEN_NEW: Macros de GEN Framework que centralizan la liberación y la reserva dinámica cuando están activados los mecanismos de control de memoria.

GEN Framework: Framework multiplataforma en C/C++ que organiza módulos, clases comunes y capas de plataforma para construir aplicaciones nativas de distintos tamaños.

GKF: Fichero de claves de GEN Framework. Formato empleado por Cipher para representar y conservar información de claves dentro de las utilidades del framework.

GPIO: General-Purpose Input/Output. Pines digitales configurables de un microcontrolador o placa que pueden actuar como entradas o salidas.

GPL: GNU General Public License. Licencia de software libre con copyleft que impone condiciones específicas al redistribuir obras derivadas.

Graphic: Módulo de GEN Framework dedicado a pantallas, canvas, bitmaps, fuentes, vídeo, gráficos vectoriales y backends de renderizado.

GZip: Formato de compresión que normalmente encapsula un flujo DEFLATE y añade cabecera y comprobación de integridad.

Handler: Identificador o referencia opaca que representa un recurso gestionado por otro sistema, como una ventana, un fichero o una conexión.

Hash: Resumen de longitud fija calculado a partir de unos datos. Sirve para comprobación, identificación o criptografía según el algoritmo utilizado.

Hilo: Secuencia de ejecución dentro de un proceso. Compartir memoria entre hilos exige sincronización y contratos claros sobre el acceso concurrente.

HTTP / HTTPS: Protocolos de aplicación utilizados en la web. HTTPS añade protección mediante TLS para autenticar y cifrar la comunicación.

I2C: Inter-Integrated Circuit. Bus serie síncrono de dos líneas utilizado para conectar sensores, memorias y periféricos cercanos.

IDE: Integrated Development Environment. Aplicación que reúne edición, navegación, compilación, depuración y otras herramientas de desarrollo.

Include: Directiva del preprocesador que incorpora declaraciones de un fichero de cabecera. También se usa el término para referirse a la dependencia declarada.

INI: Formato de configuración textual organizado normalmente en secciones y pares clave-valor. Es simple de leer y editar.

Input: Módulo de GEN Framework que abstrae botones, cursores y dispositivos de entrada para que la aplicación no dependa directamente de cada API nativa.

Instancia: Objeto concreto creado a partir de una clase. Varias instancias del mismo tipo pueden mantener estados distintos.

Interfaz: Contrato visible que separa lo que un componente ofrece de cómo lo implementa. Puede expresarse mediante clases, funciones o tipos comunes.

Interfaz gráfica: Conjunto de elementos visuales e interactivos con los que trabaja el usuario: ventanas, layouts, controles, texto, imágenes y eventos.

IP: Internet Protocol. Protocolo de red que identifica destinos mediante direcciones y permite encaminar paquetes entre sistemas.

4. J-M

JavaScript: Lenguaje de scripting integrado como una de las opciones del módulo Scripts para ejecutar lógica cargada en tiempo de ejecución.

JPEG: Formato de imagen comprimida, normalmente con pérdida, adecuado para fotografías y escenas con variación continua de color.

JSON: Formato textual de intercambio de datos basado en objetos, listas y valores. Resulta habitual en configuración y servicios web.

Layout: Descripción de la organización espacial de elementos de una interfaz. Define jerarquía, posición, tamaño y reglas de adaptación.

LGPL: GNU Lesser General Public License. Licencia con copyleft limitado que permite determinados usos como biblioteca bajo condiciones concretas de enlace y redistribución.

Licencia permisiva: Licencia de código abierto que permite reutilización amplia con obligaciones normalmente centradas en conservar avisos y atribución. MIT y BSD son ejemplos habituales.

Log: Registro persistente de hechos relevantes para auditoría o análisis posterior. A diferencia de una traza interactiva, suele conservarse y rotarse.

Lua: Lenguaje de scripting pequeño y embebible que puede integrarse para ejecutar reglas o extensiones sin recompilar toda la aplicación.

LZW: Lempel-Ziv-Welch. Algoritmo de compresión sin pérdida basado en un diccionario que se amplía mientras procesa los datos.

Macro: Sustitución controlada por el preprocesador antes de compilar. GEN Framework las utiliza para opciones, plataforma, diagnóstico y algunos patrones comunes.

Máquina de estados: Modelo que representa un sistema mediante estados y transiciones. Ayuda a expresar ciclos de vida y comportamientos reactivos de forma explícita.

MariaDB: Motor de base de datos relacional compatible en gran medida con el ecosistema MySQL y accesible desde el bloque Databases.

Memoria dinámica: Memoria solicitada durante la ejecución cuyo ciclo de vida controla el programa. Debe liberarse de forma ordenada para evitar fugas.

Módulo: Bloque funcional del framework con una responsabilidad definida, sus propias clases y dependencias controladas.

MSVC: Microsoft Visual C++. Compilador y toolchain principal de Visual Studio para aplicaciones nativas en Windows.

Multiplataforma: Capacidad de mantener una arquitectura y gran parte del código para varios destinos, generando en cada plataforma su binario y adaptaciones correspondientes.

Mutex: Objeto de exclusión mutua que permite a un solo hilo acceder a una sección crítica cada vez.

MySQL: Motor de base de datos relacional cliente-servidor soportado mediante las abstracciones del módulo Databases.

5. N-R

Namespace: Espacio de nombres que agrupa identificadores y evita colisiones entre clases, funciones o constantes de procedencias distintas.

NDK: Native Development Kit. Conjunto de herramientas de Android para compilar e integrar código nativo C/C++.

Ninja: Sistema de construcción ligero diseñado para ejecutar con rapidez las reglas generadas, por ejemplo, por CMake.

Nodo: Elemento de una estructura enlazada o jerárquica. En DataIO puede representar partes de información externa junto con valores y manejadores.

NULL: Representación tradicional de un puntero nulo. En C++ moderno suele preferirse nullptr porque expresa mejor el tipo.

Objeto: Instancia con identidad, estado y comportamiento. Las relaciones entre objetos permiten componer funcionalidades mayores.

OGG: Formato contenedor abierto utilizado habitualmente para audio comprimido y otros flujos multimedia.

OpenAL: API multiplataforma de audio posicional. El bloque Sound puede utilizarla como backend de reproducción.

OpenGL: API gráfica multiplataforma para renderizado acelerado. GEN Framework la utiliza a través de capas que aíslan la creación de contexto y la plataforma.

Open source: Software cuyo código fuente está disponible bajo una licencia que permite estudiarlo y reutilizarlo con condiciones definidas. No significa ausencia de obligaciones.

Paquete: Unidad de información transmitida o distribuida. Según el contexto puede ser un bloque de red, un conjunto instalable o un grupo de recursos.

PATH: Variable o lista de rutas donde el sistema busca ejecutables. También se usa path como término general para una ruta de fichero o directorio.

PEM: Formato textual basado en Base64 y cabeceras legibles que se usa para claves, certificados y otras estructuras criptográficas.

Pipeline: Cadena de etapas por la que pasan datos o tareas. Cada etapa recibe un resultado, lo transforma y lo entrega a la siguiente.

Plataforma: Combinación de hardware y entorno de ejecución, normalmente un procesador o microcontrolador junto con su sistema operativo. Puede existir sin sistema operativo.

Plugin: Componente cargable que amplía una aplicación sin formar parte obligatoria de su núcleo. Necesita un contrato de integración estable.

PNG: Formato de imagen raster sin pérdida, con soporte para transparencia y adecuado para interfaces, iconos y capturas.

Portabilidad: Facilidad con la que un código puede compilarse y comportarse correctamente en plataformas diferentes con cambios limitados y controlados.

PostgreSQL: Motor de base de datos relacional cliente-servidor soportado a través de las interfaces del módulo Databases.

Protocolo: Conjunto de reglas, formatos y secuencias que permite que dos componentes intercambien información de manera compatible.

Puntero: Valor que referencia una dirección de memoria u objeto. Su uso exige controlar validez, propiedad y ciclo de vida.

Puerto serie: Canal de comunicación que transmite datos de forma secuencial. UART es una tecnología frecuente para implementarlo en hardware.

Punto de entrada (main): Función desde la que el sistema inicia un programa tradicional. AppFlow puede ocultar sus diferencias de plataforma y delegar el ciclo en métodos comunes.

RAII: Resource Acquisition Is Initialization. Técnica de C++ que vincula la vida de un recurso a la de un objeto para liberarlo automáticamente al destruirse.

Raspberry Pi: Familia de ordenadores de placa única, normalmente con procesadores ARM, utilizada como ejemplo de plataforma Linux compacta.

Renderizado: Proceso que convierte una escena, geometría, texto o recursos visuales en la imagen final mostrada en pantalla o almacenada.

Repositorio: Almacén versionado de código y recursos. GEN Framework se distribuye en varios repositorios con responsabilidades distintas.

Recurso: Elemento que una aplicación necesita y debe gestionar, como memoria, ficheros, imágenes, sonidos, conexiones o identificadores del sistema.

RSA: Algoritmo criptográfico asimétrico utilizado para operaciones con claves públicas y privadas, firmas o intercambio protegido según el protocolo.

Runtime: Servicios y entorno disponibles mientras un programa se ejecuta. Puede incluir bibliotecas, cargadores, intérpretes o soporte del sistema.

6. S-U

Script: Programa o fragmento interpretado o compilado en tiempo de ejecución. Permite incorporar reglas y automatización sin reconstruir toda la aplicación.

SDK: Software Development Kit. Conjunto de cabeceras, bibliotecas, herramientas y documentación para desarrollar sobre una plataforma o producto.

Serialización: Conversión de un objeto o estructura en una representación que puede almacenarse o transmitirse y reconstruirse posteriormente.

Servidor: Componente que escucha solicitudes y ofrece datos o servicios a uno o varios clientes mediante un protocolo definido.

SHA: Secure Hash Algorithm. Familia de funciones hash criptográficas; el libro utiliza especialmente variantes de SHA-2.

Singleton: Patrón que proporciona una única instancia accesible de una clase. Es útil para ciertos gestores, aunque introduce estado global que debe controlarse.

Sistema embebido: Sistema informático integrado en un producto y dedicado a funciones concretas, a menudo con límites estrictos de memoria, energía o tiempo.

Sistema operativo: Software que gestiona procesos, memoria, dispositivos, ficheros y otros recursos de una plataforma. GEN Framework confina sus diferencias tras capas específicas.

Socket: Extremo de una comunicación de red identificado por protocolo, dirección y puerto. Puede utilizar TCP, UDP u otros transportes.

Sound: Módulo de GEN Framework dedicado a recursos, reproducción, buffers y backends de audio.

SPI: Serial Peripheral Interface. Bus serie síncrono utilizado para comunicar un controlador con sensores, memorias y otros periféricos.

SQL: Structured Query Language. Lenguaje utilizado para definir, consultar y modificar datos en bases de datos relacionales.

SQLite: Motor de base de datos relacional integrado en un fichero y enlazado dentro de la aplicación, sin servidor separado.

Stream: Abstracción de un flujo ordenado de bytes o mensajes. Permite tratar de forma parecida ficheros, sockets, puertos serie y otros canales.

SVG: Scalable Vector Graphics. Formato XML para describir gráficos vectoriales escalables mediante formas, trazados, texto y estilos.

Target: Destino concreto de una construcción o ejecución. Puede identificar una arquitectura, una plataforma, una biblioteca, un ejecutable o un canal de trazas.

TCP: Transmission Control Protocol. Transporte orientado a conexión que entrega un flujo de bytes ordenado y controla pérdidas y retransmisiones.

ThirdPartyLibraries: Repositorio que reúne las bibliotecas de terceros integradas o referenciadas por GEN Framework, junto con sus versiones, licencias y adaptaciones.

TLS: Transport Layer Security. Protocolo que autentica y cifra comunicaciones, utilizado, por ejemplo, por HTTPS.

Toolchain: Conjunto coordinado de herramientas de construcción: compilador, ensamblador, enlazador, bibliotecas, depurador y utilidades asociadas.

Traza: Mensaje de diagnóstico pensado para observar el comportamiento de una aplicación durante su ejecución. Puede incluir nivel, color, origen y bloques de datos.

UDP: User Datagram Protocol. Transporte sin conexión que envía datagramas sin garantizar por sí mismo entrega, orden o retransmisión.

UART: Universal Asynchronous Receiver-Transmitter. Periférico de hardware que implementa comunicación serie asíncrona.

Unicode: Estándar que asigna códigos a caracteres de numerosos sistemas de escritura. Necesita una codificación concreta para almacenarlos.

URL: Uniform Resource Locator. Dirección que identifica la ubicación y el método de acceso a un recurso, especialmente en servicios web.

USB: Universal Serial Bus. Estándar para conectar dispositivos, transferir datos y, en muchos casos, suministrar alimentación.

UserInterface: Módulo de GEN Framework que construye layouts, widgets, estilos y eventos de interfaz sobre Graphic e Input.

UTF-8: Codificación de Unicode de longitud variable, compatible con ASCII y muy extendida en ficheros, comunicaciones y servicios web.

7. V-Z

Vectorial: Representación gráfica basada en geometría, curvas y atributos, no en una cuadrícula fija de píxeles. Puede escalarse sin perder definición geométrica.

Viewport: Región de una superficie de renderizado en la que se proyecta una escena. Define el área visible y su transformación a coordenadas de pantalla.

WAV: Formato contenedor de audio que suele almacenar muestras PCM, aunque admite otras codificaciones.

Widget: Elemento reutilizable de una interfaz gráfica, como un botón, una etiqueta, un campo de texto o una lista.

WSL2: Windows Subsystem for Linux 2. Entorno Linux integrado en Windows mediante virtualización ligera, útil para compilar y probar toolchains Linux.

X.509: Estándar de certificados digitales que vincula una identidad con una clave pública y datos de validez firmados por una autoridad.

X25519: Mecanismo de intercambio de claves basado en Curve25519. Permite que dos partes obtengan un secreto compartido a través de un canal no protegido.

XBUFFER: Clase de XUtils que gestiona secuencias de bytes y operaciones habituales de lectura, escritura, copia y conversión.

XBYTE / XWORD / XDWORD: Tipos enteros de tamaño controlado utilizados para expresar con claridad anchuras de 8, 16 y 32 bits en el código del framework.

XEVENT: Objeto de evento de XUtils. Transporta el tipo de notificación, su origen y la información necesaria para los observadores.

XFACTORY: Infraestructura común de XUtils para crear y acceder a factorías que seleccionan implementaciones concretas.

XFILE: Abstracción de XUtils para trabajar con ficheros sin dispersar por la aplicación los detalles de cada sistema operativo.

XFSMACHINE: Clase base para implementar máquinas de estados finitos, sus estados, eventos y transiciones.

XLOG: Servicio de XUtils orientado a conservar registros persistentes y organizarlos para su consulta posterior.

XML: Extensible Markup Language. Formato textual jerárquico basado en etiquetas, atributos y contenido.

XPATH: Clase de XUtils que representa y manipula rutas de ficheros y directorios. No debe confundirse con el lenguaje XPath de XML.

XPATHMANAGER: Gestor que centraliza raíces y secciones de rutas, como configuración, recursos, gráficos, scripts o datos.

XSTRING: Clase de cadena de XUtils utilizada para texto, conversiones, formato y operaciones comunes en los módulos del framework.

XTRACE: Subsistema de XUtils que crea y envía trazas a uno o varios targets de diagnóstico.

XUtils: Módulo base de GEN Framework. Aporta tipos, cadenas, buffers, ficheros, rutas, colecciones, tiempo, eventos, hilos, trazas y otras utilidades transversales.

XVARIANT: Contenedor capaz de representar valores de tipos distintos. Resulta útil al intercambiar información genérica, especialmente con bases de datos.

XVECTOR: Contenedor dinámico de XUtils para mantener colecciones ordenadas de elementos con una interfaz común al resto del framework.

ZIP: Formato contenedor de archivos comprimidos que permite reunir varios ficheros y metadatos en una sola unidad.